

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

DOCTORAL THESIS

Mgr. Jáchym Bártík

**A Unified Framework for Multi-Model
Data Management and Adaptation**

Department of Software Engineering

Supervisor of the doctoral thesis: doc. RNDr. Irena Holubová, Ph.D.

Advisors of the doctoral thesis: Ing. Pavel Koupil, Ph.D.

prof. RNDr. Roman Barták, Ph.D.

Study programme: Computer Science

Study branch: Software Systems

Prague 2026

I declare that I carried out this doctoral thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

This thesis would never have been possible without the expertise, guidance, and support of my supervisor, Irena Holubová, as well as my advisors, Pavel Koupil and Roman Barták. I thank them for their invaluable insights, encouragement, and patience throughout my doctoral journey. Besides them, I would like to express my gratitude to the entire staff of the Department of Software Engineering at the Faculty of Mathematics and Physics, Charles University, for providing me with plenty of advice, as well as a great working environment and a friendly atmosphere.

I also owe a lot to my amazing students who, either directly or indirectly, contributed to both the presented research and my personal growth as an academic. A few of them are named on the following pages as co-authors of the included papers; this paragraph is for all the others who could not be mentioned explicitly.

Arguably the most important part of my technical expertise comes from the time spent with my friends at Flowlance. I am grateful for their patience and for all the compromises they made to accommodate my academic work, especially when deadlines were tight and issues were piling up.

I am thankful to my family for their unwavering support and understanding, to which I can always turn in times of need.

And last but not least, I would like to thank my colleague Štěpán Stenclák, because without his naivety, I would never have started this journey in the first place.

Title: A Unified Framework for Multi-Model Data Management and Adaptation

Author: Mgr. Jáchym Bártík

Department: Department of Software Engineering

Supervisor: doc. RNDr. Irena Holubová, Ph.D., Department of Software Engineering

Advisors: Ing. Pavel Koupil, Ph.D., Department of Software Engineering, and prof. RNDr. Roman Barták, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: Multi-model databases combine relational, document, graph, and other data models, but their management remains fragmented across incompatible schemas, query languages, and adaptation mechanisms. Building on an existing category-theoretic foundation for multi-model schemas, mappings, transformations, and querying, this thesis develops methods for propagating conceptual schema changes to logical schemas, and queries, as well as a self-adaptation approach that searches for better logical configurations under changing workloads. The adaptation process combines workload-driven search with learned cost estimation, allowing candidate configurations to be compared only from query plans. The thesis contributes formal models, propagation and synchronization algorithms, and an experimental validation. Together, the results show that multi-model data management can be described coherently, automated to a substantial degree, and adapted to evolving application needs.

Keywords: multi-model databases, query processing, schema evolution, self-adaptation, category theory

Contents

Preface	1
Commentary	3
0.1 Vision	6
0.1.1 Level of Automation	6
0.1.2 Challenges	7
0.1.3 Research Questions	8
0.2 Foundations	9
0.2.1 Modeling	11
0.2.2 Transformations	14
0.2.3 Querying	14
0.2.4 MM-cat framework	17
0.2.5 Related Work	18
0.3 Evolution Management	20
0.3.1 Evolution Hierarchy	21
0.3.2 Change Propagation	22
0.3.3 Level of Automation	25
0.3.4 Related Work	27
0.3.5 Contributions	28
0.4 Self-adaptation	30
0.4.1 Index Optimization	30
0.4.2 Problem Formulation	31
0.4.3 MM-mapsearch	32
0.4.4 Related Work	36
0.4.5 Contributions	37
0.5 Complementary Approaches to Multi-Model Data Management	39
0.5.1 Benchmarking & Evaluation	39
0.5.2 Schema-free Query Unification	42
0.5.3 Contributions	45
1 AI-Powered Orchestration of Multi-Model Data	47
1.1 Introduction	48
1.2 Initial Framework	49
1.2.1 Transformations	49
1.2.2 Querying	50
1.2.3 Evolution Management	51
1.3 Future Plans	53
1.3.1 AI for DB	53
1.3.2 Data for Training	54
1.4 Open problems	54
2 MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data	56
2.1 Introduction	57
2.2 Related Work	58

2.3	Categorical Conceptual Model	59
2.4	Evolution Manager <i>MM-evocat</i>	61
2.4.1	Schema Modification Operations	62
2.5	Demonstration Outline	64
3	MM-evoque: Query Synchronisation in Multi-Model Databases	66
3.1	Introduction	67
3.2	Related Work	68
3.3	Categorical Model and Querying	68
3.4	<i>MM-evoque</i>	69
3.4.1	Workflow	77
3.4.2	SMO-to-MMQL Propagation	77
3.5	Demonstration Outline	78
4	Model-Agnostic Evolution Management	79
4.1	Introduction	80
4.2	Categorical Representation of Multi-Model Data	81
4.2.1	Conceptual Querying of Multi-Model Data	82
4.3	Evolution Management Approach	82
4.3.1	Schema Modification Operations	82
4.3.2	Propagation to Data	84
4.3.3	Propagation to Queries	86
4.4	Implementation and Evaluation	90
4.4.1	Propagation to Data Instances	91
4.4.2	Propagation to Operations	93
4.5	Related Work	93
4.6	Conclusion	94
5	FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts	96
5.1	Introduction	97
5.2	Core Concepts	98
5.3	Workflow	99
5.4	Related Work	101
5.5	Demonstration Outline	102
5.6	Conclusion and Outlook	103
6	Refining Storage Strategy Through Index Selection Methods in Multi-Model Database Systems: A Survey	104
6.1	Introduction	105
6.2	Review of popular database systems	107
6.2.1	Use of indexes in database systems	107
6.2.2	Relational database systems	107
6.2.3	Graph database systems	108
6.2.4	Key-value database systems	109
6.2.5	Document database systems	110
6.2.6	Columnar database systems	110
6.2.7	Multi-Model data	111
6.2.8	Multi-Modal data	112

6.2.9	Comparison of selected database systems	113
6.3	Definitions & Notation	113
6.4	Index selection algorithms	116
6.4.1	General formulation of ISP algorithms	118
6.4.2	Linear Programming based algorithms	119
6.4.3	Greedy algorithms	122
6.4.4	Reinforcement Learning based algorithms	129
6.5	Comparison of the algorithms	132
6.5.1	Index selection in Relational DBMSs	132
6.5.2	Index selection in Graph DBMSs	134
6.5.3	Index selection in Key-Value DBMSs	134
6.5.4	Index selection in Document DBMSs	135
6.5.5	Index selection in Columnar DBMSs	135
6.5.6	Index selection in multi-model and multi-modal data . . .	136
6.6	Open questions and challenges	136
7	MM-mapsearch: Workload-Aware Mapping Selection	139
7.1	Introduction	140
7.2	Proposed Approach	141
7.2.1	Categorical Multi-Model Representation	142
7.2.2	Mapping-Space Exploration using MCTS	142
7.2.3	Plan-Structured Neural Predictor	145
7.3	Architecture and Implementation	145
7.4	Demonstration Outline	146
8	Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs	149
8.1	Introduction	150
8.2	Related Work	151
8.2.1	Repositories	151
8.2.2	Generators	152
8.3	Categorical View and Management of Multi-Model Data	153
8.3.1	Categorical Representation of Multi-Model Data	153
8.3.2	Categorical Multi-Model Data-Management Toolset	154
8.4	Multi-Model Transformation Framework	154
8.4.1	Architecture	161
8.4.2	Evaluation of the Proposed Solution	161
8.5	Conclusion	163
9	TransforMMer: A Universal Multi-Model Data Generator	164
9.1	Introduction	165
9.2	Related Work	166
9.3	Categorical Model of Data	167
9.4	TransforMMer	167
9.4.1	Sample Scenario	168
9.4.2	Workflow	170
9.4.3	Architecture	172
9.4.4	Dataset Repository	173
9.5	Demonstration Outline	173

10 DortDB: Bridging Query Languages for Multi-Model Data	
Ponds	174
10.1 Introduction	175
10.2 Basic Concepts	177
10.2.1 Unified Algebra	178
10.3 Architecture and Implementation	179
10.4 Related Work	179
10.5 Demonstration Outline	181
Conclusion	183
Bibliography	187
List of Figures	201
List of Tables	205
List of Abbreviations	207
List of Publications	209

Preface

This thesis summarizes the results of the author’s doctoral research in the field of multi-model data management. The research was conducted at Charles University in the Adaptive Data Management (ADaM) research group, particularly in collaboration with doc. RNDr. Irena Holubová, Ph.D., and Ing. Pavel Koupil, Ph.D. Some of the results are the product of collaboration with members of the Scalable Database Systems research group, led by Prof. Dr. Stefanie Scherzinger at the University of Passau. The author also participated in supervising several related master’s theses [1, 2, 3, 4].

The central claim of this thesis is that multi-model data management can support self-adaptation by combining a category-theoretic representation of schemas and mappings with evolution propagation, workload-aware configuration search, and performance estimation. This claim is supported through the results presented in a collection of ten papers published in peer-reviewed conferences and journals in the years 2022–2026. Table 1 provides an overview of these papers and their roles in the thesis. A full [list of the author’s publications](#) is included at the end of the thesis.

The thesis builds on previous work by the research group. Specifically, this theoretical foundation includes a unified categorical representation, transformations, schema inference, and querying. The original contribution then lies in evolution management, workload-aware mapping selection, supporting benchmarking infrastructure, and schema-free query unification. For each of these topics, we identify research challenges, propose solutions, and discuss the contributions of the individual papers.

This research was supported by the Czech Science Foundation (GAČR) grants no. 20-22276S and 23-07781S, the Charles University Grant Agency (GAUK) grant no. 292323, and the Bayerisch-Tschechische Hochschulagentur Akademische Projekte grant no. BTHA-AP-2023-21.

Paper	Role in thesis	Section	Topic	Venue	Year	Rank
I	supporting	0.1	vision	VLDB	2024	A*
II	core	0.3	schema evolution	CIKM	2022	A
III	core	0.3	query evolution	EDBT	2024	A
IV	core	0.3	change propagation	RCIS	2025	B
V	supporting	0.3	FD discovery	VLDB	2025	A*
VI	supporting	0.4	index selection	DKE	2026	Q2
VII	core	0.4	self-adaptation	EDBT	2026	B
VIII	boundary	0.5	dataset generation	ENASE	2025	B
IX	boundary	0.5	benchmarking	EDBT	2025	A
X	boundary	0.5	query unification	VLDB	2025	A*

Table 1 Papers of the thesis. Rankings are according to CORE/JCR.

Commentary

Data management is a broad field encompassing data modeling, storage, querying, and evolution. For years, data management research focused primarily on relational data, which is structured in tables with fixed schemas [5]. In the early 2000s, the rise of NoSQL databases [6] popularized new data models and formats, such as document, graph, key-value, and wide-column.

These models offer distinct advantages for certain types of applications and workloads, making them popular choices for handling unstructured, highly dynamic, or large-scale data, all of which are common characteristics of modern data. However, this diversity of data models comes with its own set of challenges. Moving data between databases that use different data models is much more complex and error-prone than migrating data from one relational database to another. In general, the NoSQL world lacks unified theoretical foundations, common standards, and tools that leverage them to manage data across various models [7].

Example 0.1. The *Yelp Academic Dataset*¹ consists of JSON collections representing *businesses* (e.g., restaurants), *users* (who write *reviews* and tips about the *businesses*), and the reviews and tips themselves. It also includes a social network of *friends*, as well as a set of *check-ins* and *photos* associated with the businesses.

For the rest of this thesis, we use a subset of this dataset, shown in Figure 0.1. Originally, the data was stored in a document model, but we transformed it into a multi-model representation. The **user** is a table in a relational model, the **business** is a collection in a document model, the **FRIEND** is a relationship in a graph model, and the **review** is a column family in a wide-column model. Crucially, the data from different models is linked via the same identifiers, allowing us to combine results from multiple queries. □

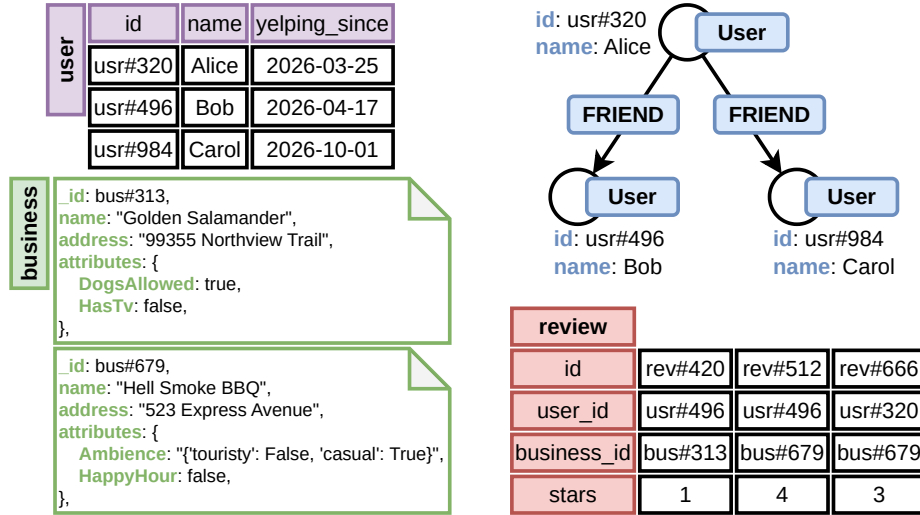


Figure 0.1 Multi-model data (running example). A subset of the Yelp Academic Dataset from Example 0.1, organized into different data models—relational (purple), document (green), graph (blue), and wide-column (red)—and connected via the same identifiers.

The various data models in Example 0.1 illustrate the advantages of each

¹<https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>

model for different types of data. For example, the relational model is suitable for structured data with fixed schemas, while the document model is better suited to semi-structured data with flexible schemas (e.g., the many different options in the `attributes` field). The graph model excels at complex queries, such as finding the shortest path between two users in a social network, and the wide-column model is optimized for high-throughput scenarios. On the other hand, this specialization comes with a cost: even though we can combine results from multiple queries, doing so is not straightforward and, if done incorrectly, can introduce semantic inconsistencies.

In real-world scenarios, complex use cases involving multiple data models and, eventually, database systems are becoming increasingly common. Such data is often referred to as **multi-model data** (in contrast to the traditional **single-model data** approach).

The architectural decision to use a separate database system for each model is called **polyglot persistence** [6]. It is a common approach in modern applications as it allows developers to choose the optimal database for each part of the application based not only on the data model but also on other factors, such as scalability, consistency, and availability. **Polystores** [8] address polyglot persistence by providing a unified interface for querying and managing data across multiple databases. However, they still have to deal internally with the heterogeneity of the underlying databases. Conversely, **multi-model databases** [7] combine multiple data models into a single database system. This tighter integration can reduce some coordination overhead and provide a more uniform interface for users. On the other hand, it is less flexible and more difficult to implement—to the point that no multi-model database actually supports all commonly used data models [9].

These challenges motivate the unified approach to multi-model data management developed in this thesis.

Outline

The rest of the [Commentary](#) chapter provides a high-level overview of the thesis:

- Section [0.1](#) formulates the vision and research goals of our overall research (Paper [I](#)) and the challenges of multi-model data management.
- Section [0.2](#) defines the common terminology and describes our previous work on modeling and querying multi-model data.
- Section [0.3](#) proposes an approach to evolution management (Paper [II](#)), which is then extended with propagation to queries (Paper [III](#)) and the ability to choose between different propagation strategies (Paper [IV](#)). This is enabled by a novel take on functional dependency discovery (Paper [V](#)).
- Section [0.4](#) first reflects on our survey of index selection in heterogeneous database systems (Paper [VI](#)) and then proposes an algorithm that allows the system to automatically choose a suitable multi-model configuration for a given workload (Paper [VII](#)).

- Section 0.5 discusses generating synthetic multi-model data and queries (Papers VIII, IX), which are crucial for benchmarking and testing the proposed approach. Then it explores an alternative approach to querying multi-model data (Paper X).

The following ten chapters contain the individual papers adapted to the thesis format. Finally, [Conclusion](#) summarizes the contributions of this thesis and outlines future research directions.

0.1 Vision

When it comes to data (and for that matter, any kind of data, not just multi-model), what we really want is to extract information from it—i.e., to ask questions about it and obtain answers—in a process commonly known as **querying**. In the context of multi-model data, this process naturally extends across the boundaries of different models and, in the case of polyglot persistence, across database systems as well.

Besides this primary goal, several additional milestones help us achieve it. We need to be able to **model** data so that we can reason about its structure and semantics—a necessary prerequisite for querying and for all of the capabilities described below. Sometimes, data is modeled first and created later. In other cases, we want to **infer** the schema from the data.

Both modeling and inference are closely related to the process of **evolution**. Whenever the schema changes, we have to propagate those changes to both the data and the queries that depend on it. In a multi-model data world, this might require us to **transform** the data between different models.

Lastly, our motivation concerns not only the correctness and completeness of multi-model data management, but also its efficiency. The primary functionality of a database system—querying data—should be as efficient as possible. Multi-model data offers additional opportunities to optimize querying by choosing the most appropriate combination of models for the task at hand. Doing so manually is very difficult, so we want the system to be able to adapt its internal data representation—in a process called **self-adaptation**—to the current workload and data characteristics.

0.1.1 Level of Automation

Data management can be automated to various degrees. More automation is generally beneficial, as it reduces the amount of manual work and expertise required from users. It also allows the system to find solutions that might not be obvious to users or that would be too time-consuming to identify manually. Most importantly, an automated process can reduce manual effort and the risk of certain human errors.

The price of this is increased system complexity and potentially reduced control over the process. Therefore, we introduce automation gradually, following three distinct levels based on who the primary actor is [10]:

- **User-driven** – Users explicitly *model* the data by creating a schema. They can create *queries* over the schema and *transform* the data between models. They can also *evolve* the schema, and the system will propagate the changes to the data and queries.
- **Data-driven** – The system automatically *infers* the schema from the data. Whenever the structure of the data changes—either due to user actions or another process—the system *infers* those changes and updates the schema accordingly.
- **System-driven** – The system uses *self-adaptation* techniques to proactively analyze the workload and data characteristics so that it can decide on its

own to *transform* the data into a new configuration of models that is most suitable for the current workload.

These levels are not strictly disjoint; they describe the dominant source of decisions and control.

0.1.2 Challenges

The main source of complexity in multi-model data management is the heterogeneity of the participating models. Each of them embodies its own unique, and often contradictory, assumptions about how data is structured, accessed, and queried. This makes it difficult to reason about the data as a whole and to design algorithms and systems that can operate across multiple models. In the [NoSQL](#) world, this heterogeneity is further amplified by the lack of common standards and tools for managing data across different models [11].

Some models (document, wide-column, key-value) are aggregate-oriented and naturally group related information into nested structures, while others (relational, graph, array) are aggregate-ignorant and prefer decomposition into flat records connected by foreign keys or references. Another important division is between schema-less models (document, key-value, wide-column), which allow flexible or only partially explicit structure, and schema-full models (relational), which require a fixed schema. This diversity is not limited to the data structures themselves. It also affects the associated query languages, integrity assumptions, and optimization techniques. Flat relational data and hierarchical [JSON](#) documents call for different operators, just as table joins and graph traversals reflect different views of connectivity.

The theoretical data models have to be implemented in actual database systems, which adds another layer of complexity. Many [NoSQL](#) systems are distributed by design, enabled by relaxed consistency and availability requirements ([BASE](#)). This stands in stark contrast to traditional centralized relational systems, which are generally centralized and provide strong consistency guarantees ([ACID](#)). Differences in scalability mean that there is usually no “one-size-fits-all” solution—the advantages of some databases or models become apparent only at certain scales, making any optimization approach even more dependent on the workload. On top of that, each database system has its own performance characteristics, leading to specific design decisions. For example, while normalization is a common practice in relational databases, document databases often prefer denormalization and redundancy to optimize read performance.

In fact, more than half of the 50 most popular database systems [9] are multi-model, i.e., they support multiple data models. Therefore, we have to consider not only different data models but also different combinations of models and their implementations across various database systems. The vast majority of these databases are not *natively* multi-model, but rather started as single-model systems and later added support for other models. This means that multi-model support is often limited to a subset of the features of the additional models. Internally, the secondary models are typically implemented as interfaces to the primary model, which can lead to performance and consistency issues.

0.1.3 Research Questions

The vision described above is based on Paper I and captures the long-term goals of our research. Let us now refine it into concrete research questions. Some of them have already been addressed in previous work by our research group [12, 13, 14, 15]:

- **Modeling** – How can heterogeneous data models be represented in a common semantic framework without erasing their model-specific structure?
- **Transformations** – How can data be transformed between different models without the need to write custom transformation rules for each pair of models?
- **Querying** – How can queries be unified across different data models, supporting a common, practically useful subset of features of each model, and allowing for cross-model queries and optimizations?

We use these answers as a foundation for the rest of our research. In this thesis, we address the following questions:

- **Evolution** – How can changes be propagated across conceptual and logical schemas, data, and queries in a multi-model environment? How can uncertain or ambiguous evolution decisions be supported by data-derived semantic evidence?
- **Self-adaptation** – How can a multi-model system choose suitable representations for a workload without exhaustively materializing and benchmarking all alternatives?
- **Benchmarking** – How can such systems be evaluated when real multi-model benchmarks and workloads are scarce?
- **Schema-free Query Unification** – How can multi-model queries be unified when a shared conceptual schema is unavailable?

Many of these problems are common to single-model data management, but they are significantly more challenging in the multi-model world. In the following sections, we describe the specific challenges that we address, as well as our contributions to solving them.

0.2 Foundations

Our work is based on a preexisting approach to multi-model data modeling [12, 13], querying [14, 15], and inference [16, 17].² In this section, we introduce its main concepts and describe how it evolved into a comprehensive multi-model data management framework called **MM-cat**. We first clarify the terminology and the levels of abstraction used throughout the thesis. Next, we briefly introduce the categorical ideas on which the approach is built. We then show how the same conceptual representation supports both modeling and querying, which is the key reason it also later serves as the basis for evolution management and self-adaptation.

Common terminology

We distinguish three abstraction levels commonly used in database design: **conceptual**, **logical**, and **physical**. There is no universal agreement on their exact definitions; in this thesis, we use the following interpretation. A conceptual schema is a *platform-independent* description of the domain of interest. It captures business concepts and their relationships without committing to any particular database technology. Logical schemas are *platform-specific* descriptions of how selected parts of the conceptual schema are represented in concrete data models. For example, a classical relational representation is flat, whereas a document representation can naturally express nesting. Moreover, particular database systems may further restrict or extend these schemas through their specific features. Finally, a physical schema describes the specific data structures used to store the data and implement indexes. In this thesis, we focus on the conceptual and logical levels; however, we also discuss physical aspects when they are relevant to the overall approach.³

Each data model, or even each database system, has its own terminology for the concepts and data structures it uses. Let us define the following unifying terms:

- **kind** – a class of items (e.g., a relational table, a collection of documents, or a collection of nodes/edges with the same label),
- **attribute** – a field of a kind (e.g., a column in a table, a field in a document, or a property of a node/edge).

These terms unify the concepts from the logical level. On the physical level, we have:

- **record** – an item of a kind (e.g., a table row, a document, or a node/edge),
- **value** – a value of an attribute in a record (e.g., a cell in a table row, a value of a field in a document, or a value of a property of a node/edge).

Finally, the terminology on the conceptual level is defined in a way that abstracts away any model-specific details:

²Unless explicitly stated otherwise, we use “we” and “our” to reference the previous work of our research group on these topics.

³These levels can be roughly mapped to the terminology defined by **MDA** [18]—the conceptual level corresponds to the **PIM**, while the other two levels correspond to the **PSM**. Nevertheless, we do not use **MDA** terminology in the rest of the thesis.

- **simple type** – a value-carrying element (e.g., id, name, email),
- **complex type** – a structure consisting of simple types (e.g., user, business, review),
- **projection** – a directed connection between a complex type and another (simple or complex) type (e.g., a user has a name, a review is about a business),
- **properties** [of a complex type] – a set of all simple types that are connected to the given complex type via a projection.

While a kind has to correspond to a complex type, an attribute can correspond to either a complex type (e.g., a nested document in a document database) or a simple type (e.g., a column in a relational table). Relationships can be expressed in many ways (e.g., references, embedding), so we do not define a single term for them.

For illustration, let us show how these terms correspond to the classical [ER](#) model [19] on the conceptual level:

- complex type – entity/relationship type,
- simple type – attribute of an entity/relationship type,
- projection – edge from an entity/relationship type to an attribute or another entity/relationship type.

The most significant difference is that we use complex types for both entity and relationship types because some data models (e.g., graph) treat relationships as first-class citizens. In general, we do not use [ER](#) terminology in the rest of the thesis.

Category Theory

Category theory provides a concise language for describing structures and the relationships between their parts. In this thesis, we use it as a modeling tool—it lets us describe data from different models in a common formalism while still preserving enough structure for querying, evolution, and transformation.

Let us start with a definition. A **small category** $\mathcal{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of **objects** $\mathcal{O}$, a set of **morphisms** $\mathcal{M}$, and a composition operation $\circ$ over morphisms. A morphism $f \in \mathcal{M}$ is a directed connection between two objects $A, B \in \mathcal{O}$, denoted $f : A \rightarrow B$. For each object $A \in \mathcal{O}$, there is an **identity morphism** $1_A : A \rightarrow A$. Finally, the composition operation $\circ$ is associative and has the identity morphisms as its neutral elements.

In full generality, a **category** can have *collections* of objects and morphisms instead of *sets*. In this thesis, however, we only work with small categories. For intuition, it is often helpful to picture such a category as a directed multigraph with objects as nodes and morphisms as edges. However, we still have to keep in mind that a category is not given only by the nodes and edges; it also explicitly captures which paths can be composed and guarantees that composition behaves consistently.

In our setting, morphisms directly specified by the schema are called **base morphisms**, while morphisms obtained by composition are called **composite morphisms**. This distinction is practical rather than purely theoretical. Base morphisms capture explicitly modeled projections, whereas composite morphisms allow us to refer to longer semantic paths through the schema.

A second key notion is a **functor**. A functor provides a structure-preserving mapping between categories. It is a function that maps all objects from the first category to objects in the second category (and similarly for morphisms), such that both identity morphisms and composition of morphisms are preserved.

The categories do not have to be the same size—for example, we can map multiple objects to a single object and the morphisms between them to the identity morphism on that object. Alternatively, we can map two objects connected by a base morphism to two objects connected by a composite morphism, which effectively expands the two objects into a more complex structure. In any case, the preservation of identity and composition ensures structural consistency of morphisms in both categories.

0.2.1 Modeling

The basic idea of our approach is to create a *unified model* of multi-model data at both the conceptual and logical levels, and to use it as the basis for all operations on the data, including querying and evolution. This unified model is based on category theory, enabling us to capture the semantics of all *supported*⁴ data models consistently.

The conceptual schema is a small category called **schema category**. Its objects represent the concepts in the domain of interest—complex and simple types—while its morphisms represent projections between them. An object represents a simple type if there is no morphism from it to any other object; otherwise, it represents a complex type.

Example 0.2. Figure 0.2 shows a schema category that models the data from Figure 0.1. Categorical objects (representing complex and simple types) are displayed as ovals. Each object has an internal identifier (not shown in the figure for simplicity) and a label. By convention, complex types have capitalized labels (e.g., **User**), while simple types do not (e.g., **id**).

Morphisms (representing projections between types) are displayed as arrows. For convenience, we identify each of them by a unique integer **signature**. For example, the morphism with signature 1 corresponds to the projection between the complex type **Review** and its property **id**. Morphisms can also have human-readable labels, but they are not displayed in the figure for simplicity. Identity morphisms are omitted for the same reason, although they are present internally (as required by the definition of a category). $\square$

This definition of schema category slightly differs from the original [12, 13], which required that each morphism had a counterpart in the opposite direction (called **dual morphism**) and allowed each morphism to specify **cardinality** in

⁴The supported models are: relational, document, graph, wide-column, key-value, array. In general, our approach is not limited to these models in particular, but in the context of this thesis, we focus only on them because they are the most widely used and studied.

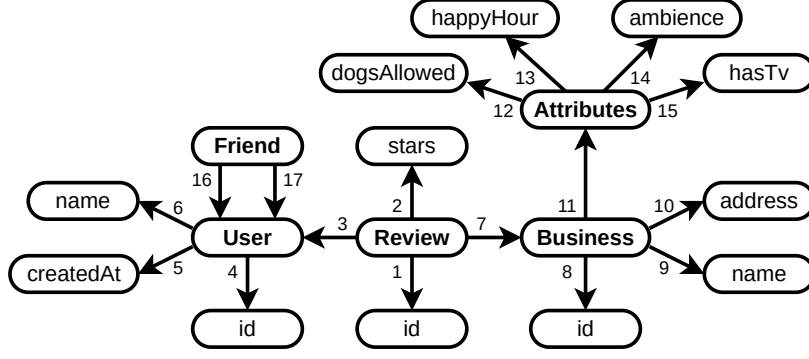


Figure 0.2 Schema category from Example 0.2, modeling the data from Figure 0.1 on the conceptual level.

the same way as an ER model would (e.g., $0..*$). This is intentional, as we⁵ refined the original definition in Paper II.

The current version does have the concept of dual morphisms, and the cardinalities are limited to $0..1$ and $1..1$ *in the direction of the morphism*. For example, in Figure 0.2, we can immediately see that each instance of **User** can have at most one instance of **name** (even without providing any further information). This limitation is crucial to ensure that the conceptual schema can be mapped to any data model, because it requires each N:N projection to be decomposed to an intermediate complex type with two N:1 projections. A more fine-grained cardinality control is achieved through categorical concepts such as **monomorphism** and **epimorphism**.

On the logical level, each kind has its own logical schema. A logical schema is also a schema category, but it is defined over a specific data model. Therefore, its objects and morphisms have very specific semantics; e.g., in a relational model, there would be a complex type representing the table, simple types representing its columns, and projections between them, meaning that the columns belong to the table. Because the underlying kinds are always trees (e.g., a row is just a tree with depth 1), the schema categories of logical schemas are also trees with a distinguished **root object**.

The logical schema category has to conform to the constraints of the underlying data model. For example, in a traditional relational model, columns cannot contain array values, so there cannot be a morphism from a “column object” (i.e., an object representing a column) to a table object. However, in a document model, there is no such restriction, so we can have a morphism from a field object to a document object, representing the fact that the field is an array nested inside the document. We can also rename objects in the logical schema to conform to the naming conventions of the underlying database system or to avoid name clashes with attributes of different kinds.

Finally, each logical schema is connected to the conceptual schema via a **mapping**, which is a functor that maps objects and morphisms between the two schema categories (from the logical to the conceptual). This allows us to preserve the semantics of the conceptual schema while still leveraging the specific features of the underlying data model.

⁵This “we” refers to the work presented in this thesis.

Example 0.3. Figure 0.3 displays a logical schema category for the wide-column database from Figure 0.1 (in red), mapped to the conceptual schema category from Figure 0.2. The root object of the logical schema is `review`, which is mapped directly to the `Review` object in the conceptual schema.

The column family `review` cannot have a column `user`, because `User` is a complex type and the wide-column model does not support nested structures. However, we can **embed** the simple type `id` into `review` by defining the object `user_id`, mapped to the object `id` from the conceptual schema, and the morphism 3.4, mapped to the morphism from `Review` to `id` in the conceptual schema. The dot symbol represents a composite morphism—in this case, morphism 3.4 corresponds to the composition of morphisms 3 and 4.

Because the schema category of a logical schema is always a tree, we can express both it and its mapping to the conceptual schema in a single JSON-like structure. It shows the root object, its attributes, and the morphism signatures to it on the right side of the figure. $\square$

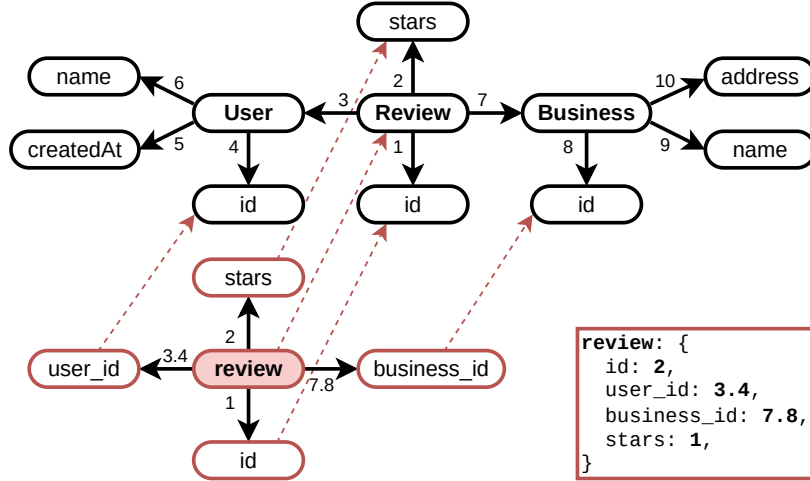


Figure 0.3 A mapping between a logical schema for the wide-column database from Figure 0.1 and the conceptual schema from Figure 0.2. Complex types `Friend` and `Attributes` are omitted for simplicity.

Example 0.4. Logical schemas of other kinds from Figure 0.1 can be mapped to the conceptual schema in a similar way. Figure 0.4 shows the logical schema for the relational database (in purple), the document database (in green), and the graph database (in blue). The mappings are omitted for simplicity, but they are similar to the one in Figure 0.3.

Note that the `Attributes` complex type is represented by the `attributes` document, which is nested inside the `business` document. This is a common pattern in document databases, where related data is often stored together in a single document for performance reasons. In addition, the graph database contains two logical schemas—one for the node label `User` and one for the edge label `FRIEND`. Lastly, the column `yelping_since` in the relational database is mapped to the simple type `createdAt`, demonstrating that objects in the conceptual schema can be renamed as needed. $\square$

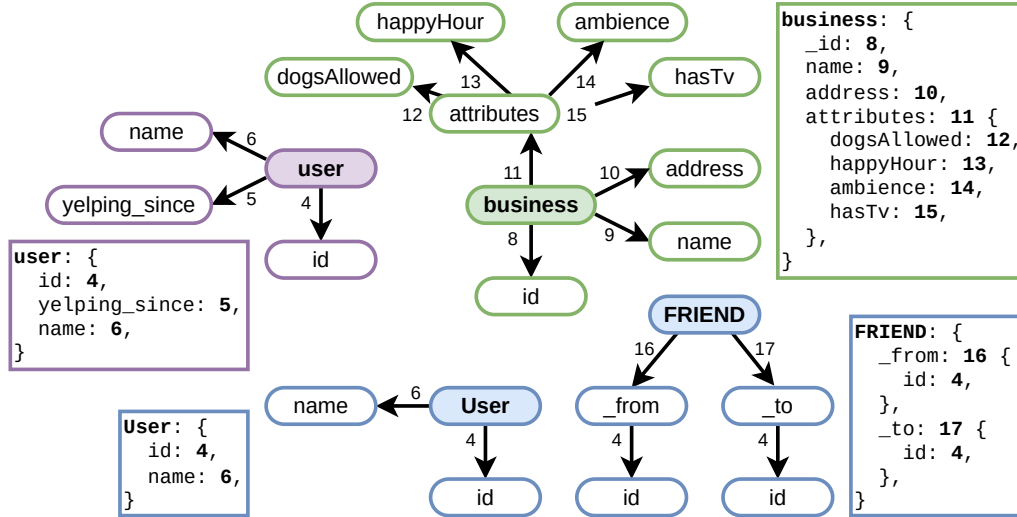


Figure 0.4 Logical schemas for the relational, document, and graph databases from Figure 0.1.

0.2.2 Transformations

In our approach, transformations are a direct consequence of modeling. We defined the **instance category** [12, 13] to have the same structure as the schema category, while also containing the actual data records. We also developed algorithms that leverage the above-mentioned mappings between the conceptual and logical schemas to transform data from a database to an instance category and back [13]. This allows us to transform data between any two models by first converting it to the instance category and then to the target model. The mappings ensure that the semantics of the data are preserved during this process.

The transformation algorithms are mostly database-independent, but they have a few database-specific parts, namely the concrete data structures and the **DDL** and **DML** statements. Therefore, for each supported database, we must construct a **wrapper** that implements all the database-specific functionality. The same approach is used elsewhere as well—for example, during querying, which will be explained later, the algorithms are database-independent; only the translation into the native query languages is database-specific and implemented in the wrapper. Evolution management also works in the same way. Crucially, the underlying algorithms are general enough to support all data models considered in this thesis; the wrappers handle only the technical details of each database.

0.2.3 Querying

Although most relational databases use the standard query language (**SQL** [20]), there are still many differences in the syntax and semantics of the language across database systems. For example, some databases support specific features such as window functions or recursive queries, while others do not. This can make it difficult to write queries that work across multiple databases, even if they all use **SQL**.

In the case of **NoSQL** databases, the situation is even more complex. Each database system has its own query language, tailored to its specific data model.

From SQL to XQuery [21], these languages differ substantially in both syntax and semantics. Even within the same data model, query languages such as SPARQL [22] and Cypher [23] for the graph model offer very different features [11].

There are attempts to unify query languages across different data models, most notably by extending SQL with features tailored to specific data models. However, these extensions are often limited in scope and do not cover all possible queries across all supported data models. For example, SQL/JSON [24] allows querying both relational and JSON data, but it does not support other data models, such as graphs. Similarly, SQL/PGQ [25] supports querying property graphs but not documents.

In a polyglot persistence system, querying multi-model data requires running multiple queries in different query languages and manually combining their results, a complex, error-prone task. Moreover, mastering multiple query languages takes significant time and effort, and it is unrealistic to expect users to be proficient in all of them. This is a major barrier to the efficient use of multi-model data. To address this issue, we want to be able to query multi-model data using a single, unified query language. Such a language should be expressive enough to cover queries across all supported data models and recognize all possible data structures returned by those queries.

Lastly, one of the primary motivations for using multi-model data is the performance gain from selecting the best model for a given task. Therefore, to fully realize these benefits, the overhead of using the unified query language should be minimal, ideally negligible compared with a native solution.

MMQL

These requirements led us to develop the **multi-model query language** (**MMQL**) [14, 15], a query language over the schema category, making it independent of both the underlying data models and the specific databases. After parsing, the query engine splits the query into maximal subqueries that can be executed on a single database, translates them to the native query languages, executes them, and finally combines the results.

The syntax of MMQL is based on SPARQL [22], a well-known and widely used query language for graph data, and the schema category can be represented as a multigraph. Each triple pattern in the query corresponds to a morphism in the schema category, together with its source and target objects. However, the similarity to SPARQL is purely syntactic, as both objects and morphisms in the schema category can have very different semantics depending on the underlying data models and databases. For example, a single morphism can represent a join in a relational database, a relationship kind in a property graph database, or a nested document in a document database.

A single **query plan** is created by matching the patterns in the query to the available logical schemas via the mappings from the schema category. Because of redundancy, more than one combination might be possible. If that happens, multiple query plans are created, and the best one is chosen based on the estimated cost.

Example 0.5. Returning to the schema category from Figure 0.2, we can query the following information: *Find all businesses that allow dogs. For each business, return its*

name and whether it has a TV available. This process is displayed in Figure 0.5.

First, we use the **WHERE** clause to express the pattern we want to match in the logical schemas. The names of the variables (prefixed with the ? symbol) are arbitrary, as the triples are identified by the signatures of the morphisms. The **true** literal acts as an implicit filter. Next, the **SELECT** clause describes the structure of the result, which is an object with two fields—**name** and **hasTV**.

In the example, the **WHERE** pattern is matched to the logical schema of the document database (in this case, MongoDB), so the query engine translates it directly to the native query language. After executing the query, the results are projected to the structure defined in the **SELECT** clause and returned to the user. □

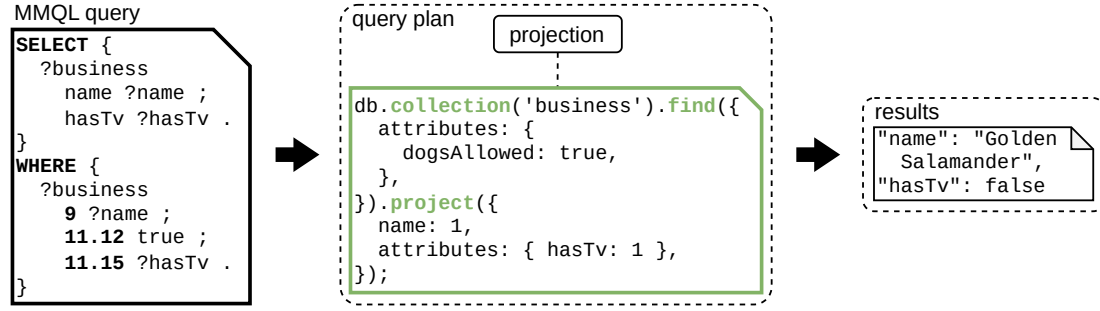


Figure 0.5 A **MMQL** query from Example 0.5 and its translation to the MongoDB query language [26].

Example 0.6. Figure 0.6 shows another query: *Find all reviews with at least 3 stars about a business with id bus#679, ordered by the number of stars (ascending). For each review, return the stars and the author’s name.*

This time, we need **FILTER** and **ORDER BY** clauses to filter and order the results. There is no logical schema that contains all the required information, so the query engine splits the query into two subqueries—one for the wide-column database and one for the relational database. The query engine then joins the results of both subqueries based on the **id** of **User** and projects them into the output structure. □

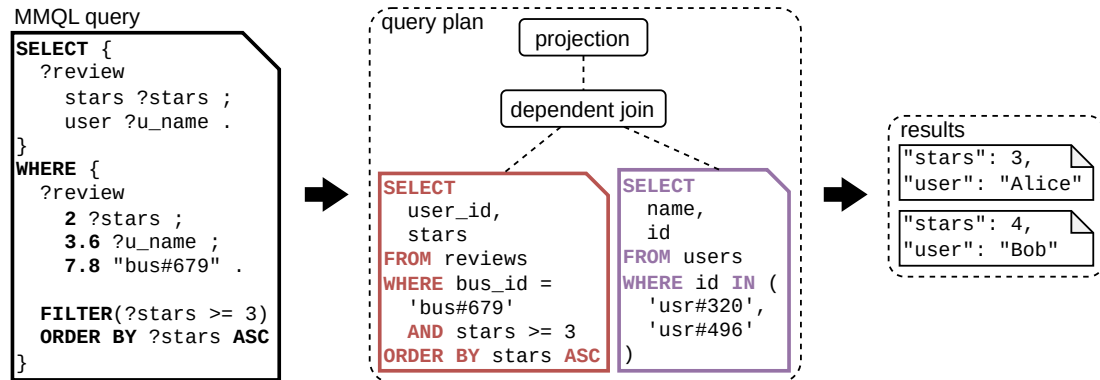


Figure 0.6 A **MMQL** query from Example 0.6 and its translation to **SQL** for a relational and a wide-column database.

Optimizing query plans is crucial to the performance of any database system. In our case, it is even more important, as we have to manage multiple databases simultaneously. We do not try to optimize the native queries themselves, as

that is the responsibility of the underlying systems. Instead, we⁶ have explored, implemented, and evaluated several techniques to improve the performance of the **MMQL** query engine [27] (under review):

- **Predicate pushdown** – In general, it is more efficient to filter the data as early as possible, ideally in the underlying databases, so we push predicates down along the query plan tree. Not all operations can be pushed down to the databases (e.g., cross-database joins), so they are executed in the query engine. In some cases, the databases do not support certain operations (e.g., aggregation) or perform them inefficiently (e.g., lookups in MongoDB), so we have to execute them ourselves as well.
- **Dependent joins** – If one side of a join depends on the results of the other side, we might be able to push this as a “filter” to the database. For example, when two kinds are joined on a key, we can query one of them first and then use the possible key values as a filter for the second one. This is especially beneficial when the first kind is already heavily filtered by other predicates. This is shown in Figure 0.6.
- **Cost estimation** – The cost of plans and subplans is the most important factor for plan creation and selection. We have implemented cache-based cost estimation, allowing us to reuse the results of previous queries and to learn from them over time. We have also implemented a simple cost model based on the database-estimated costs and the number of records processed by each operation, used when there is no information in the cache.

0.2.4 MM-cat framework

The **MM-cat** framework is an umbrella term for a family of tools based on the unified representation described in Section 0.2.1. Together, they provide a proof-of-concept implementation that allows us to test the proposed approach, evaluate its feasibility, and demonstrate its capabilities.

- **MM-cat**⁷ [12] enables multi-model data modeling and data transformation across various models.
- **MM-evocat** (Paper II) adds the ability to evolve conceptual schemas and propagate changes.
- **MM-infer** [16] (later extended in Papers VIII, IX) allows inferring conceptual and logical schemas from the data.
- **MM-quecat** [14] enables querying multi-model data using **MMQL**.
- **MM-evoque** (Paper III) allows propagating changes from conceptual schemas to queries.
- **MM-mapsearch** (Paper VII) explores the concept of self-adaptation.

⁶This “we” refers to the work presented in this thesis.

⁷The first tool has the same name as the entire framework.

The overall architecture of the system is shown in Figure 0.7. There are three levels of abstraction—conceptual, logical, and physical—each with its own artifacts: schema categories, mappings, and data. The arrows show interactions between various parts of the system enabled by the tools. For example, MM-infer observes the data and infers both the conceptual and logical schema categories. The MM-cat tool has two distinct components, *modeling* and *transformations*, which are displayed separately. The colors of the arrows do not have any particular meaning; they are used only to distinguish between different interactions. Although the tools roughly correspond to the capabilities described in Section 0.1, in practice they are better understood as different *use cases* of a unified system rather than as separate tools.

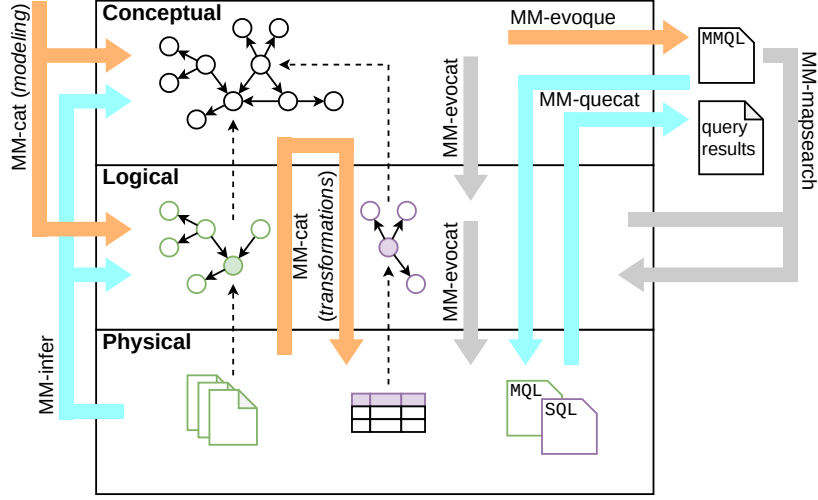


Figure 0.7 Architecture of the MM-cat framework, showing the main tools and their interactions.

0.2.5 Related Work

Traditional conceptual modeling is based, among other approaches, on [ER](#) [19] or [UML](#) [28]. These formalisms remain important because they provide a technology-independent description of the domain, but they were not designed as a unifying framework for heterogeneous multi-model systems with explicit cross-model mappings, embedding, or redundancy. They are therefore useful mainly as inspiration for the conceptual level, not as a complete solution to the broader problem addressed in this thesis.

Besides these classical approaches, several proposals target multi-model data more directly. Probably the closest practical line is represented by **U-schema** [29] and its extensions, such as the **Orion** language [30, 31] for evolution and **SkiQL** [32] for schema querying. Their main difference from our framework is the level of unification they choose. U-schema primarily unifies data models at the *logical* rather than the *conceptual* level. This leads either to an inability to tailor the representation to the specific features of each data model (if we use a single logical schema across multiple databases) or to redundancy and a loss of semantics (if we use multiple disconnected logical schemas). Our approach keeps a platform-independent conceptual schema and connects it to specialized logical

representations through explicit mappings. This separation gives us more freedom to tailor each logical schema to the capabilities of a concrete database system while still preserving one shared conceptual view.

A second difference lies in the querying capabilities. SkiQL is a *schema* query language, meaning that it can query only the structure of the data, not its content. This is a very important distinction, as our focus is the opposite—we query only the contents of the data, not its structure. However, this is not a limitation of our approach, as we can still obtain the structure from either our internal model or the data (i.e., by inference).

There is also a more theoretical category-theoretic approach to heterogeneous data management [33]. A recently published version [34, 35] is much more rigorous and comprehensive while also exploring querying multi-model data. It introduces categorical algebra and calculus as an extension of relational algebra and calculus via multi-model predicates. These works provide valuable formal foundations for representing multiple models and reasoning about transformations between them. Compared with our work, however, they focus more on the formal apparatus itself than on an integrated toolset for modeling, querying, schema inference, and evolution over practical multi-model systems.

Finally, related work also comes from multi-model databases, polyglot persistence, and query federation. Existing systems often provide a common access layer over heterogeneous sources or extend one dominant query language to cover additional models. A broad overview of this landscape is given in [11]. On the standards side, SQL has been extended toward particular combinations of models, for example through SQL/XML [36], SQL/JSON [24], and SQL/PGQ [25]. These standards are important, but each covers only a subset of the multi-model problem. Similarly, polystore and mediator-style systems [37, 38, 39] show how queries can be executed across heterogeneous backends, yet they usually treat local representations as given rather than deriving them from an explicit conceptual schema and its mappings. In contrast, our framework combines conceptual modeling, logical representation, mappings, and querying within one coherent foundation.

0.3 Evolution Management

Everything is subject to change, and multi-model data is no exception. Changes can occur in any part of the system, such as the conceptual or logical schema, the data itself, or the queries. **Evolution management** involves detecting these changes, formally describing them, and propagating them to all affected components to maintain the system’s consistency and correctness. The **change propagation** is especially challenging in a multi-model environment, where component relationships can be complex, and changes can have far-reaching consequences. Based on the related work (Section 0.3.4), we identified the following research challenges:

E1 *Model-agnostic evolution* Changes in a multi-model system should be described and propagated in a way that is independent of the underlying data models, yet still precise enough to preserve consistency and correctness. Existing systems usually handle evolution separately for each supported model or for a narrow family of models. In contrast, a genuinely multi-model solution needs a shared conceptual view in which analogous changes can be treated uniformly despite differences in structure, constraints, and redundancy patterns across the participating models.

E2 *Evolution language* Given such a shared view, changes should be expressed in a formal language that can be understood by both users and the system. On the one hand, the language should provide a small set of well-defined atomic operations with explicit preconditions and postconditions, so that propagation remains analyzable and correct. On the other hand, users rarely think in such low-level steps, so the same language should also support higher-level operations that better capture restructuring intent while still being reducible to the atomic core.

E3 *Propagation to data* Once the schema of the data is changed, the data must be changed as well to maintain consistency. The most straightforward approach would be to load all data into some universal representation, apply the change, and then transform it back into the original model. However, this is not very efficient, especially for large datasets. Instead, we should be able to modify the data directly in the database via a sequence of [DDL/DML](#) statements—ideally, we would find the most efficient sequence of statements for the given change. Nevertheless, the completeness and correctness of the propagation are the most important qualities and should be guaranteed even if the chosen sequence is not optimal.

E4 *Change detection* High variance and dynamicity are common characteristics of modern data. We should be able to detect changes in the data and keep the schemas up to date. Additionally, multi-model data naturally supports redundancy, both intra-model (i.e., some NoSQL databases promote denormalization) and inter-model (i.e., the same data can be represented in multiple models). Modifications to the data in one model (or part of it) should be reflected in the other models (or parts of them) to maintain consistency.

E5 *Propagation to queries* A dataset consists not only of data, but also of queries defined over the data. When the schema changes, queries must be updated to

ensure they continue to return the intended results. This is particularly challenging in a multi-model environment where queries may span multiple data models and their relationships can be complex.

E6 *Evolution automation* Some changes can be propagated automatically, while others require additional information (e.g., specific domain knowledge) to correctly interpret the user’s intent. However, they should still be automated as much as possible. The user must be able to intervene at any point, but the system should decide when to explicitly prompt for user input and when to automatically select the best option among the admissible alternatives.

E7 *Migration strategy* There are multiple strategies for propagating changes to the data (e.g., **eager**, **lazy** [40], and **hybrid** [41]) and for reconstructing queries [42, 43, 44]. The best strategy depends on the specific situation, such as the size of the data, the frequency of queries, the expected performance impact, etc. The system should support multiple strategies to accommodate a wide range of use cases, and it should also be able to choose the most appropriate strategy based on the current workload and data characteristics.

0.3.1 Evolution Hierarchy

A change in one component that is propagated to another component is also a change, so it may require further propagation, leading to a cascade of changes that can be difficult to manage. This is especially true in a multi-model data world, where there are many different components and relationships between them.

Our approach starts by defining a clear hierarchy of components and a set of rules for propagating changes between them. At the top of the hierarchy (see Figure 0.8) is the conceptual schema. The schema can be edited via atomic *schema modification operations* (**SMOs**), which are either provided directly by the user or automatically inferred from data changes (as discussed in Section 0.1.1).

Next, the logical schemas can also be edited via **SMOs** (different from those used for the conceptual schema), which are automatically generated by propagating changes from the conceptual schema. The operations can also be created directly—either by the same use cases as the conceptual changes (i.e., user-specified or data-driven), or by the last use case from Section 0.1.1 (i.e., self-adaptation).

Changes in the logical schemas are propagated to the data via **DDL** and **DML** statements. However, these changes do not affect **MMQL** queries, as the queries are defined over the conceptual schema and translated into native database queries only at execution time. Therefore, the queries are updated directly by the conceptual **SMOs**.

This top-down approach (i.e., from the most abstract components to the most concrete ones) is not the only way to propagate changes. A bottom-up approach is also possible—either originating from the data or from the queries. The former is achieved by observing changes in the data (e.g., a new field is added to a document collection) and inferring the corresponding **SMOs** that would have caused those changes. These **SMOs** are then applied to the conceptual and logical schemas, leading to the top-down propagation to the data and queries described above. The latter concerns only changes in the query *workload*, not in the **MMQL**

query contents—the queries depend on the conceptual schema, not the other way around. The system can observe shifts in the workload (i.e., the frequency of certain queries) and use that information to trigger self-adaptation, leading to further changes in the logical schemas and data.

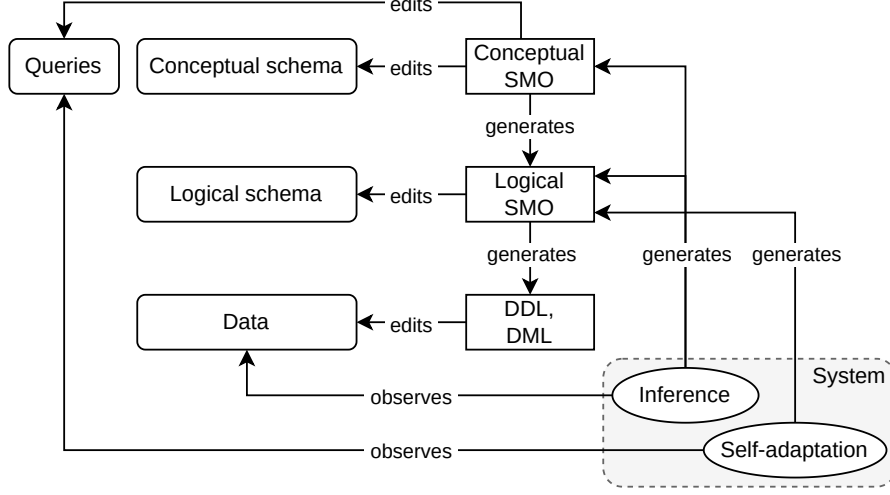


Figure 0.8 Hierarchy of components for evolution management. All changes are mediated by atomic operations (SMO, DDL, DML), which are propagated downward. Direct changes originating outside the system (i.e., data, query workloads) are observed and propagated upward by inferring the corresponding atomic operations.

0.3.2 Change Propagation

The SMOs were proposed in Paper II and formally defined in Paper IV. For clarity, we divide them into three groups based on the level of abstraction:

- **basic** – atomic operations that work with individual objects and morphisms, e.g., `create`, `delete`, and `move`,
- **composite** – operations that work with frequent higher-level patterns, such as properties, relationships, arrays, sets, or maps (e.g., `createProperty` or `deleteMap`),
- **complex** – operations that capture larger restructurings of existing schema fragments that involve coordinating several intermediate choices, e.g., `group`, `join`, or `asMap`.

This division is important for propagation—users can work with expressive high-level commands, but the system can still reduce them to a sequence of basic operations with explicit preconditions and postconditions. High-level operations capture the user’s intent in greater detail, allow for more powerful optimizations (a complex operation can usually be decomposed into several ways, some of which are more efficient than others), and also offer better error handling (if one atomic operation fails, we can roll back the whole complex operation instead of having to deal with a partially applied change).

Once the change is expressed in this form, the system propagates it along the hierarchy described above. In Papers II, IV, we further distinguish between two types of operations:

- **light** – the conceptual schema and mappings are updated, and the logical schemas and data remain unchanged,
- **heavy** – the logical schemas and data are updated as well.

This distinction is useful because not every conceptual change must immediately trigger a full data migration—some operations can remain as logical adjustments until a heavier form of propagation is required.

Toward queries, we build on the approach from Paper III, where schema changes are reflected in **MMQL**. More specifically, each query is analyzed through its selection and projection categories, which capture the graph patterns and returned structure over the conceptual schema. The propagation then evaluates how a given basic **SMO** affects these categories and related clauses such as **FILTER**, **DISTINCT**, **ORDER BY**, or aggregations. As a result, some changes have no observable effect, some lead to a restricted rewrite with potentially changed semantics or performance, and some are too destructive to be repaired automatically. In the last case, the system must either ask for additional guidance or switch to a different propagation strategy.

Example 0.7. Starting with the conceptual schema and the wide-column logical schema (see Figure 0.3), users may realize they want to be able to delete reviews. To do that, they use the **createProperty** operation to add a property **deletedAt** to the complex type **Review** in the conceptual schema and connect them via a new morphism (18). This triggers the creation of the corresponding attribute in the logical schema via the **updateMapping** operation and the addition of the attribute to the data via the corresponding **DDL** statements (Figure 0.9).

Although **Review** is used in the **MMQL** query from Figure 0.6, the change is not propagated to the query because adding a property does not affect the query results. However, users might decide to add the property to the query manually to filter out deleted reviews (Figure 0.10). □

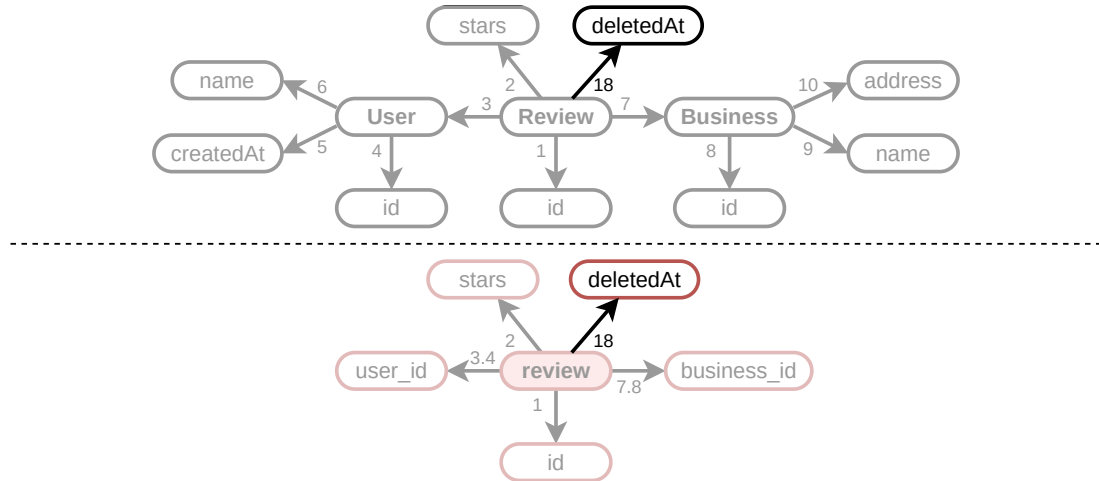


Figure 0.9 Schema from Figure 0.2 is extended via the **createProperty** operation. The change is propagated to the logical schema.

Example 0.8. A more complex use case is displayed in Figure 0.11. The properties of the **Attributes** complex type represent various characteristics of businesses. In such a dynamic environment, it is common for new attributes to be added to the

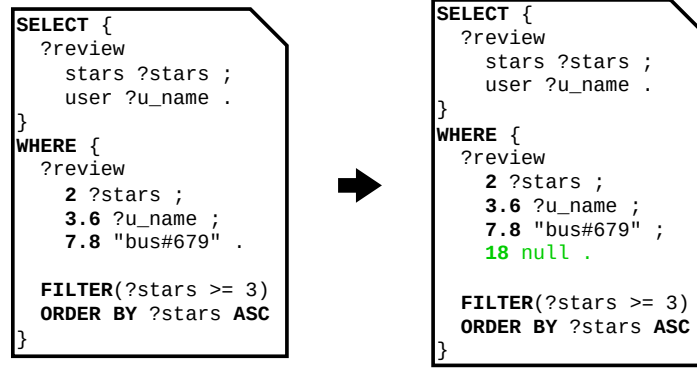


Figure 0.10 The MMQL query from Figure 0.6 is manually updated to use the new `deletedAt` property added in Figure 0.9.

data. To do so quickly, users might convert `Attributes` into a key-value map using the `asMap` operation. The `Attribute`, `key`, and `value` objects are created, together with morphisms 19, 20, and 21. Note that morphism 19 has the opposite direction from the previous morphism 11. This is because in the original schema, there was a `*:1` relationship between `Business` and `Attributes`. Now, there is a `1:N` relationship between `Business` and `Attribute` (one business can have multiple attributes).

After that, the old `Attributes` complex type is deleted. The logical schema is updated accordingly. This time, the change affects the query shown in Figure 0.5, because `Attributes` was used in it. The query is rewritten to use the new objects instead, and the previous attributes are replaced with explicit string literals that filter the `key` property (Figure 0.12). □

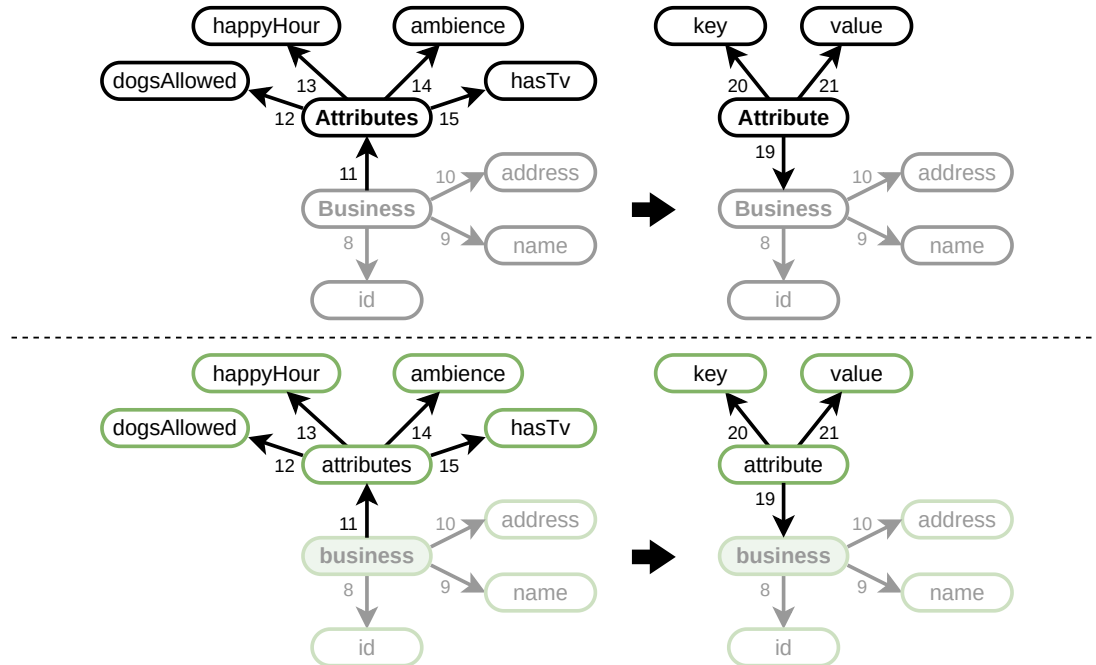


Figure 0.11 Schema from Figure 0.2 is transformed via the `asMap` operation. The change is propagated to the logical schema.

Because propagation is defined over atomic operations, we do not have to execute the entire change in a single step. Composite and complex operations can be decomposed, analyzed, and then propagated incrementally, allowing the process

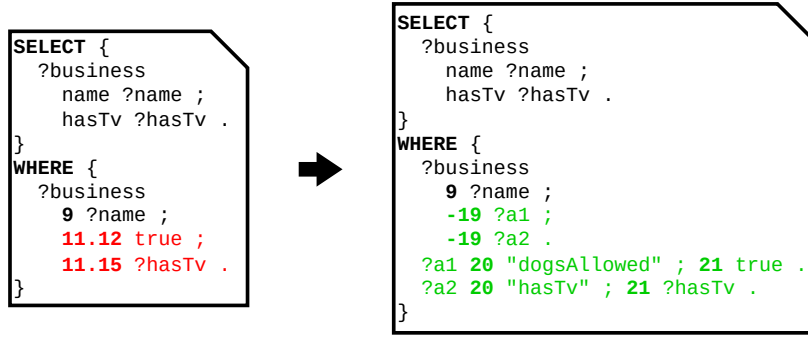


Figure 0.12 The MMQL query from Figure 0.5 is automatically fixed so that it uses the new Attribute map created in Figure 0.11.

to interleave with additional user input whenever an intermediate choice requires domain knowledge. The same decomposition is useful for scheduling—individual propagation steps can be postponed to a more suitable moment (e.g., when the system is underutilized) without losing track of the overall intent of the change. Regarding the data itself, our current implementation assumes eager migration, i.e., all affected records are updated immediately. Nevertheless, the framework is not tied to this choice. In principle, it can also support lazy or hybrid strategies by maintaining access to older versions of the conceptual schema, logical schemas, and queries until the migration is completed.

0.3.3 Level of Automation

As mentioned in Section 0.1.1, the process of change propagation can be automated to different degrees. Higher automation reduces the amount of manual work and expert intervention required from users, but it also places stronger demands on the metadata, propagation rules, and decision mechanisms available to the system. In our work, automation is therefore not treated as an all-or-nothing property. Instead, we distinguish three levels based on how much of the propagation process can be determined and executed by the system itself [10].

Rule-based Adaptation

In the first level, the user or the change-detection mechanism specifies *what* change should happen, while the system deterministically executes *how* the change is propagated. This is the role of the SMOs and their propagation rules, illustrated by Examples 0.7 and 0.8.

The key property of this level is that the individual propagation steps are explicitly defined in advance. In Paper III, we showed how this idea can be extended to queries—a change is decomposed into basic SMOs, the affected MMQL query is analyzed through its selection and projection categories, and the impact of each SMO on query constructs is evaluated. This allows the system to distinguish between changes that preserve query behavior, changes that require a restricted rewrite, and changes that are too destructive to synchronize automatically. While the first two cases can be handled systematically, the last one is reported back to the user as a point requiring additional guidance.

Learning-based Adaptation

This level builds on top of the previous one by retaining the deterministic propagation rules while allowing the system to decide *which* admissible strategy to choose when several alternatives exist. In Paper IV, this problem appears mainly for information-capacity-decreasing changes, such as deleting a property used by existing queries. We investigated several synchronization strategies for such cases, in particular, phantom-based reconstruction and replacements based on functional dependencies or relaxed functional dependencies. Their selection depends on multiple criteria, such as whether the query can still be reconstructed, how much the new query increases in complexity, how expensive the required metadata is to maintain, and how faithfully the new query preserves the original results.

At this point, learning and heuristic decision-making become especially useful. The rule-based layer can enumerate admissible repairs, but it cannot, by itself, decide which trade-off is best for a given workload and system state. For example, a frequently executed query may justify a memory-intensive phantom representation, whereas a rarely executed query may be better served by a cheaper but less faithful rewrite based on functional dependencies. To make such decisions reliably, the system needs high-quality information about the structure of the data. In the case of query propagation, functional dependencies are especially important because they indicate which deleted values can be reconstructed from the remaining attributes and with what loss of precision.

This is the main motivation behind our new approach, which is designed not only to discover functional dependencies but also to distinguish between genuine, fake, and ghost functional dependencies. The approach, together with a proof-of-concept implementation called **FDepHunter**, is described in Paper V.

Rather than treating dependency discovery as a fully automatic black-box task, FDepHunter provides an interactive workflow in which domain experts iteratively refine the discovered dependencies. Technically, it incrementally builds an Armstrong relation [45] enriched with positive and negative examples, which approximate the relevant properties of the complete dataset while allowing several questionable dependencies to be refined at once. FDepHunter provides a potential source of higher-quality semantic information in situations where purely automatic profiling would be too unreliable for downstream adaptation tasks.

Self-adaptation

The third level moves one step further—the system no longer only chooses how to propagate a given change, but can also decide that a change should be initiated in the first place. In this case, workload observations and performance estimates serve as triggers for structural modifications to the logical schemas and data layout. From the perspective of evolution management, this means that propagation is no longer merely reactive, but can become part of a broader optimization loop. Because this topic goes beyond the synchronization of already given changes, it is discussed separately in Section 0.4.

0.3.4 Related Work

Evolution management is challenging even in a single-model setting. Relational approaches typically benefit from a mature theory behind them, but they usually assume only one logical model and query language, as illustrated, for example, by the relational setting of [46]. NoSQL-oriented work broadens the range of supported structures and migration strategies, but it is still usually tied to a single family of models or a single implementation platform. Moreover, most existing work focuses on schema evolution and data migration, while propagation to queries is often treated as a separate problem. In contrast, our approach integrates all these aspects into a single framework that can handle multiple models and their interactions.

Among the more relevant systems, **Darwin** [47] is important because it treats schema evolution as a practical migration problem rather than merely a modeling exercise. It supports lazy and hybrid migration strategies, as well as version jumps, unlike many other approaches that only support eager migration. However, its focus remains on aggregate-oriented NoSQL stores and on propagation to stored data. It does not provide a model-agnostic conceptual layer in the sense used in this thesis, nor does it address query synchronization across a shared multi-model representation.

Another closely related approach is based on **U-Schema** and the **Orion** language, already introduced in Section 0.2.5. For the purposes of this chapter, the key point is that Orion provides a useful taxonomy of schema modifications across several models, but it remains centered on a unified representation at the logical level and does not integrate query propagation in the same sense as our MMQL-based approach. As a result, it does not treat evolution as one connected process spanning the conceptual and logical schemas, mappings, data, and queries.

This last point becomes particularly important once queries are considered. **Query rewriting** [48] broadly transforms a query into an equivalent or otherwise suitable form, often for optimization or mediation, whereas **query adaptation** [49] reacts to changing runtime conditions and execution plans. Our problem is different—we need *query synchronization*, i.e., adaptation of queries after schema evolution. Existing work in this area is much less common and usually remains single-model (especially relational [50]), with only partial extensions toward NoSQL databases [51] or polystores [8]. These approaches show that propagation to queries cannot be ignored, but they rarely combine it with model-agnostic schema evolution and direct propagation to data.

Overall, the existing literature is strong in isolated parts of the problem—some approaches focus on schema evolution, others on migration strategies, query rewriting, and workload-aware tuning. What is still largely missing is a unified treatment of evolution as propagation across a genuinely multi-model representation with explicit conceptual and logical schemas, mappings, data instances, and queries. Our approach builds on a shared categorical representation, supports all major model combinations through explicit mappings, and extends propagation not only to data but also to queries, including difficult cases with decreasing information capacity.

0.3.5 Contributions

1 *Unified Evolution Model* We proposed a novel approach to evolution management in multi-model data management systems. It is based on a unified categorical representation of multi-model data, which allows us to include all supported data models and their combinations within a single framework. Thanks to the shared abstraction of logical models, we can express analogous modifications uniformly, rather than redefining them separately for each model, thereby improving both the clarity of the overall evolution process and the compactness of the operation set (Paper IV).

2 *Atomic Propagation Operations* We defined a set of atomic operations for modifying the conceptual and logical schemas, data, and queries (E1). These operations can be propagated across all components of the system systematically (ensuring consistency and correctness) and with varying levels of automation (reducing the manual work and expertise required from users) (E6).

3 *Higher-level Evolution Operations* Besides atomic operations, we also defined more user-friendly composite and complex operations (E2), which are reduced to atomic operations during propagation. They can be used to express common changes in a more natural way while still being propagated through the system consistently.

4 *Query Synchronization* Another contribution is the extension of our evolution-management approach to query synchronization (E5). When schema changes propagate through the system, the affected queries are analyzed to determine whether they remain valid, can be rewritten unambiguously, require an approximation of the original semantics, or need additional user input. We focused especially on situations with decreasing information capacity, where some information may be removed or become unavailable after the change. For these cases, we investigated several rewriting and synchronization strategies in the multi-model context and showed how they fit into the broader evolution process.

5 *Prototype Validation* Finally, we validated the proposed ideas with a proof-of-concept implementation of the tools MM-evocat (Paper II) and MM-evoque (Paper III). The implementation and experimental evaluation demonstrated that the evolution process can be handled systematically, reducing manual, error-prone intervention when propagating changes across schemas, mappings, data, and queries. The experiments then compared the efficiency of the considered synchronization strategies and provided initial evidence of the approach’s practical feasibility.

6 *Dependency-guided Adaptation* Another, more focused contribution is FDepHunter (Paper V), which strengthens the learning-based component of our approach by providing more accurate information about functional dependencies in real-world data. Although this is only a partial solution, it fits the broader evolution framework naturally—such dependencies provide exactly the kind of semantic insight needed for ambiguous query synchronization tasks and other

situations where purely rule-based propagation is no longer sufficient. In this sense, FDepHunter is both a standalone tool for functional dependency discovery, and a supporting component that improves the system's ability to make informed adaptation decisions.

0.4 Self-adaptation

With the categorical representation of multi-model data and the capabilities for querying and evolution, we have a solid foundation for managing multi-model data. Now we can focus on optimizing it.

Before developing our own search-based solution, we first studied a narrower but closely related workload-aware optimization problem at the physical level, namely index selection. In Paper VI, we surveyed state-of-the-art index-selection methods, compared how major database systems implement and use indexes, and evaluated how far existing approaches transfer from relational systems to NoSQL, multi-model, and multi-modal⁸ settings. This gave us a concrete picture of what current automated tuning methods can and cannot do once the logical representation is already fixed.

0.4.1 Index Optimization

The survey begins by noting that index selection is one of the most important physical design decisions for workload performance, yet the existing literature remains primarily focused on relational databases. We therefore analyzed representative relational, graph, key-value, document, wide-column, multi-model, and multi-modal systems and compared both the index types they support and the workloads for which these indexes are useful. Based on that background, we reviewed state-of-the-art index-selection algorithms and assessed their broader applicability outside the relational setting.

The main conclusion was not that existing methods can be transferred directly, but rather that such transfer is limited and uneven. Many algorithms assume capabilities that are natural in relational systems but unavailable or much weaker elsewhere, such as rich what-if optimization interfaces, stable cost models, or a fairly uniform space of supported index types. At the same time, the survey confirmed that heterogeneous systems still face the same underlying need: they must choose workload-aware physical structures under storage and maintenance constraints, often for much more diverse query patterns than those addressed in classical relational tuning.

Two observations from that paper were especially important for the present chapter. First, cost estimation is the central technical bottleneck. Existing index-selection methods depend heavily on the ability to estimate the effects of many alternative configurations cheaply, without fully deploying them. Second, physical tuning alone is not enough for our setting. The survey assumes that the logical representation and database placement are already given, whereas in a multi-model environment, these choices may themselves dominate performance. This led us to ask whether the optimization problem should be formulated at a higher level of abstraction, over alternative logical representations of the same conceptual schema.

In that sense, Paper VI serves primarily as motivation and inspiration. It showed us both the value of workload-aware automated optimization and the limits of staying purely at the physical level. Therefore, we decided that, instead of

⁸Multi-modal systems combine different *modalities* (e.g., text, image, video, and geospatial data).

selecting indexes for a single fixed representation, we should search over alternative mappings, embeddings, and redundancies of the same conceptual schema across multiple databases. The richer physical-design space identified in A7 remains important future work, but it is not the main point of our optimization approach.

0.4.2 Problem Formulation

We assume a fixed conceptual schema, a workload (a set of queries weighted by their frequencies), and a set of available database applications. Let us define a **configuration** as a set of logical schemas of these databases that, if materialized with the data, would allow the system to answer all queries in the workload. The goal is to find a configuration that minimizes the overall query latency. Based on this problem definition and the insights from the survey, we identified the following research challenges:

A1 *Search space* The space of possible configurations is, in principle, unbounded and, even after removing obviously equivalent or strictly worse alternatives, still grows exponentially with the size of the conceptual schema. The system, therefore, needs a deterministic finite representation that still captures the main structural choices relevant to performance, especially model assignment, embedding, and cross-model redundancy, together with their maintenance costs.

A2 *Search strategy* Even in a finite state space, the number of configurations grows too quickly for exhaustive evaluation, so the system must explore alternatives systematically while balancing exploitation and exploration. This is particularly important in multi-model settings, where small changes may temporarily place jointly queried kinds in different databases, resulting in cross-model queries. Those are generally more expensive than single-model ones, thus creating local maxima that can be escaped only by larger structural changes.

A3 *Latency estimation* To evaluate a configuration, the system has to at least estimate the latency of the queries in the workload. A naive solution (executing the queries) is too expensive. A much more efficient approach is required to make the search process feasible. Specifically, this solution cannot require the actual materialization of the configuration, as that, by itself, is too expensive.

A4 *Cross-model queries* Given the nature of the multi-model environment, some queries may require data from multiple models. Therefore, the system has to be able to estimate the latency of cross-model queries, taking into account the cost of data transfers and transformations between models.

A5 *Migration cost* Migration to the final configuration is usually very expensive, as it may involve large data transfers and downtime. The performance gain of the new configuration must be significant enough to justify the migration cost. The system has to be able to estimate the cost of migrating to (and maintaining) a configuration.

A6 *Workload prediction* In practice, workloads are not static. They evolve over time and may contain both periodic trends and one-off anomalies. A truly autonomous system should reason not only about the present workload but also

about its likely future development, so it can avoid overfitting to transient patterns and prepare for upcoming changes in advance.

A7 Richer search space Database applications offer a wide range of physical design choices beyond the logical model, such as indexes, materialized views, partitioning schemes, replication strategies, knob tuning, etc. These choices are not currently covered by the categorical representation, but they can significantly impact query performance.

0.4.3 MM-mapsearch

Motivated by the research challenges above and by the limitations revealed by the index-selection survey, we propose the following principles:

- A **state** is an element of a finite discrete **state space** represented by a matrix. It maps the conceptual schema and the databases to a configuration. The state space is then a graph of all possible states connected by the available transitions.
- The state space is explored using an algorithm that evaluates the states both directly (by estimating the latency of the corresponding configurations) and indirectly (by the values of neighboring states).
- There is a model that, given a state and a workload, can cheaply estimate the total latency of the corresponding configuration without actually materializing it.

It is important to note that, while these principles are general and most likely applicable in other approaches to the problem of self-adaptation, they can be enabled only by a sufficiently robust representation of multi-model data.

In our approach, the categorical representation provides a unified view of the conceptual schema, the logical schemas, and the mappings between them. It allows us to describe queries (and thus the workload) independently of the actual configurations, as well as to reason about cross-model queries. It also defines a set of **SMOs** that provide a natural way to transform one configuration into another, which is crucial for defining state transitions. Lastly, we can use it to migrate data to the final configuration.

An in-depth description of the proposed approach, together with a proof-of-concept implementation called **MM-mapsearch**, is provided in Paper [VII](#). The following sections contain a short summary and a few improvements we have made since the publication of that paper.

State Space

In our approach, we consider complex types, not simple types, to be the most basic building blocks. That is, if a complex type is mapped to a kind, all of its properties are mapped to the attributes of that kind. Then, a configuration tells us in which database(s) each kind is stored and into which other kind (if any) it is embedded. This limitation allows us to define a finite state space, as there is

only a finite number of ways to assign the kinds to the available databases and to embed them into each other.

Paper VII describes a simple representation of a state consisting of two vectors, both of which have the same number of elements as there are complex types in the conceptual schema. The first vector contains the database to which each complex type is assigned. The second vector contains the kind into which each complex type is embedded (or `none` if it is not embedded into any other kind).

This representation of a state has one major issue: it does not support cross-model redundancy, so if kinds used in a query are stored across multiple databases, the query must be cross-model, resulting in a significant performance penalty. Such a penalty is likely to outweigh the benefits of using multiple databases, so the search process would lead to single-database configurations. To address this issue, we have replaced the previous two vectors with a **state matrix**. Its columns correspond to the complex types, while its rows correspond to the databases. The values are one of:

- 0 (i.e., `false`), if the kind (for the given complex type) is not stored in this database at all,
- 1 (i.e., `true`), if the kind is stored in this database, but not embedded into any other kind, and
- the identifier of another kind, if the kind is stored in this database and embedded into that other kind.

This allows us to represent both cross-model redundancy and embedding while keeping the state space finite. The number of possible states is further reduced by a handful of constraints, such as that a kind cannot be embedded into itself, or that all complex types must be represented by at least one kind. The resulting state space is still large, but it is manageable and can be explored systematically.

Example 0.9. After the evolution from Examples 0.7 and 0.8, the conceptual schema has 5 complex types: `User`, `Friend`, `Review`, `Business`, and `Attribute`. Figure 0.13 shows the simplified schema category (all properties are omitted), the initial configuration (based on Figure 0.1), and the corresponding state matrix.

The optimizations are closely tied to the specific databases rather than to general data models, so we assumed the following options: *PostgreSQL* (relational), *Cassandra* (wide-column), *Neo4j* (graph), and *MongoDB* (document). We also had to extend the logical schema `User` in *Neo4j* with the `yelping_since` attribute (it was not in the original schema), because, as noted above, each kind has to be stored with all properties of its corresponding complex type. $\square$

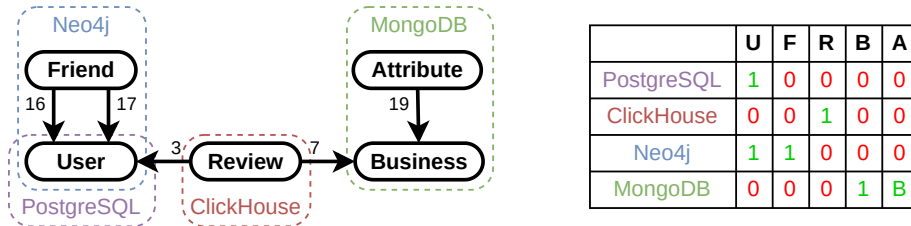


Figure 0.13 The initial configuration of the running example and its state matrix. Columns are labeled with the first letter of the corresponding complex type.

Search Strategy

Although the state space is finite, it is still too large for exhaustive evaluation. We considered several search strategies, including greedy search, genetic algorithms, and integer linear programming. Ultimately, we chose **MCTS**, but we are still exploring other options.

The main idea behind **MCTS** is to incrementally build a tree of evaluated states. At each step, the algorithm first selects the most promising path from the root to a leaf node, balancing exploitation and exploration. Then, it expands the tree by applying a migration step to the selected leaf node, thereby creating a new child node. A corresponding configuration is evaluated using the latency estimation model, and the result is backpropagated up the tree to update the values of the nodes along the path.

A more detailed description of the algorithm can be found in Section 7.2.2. The standard version of **MCTS** is designed for tree-structured search spaces (e.g., a game of chess), so we had to make some adjustments. For instance, we use a **DAG** instead of a tree, as the same configuration can be reached by different sequences of migration steps.

Example 0.10. Figure 0.14 shows a simplified version of the searched **DAG**. The left part shows the initial state from Figure 0.13 (node with label 0) and all its possible modifications of **User**. That is, we can either remove it from PostgreSQL (and get to state 1), or add it to ClickHouse (2) or MongoDB (3). Notably, we cannot remove it from Neo4j, as that would require removing **Friend** first, because it is represented there as a relationship. For the same reason, we cannot embed it into **Friend** in Neo4j. Next, nodes 4–6 are all the states reachable by two consecutive modifications of **User**. As we can see, we can get to them in multiple ways.

The right part shows the first modification of **Friend**: we can add it to all databases where it is not already stored (states 7–9). However, we cannot remove it from Neo4j, because then it would not be stored in any database. We also cannot embed it into **User** in either Neo4j or PostgreSQL, because they do not support array values,⁹ and one user can have multiple friends. $\square$

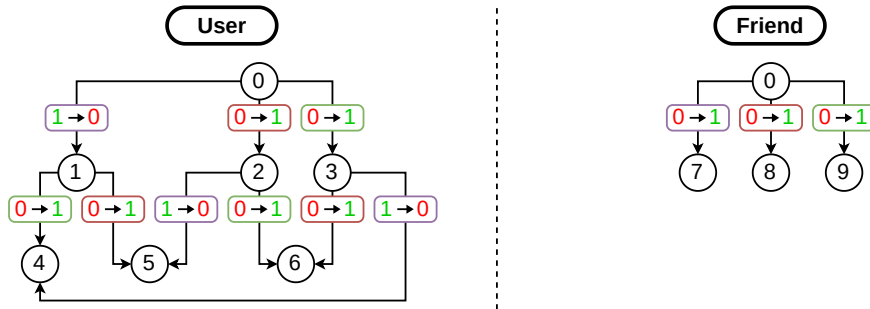


Figure 0.14 First few steps of the search process from the initial configuration (0), either by modifying **User** (left) or **Friend** (right). Colors around the steps correspond to the databases from Figure 0.13.

⁹Technically, PostgreSQL *does* support arrays, but we use it here as an example of a generic relational database.

Latency Estimation

Similarly to the search strategy, we treat latency estimation as a black-box function that can be implemented in various ways. So far, we have tried two approaches: a plan-structured neural network (see Section 7.2.3) and a random decision tree.

Both approaches estimate query latency from the query plan, which must be obtained from the database. Although current databases still require the configuration to be materialized in order to obtain the query plan, there is no fundamental reason why this should be the case. In general, databases generate query plans based on the schema and data statistics. This information can be obtained once (e.g., from the database where the data is currently stored) and then used as input to the planning engines of other databases.

Although this is still very much an open research question, we are optimistic that it can be solved in the near future, which would make the search process much more efficient. Until then, we can use explicit materialization to train, test, and validate the latency estimation models.

Once we can estimate query latency within individual databases, we can extend this to cross-model queries by leveraging the query plans generated by the MMQL query engine. A simple cost model for MMQL queries was proposed and implemented in [2]. It uses the estimated latencies of individual subqueries, along with other metadata obtained similarly (e.g., the size of intermediate results), to estimate the overall latency of cross-model queries.

Lastly, we can also use the latency estimation model to estimate both the cost of migrating to a new configuration and the cost of maintaining it. Our approach is to define a set of INSERT, UPDATE, and DELETE queries that represent the expected workload of the migration and maintenance processes, and then estimate their latency in the same way as for the workload queries.

However, this is still a work in progress, as many other factors can affect migration costs. For example, the cost of downtime depends not only on the latency of migration queries but also on the “business” cost of downtime, which, in turn, depends heavily on the specific application and its users. Nevertheless, the proposed cost model is a good starting point for a more general solution to this problem, as it provides all the “technical” information needed to estimate migration costs, while leaving the “business” part to the user.

Example 0.11. Currently, the workload consists of two queries (Examples 0.5 and 0.6). Neither of them uses the **Friend** complex type, so the algorithm will try to optimize it away. We can fix that by adding a third query that uses **Friend**, but for the sake of this example, we will keep the workload as it is.

The precise result is impossible to predict, as it depends on the actual query latencies in the individual databases. However, we can reasonably expect the algorithm to move both **User** and **Review** to the same database, as that would eliminate the cross-model query. Thus, the final configuration will most likely consist of two independent parts—the fastest database for the query from Example 0.5 (containing **Business** and **Attribute**) and the fastest database for the query from Example 0.6 (**User** and **Review**). □

In real-world scenarios, both data and workloads are much more complex than

in Example 0.11. Such a clear separation of the workload into independent parts is quite rare, so the process will usually result in either several cross-model queries or redundant kinds. The balance between these two options is determined by the relative overhead of cross-model querying vs. the additional cost of migration and maintenance of redundant data.

0.4.4 Related Work

We explored related approaches from three directions. First, physical-design methods such as index selection, materialized-view selection, and related forms of workload-aware tuning optimize performance by modifying structures within a single database system (and usually a single logical model). As already discussed in Sections 0.2.5 and 0.3.4, adjacent work also exists on multi-model representation and evolution. What is still missing in them, however, is a formulation in which the choice of logical model, embedding, and redundancy becomes the optimization space itself.

This difference matters because the logical representation alone can have a major impact on workload latency [52]. In a multi-model setting, the problem is even harder—the same conceptual data may be placed in different stores, embedded in different ways, or duplicated across models, and each such choice changes not only local operator costs but also the need for cross-model execution. Traditional physical design approaches usually assume that the model and the storage platform are already fixed. Our setting starts one level above that assumption and treats alternative mappings themselves as the objects to be compared.

The second relevant direction is workload-aware and learning-based tuning. Recent work shows that learned methods can support latency prediction, data-structure design, or system-parameter tuning [53, 52, 54, 55]. These approaches are highly relevant to our problem because they demonstrate how search can be guided by predicted workload behavior rather than solely by hand-crafted heuristics. At the same time, they usually optimize an existing configuration (e.g., by tuning knobs, choosing physical structures, or estimating plan costs), but they do not explicitly search over heterogeneous mappings of a single conceptual schema across multiple database models.

The third relevant direction is multistore and polystore research, which addresses interoperability and query execution across heterogeneous backends [39, 38, 56]. These systems demonstrate how multiple stores can be combined within a single application and how queries can cross system boundaries. However, they typically treat the available placement of data as given, or at least outside the main optimization problem. Likewise, evolution-oriented approaches such as Darwin [47], MigCast [57], and related NoSQL migration work, as well as the categorical and logical-level approaches discussed in the previous chapter, are highly relevant as enabling technology, but they primarily address migration, schema modification, or impact analysis rather than workload-driven search over alternative multi-model layouts.

Our approach combines elements that these lines of work usually keep separate. It assumes a fixed conceptual schema, makes mappings first-class objects, defines an explicit search space of alternative configurations, reasons about migration between them, and evaluates candidates with respect to a weighted workload. In

contrast to classical physical-design tuning, the search space here includes not only indexes or access paths, but also model assignment, embedding, and cross-model redundancy.

0.4.5 Contributions

1 *Index-selection Survey* We conducted a survey of index selection in heterogeneous database systems (Paper VI). This survey compared indexing strategies across relational, graph, key-value, document, wide-column, multi-model, and multi-modal systems, assessed how existing index-selection methods transfer beyond the relational setting, and identified the main obstacles to such transfer. In particular, it showed that current approaches depend heavily on cost-estimation capabilities and usually assume that the logical representation is already fixed, which helped us delimit the scope of existing physical-design methods and motivated the search for a higher-level optimization approach.

2 *Problem Formulation* We formulated self-adaptation as a workload-aware optimization problem over the logical schemas and their mappings to a fixed conceptual schema. Instead of treating different data models resp. databases individually, our approach represents alternative configurations of the same conceptual schema in a common search space. This makes it possible to systematically compare heterogeneous designs and reason about adaptation at the level of the whole multi-model system rather than one store at a time (Paper VII).

3 *Finite Configuration Space* The categorical framework allowed us to define a state representation that captures the essential aspects of a configuration, including model assignment, embedding, and cross-model redundancy. This representation is both expressive enough to cover the most important design choices and compact enough to keep the search space finite (A1). It also provides a principled way to migrate data between configurations.

4 *MCTS-based Optimization* On top of this representation, we introduced an automated optimization process that derives configuration changes from weighted workloads and explores them using MCTS. This algorithm enables us to systematically explore the search space, favors promising regions, and reduces the risk of being trapped in local maxima (A2).

5 *Learned Cost Estimation* Because evaluating every configuration directly on the databases would be too expensive, we complement the search with learned performance estimation. In particular, we extend a plan-structured neural predictor [53] for non-relational databases, so that configurations can be evaluated solely from the query plans of the workload queries (A3). However, a method that would not require full materialization to obtain query plans remains future work. A similar approach is used to estimate cross-model queries (A4) and the cost of migrating to and maintaining a configuration (A5).

6 *Prototype Validation* Lastly, we implemented these ideas in a proof-of-concept prototype MM-mapsearch that integrates workload analysis, generation of schema variants, neural latency prediction, and an interactive interface for inspecting the recommended alternatives. Together, these results provide an initial practical foundation for automatic adaptation of multi-model data layouts under evolving workloads.

0.5 Complementary Approaches to Multi-Model Data Management

This section collects two complementary lines of work that support the practical side of the thesis. The first concerns evaluation—if our modeling, querying, and self-adaptation techniques are to be useful, we need datasets and workloads that are both realistic and sufficiently diverse. The second concerns a more lightweight approach to cross-model querying. While the main line of the thesis is based on explicit schemas and mappings, we also wanted to explore a more pragmatic route focused on rapid interoperability over already existing heterogeneous data.

0.5.1 Benchmarking & Evaluation

A novel approach can be interesting and theoretically sound; however, to be useful in practice, it must be evaluated on realistic datasets and workloads. In the context of multi-model data management, this is particularly challenging, as there are only a few real-world multi-model datasets and queries.

Related Work

Most established benchmarks are single-model. Classical representatives such as TPC-H [58] and TPC-DS [59] target relational workloads, XMark [60] and DeepBench [61] focus on hierarchical or document-style data, and graph-specific benchmarking is covered by suites such as LDBC-SNB [62] or by surveys of graph generators [63].

Only a few benchmarks explicitly target multi-model systems. BigBench [64] combines relational, semi-structured, and unstructured data; UniBench [65] focuses on a multi-model workload within a single integrated scenario; and M2Bench [66] broadens the supported model combinations for analytical workloads. These tools are valuable, but they do not directly address all the possible combinations of multiple models, embeddings, and cross-model query plans that matter in our setting.

Also, the presented datasets consist of only a few kinds (~ 10) and a small number of relatively simple queries (10–30). This makes it difficult to evaluate the approaches proposed in the previous sections—the state space of possible configurations for self-adaptation grows exponentially with the number of kinds, so we need to benchmark them on realistic schemas with substantially more kinds and structural variation than current multi-model benchmarks provide. Similarly, the query-optimization algorithms require complex cross-model queries to test both their effectiveness and efficiency. Moreover, each of these benchmarks provides only a single version of both the schema and the queries, which is not sufficient for evaluating evolution management.

Research Challenges

Existing practice usually falls back to two imperfect strategies. The first is to generate a synthetic dataset, which is often not realistic enough and takes a lot of time to design and implement. The second is to start from an existing dataset and transform it into the required representation. This means writing

a new custom transformation script for each dataset and target scenario. Such scripts are difficult to reuse, hard to validate semantically, and often too ad hoc to support reproducible benchmarking across many model combinations. Based on these limitations, we identified the following research challenges:

B1 *Benchmark coverage* Evaluation of multi-model systems should cover a diverse set of relevant combinations of data models, embeddings, and redundancy patterns. For instance, the approaches discussed in this thesis require evaluation across many alternative logical representations of the same data on the conceptual level, often at much larger schema scales and with more complex cross-model interactions than current multi-model benchmarks provide.

B2 *Evolution variants* When testing evolution management, we need to evaluate the system’s ability to propagate changes of the conceptual schema. The benchmark should therefore provide multiple variants of the same dataset, each corresponding to a different change on the conceptual level.

B3 *Reusable derivation* The system should be able to derive multi-model datasets from existing data in a universal and repeatable way. We needed a representation-independent process that preserves the meaning of the original data, records the applied transformations, and systematically generates multiple multi-model variants.

B4 *Benchmarking queries* A useful benchmark consists not only of data, but also of queries that stress the intended capabilities of the evaluated system. In the multi-model setting, these workloads should remain comparable across alternative representations and across evolution steps, including cases where only the logical mapping changes and cases where the conceptual schema itself evolves.

TransforMMer

The main idea behind our approach, proposed in Paper [VIII](#) (and further developed in [\[67\]](#) (under review)), is to use the categorical framework described in previous sections to automate the transformation of a given dataset into a multi-model representation.

Not all data models are schema-full, so the first step is to infer the schema of the input dataset. For that, we use an existing approach proposed by our research group [\[16, 17\]](#). Unlike most of our previous work, it is not based on category theory, so we adapted it to the categorical framework. It can infer both conceptual and logical schemas (represented by schema categories), as well as the mappings between them.

The next step requires limited manual refinement, because some semantic decisions cannot be recovered from the data alone. For example, the inference algorithm can identify candidates for references and identifiers, but it cannot always determine which of them are actually intended by the users or the application logic. Therefore, users confirm the correct ones and add any semantically missing identifiers or references. Another common task is the unification of structurally similar schema fragments—e.g., if a document contains multiple nested subdocuments under different keys, the inference algorithm will naturally treat them as separate kinds even when they are semantically the same.

Crucially, at this stage we allow only changes that are performed via *light* operations (see Section 0.3.2). This way, we can easily update the previously inferred mappings to make them consistent with both the refined conceptual schema and the original logical schema, so we can use them to import the data into the instance category corresponding to the refined conceptual schema.

Finally, users can choose the desired logical representation of the data, and the system will automatically transform the instance category into that representation. Users can also choose multiple different representations, or change them over time as workloads or design goals shift. This is a major advantage of our approach—it allows us to present a single dataset in many combinations of data models while preserving semantic equivalence to the original data, and it also enables us to simulate migrations from one logical configuration to another.

Example 0.12. Let us start with an already existing dataset. Again, we will use the Yelp data in multi-model format (as shown in Figure 0.1), although the initial representation can be arbitrary. The first part of Figure 0.15 shows the inferred conceptual schema, consisting of several disconnected components. The three-dot symbol is used as an abbreviation for properties that are not relevant to the example.

The system can automatically identify reference candidates; in this case, `user_id` refers to `id`, and `business_id` refers to `_id`. Users can then refine the schema by confirming the reference candidates. This operation can be represented via a functor between the old and the new schema, mapping morphism 2 to 6.1 and morphism 3 to 7.4. Note that some objects were renamed; this is only a cosmetic change that users can make to improve consistency and readability.

The final conceptual schema can be mapped to any number of logical representations. Let us focus only on the **Business-Attributes** part. Users have chosen to map it to a relational representation with two tables, another relational representation with one table (and all properties of **Attributes** embedded as columns), and a document representation with one collection and a nested subdocument for **Attributes**. The system will then transform the data into all three representations. $\square$

This approach is not limited to various logical representations of the same conceptual schema—we can also leverage the evolution-management techniques described in Section 0.3.2 to simulate evolution of the conceptual schema. Users can apply any of the **SMOs** to generate multiple different versions of the conceptual schema. The system will take care of propagating the changes to the logical schemas and to the data, so that all representations remain semantically aligned with each other and with the original data.

Queries are generated in a similar way. First, users employ **MMQL** to define a set of queries over the conceptual schema, which are then evolved together with the schema. Finally, we use the **MMQL** query engine to generate query plans, which transform the queries into the chosen logical representation without executing them. This gives us comparable workload variants not only across different logical representations, but also across successive conceptual versions of the benchmark.

A proof-of-concept implementation of this approach is described in Paper IX. We used it in [67] to compare the performance of different logical representations in several multi-model scenarios, including evolution of the conceptual schema.

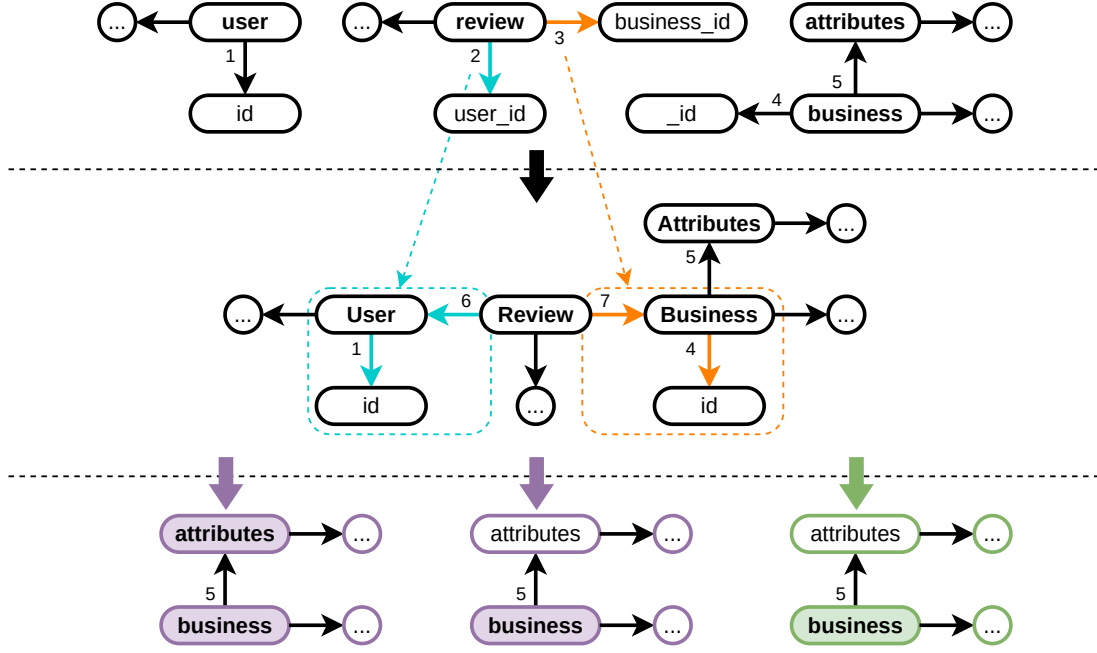


Figure 0.15 Benchmark creation steps from Example 0.12: inferred conceptual schema, refined conceptual schema, and three alternative logical representations.

0.5.2 Schema-free Query Unification

This problem is motivated by highly dynamic settings—such as data lakes, browser-side data integration, or loosely coupled collections of services—where heterogeneous data already exists in multiple forms and users need to query it quickly, often without first constructing a stable conceptual schema and explicit mappings [7, 68]. In such environments, the main obstacle is not only the coexistence of multiple data models, but also the coexistence of multiple query languages and execution paradigms.

Related Work

Existing approaches to multi-model querying usually follow one of two broad strategies. Polystores tend to expose a unified access layer over heterogeneous back-end systems, but often keep model-specific languages and execution engines underneath. Multi-model databases, on the other hand, commonly extend one dominant language, usually **SQL**, with constructs for additional models [11]. This includes standards and systems such as SQL/XML [36], SQL/JSON [24], SQL/PGQ [25], Myria [69], Apache Drill [70], or CloudMdsQL [71]. All of these are important steps toward practical interoperability, but they usually retain one privileged top-level language and support only limited forms of embedding.

There are also systems that relax this restriction only partially. Apache AGE [72, 73], for example, embeds Cypher into PostgreSQL through the `cypher()` function, but only in the Cypher-to-SQL direction. Systems such as OrientDB [74] or ArangoDB [75] support several models as well, yet they do so mainly through proprietary syntax or a system-defined language. BigDAWG [76] comes closer to language plurality by allowing different query islands, but it still separates them into predefined access modes rather than supporting unrestricted recursive

combinations inside one query.

Research Challenges

Our goal was not to define yet another universal user language, but to preserve existing ones and connect them through a shared algebraic layer. We identified the following research challenges:

S1 *Language interoperability* Different query languages are designed for different data models and therefore have different operators, semantics, and execution paradigms. In a heterogeneous environment, the system should allow users to combine established query languages without limitations on the top-level language, or on the nesting depth or direction.

S2 *Unified algebra* If multiple query languages are to interoperate in a principled way, they need a shared internal representation that captures their essential operators. Such a representation must be expressive enough to serve as a common compilation target, but also simple enough to support systematic reasoning and execution. The challenge is to design a unified algebra that preserves the core semantics of these operators while enabling mixed queries to be analyzed as a single plan.

S3 *Schema-free querying* In a highly dynamic environment, the system should be able to query the data even when no stable schema and no explicit mappings have been prepared in advance. Instead, it should recover the structure from the query and the accessed data fragments to make the mixed query meaningful and executable.

DortDB

In Paper [X](#), we addressed these problems with a novel approach called **DortDB**. Rather than introducing yet another user-facing universal language, it treats interoperability itself as the main goal: [SQL](#) [20], [XQuery](#) [21], and [Cypher](#) [23] can be nested into one another recursively, and the mixed query is then translated into a *unified algebra* [77] (under review) covering relational, hierarchical, and graph-style operations [78, 79, 80]. This allows mixed-language cross-model queries to be formulated without committing to a single privileged language, as illustrated later in Example [0.13](#). It also keeps the setup lightweight—rather than requiring a stable shared schema in advance, DortDB derives the necessary local structure from the query and the accessed data fragments.

The unified algebra is the key contribution of this approach. Internally, it acts as a common compilation target over a schema-agnostic object representation of the data. It integrates tuple-oriented operators known from relational algebra, operators for item-oriented and hierarchical processing inspired by [XQuery](#), and path-oriented operators needed for graph traversal. As a result, nested fragments written in different languages are no longer treated as opaque foreign calls; instead, they become parts of one logical plan tree whose operators can be inspected, rearranged, simplified, and executed uniformly, even though the system starts without an explicit conceptual schema.

Example 0.13. The query from Example 0.6 uses SQL for both the **User** and the **Review** data. However, the DortDB query engine allows us to express the same query in multiple different ways. E.g., we can express the main query in **SQL**, while the **Review** data is obtained from a nested XQuery subquery:

```
SELECT r.stars, user.name
FROM user
JOIN (
  LANG xquery
  for $review in $reviews/Review
  where $review/business_id = 'bus#679' and $review/stars ge 3
  return $review
) r ON r.user_id = user.id
ORDER BY r.stars ASC
```

Alternatively, we can use Cypher in the main query for **Review**, while the **User** data is obtained from a nested SQL subquery:

```
MATCH (review:Review)
WHERE review.business_id = 'bus#679' AND review.stars >= 3
UNWIND (
  LANG sql
  SELECT user.id, user.name
  FROM user
  WHERE user.id = review.user_id
) AS u
RETURN review.stars AS stars, u.name AS name
ORDER BY stars ASC
```

Note the **LANG** keyword, which allows us to switch between languages at any point in the query. In practice, the choice of which language to use for which part of the query can be guided by users' familiarity with the languages, the expected performance of the underlying engines, or other practical considerations. The DortDB query engine will then translate the entire query into the unified algebra and optimize it as a whole. The final plans are displayed in Figure 0.16. $\square$

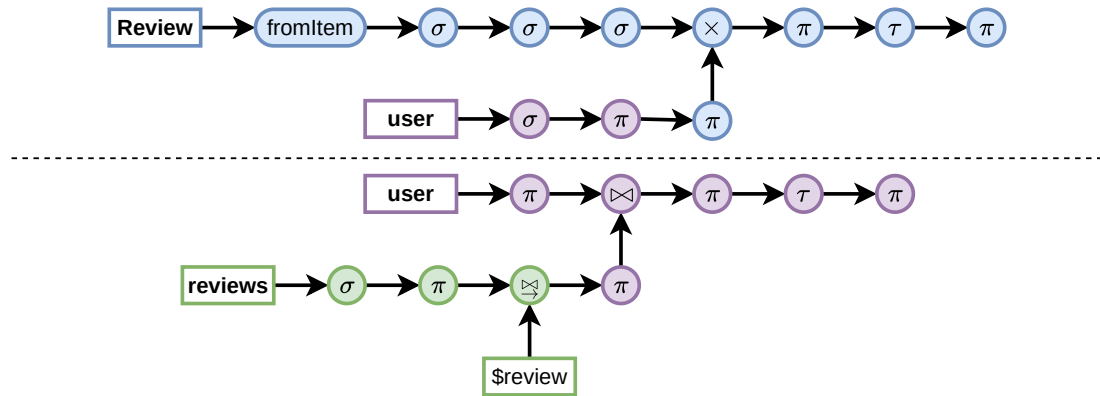


Figure 0.16 Unified algebra representation of the queries from Example 0.13. The colors correspond to the original languages—**SQL** (purple), **Cypher** (blue), and **XQuery** (green). Some operations (e.g., calculations, tree joins) were omitted for clarity.

DortDB should not be seen as a replacement for the categorical framework, but rather as a complementary point in the design space. Our main line of work starts from an explicit conceptual schema, derives logical schemas and mappings, and uses this structure to support querying, evolution, and eventually self-adaptation. DortDB starts much lower and more pragmatically—it assumes heterogeneous data is already present, keeps the representation intentionally schema-free, and focuses on making cross-model querying workable with minimal upfront setup. The price is that it does not provide the same semantic guarantees, principled evolution machinery, or explicit reasoning about equivalence between alternative representations. The benefit is speed of adoption and flexibility in dynamic or exploratory settings.

This makes DortDB especially relevant in scenarios where a fully modeled solution would be too costly or too premature. It can be understood as a universal mediator layer for rapid integration of existing sources, prototyping of cross-model workloads, or interactive analysis over data whose structure is still evolving. At the same time, it broadens the thesis by exploring a more pragmatic branch of multi-model querying, focused on language interoperability rather than on explicit shared semantics. Once the data and workloads stabilize, the schema-first categorical approach offers stronger guarantees and deeper automation. Conversely, DortDB shows that there is practical value in supporting heterogeneous querying even before such a full semantic model is available.

These ideas were also validated, as well as evaluated in terms of performance [77], in a proof-of-concept prototype. For now, the system is intentionally in-memory and read-only, allowing us to focus on core interoperability and optimization problems without being distracted by storage or transaction management. However, it already supports the whole unified algebra, thus covering a large portion of the most commonly used operators from SQL, XQuery, and Cypher.

0.5.3 Contributions

1 *Benchmark Derivation* TransforMMer contributes a transformation-driven approach to benchmark engineering for heterogeneous information systems (Paper VIII). Instead of treating a benchmark as one fixed dataset, it derives families of semantically aligned multi-model variants from existing data (B1). The categorical basis ensures reusable derivation in a controlled, repeatable way (B3).

2 *Benchmark Evolution* The same framework also supports benchmark evolution as a first-class concern. By versioning both conceptual and logical variants and by evolving queries together with the schema, it makes it possible to evaluate propagation scenarios (B2) and benchmark queries (B4) [67].

3 *Prototype Validation* There is a proof-of-concept tool called TransforMMer, described in Paper IX and further evaluated in [67]. Overall, this work is essential for evaluating the other two main contributions of the thesis—evolution management and self-adaptation—as it provides a systematic way to generate realistic multi-model datasets and workloads, including evolution scenarios.

4 *Language Interoperability* DortDB contributes a lightweight alternative for situations where explicit shared semantics would be too costly or limiting. Its main idea is to preserve established query languages (for now [SQL](#), XQuery, and Cypher) and allow them to interoperate symmetrically, so that they can be nested into one another without a privileged top-level language ([S1](#)). It is enabled by a unified algebra [\[77\]](#) that serves as a common compilation target for mixed queries ([S2](#)), allowing for seamless integration and comprehensive optimization.

5 *Schema-free Querying* At the same time, DortDB keeps the setup intentionally schema-free. The required local structure is derived from the query itself, enabling heterogeneous querying even in dynamic, loosely integrated settings ([S3](#)). The approach, together with a proof-of-concept prototype, is presented in Paper [X](#). Within the thesis, it complements the main schema-centric line by offering a practical alternative for specific use cases, and by exploring what can already be achieved at the query-processing level before a full semantic model is available.

Paper I

AI-Powered Orchestration of Multi-Model Data

Jáchym Bártík¹

Presented at the PhD Workshop of the 50th International Conference on Very Large Data Bases (VLDB 2024). URL: https://vldb.org/2024/files/phd-workshop-papers/vldb_phd_workshop_paper_id_2.pdf

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

Multi-model databases are an increasingly popular solution to today's data management challenges of Big Data. However, their inherent complexity and lack of standardization stand in the way of their widespread adoption. In our research, we focus on reducing the complexity by automating the management of such databases. The goal is to provide a robust framework capable of unified modelling, transformation, querying, and evolution management of multi-model data and to leverage AI techniques to optimize data distribution among the database systems.

1.1 Introduction

More than 2/3 of the 50 most widely used database management systems (DBMSs)² fall under the category of *multi-model*³. The multi-model data is organised in various mutually interlinked formats and models, often with contradictory features [7]. In addition, its structure may change over time, and its size can grow to the extremes of Big Data. These aspects create one of the most complex challenges of effective data management.

As handling such a complex task manually is impossible, we focus on the automatic management of dynamic multi-model Big Data. We want to create a robust framework capable of accepting different types of data, queries, changes, and propagation strategies. Based on such rich input, the system will learn to provide self-adapting evolution management, ensuring a complete, correct, and efficient propagation of changes. Particularly, it will support the following features:

- *Multi-Model Modeling*: We need to model the data in one unified and formally backed schema. The model can either (1) be created manually or (2) automatically inferred from sample data. We can also combine these approaches, i.e., infer a reasonable schema and then manually improve it.
- *Multi-Model-to-Multi-Model Transformations*: Transformation from one model to another is a simple process. But, we must be able to migrate the data between different combinations of models represented by different database systems.
- *Cross-Model Querying*: We need to query over the whole dataset, not just a single database system. Also, the queries should be independent of the underlying data models so that we can use the same query language for the whole system and thus not force the user to learn different languages.
- *Multi-Model Evolution Management*: As we have mentioned, each system evolves over time, whereas in the case of multi-model data, the evolution must cover all combined models. Primarily, we want to be able to update the model, the data itself, and the queries. And, when possible, automatically.

Several solutions have already implemented these features, many of which are widely used. However, they all have one thing in common: they are either

²<https://db-engines.com/en/ranking>

³I.e., consisting of multiple data models (relational, document, graph, ...).

tightly coupled with the underlying database systems or too limited to fully model multi-model data. For example, the UML and ER models are industry standards. But, they cannot generally model complex properties, maps, or graphs.

Outline In Section 1.2, we discuss the current functionalities of our framework consisting of a family of tools. In Section 1.3, we describe our planned steps. In Section 1.4, we outline the open problems.

1.2 Initial Framework

In our research group, we have proposed several solutions to selected aspects of unified and efficient multi-model data management. We have also implemented tools for their experimental verification. This toolset represents the initial framework we currently intend to enhance by exploiting AI to automate data management.

First, we needed a sufficiently abstract approach to handle all the conflicting requirements because we deal with varied data models and database systems. Therefore, we proposed a system-independent representation based on category theory [13]. We can view a *category* as a directed multigraph for simplicity. The nodes (called *objects*) represent entities, and the edges (called *morphisms*) represent relationships between them. For example, object A represents a User and object B represents a Name. Then, we can have a morphism $f : A \rightarrow B$, meaning that a User has a Name. We can create structures representing arrays, sets, weak-entity types, etc. In our framework, we call such category a *schema category*.

This unifying representation enables us to “grasp” any combination of models and to process it in a system-independent manner. When a particular operation has to be done at this abstract level, it is propagated to the underlying database system.

Example 1.1. An example of a schema category can be found in Fig. 1.1. The schema category is mapped only to the relational database model (denoted using the violet colour). On the other hand, in Fig. 1.2, we can see the same schema category after an evolution of the mapping. It is now mapped to relational (violet) and document (green) models. $\square$

Our toolset involves two tools that enable the creation of a schema category: *MM-cat* [12] enables the modelling of the schema category manually, as well as the creation of its decomposition and mapping of the selected components to particular logical models. *MM-infer* [16] enables one to infer a schema category from the given sample multi-model data (semi-)automatically.

1.2.1 Transformations

As indicated in the introduction, not only do we need to be able to model the multi-model data using a unified representation, but we also need to be able to transform them. In particular, we need a support for transforming any combination of the (sub-)models to any other combination of (sub-)models.

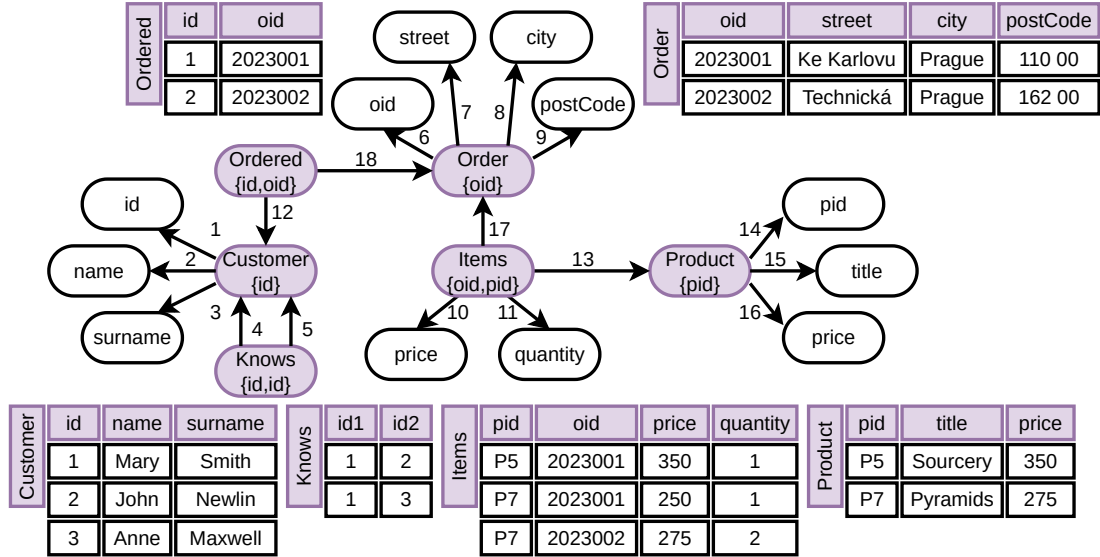


Figure 1.1 A sample schema category. Each violet node represents a complex object. In this case, all of them are mapped to respective tables of the relation model, e.g., PostgreSQL.

For this purpose, as a part of *MM-cat*, we have developed algorithms that leverage mapping between the schema category and the logical models to transform data between them [13]. To work with each database system in the same way, we have to create wrappers for each of them. Then, we can use the same algorithm to transform data between any two database systems.

This is an important distinction from other approaches. For example, the ETL (*Extract-Transform-Load*) process is usually tightly coupled with the underlying database systems. In our solution, we can use the same algorithm to transform data between any two database systems, even multi-model ones. Another example are *data lakehouses* which can store data in multiple (single-model) formats, but they do not provide a unified way to work with them.

1.2.2 Querying

Querying the data is an essential feature of any database. However, this is a much more challenging task in multi-model databases as each system has its own set of supported models and their combinations [7] and a specific query language [11]. There is no general standard for multi-model querying except for the standards SQL/XML [36] and SQL/JSON [24] for relational/document models.

We have proposed the *Multi-Model Query Language* (MMQL), a query language based on the SPARQL syntax that enables one to query over the schema category. Then, within a tool called *MM-quecat* [14], we developed the query-evaluation algorithm that uses a similar approach as the transformation algorithms: First, the query is parsed and mapped to the schema category. Then, we use the mappings to split it into *query parts* that can be executed in the specific underlying systems. We perform the maximum amount of work in the databases, minimising cross-database joins. Finally, we combine the intermediate results.

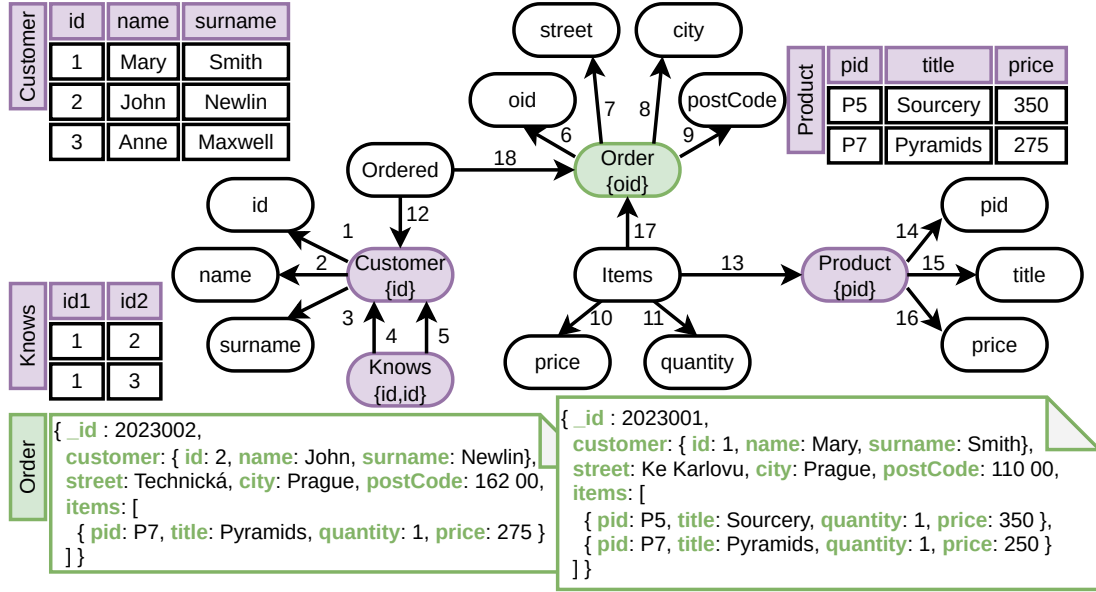


Figure 1.2 The example schema category from Fig. 1.1 after an evolution. The schema category did not change, but the affected tables (*Items* and *Orders*) were replaced by a collection of documents (*Order*), e.g., in MongoDB. The change introduced redundancy to query orders more efficiently.

The global workflow with the indicated functionalities of the initial framework is depicted in Fig. 1.3. First, the schema category is inferred from sample data or modelled manually. Then, it has to be decomposed into the system-specific sub-models. Finally, the user can specify queries over the schema category. All these actions require user input. On the other hand, all data transformations and query resolutions are automated.

1.2.3 Evolution Management

On the most basic level, evolution means changing a database schema over time. However, when the schema changes, the data and the queries must also be updated. Thus, in the context of our framework, we do not see evolution as just another feature but as a fundamental quality of each part of the system.

The change usually starts with a modification of the schema category. First, we have identified several key *schema modification operations* (SMOs) [81], e.g., creating a new object or a new morphism. Then, we can build more complex operations from them, such as grouping several objects into a new one or moving objects around the graph. Then, we again leverage the mappings to propagate the changes to the system-specific models and the data. Lastly, the changes are propagated to the queries in two ways. On the one hand, we have to update the queries to reflect the changes in the schema. On the other hand, we use the mappings to decompose and translate the queries into the system-specific languages, which are also affected by the changes.

However, the change might start with the mappings (compare Figs. 1.1 and 1.2). For example, we might want to alter the way we store data in a specific database system or add a new system altogether. In that case, we have to update the mappings and then propagate the changes to the queries, though the schema

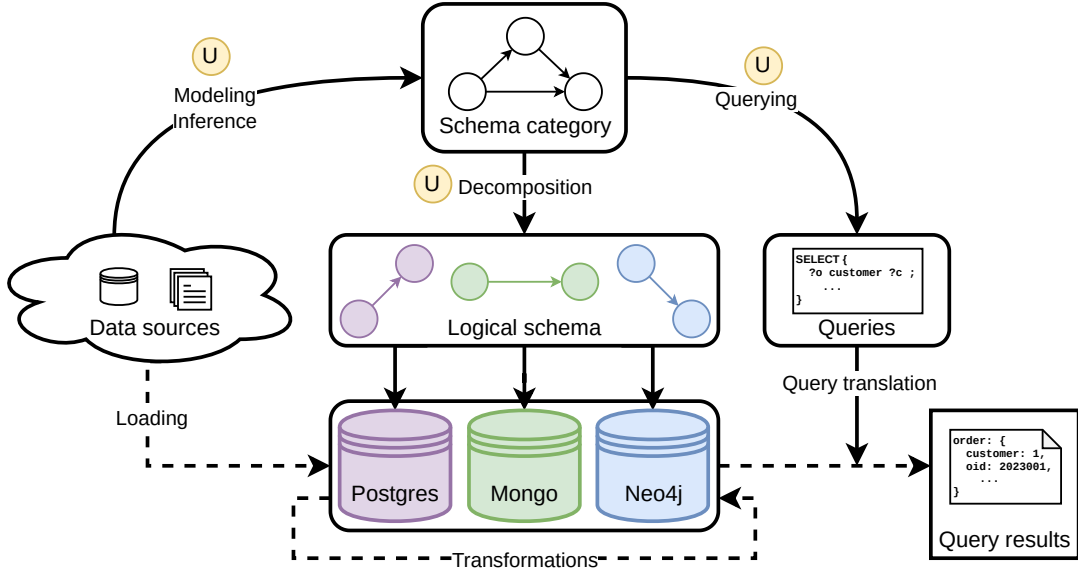


Figure 1.3 Workflow of the initial framework. The full arrows represent the flow of information with the U symbols marking user inputs. The dashed lines represent the flow of data.

category is unaffected.

Example 1.2. The evolution process is depicted in Fig. 1.4. The user changes the schema category, the mappings, or both. The changes are then automatically propagated to the rest of the system.

The propagation of the changes to the data can be, in some cases, mediated by the querying feature. For example, suppose we want to split an object (e.g., an Address) into two (e.g., a Street and a City) based on a specific rule. In that case, we may first create the new objects and then internally use a query with a `SPLIT` function to extract the data about the street and the city from the original object. $\square$

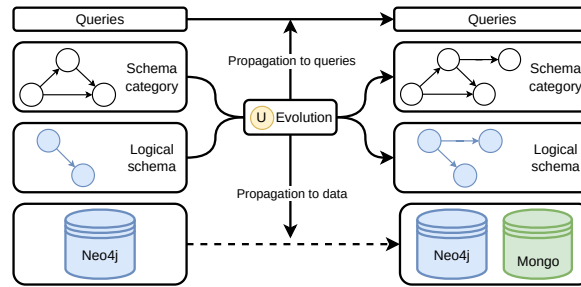


Figure 1.4 Evolution in the framework.

Currently, evolution is managed in the framework by two tools: *MM-evocat* [82] (dealing with propagation of SMOs to logical schemas and data instance) and *MM-evoque* [83] (dealing with propagation of SMOs to MMQL and system-specific query languages).

1.3 Future Plans

Currently, we have a solid theoretical background and proof-of-concept tools to integrate and extend to the final robust solution. It may seem that the only work left is to optimize the algorithms and finish the implementation. And that would be the case in the world of single-model databases, where we can rely on a skilled database administrator to manage the whole system. In the case of multi-model data, however, we still have to deal with different database systems, models, optimization strategies, etc. Our framework can shield the users from all databases' complexity and specific (often contradictory) features. But the principal question now becomes: “*How to optimally distribute data between the particular data models and respective database systems with requirements changing over time?*”.

We cannot rely only on human experts for more complex use cases. Therefore, our main future aim is an AI-powered solution. The general idea is that the user would design the system-independent schema while the framework would decide how to distribute the data among the available database systems. And over time, the framework will propose modifications to the schema category.

Generally, there are three possible approaches – *rule-based*, *search-based*, and *model-based*. The first relies on a hard-coded rule set to generate optimal database settings [84]. The second one (e.g., [85]) searches the space of all possible configurations. The last one utilizes novel techniques such as deep reinforcement learning (as discussed in Section 1.3.1). However, to our knowledge, none of these approaches has been considered for multi-model databases. The rule-based strategy is not very flexible and extensible, so it would not be easy to cover all possible scenarios of the multi-model world. Similarly, the search-based one would be too expensive as the search space grows exponentially with the number of models and database systems. Hence, the model-based one is the most promising because it should be able to adapt to all possible scenarios, although it would require a lot of data to train the model.

1.3.1 AI for DB

The cooperation of AI with database systems can be divided into two categories [86]. The *DB4AI* approaches use databases to improve AI models. For example, the AI-native DBMS openGauss [87] supports native AI computing engine, model management, AI operators, native AI execution plan, etc.

Conversely, the *AI4DB* techniques leverage AI to improve DBMSs. These approaches include learning-based methods that utilize reinforcement or deep learning to optimize database functionality. Examples include configuration tasks such as index selection [88], partitioning advisory [89], and general knob tuning [90]. Additionally, AI4DB techniques extend to optimizing query performance [91], join selection [92], and influencing database design through learned indexes [93] or key/value design [94]. The applications also involve predicting, e.g., query arrival rates [95].

Our planned extension of the framework falls into the second category. We aim to optimize the management of multi-model databases by exploiting AI. All user inputs necessary in the current framework are depicted in Figs. 1.3 and 1.4.

The schema category has to be modelled by the user (unless we can infer it from the data, which is still not fully automatic). Similarly, only the user knows what to query (but there are already techniques to infer this information from the application code, e.g., various ORM frameworks). Hence, we will primarily focus on automating the decomposition process and the evolution (in the form of a gradual optimization of how the schema is decomposed).

1.3.2 Data for Training

The quality of an AI model is highly dependent on the quality of training data. In our case, the expected data will be primarily logs from real-world databases. A member of our research group is already working on a wrapper for commonly used database systems that collect metadata for each query, ranging from the execution time, through the number of entities, to the percentage of entities that were filtered out in each step of the execution plan. The dataset collected during the process will have a value on its own. It can be used to train various AI models utilised for data management purposes or just as an insight into the performance of underlying database systems of our toolset.

There are also many open datasets and even some query datasets. But they usually lack the statistical information we need. Moreover, they tend to be not very diverse, usually focusing on a single single-model database system, which is quite the opposite of what we need. Hence, another task being solved by a member of our research group is a tool for transforming single-model real-world data sets to pseudo-realistic multi-model versions.

Both these approaches solve us the core problem of AI-based approaches – where to gain high-quality training data.

1.4 Open problems

Besides identifying the data management tasks where AI can be utilised to lower user interaction and to select the optimal approach, we face more specific challenges.

Extensible Approach Most contemporary AI4DB approaches are based on the ER model, thus suitable for relational databases only. The question is how challenging it will be to adapt them to the multi-model world. Generally, we want to adjust the existing approaches for the graph model because the schema category is, in essence, a graph. There are some similarities that we can exploit (e.g., both models are aggregate-ignorant), but there are also many differences that we must consider.

Information Aggregation Primarily, we want to use the above-specified database logs as training data as they are not extensive. However, it is unclear what parts of the logs are the most important. Also, we are not sure if we need all the logs or just some kind of aggregation (e.g., the average execution time for each type of query). If we aggregate the data, we can fit much more information into the model, but we might lose some essential details.

Data Quality To clean the real-world data sets, we will use verified techniques like anomaly detection to improve the data quality. Again, there are not many applications of such techniques in the multi-model world yet. Each data model brings challenges, and their solutions might require decisions from domain experts. For example, it is a common practice to remove a whole row if we find a single anomaly. However, if we apply the same approach to a document database with large documents, we might remove many otherwise valid data.

Model Scale There are two options for the scale of the AI model. We can create a single general model trained on statistical data from many multi-model database instances. Alternatively, each instance can have its model and be trained on its data. The first option is more efficient in justifying a larger model. On the other hand, the second one might give us better results because of its customization. There is also a third option – create a single general model but fine-tune it for each instance.

In each of the cases, we plan to compare the most promising approaches experimentally and identify optimal approaches together with their parameters.

Paper II

MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data

Pavel Koupil¹, Jáchym Bártík¹, Irena Holubová¹

Published as Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of CIKM 2022*. Atlanta, GA, USA: ACM, 2022, pp. 4892–4896. ISBN: 9781450392365. DOI: [10.1145/3511808.3557180](https://doi.org/10.1145/3511808.3557180)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

In this paper, we focus on the problem of evolution management of multi-model data. With the changing user requirements, the schema and the data need to be adapted to preserve the expected functionality of a multi-model application. We introduce a tool *MM-evocat* based on utilising the category theory. We show that the core of the tool, i.e., the categorical representation of multi-model data, enables us to grasp all the specifics of the individual models and their possible combinations. Its simple but powerful formal basis enables unique and robust support for evolution management.

2.1 Introduction

Currently, there exist 388 database management systems (DBMSs)². Considering the 50 most popular representatives, we get 60 % systems that can be denoted as *multi-model*.

Example 2.1. Consider an example of a multi-model scenario as shown in Figure 2.1. The relational data (purple) represents customers and their addresses, whereas the document data (green) is redundant to the relational data. Relationships between customers are captured in the graph (blue). $\square$

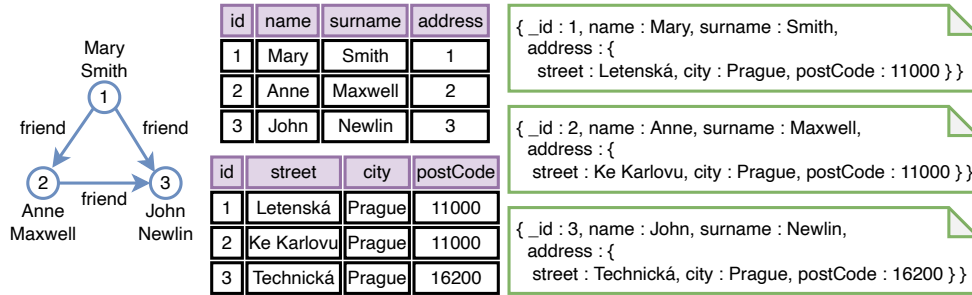


Figure 2.1 An example of multi-model data

According to our extensive survey [7], the features of existing multi-model databases vary significantly. This status is given by the fact that (1) they are based on the distinct original core single model as well as distinct target application domains, (2) there is no acknowledged standard on how to support a combination of models, cross-model querying, multi-model indices, etc., and (3) the combined models have distinct, often contradictory features.

The described variety of multi-model DBMSs lacking standards and the general complexity of multi-model applications bring many new challenges to data management. One of them, our aim in this paper, is *evolution management* of multi-model data. As user requirements or the environment change, the data structures evolve, and the whole system must be adapted to preserve the same functionality (and efficiency). We can rely on a skilled database administrator in simpler applications, but it is a complicated and error-prone task in complex cases where multi-model data undoubtedly belong.

²<https://db-engines.com/en/ranking>

Example 2.2. Consider again Figure 2.1. We may want to merge customers with their addresses into a single table in the relational model to speed up respective queries. $\square$

To address the indicated problems, we extend our previous research results in evolution management [96, 97] and categorical representation of multi-model data [13]. We introduce a tool called *MM-evocat*³, which enables to perform user-specified changes over an abstract multi-model schema and propagates them across all affected sub-models to the logical schemas as well as data instances. The main contributions can be summarised as follows:

- The unifying categorical representation enables us to represent all popular data models (relational, key/value, document, column, and graph) and all types of their combination (embedding, references, and redundancy).
- The representation enables a full decomposition of the data structures to a categorical graph. Hence, it does not impose any initial aggregation of more complex structures, making evolution management robust and efficient.
- Apart from the standard core *schema modification operations* (SMOs), we introduce operations with integrity constraints (ICs), cardinalities, and groups of properties.
- Besides the most natural propagation from the conceptual to the logical level (so-called *heavy operations*), we also introduce *light operations* which change only the conceptual level and its mapping to the logical level.
- We present a proof-of-concept implementation *MM-evocat*. Using real-world data, we demonstrate its features and show how much manual and error-prone work the approach saves.

Paper Outline In Section 2, we review related work. Section 3 introduces the categorical representation of multi-model data. Section 4 describes the evolution management approach and its implementation. In Section 5, we outline its demonstration.

2.2 Related Work

There exists a few multi-model approaches with various limitations.

DaemonX [98] assumes the design of a platform-independent model (PIM) and then mapping its selected parts to the respective single-model platform-specific models (PSMs) involving XML, relational, business-process, and REST models. The authors focus on the correct and complete propagation of changes. The idea of the framework is general and extensible. However, they do not consider proper multi-model data because neither cross-model references nor mutual embedding of models is supported.

Darwin [99, 100] supports only aggregate-oriented NoSQL data stores (namely *MongoDB*, *Couchbase*, *Cassandra*, and *ArangoDB*). Beyond extracting a schema summary from a NoSQL data instance, users can derive the historical series

³<https://www.ksi.mff.cuni.cz/~koupil/mm-evocat/>

of schema versions and semi-automatically declare the changes between two consecutive schemas. *MigCast* [57, 101] extends *Darwin* with an adviser for evaluating different data migration strategies w.r.t. their costs (such as monetary costs for operating in the cloud).

MM-evolver [97] supports evolution management of aggregate-oriented and aggregate-ignorant models, including references (in a limited way). However, the level of abstraction of the multi-model data is insufficient, and its further extensions are complicated.

The *ORION language* [30] was designed for evolution management of *U-Schemas* [29], a representation of multi-model data. Although the authors try to ensure unification, they involve special constructs covering model-specific features. Also, the models are handled relatively separately, not as one unit. Consequently, the schema modification language is complex, as it must consider all the constructs and their specifics. As in the previous case, further approach extensions will not be straightforward.

The idea to exploit category theory for this purpose is not new. However, the existing approaches are still limited too. Paper [102] utilises category theory to represent (object-)relational data and considers only basic operation (adding and deletion). The authors of paper [33] utilise category theory to represent multi-model data (in particular relational, JSON document, and graph) and to express the data migration process between the models formally.

2.3 Categorical Conceptual Model

Let us first remind the basic notions of the category theory. A *category* $\mathbf{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of objects $\mathcal{O}$, set of morphisms $\mathcal{M}$, and a composition operation $\circ$ over the morphisms. Each morphism is modeled as an arrow $f : A \rightarrow B$, where $A, B \in \mathcal{O}$. We must also ensure *transitivity* ($g \circ f \in \mathcal{M}$ for any $f, g \in \mathcal{M}$, $f : A \rightarrow B$, $g : B \rightarrow C$), *associativity* (requiring $h \circ (g \circ f) = (h \circ g) \circ f$ for any suitable $f, g, h \in \mathcal{M}$), as well as introduce an *identity* morphism 1_A for each object A such that $f \circ 1_A = f = 1_B \circ f$ for any $f : A \rightarrow B$. A category can be visualised as a multigraph, where objects act as vertices and morphisms as directed edges.

Schema Category The core of conceptual modelling of multi-model data in *MM-evocat* forms so-called *schema category*.⁴ To simplify the understanding, we will explain it using the terms known from the ER model⁵, though we do not need to distinguish them, as they are all treated in the same way.

Schema category $\mathbf{S}$ is defined as a tuple $(\mathcal{O}_\mathbf{S}, \mathcal{M}_\mathbf{S}, \circ_\mathbf{S})$. Objects in $\mathcal{O}_\mathbf{S}$ correspond to entity types, attributes, and relationship types. Attribute/relationship/hierarchy morphisms in $\mathcal{M}_\mathbf{S}$ connect appropriate pairs of objects. The explicitly defined morphisms are denoted as *base*, those obtained via the composition $\circ$ as *composite*.

Each object $o \in \mathcal{O}_\mathbf{S}$ is internally modelled as a tuple $(key, label, superid, ids)$, where $key \in \mathbb{O} \subseteq \mathbb{N}$ is an automatically assigned internal identity, *label* is an

⁴The *instance category* [13] was also introduced for unified representation of data.

⁵An ER model can be automatically transformed to a schema category. [103]

optional user-defined name (e.g., name of the corresponding entity type) or $\perp$ when missing, $superid \neq \emptyset$ is a set of attributes forming the actual data contents of a given object, and $ids \subseteq \mathcal{P}(superid)$, $ids \neq \emptyset$ is a set of particular identifiers (each modelled as a set of attributes) allowing us to uniquely distinguish individual data instances. It holds that $\bigcup_{id \in ids} id \subseteq superid$ (equality for entity or attribute objects).

Each morphism $m \in \mathcal{M}_{\mathbf{S}}$ is a tuple $(signature, dom, cod, min, max)$. $signature \in \mathbb{M}^*$ enables us to distinguish all morphisms except the identity ones. $\mathbb{M}^*$ is the set of all possible sequences of symbols from $\mathbb{M}$ connected using “.”, $signature \in \mathbb{M}$ is used for the base morphisms, $signature \in \mathbb{M}^* \setminus (\mathbb{M} \cup \{\varepsilon\})$ is used for the composite morphisms allowing their decomposition to base morphisms, $signature = \varepsilon$ is used for identity. dom and cod represent the domain and codomain of the morphism. Cardinalities $min \in \{0, 1\}$ and $max \in \{1, *\}$, $min < max$, allow us to express constraints on min/maximal numbers of occurrences. Identity morphism for $o \in \mathcal{O}_{\mathbf{S}}$ is defined as $1_o = (\varepsilon, o, o, 1, 1)$.

For non-identity morphisms $m_1 = (signature_1, dom_1, cod_1, min_1, max_1)$ and $m_2 = (signature_2, dom_2, cod_2, min_2, max_2) \in \mathcal{M}_{\mathbf{S}}$, their composition $m_2 \circ_{\mathbf{S}} m_1 = (signature_2 \cdot signature_1, dom_1, cod_2, \min(min_1, min_2), \max(max_1, max_2))$.

Example 2.3. Figure 2.2 depicts the ER (a) and categorical (b) conceptual model of sample data from Figure 2.1. $\square$

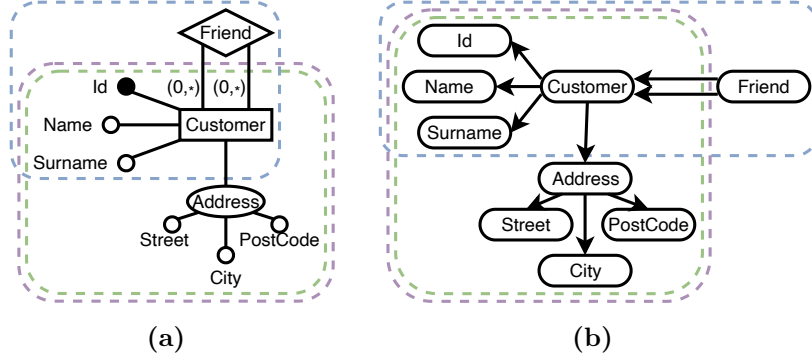


Figure 2.2 ER (a) and categorical (b) schema of sample data

Mapping To “glue” the categorical conceptual schema with the respective logical schemas in the underlying DBMSs, the schema category is decomposed and mapped to particular model-specific data structures. Since the terminology within the particular popular models differs, in [12] we provide the unification of respective model-specific terms. A *kind* corresponds to a class of items represented in each model, e.g., a relational table or a collection of JSON documents. A *record* corresponds to one item of a kind, e.g., a table row or a JSON document. Depending on a particular model, a record consists of simple or complex *properties* having their *domains*.

At the logical level, a transition between two distinct models can be expressed either via (1) *inter-model references* or by (2) *embedding* one model into another (e.g., columns of type JSON in tables of the relational model of *PostgreSQL*⁶).

⁶<https://www.postgresql.org/>

Another possible combination of models is via (3) *multi-model redundancy*, i.e., storing the same data fragment in two or more distinct models.

The decomposition of a schema category $\mathbf{S}$ (depicted in Figure 2.2 (b) using colours), eventually partial or overlapping, is defined via a set of *mappings*, as also formally defined in [13] (and omitted for paper length). Each mapping describes where and how data instances of a subgraph of $\mathbf{S}$ (following particular conditions) are stored in a kind of a particular DBMS. For each kind, the mapping specifies the respective DBMS, its name, its root object in $\mathbf{S}$, and an *access path* which recursively (i.e., in the same way) describes the structure of a kind, i.e., its (simple or complex) properties, relatively to the root object. The description is rich enough to cover various specifics of the underlying models and their combinations, such as properties with user-defined, anonymous, or dynamically-derived names; properties *inlined* from more distant parts of the categorical graph (via composite morphisms); auxiliary properties used, e.g., for logical grouping of a set of properties; order-preserving/order-ignoring sets of subproperties etc. We also introduce a JSON-like representation of an access path for easier understanding.

Example 2.4. In Figure 2.3, the mapping is depicted for kinds (tables) *Address* and *Customer* using the dashed arrows. $\square$

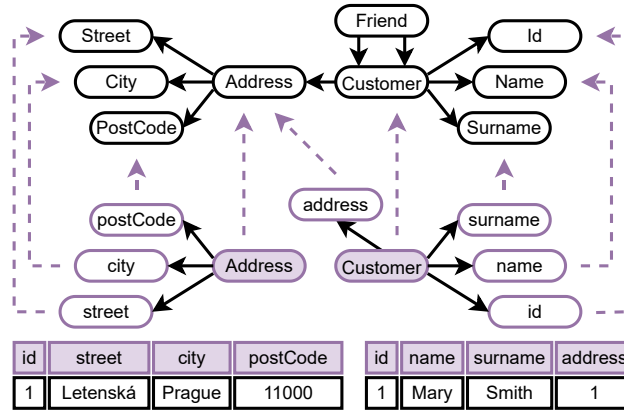


Figure 2.3 Relational-to-conceptual mapping

2.4 Evolution Manager *MM-evocat*

MM-evocat is an extensible modular framework whose core forms the introduced categorical conceptual model. It enables the creation of a categorical model, its decomposition and mapping to any combination of the supported models, and the application of user-specified modifications. The architecture of *MM-evocat* is depicted in Figure 2.4. At the lowest, platform-specific (green) level, there are the particular DBMSs, each with respective wrappers ensuring the unified communication with the DBMS and unifying representation of the multi-model data. Currently it supports the following models and DBMSs: *PostgreSQL* (relational and document, i.e., multi-model), *Neo4j*⁷ (graph), and *MongoDB*⁸ (NoSQL document). The blue logical-to-conceptual mapping represents the “glue” between

⁷<https://neo4j.com/>

⁸<https://www.mongodb.com/>

the logical schemas and the conceptual schema at the (red) platform-independent level. Over the conceptual schema, the user can specify evolution/migration operations or conceptual queries. (We also proposed an approach for inference of the categorical conceptual schema [16] from the sample data.)

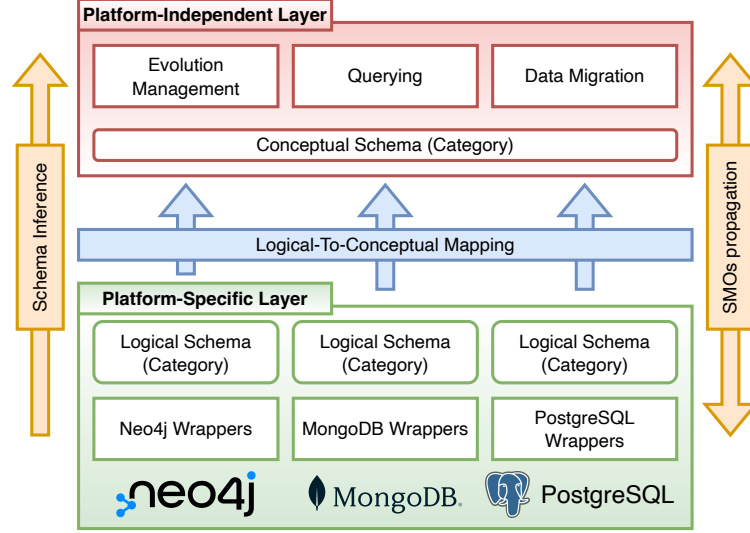


Figure 2.4 Architecture of *MM-evocat*

In Figure 2.5 we provide a screenshot of *MM-evocat* with the same categorical model as in Figure 2.2 (b).

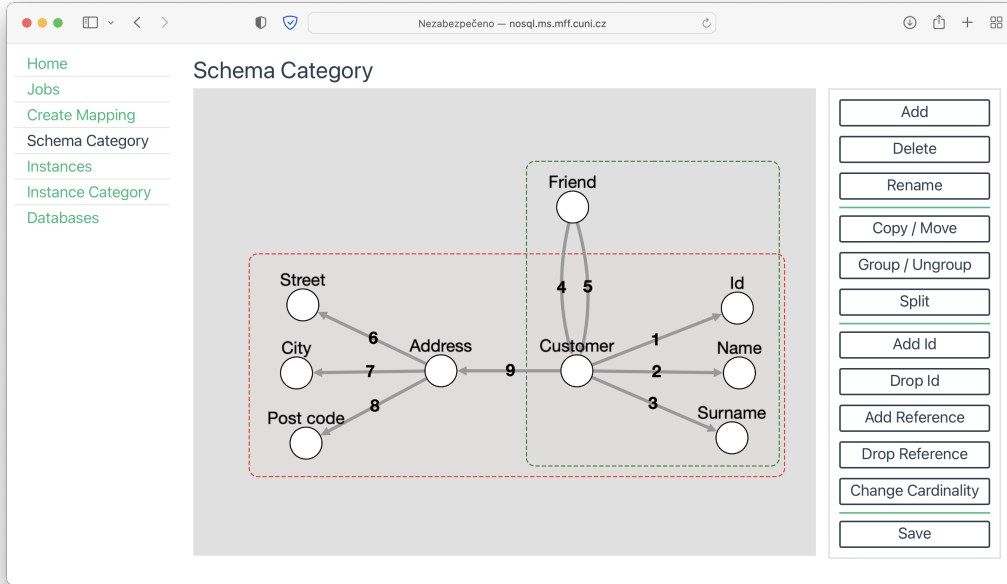


Figure 2.5 A screenshot of *MM-evocat*

2.4.1 Schema Modification Operations

For specification of user-required changes, we have defined the *Multi-Model Schema Evolution Language* (MMSEL), i.e., a set of *Schema Modification Opera-*

tions (SMOs) that enable to create and delete any allowed schema. Figure 2.6 shows its EBNF grammar.

```

MMSEL := ADD | DELETE | RENAME | COPY | MOVE | GROUP | UNGROUP | UNION |
        SPLIT | ADD_ID | DROP_ID | ADD_REF | DROP_REF | CHANGE_CARDINALITY

ADD      := "add"      PROP [("to" PROP) | ("=" {val} "to" PROP) [SELECTION]]
DELETE   := "delete"   PROP [SELECTION]
RENAME   := "rename"   PROP "to" {name} [SELECTION]
COPY     := "copy"     PROP "to" PROP [SELECTION]
MOVE     := "move"     PROP "to" PROP [SELECTION]
GROUP    := "group"    PROP["", "PROP"* "to" PROP [SELECTION]
UNGROUP  := "ungroup"  PROP [SELECTION]
UNION    := "union"    PROP["", "PROP"* "to" PROP "separator" {sep} [SELECTION]
SPLIT    := "split"    PROP TO PROP["", "PROP"* "separator" {sep} [SELECTION]
ADD_ID   := "addId"    PROP["", "PROP"* "to" PROP
DROP_ID  := "dropId"   PROP["", "PROP"* "from" PROP
ADD_REF  := "addRef"   PROP["", "PROP"* "from" PROP "to" PROP
DROP_REF := "dropRef"  PROP["", "PROP"* "from" PROP "to" PROP
CHANGE_CARDINALITY := "changeCardinality"
                    (PROP1->PROP2      "to" PROP1<-PROP3->PROP2) |
                    (PROP1<-PROP3->PROP2 "to" PROP1->PROP2) ["separator" {sep}] [SELECTION]

SELECTION := "where" COND ["and" | "or" COND]*
COND      := PROP "=" {val}
PROP, PROP1, PROP2, PROP3 := [{schemaId}]."{label}" | [{schemaId}]."{key}"
<-, ->    := [{schemaId}]."{signature}"

```

Figure 2.6 EBNF syntax of MMSEL (Non-terminals are capitalised. Terminals are in quotation marks. Variables are in curly brackets.)

As we can see, we naturally involve core operations to *add*, *delete*, or *rename* a property and common operations to *copy* or *move* a property to another property. We also support the creation of a new property from a set of properties, either creating a complex parent property (*group*) or a simple property containing their values using a specified separator (*union*), including their inverse operations (*ungroup*/*split*). In addition, we enable to work with identifiers (*addId*/*dropId*) and references (*addRef*/*dropRef*). Finally (and also uniquely), we enable the modification of cardinalities in both directions (also covering the change between a simple type and an array), i.e., from more to less restricted and vice versa (*changeCardinality*).

Regarding the propagation, the operations can be further divided into heavy and light, depending on user requirements. A *heavy operation* is an SMO propagated from the conceptual level to the mapping, logical level, and data instances. Conversely, a *light operation* only triggers a propagation to the mapping. Each SMO can be heavy. Operations *add*, *rename*, *group*, *addId*, and *addRef* can be also light. (Note that this behaviour is different from the *eager* and *lazy* migration considered in [57, 101]. These classes reflect whether the operation is propagated immediately or the propagation can be delayed until the change is needed.)

A sample heavy operation is provided in Figure 2.7. We apply operation *ungroup Address* to extract subproperties *Street*, *City*, and *PostCode* from kind *Address* and to store them directly to kind *Customer*. This part of the schema

is stored redundantly in both relational (purple) and document (green) models. In the relational model the respective kinds (tables *Address* and *Customer*) are merged into one kind (table) *Customer*. Conversely, we only remove one level from the hierarchical JSON data in the document model.

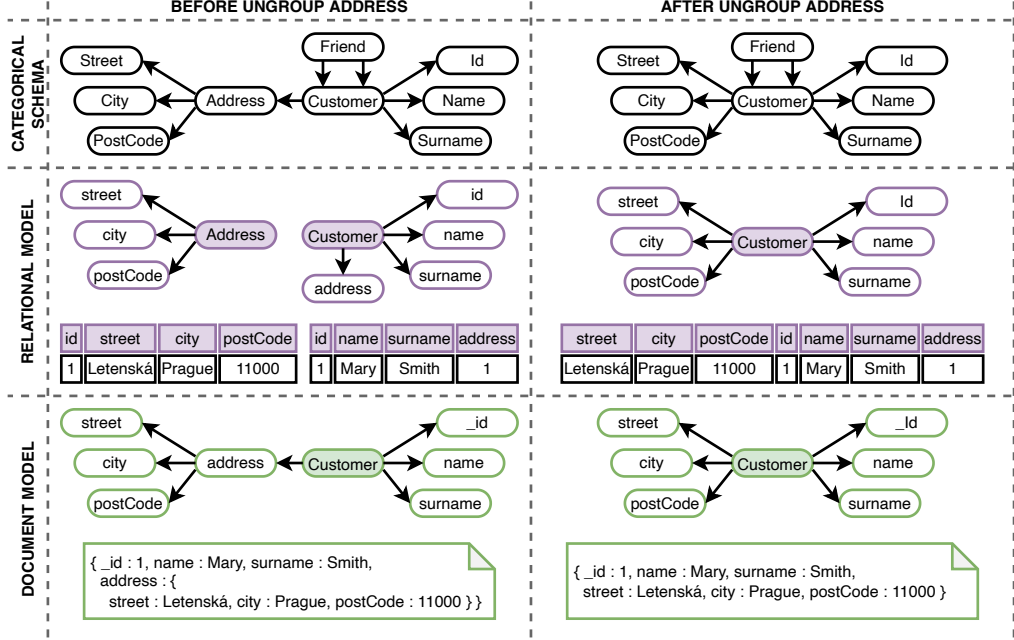


Figure 2.7 Heavy operation ungroup Address

A sample light operation is provided in Figure 2.8. Operation *group Name, Surname to Personal* creates a new complex property *Personal* consisting of existing properties *Name* and *Surname*. The change in the categorical schema is propagated to the mapping; however, the logical level remains unchanged. We also provide an example of a conceptual query over the schema category, which reflects the change at the conceptual level. Conversely, the graph query over the graph (logical) model remains unchanged.

2.5 Demonstration Outline

Our demonstration will be based on three collections of the *Yelp Academic Dataset*⁹ of JSON documents – *Review*, *Tip*, and *User* – transformed into the three supported data models, i.e., graph, JSON document, and relational. Using the data, we will gradually modify the datasets to demonstrate:

1. the advantages of the categorical representation (decomposition, unifying representation, cross-model transitions etc.),
2. the rich set of SMOs, i.e., the new operations as well as the difference and applicability of light and heavy operations,
3. the number of affected kinds/properties and consequently the amount of saved manual and error-prone work, and

⁹<https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>

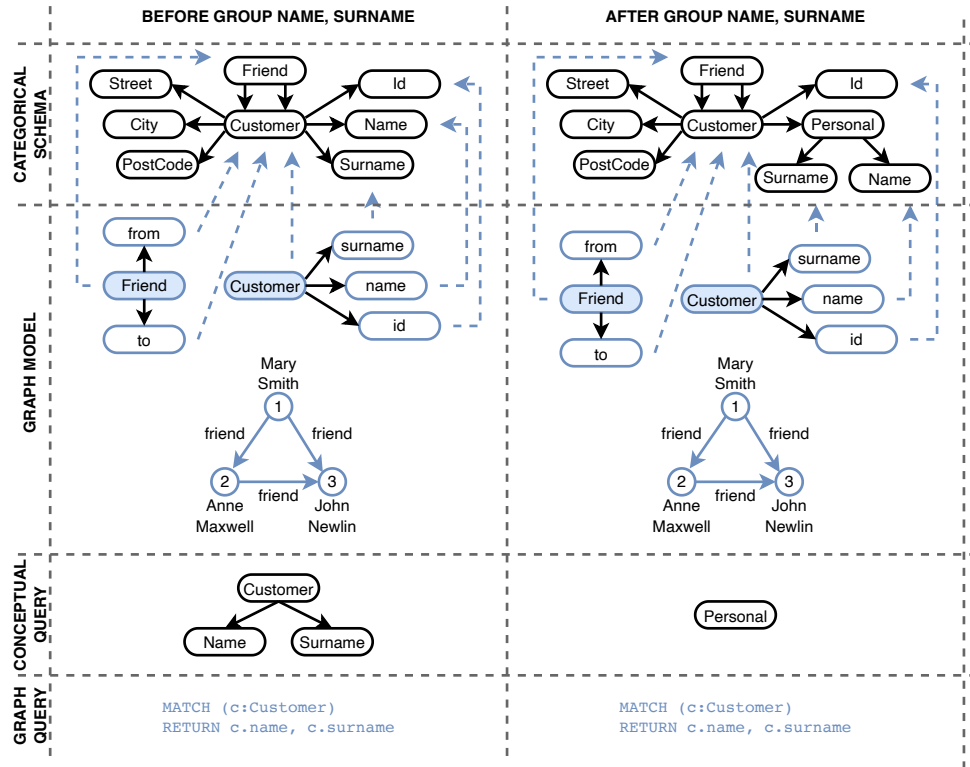


Figure 2.8 Light operation group name, surname to Personal

- the extensibility for future work, i.e., the efficient propagation to queries or the utilisation of AI for self-adaptability.

Paper III

MM-evoque: Query Synchronisation in Multi-Model Databases

Pavel Koupil¹, Jáchym Bártík¹, Irena Holubová¹

Published as Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evoque: Query Synchronisation in Multi-Model Databases”. In: *Proceedings of EDBT 2024*. OpenProceedings.org, 2024, pp. 818–821. DOI: [10.48786/edbt.2024.78](https://doi.org/10.48786/edbt.2024.78)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

As multi-model databases become increasingly prevalent, the evolving multi-model schemas pose a significant challenge to query validity. This demo paper introduces *MM-evoque*, a novel tool designed to propagate schema changes across multi-model databases and synchronise not only data but also respective queries to remain valid and performant.

To cover all the popular models and their combinations, *MM-evoque* is based on the categorical representation of multi-model data. To cover non-standard multi-model query languages of the underlying database systems, we use a conceptual multi-model query language MMQL. This demo paper introduces the functionality of *MM-evoque* in several types of use cases.

3.1 Introduction

Unlocking the power of multi-model database management systems (DBMSs)² is not just about keeping pace with the present majority; it is a strategic leap into the future of data complexity. Regrettably, our extensive survey [7] reveals significant variations in multi-model database management systems (DBMSs) even in the core aspects – the supported models, the way they are combined, as well as the multi-model query languages. This situation brings many new challenges to multi-model data management. We have already proposed several solutions based on the categorical representation of multi-model data, the so-called *schema category* [13]. This abstract graph representation backed by the formalism of category theory enabled us to build a family of tools for modelling and transformations [12], schema inference [16], data migration under evolving schema [82], or querying using SPARQL-like query language MMQL [15].

In this paper, we further extend our toolset with a new complementary member called *MM-evoque*³. It extends the functionalities we already provide with an important novel feature – propagation of changes in the multi-model schema to respective queries. The main contributions can be summarised as follows:

- Since we use the unifying categorical representation, we support any combination of existing popular data models (relational, key/value, document, column, array, and graph) that can be mutually embedded, referenced, or overlapped (i.e., redundancy is naturally supported too).
- The graph categorical representation can be queried using a SPARQL-like query language MMQL that is then decomposed and translated to a set of Database-Specific Languages (DSLs) of underlying DBMSs where they are executed. The results are combined into the final result.
- We introduce a set of schema-modification operations (SMOs) to modify the categorical schema. As each schema is represented as a sequence of operations creating it, the user can return to any previous version.

²<https://db-engines.com/en/ranking>

³<https://www.ksi.mff.cuni.cz/~koupil/mm-evoque/> (including a demo video)

- The schema changes are propagated to underlying data and respective queries over the data. We ensure synchronisation of the queries when needed and possible. If the correction is ambiguous or cannot be done automatically, the user is informed at least.
- We present a prototype implementation of *MM-evoque* demonstrating the indicated advantages in several use cases and shielding the user from the implementation specifics of particular DBMSs.

Paper Outline In Section 2, we review related work. Section 3 introduces the categorical representation of multi-model data and the categorical query language MMQL. Section 4 describes the presented tool and a bigger example. In Section 5, we outline its demonstration.

3.2 Related Work

The evolution management is challenging even for a single model. There exist approaches, e.g., for relational DBMSs [46], NoSQL systems [47], and first approaches for multi-model DBMSs [82]. But they focus primarily on data adaptation/migration.

Considering also queries, we can encounter related approaches with different aims and motivations. *Query rewriting* [48] focuses on transforming a query to a more efficient version. *Query adaptation* [49] focuses on adapting query plans to changing environmental conditions at runtime. In contrast, in this paper, we aim at *query synchronisation* reflecting the evolution of the schema, in particular of multi-model data. Several approaches also exist here, though subtly less represented. Most of them focus on a single model (relational or XML) [50]; some consider aggregate-oriented NoSQL models [51], or polystores [8].

From its beginning, our approach targets all popular models and existing types of their combinations. It provides an intuitive conceptual graph layer and SPARQL-like querying, both covertly backed by the category theory.

3.3 Categorical Model and Querying

Let us first remember the basic notions of category theory. A *category* $\mathbf{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of objects $\mathcal{O}$, set of morphisms $\mathcal{M}$, and a composition operation $\circ$ over the morphisms ensuring transitivity and associativity. Each morphism is modelled as an arrow $f : A \rightarrow B$, where $A, B \in \mathcal{O}$, $A = \text{dom}(f)$, $B = \text{cod}(f)$. And there is an *identity* morphism $1_A \in \mathcal{M}$ for each object A . The key aspect is that a category can be visualised as a multigraph, where objects act as vertices and morphisms as directed edges.

Schema Category The *schema category* [13] forms the core of conceptual modelling in all our tools. It is defined as a tuple $\mathbf{S} = (\mathcal{O}_\mathbf{S}, \mathcal{M}_\mathbf{S}, \circ_\mathbf{S})$. Objects in $\mathcal{O}_\mathbf{S}$ correspond to the domains of the ER model’s entity types, attributes, and relationship types. Morphisms (of semantics “has a property”, “has an identifier”, “has a role”, or “is a”) in $\mathcal{M}_\mathbf{S}$ connect appropriate pairs of objects.

The explicitly defined morphisms are denoted as *base*, obtained via the composition $\circ$ as *composite*.

Since the terminology within the particular models differs, we use a unification of respective model-specific terms [13]. A *kind* corresponds to a class of items represented in each model, e.g., a relational table or a collection of JSON documents. A *record* corresponds to one item of a kind, e.g., a table row or a JSON document. Depending on a particular model, a record consists of simple or complex *properties* having their *domains*.

The decomposition of a schema category $\mathbf{S}$, eventually partial or overlapping, is defined via a set of *mappings* [13]. For each kind, the mapping specifies where and how its records are stored in a selected single-/multi-model DBMS using a so-called *access path*, which recursively describes the structure of a kind.

Example 3.1. In Figure 3.5 on the left, we can see a schema category describing customers, their friendships, products, and orders. It is decomposed and mapped to relational (violet) and document (green) models. $\square$

Multi-Model Query Language (MMQL) The MMQL [15] is a SPARQL-like model-agnostic query language based on pattern matching. The MMQL expression consists of a projection (**SELECT**) that defines the structure of the result and a selection (**WHERE**) where graph patterns can be defined along with filter conditions (**FILTER**). We also support common constructs such as **ORDER BY**, **LIMIT**, or aggregation functions. Graph patterns are expressed using triples $p : s \rightarrow o$, where o, s (e.g., variables or constants) correspond to objects from $\mathcal{O}_{\mathbf{S}}$ and p maps to morphisms in $\mathcal{M}_{\mathbf{S}}$. For query q , patterns in selection form a *selection category* $\mathbf{S}_q^\sigma$ (a subcategory of $\mathbf{S}$) and patterns in projection form a *projection category* $\mathbf{S}_q^\pi$ (a subcategory of $\mathbf{S}_q^\sigma$).

Example 3.2. Figure 3.5 on the right shows two MMQL queries q_1 : “For each pair of friends, return their names and surnames” and q_2 : “Find the most expensive order ordered by Anne and return its price”. $\square$

Multi-Model Schema Evolution Language (MMSEL) The MMSEL [82] consists of 8 basic *schema modification operations* (SMOs) – see Table 3.1 – that enable the gradual creation (and deletion) of any *consistent* schema category. Hence, each operation is assigned with prerequisites that must be fulfilled.

In addition, we have operations *addMapping()* and *deleteMapping()*, which enable one to add/delete mapping of a kind, i.e., data migration.

Basic SMOs form more user-friendly *composite SMOs*, such as, e.g., (un)group or join/split [82]. *Correct* composition ensures that we only need to define the propagation of basic SMOs.

3.4 MM-evoque

MM-evoque was built on categorical representation, MMQL, and MMSEL to support query synchronisation. Currently, it supports the following models/DBMSs: *PostgreSQL*⁴ (relational and document, i.e., multi-model), *Neo4j*⁵

⁴<https://www.postgresql.org/>

⁵<https://neo4j.com/>

(graph), *MongoDB*⁶ (JSON document), and *Apache Cassandra*⁷ (columnar model).

In Figure 3.1, we provide a screenshot of *MM-evoque*. On the left, we can see MMQL queries synchronised with the schema category from Figure 3.5 visualised on the right.

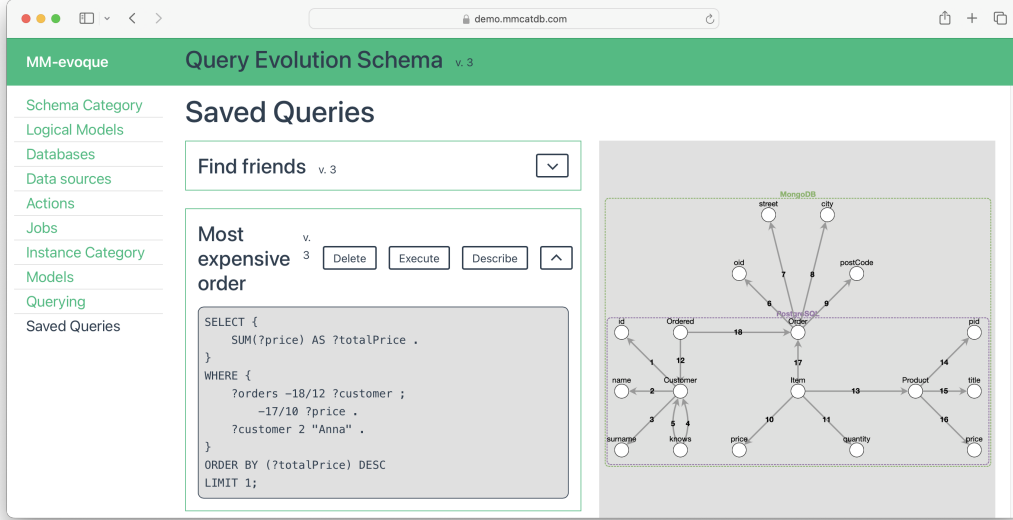


Figure 3.1 A screenshot of *MM-evoque*

Example 3.3. Consider Figures 3.2–3.5 and queries q_1 , q_2 from Example 3.2.

Figure 3.2 shows a naive relational schema with several redundant values in table *Orders* (version 1). We also depict the mapping of $\mathbf{S}$ to the relational model, where the violet objects represent kinds (tables), and their neighbours represent their properties (columns). The efficiency of q_1 and q_2 is satisfactory for low numbers of customers and products.

In Figure 3.3, we decided on the normalisation of the data. We applied the respective SMOs to $\mathbf{S}$ (version 2) and triggered their propagation to data [82]. Their propagation to queries ensured by *MM-evoque* does not change q_1 . But for q_2 , it adds the new relationship tables to the selection of MMQL; thus, the SQL translation contains more joins.

In Figure 3.4 (version 3A), we try to use the document model instead. Hence, the schema category $\mathbf{S}$ remains unchanged, but the data is migrated from the relational to the document model (green). Queries q_1 and q_2 are slightly (technically) modified since we assume that every graph pattern must start in the root of a kind. However, we still have a problem with data redundancy (friends of customers and products) in this solution.

In Figure 3.5 (version 3B), we utilise the multi-model paradigm and migrate the data to a combination of relational (violet) and document (green) models. q_1 and q_2 are synchronised, respectively, and we gain a good performance for both.

□

⁶<https://www.mongodb.com/>

⁷https://cassandra.apache.org/_/index.html

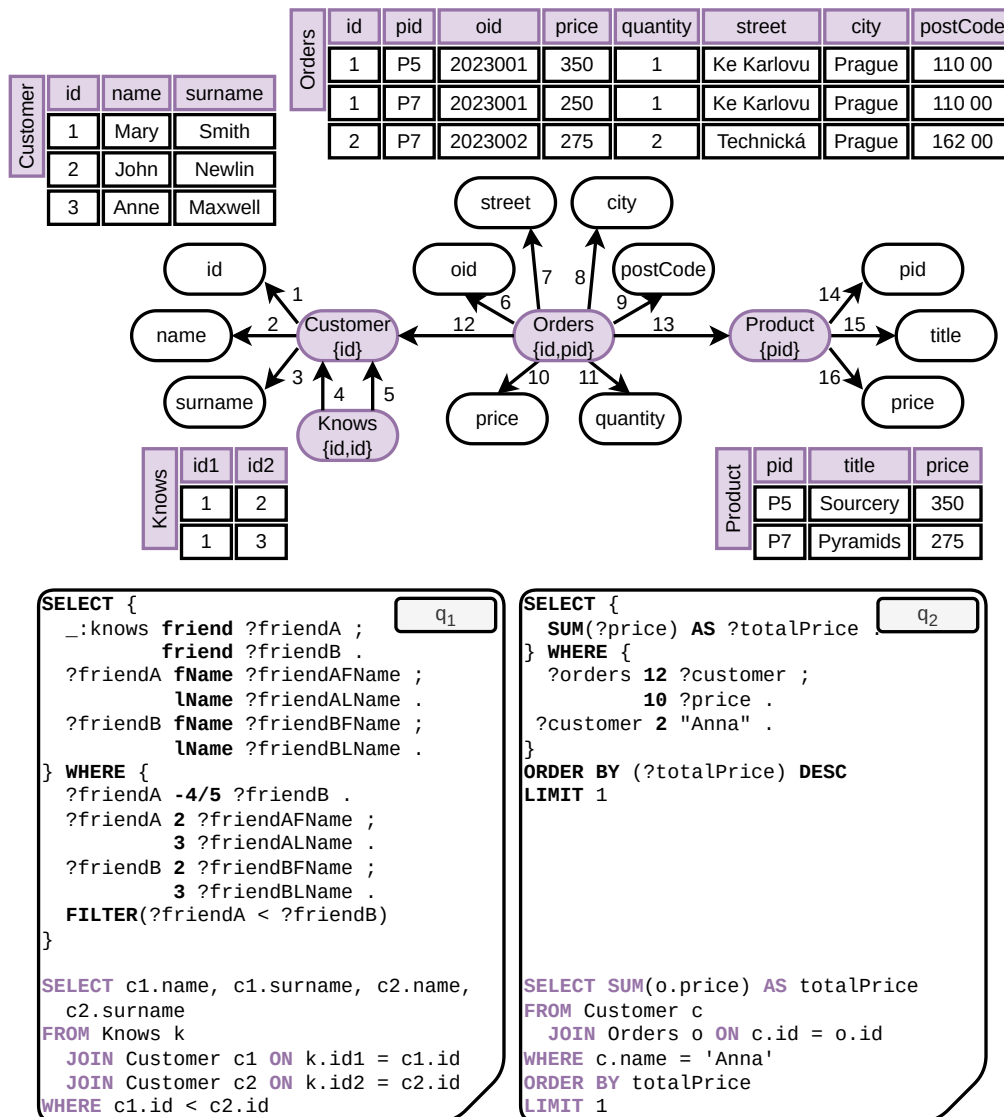


Figure 3.2 Version 1 – a naive mapping to the relational model (PostgreSQL)

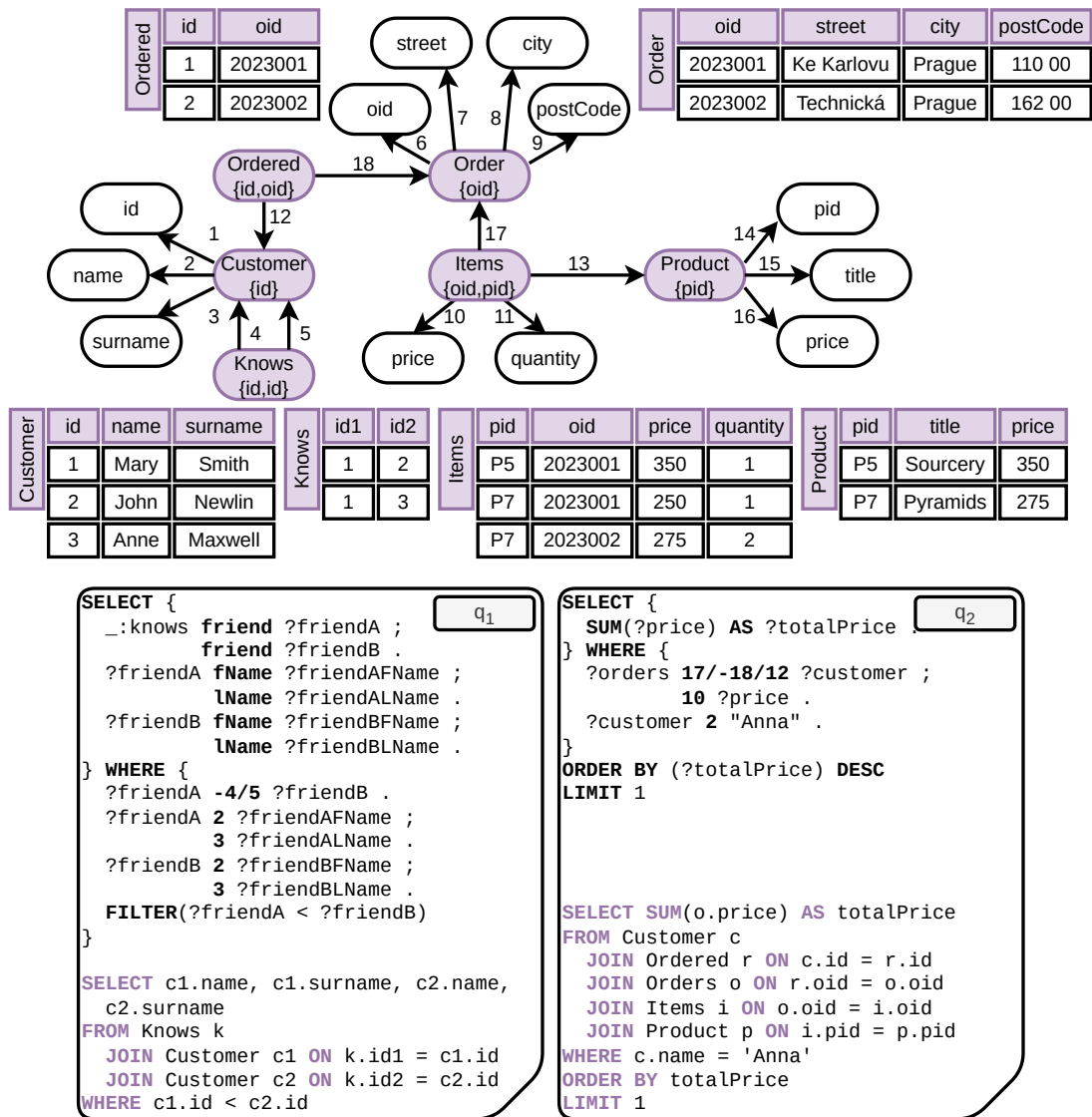


Figure 3.3 Version 2 – a mapping to a normalized relational schema (PostgreSQL)

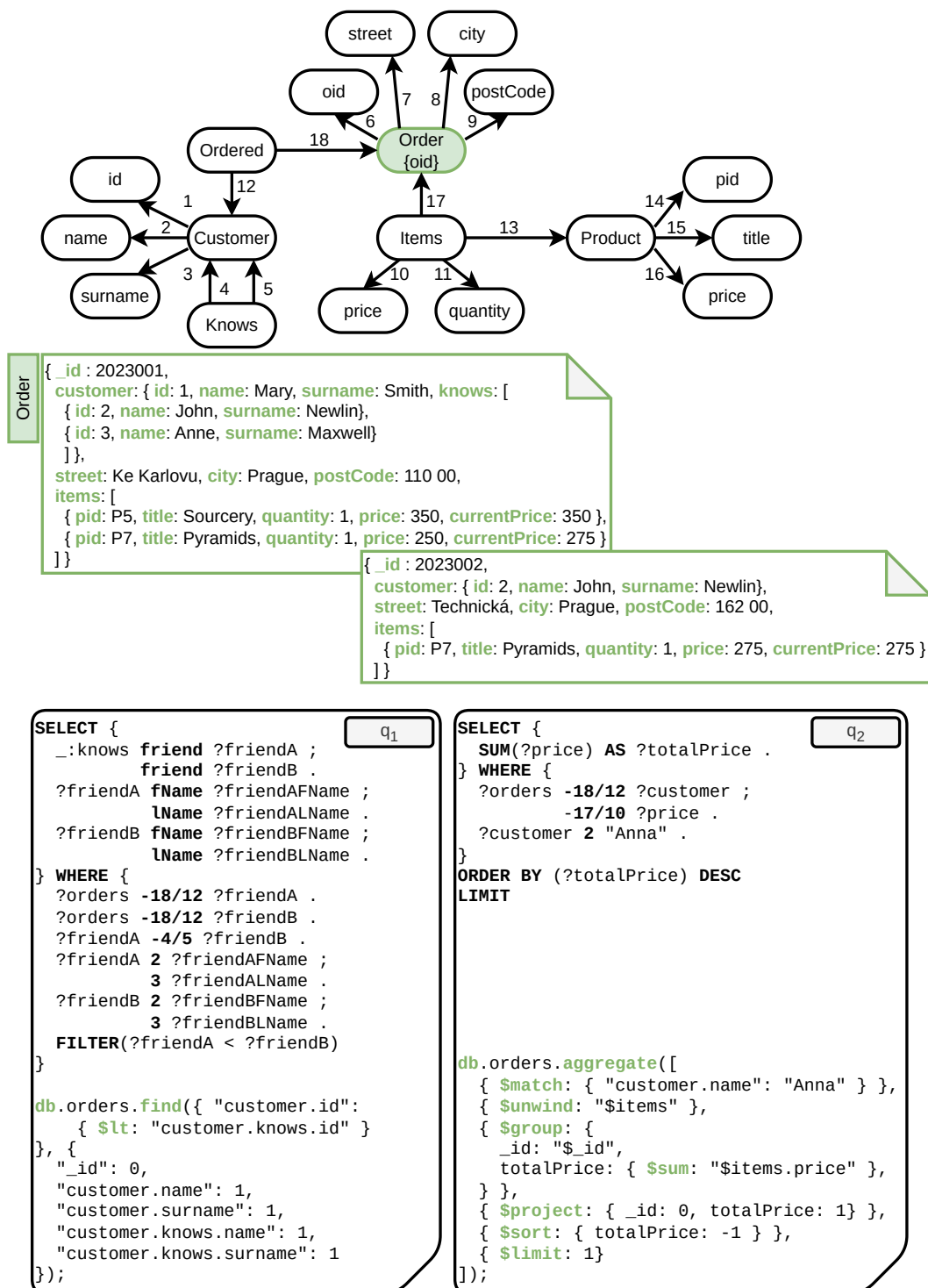


Figure 3.4 Version 3A – a mapping to the document model (MongoDB)

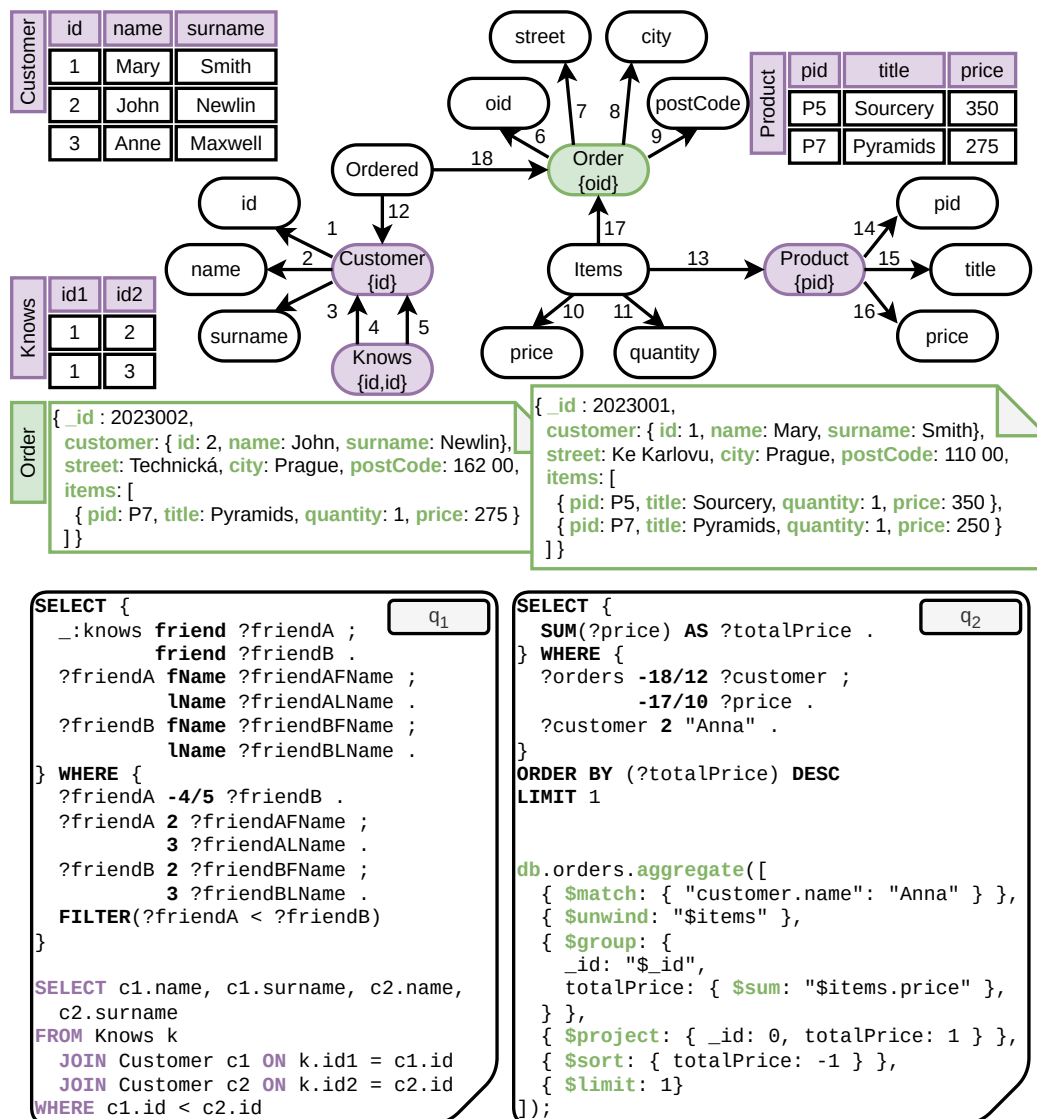


Figure 3.5 Version 3B – a multi-model mapping (PostgreSQL and MongoDB)

SMO	Behaviour	Prerequisite
$create(O)$	Creates object O with a unique identifier and default parameters.	–
$create(m, D, C)$	Creates morphism $m : D \rightarrow C$ with a unique signature and default parameters.	$D, C \in \mathcal{O}_{\mathbf{S}}$
$delete(O)$	Deletes object O .	$O \in \mathcal{O}_{\mathbf{S}}$, O can participate only in $1_O \in \mathcal{M}_{\mathbf{S}}$
$delete(m)$	Deletes morphism m .	$m \in \mathcal{M}_{\mathbf{S}}$
$rename(O, n)$	Changes the name of object O to n .	$O \in \mathcal{O}_{\mathbf{S}}$
$move(m, p)$	Sets $dom(m)$ to $cod(p[last])$, i.e. to the end of a path p .	$m \in \mathcal{M}_{\mathbf{S}}$, $dom(p[first]) = dom(m)$
$copy(O)$	Creates a copy O' of object O (with a new unique identifier)	$O \in \mathcal{O}_{\mathbf{S}}$
$copy(m, D, C)$	Creates a copy $m' : D \rightarrow C$ of morphism m (with a new unique signature)	$D, C \in \mathcal{O}_{\mathbf{S}}$, $D = copy(dom(m))$, $C = copy(cod(m))$

Table 3.1 Basic SMOs

	$create(O)$	$create(m, D, C)$	$delete(O)$	$delete(m)$	$rename(O, n)$	$move(m, p)$	$copy(O)$	$copy(m, D, C)$	$addMapping()$ $deleteMapping()$
S_q^σ	0	0	0 ∞	0 1 ∞	0	0	0	0	0 1
VALUES	0	0	0 ∞	0 1	0	0	0	0	0 1
FILTER	0	0	0 ∞	0 1	0	0	0	0	0 1
S_q^π	0	0	0 ∞	0 1	0	0	0	0	0 1
DISTINCT	0	0	0 ∞	0 1	0	1	0	0	0 1
ORDER BY	0	0	0 ∞	0 1	0	0	0	0	0 1
LIMIT	0	0	0	0	0	0	0	0	0 1
Aggregations	0	0	0 ∞	0 ∞	0	∞	0	0	0 1

Table 3.2 Basic SMOs and their impact on MMQL query constructs

3.4.1 Workflow

Having the old and new versions of the schema category, denoted $\mathbf{S}$ and $\mathbf{S}'$, and the MMQL query q to be synchronised with the changes, the workflow of *MM-evoque* is as follows:

1. The sequence of basic SMOs which changed $\mathbf{S}$ to $\mathbf{S}'$ is determined (as composite SMOs consist of basic ones).
2. MMQL query q is translated into a set of constructs. The selection category $\mathbf{S}_q^\sigma$ and projection category $\mathbf{S}_q^\pi$ are created according to respective graph patterns of q .
3. Using Table 3.2, we determine whether q can be synchronised as maximum of $\text{impact}(\text{smo}, \text{construct})$ from all pairs (SMO, affected MMQL construct) occurring in q . If the result is 0, we can synchronise q and preserve its original behaviour. If the result is 1, we can synchronise q , but its behaviour may change (see Section 3.4.2). Otherwise, the changes are too critical that we cannot easily adapt the query.
4. If the user agrees with the proposed changes, we apply the SMOs to $\mathbf{S}_q^\sigma$ and $\mathbf{S}_q^\pi$. We use the fact that both are subcategories of $\mathbf{S}$, so we identify SMOs that affect them.
5. The modified query q' is translated to a new version of DSL queries.

3.4.2 SMO-to-MMQL Propagation

Let us discuss the impact of SMOs from Table 3.1 to $\mathbf{S}_q^\sigma$ and $\mathbf{S}_q^\pi$.

Operation $\text{create}(O)$, i.e., creating a new object $O \in \mathcal{O}_{\mathbf{S}}$, is not propagated to $\mathcal{O}_{\mathbf{S}_q^\sigma}$, i.e., it does not impact σ . Similarly, operation $\text{create}(m, D, C)$, i.e., forming an association between two objects $D, C \in \mathcal{O}_{\mathbf{S}}$, does not affect $\mathcal{M}_{\mathbf{S}_q^\sigma}$.⁸

The $\text{delete}(O)$ prerequisite ensures that O cannot participate in non-identity morphisms. So, we delete a whole kind with a single property. This has a critical impact on σ if $O \in \mathcal{O}_{\mathbf{S}_q^\sigma}$.

Operation $\text{delete}(m)$ is propagated as follows: (i) We try to modify $\mathbf{S}_q^\sigma$ by replacing m with $m' \in \mathcal{M}_{\mathbf{S}}$, s.t. m and m' form a *comutative diagram* (i.e., the semantics is preserved). (ii) If there is no such m' , we remove all unreachable subgraphs of $\mathbf{S}_q^\sigma$. (iii) The unreachable variables are also removed from $\mathbf{S}_q^\pi$, filtering, aggregation, and result modifiers. Removing an unreachable variable may cause, e.g., a result containing more records when filtering, a result differently ordered using `ORDER BY`, etc.

Operation $\text{rename}(O, n)$ is not propagated to σ as there is no binding between object names and MMQL variable names.

Operation $\text{move}(m, p)$ only changes the morphisms, leaving the MMQL variables unchanged. Thus, the move operation has no impact on the evaluability of q except for the clause `DISTINCT` and aggregation functions. In the former case, a different result may be returned. In the latter case, the root of the aggregation may be changed. Nevertheless, the modification of $\mathbf{S}_q^\sigma$ may be significant in the

⁸Note that the attributes in the projection of σ must be explicitly specified.

DSL statement, where, e.g., the join or graph traversal may be more complex as we change a base $m \in \mathcal{M}_{\mathbf{S}}$ to a composite one.

Operations $copy(O)$ and $copy(m, D, C)$ do not impact σ . However, the second one could simplify $\mathcal{M}_{\mathbf{S}_\sigma}$ by replacing a composite morphism with a base morphism m' .

The propagation of $addMapping()$ / $deleteMapping()$ depends on the underlying logical representation of the data. The following complications may arise: (i) We do not change $\mathbf{S}$, but only the mapping. Hence, there should not be any changes in q . But, as depicted in Figure 3.4, we might need to synchronise the query with the newly defined kinds, as every graph pattern must start in the root of a kind. (ii) MMQL statement does not change, yet target DSL does not allow particular MMQL construct. Then, such construct is evaluated utilising *MM-quecat* [14] as a so-called *postponed statement*, which is not propagated into the target DSL.

3.5 Demonstration Outline

In our presentation, we will first demonstrate the described functionality of *MM-evoque* using the example depicted in Figures 3.2– 3.5. Next, we will demonstrate additional features, such as the representation of the schema category using a sequence of operations or the support for versioning and redundancy.

As the tool is currently a prototype part of a robust research aim of efficient unifying multi-model data management, we will also introduce the broader context and discuss the open problems and challenges.

Paper IV

Model-Agnostic Evolution Management

Pavel Koupil¹, Jáchym Bártík¹, Irena Holubová¹

Published as Pavel Koupil, Jáchym Bártík, and Irena Holubová.
“Model-Agnostic Evolution Management”. In: *Proceedings of RCIS 2025*. Vol. 547.
LNCS. Springer, 2025, pp. 245–261. DOI: [10.1007/978-3-031-92474-3_15](https://doi.org/10.1007/978-3-031-92474-3_15)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

This paper tackles a critical challenge in data management: adapting to evolving user requirements, a concept we term *evolution management*. Our approach specifically addresses the evolution management of multi-model data. By leveraging category theory, we provide an abstract representation of the combined models. We extend this categorical framework by introducing schema-modification operations of varying types and complexities. And, we propose corresponding propagation strategies to adapt queries, with a particular focus on mitigating the impact of *decreasing information capacity*. Through experimental verification, we demonstrate significant reductions in effort across different strategies and their effects on querying.

4.1 Introduction

More than two-thirds of the 50 most widely used database management systems (DBMSs)² fall into the category of *multi-model*, i.e., supporting multiple models as first-class citizens. However, due to the absence of established standards governing which models should be combined and how or how to query the multi-model data, the features of existing multi-model databases exhibit significant variations in the supported models, the way they are combined, the multi-model query languages, etc. [7]. In addition, combining different models, such as aggregate-oriented and aggregate-ignorant, flat and structured, or schema-full and schema-less, naturally introduces conflicting features requiring specific handling.

This situation presents numerous challenges across all aspects of data management. In this paper, we focus on the situation when user requirements change, resulting in the need to adapt the data structures, so-called *evolution management*, i.e., propagation of the user-required changes in the schema to all affected parts of the system while preserving its integrity and efficiency. We must also consider the cross-model redundancy and the contradictory features of the combined models/DBMSs. Our approach is based on our unifying abstract categorical representation of multi-model data [13], which covers all popular data models and their combinations, including cross-model redundancy. In addition, its natural graph representation shields the user from the technical details of the underlying models and DBMSs.

The key and, to the best of our knowledge, genuine contributions of this paper to the evolution management of multi-model data are as follows:

- For the categorical model of multi-model data, we formally define three types of schema-modification operations (SMOs) – *basic*, *composite*, and *complex* – enabling natural and user-friendly model modifications. In addition, our set is among the most comprehensive among related solutions, while model abstraction eliminates redundant operations for analogous concepts across underlying logical models, ensuring both expressiveness and conciseness.
- We extend our multi-model evolution-management approach by synchronizing multi-model queries, i.e., ensuring that the queries provide the same

²<https://db-engines.com/en/ranking>

results after applying SMOs. We primarily focus on the challenging aspect of *decreasing information capacity*, i.e., the possible loss of information. We utilize several approaches to the multi-model context to solve the problem.

- Through a proof-of-concept implementation, we demonstrate how our approach reduces manual, error-prone work. And, we experimentally evaluate the efficiency of the distinct synchronization strategies.

Outline Section 4.2 introduces the categorical representation of multi-model data. Section 4.3 describes the evolution management strategy, the SMOs, and their propagation. Their implementation and experiments are described in Section 4.4. Section 4.5 overviews related work, and Section 4.6 concludes.

4.2 Categorical Representation of Multi-Model Data

First, let us introduce our categorical representation of multi-model data. To unify the terminology of distinct models, we use the following terms: A *kind* corresponds to a class of items (e.g., a relational table or a collection of documents). A *record* corresponds to an item of a kind (e.g., a table row or a document). A record consists of simple/complex *properties* having *domains*.

Let us also remember the basic notions of category theory. A *category* $\mathbf{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of objects $\mathcal{O}$, set of morphisms $\mathcal{M}$, and a composition operation $\circ$ over the morphisms ensuring transitivity and associativity. Each morphism is modeled as an arrow $f : A \rightarrow B$, where $A, B \in \mathcal{O}$, $A = \text{dom}(f)$, $B = \text{cod}(f)$. And there is an *identity* morphism $1_A \in \mathcal{M}$ for each object A . Apparently, a category can be visualized as a multigraph, with objects represented as vertices and morphisms as directed edges.

The core and the integration point of all our solutions is an abstract representation of multi-model data called *schema category* [13]. It is defined as a tuple $\mathbf{S} = (\mathcal{O}_{\mathbf{S}}, \mathcal{M}_{\mathbf{S}}, \circ_{\mathbf{S}})$. Objects in $\mathcal{O}_{\mathbf{S}}$ correspond to the domains of the ER model’s entity types, attributes, and relationship types. Morphisms in $\mathcal{M}_{\mathbf{S}}$ (of semantics “has a property”, “has an identifier”, “has a role”, or “is a” and respectively labels) connect appropriate pairs of objects.

The decomposition of a schema category $\mathbf{S}$, eventually partial or overlapping, is defined via a set of *mappings*. For each kind, the mapping specifies where and how its records are stored in a selected single-/multi-model DBMS utilizing a concept of *functor*, which mutually maps corresponding objects of logical and conceptual schemas and preserves the hierarchical structure of a kind.

Example 4.1. Consider a part of the sample multi-model scenario in Figure 4.1(c). In the categorical schema, kinds *User* and *Business* (whose root objects are highlighted with colors) are mapped to the respective relational table (purple) and document collection (green). The kind *User* has only simple properties, e.g., *name* or *fans* (we omit the most common morphism label *#property* for morphisms of semantics “has a property”). In the curly braces, we can see the identifiers of the kinds, e.g., *user_id* for kind *User*. The kind *Business* also has a complex property *address* consisting of several simple properties (e.g., *city* and *street*). There is also kind *Tip*, which represents the relationship of kinds

denoted using the respective morphisms with label *#role*, of semantics “has a role” (i.e., *User* and *Business*). Kind *Friends* represents an array of pairs of objects denoted using morphisms with label *#role*, whose position is given by the simple property *_INDEX*. Last but not least, complex property *attributes* of kind *Business* represents a map of (*key*, *value*) pairs associated with kind *Business*. $\square$

4.2.1 Conceptual Querying of Multi-Model Data

The categorical representation can be viewed as a graph. Thus, it can be queried using a graph query language. For this purpose, we introduced the *Multi-Model Query Language* (MMQL) [15] utilizing the well-known SPARQL notation.

An MMQL expression consists of a projection (**SELECT**) that defines the structure of the result and a selection (**WHERE**), where graph patterns can be defined along with filter conditions (**FILTER**). We also support result modifiers such as **ORDER BY**, **LIMIT**, or aggregation functions. Graph patterns are expressed using triples $p : s \rightarrow o$, where o, s (e.g., variables or constants) correspond to objects from $\mathcal{O}_{\mathbf{S}}$ and p maps to morphisms in $\mathcal{M}_{\mathbf{S}}$. In other words, instead of RDF graph data, we query the schema category $\mathbf{S}$. (Sample MMQL queries are provided in Figure 4.2(a) and explained in Example 4.3.)

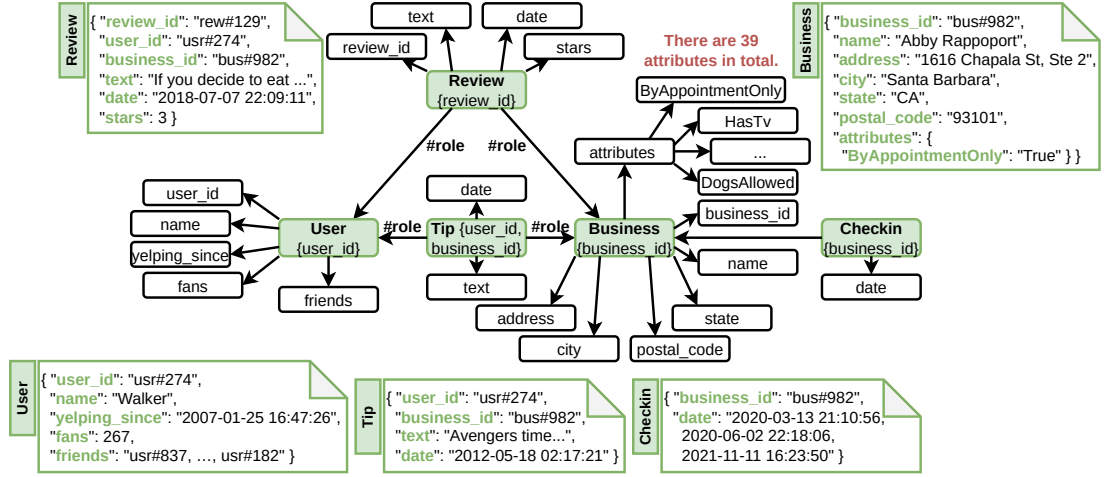
4.3 Evolution Management Approach

A schema category $\mathbf{S}$ represents the conceptual model of a given multi-model use case, abstracting from the technical details of underlying logical models. In addition to their specific features, the logical schemas can overlap, i.e., parts of the data can be represented redundantly using distinct logical models. Users interact with $\mathbf{S}$ represented as a graph while mapping to logical schemas ensures information preservation. This mapping can be inferred from sample data instances [17] or created manually for selected parts of $\mathbf{S}$ using the operations *createMapping()*, *updateMapping()*, and *deleteMapping()*.

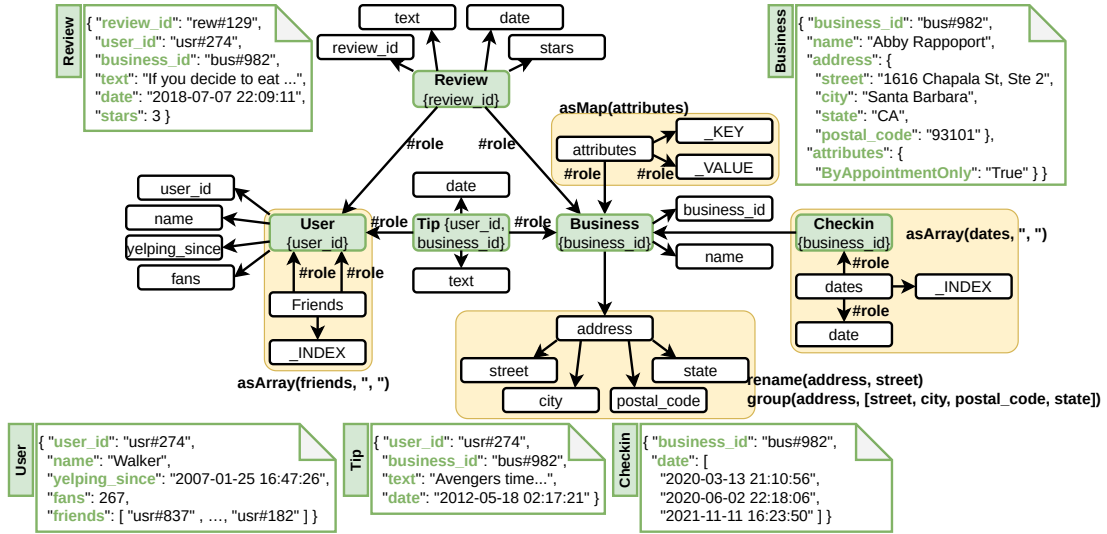
4.3.1 Schema Modification Operations

For the specification of user-required changes, we define the *Multi-Model Schema Evolution Language*, i.e., a set of schema modification operations (SMOs) that enables creating and deleting any allowed categorical conceptual schema.

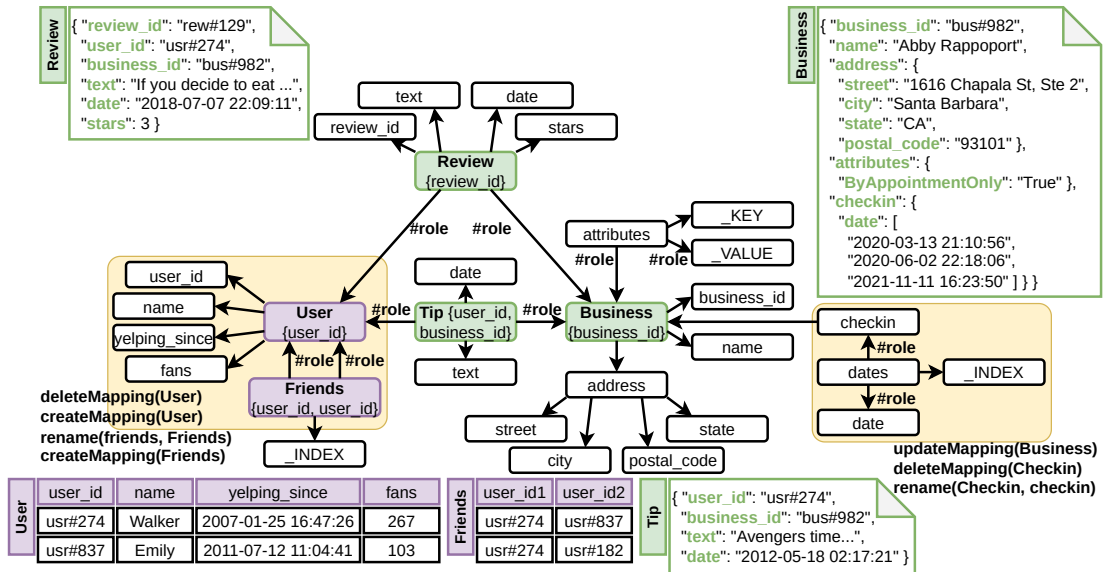
We divide the SMOs for the conceptual layer into three groups: *Basic* SMOs (see Table 4.1) are atomic and low-level, working with a single object or morphism. They may, however, affect multiple objects or morphisms (and even use other basic operations). To guarantee correct propagation, basic SMOs must satisfy the prerequisites (as indicated in Table 4.1). The *composite* SMOs (see Table 4.2) and the *complex* SMOs (see Table 4.3) involve multiple objects or morphisms and usually require more parameters than the basic ones. For example, the operations *split()/join()* require a custom user input, which none of the basic SMOs can provide. Composite SMOs enable working with the categorical model using higher-level constructs, i.e., creating/deleting common structures, such as simple/complex properties, relationships, arrays, sets, or maps. As such, these



(a) Version 1: Semi-automatically inferred unnatural schema.



(b) Version 2: More realistic schema: Attributes forming an *address* are grouped together. A large varying set of *attributes* is transformed into a map.



(c) Version 3: The transformation of the single-model schema and data to multi-model.

Figure 4.1 Schema evolution and its propagation to data

operations correspond to the composition of basic SMOs. Complex SMOs aim to mediate transformations on existing parts of the schema category $\mathbf{S}$.

Example 4.2. Consider the examples from Figure 4.1 illustrating categorical schema automatically inferred [17] from the *Yelp Academic Dataset*³, subsequently modified using SMOs and changes propagated to data. SMO-targeted areas are highlighted in yellow.

Version 1 in Figure 4.1(a) maps the inferred schema solely to the input document (green) model. There are kinds *User*, *Tip*, *Business*, *Checkin*, and *Review*. Relationship objects *Review* and *Tip* include morphisms labeled *#role* indicating participants, while *Checkin* is a weak-entity-like kind identified by *business_id* of kind *Business*.

As we can see, the automatic schema inference led to unnatural structures. For instance, kind *Business* includes a complex property *attributes*, comprising 39 optional properties, each found in 19 to 119,765 source documents. Hence, we transformed property *attributes* into a map (using SMO *asMap()*). Also, kind *Business* contains several properties forming an address (e.g., *state*, *city*, etc.). So, we renamed *address* to *street* (using SMO *rename()*) and grouped related properties into a complex property *address* (using SMO *group()*). Furthermore, we converted properties *friend* and *date* into arrays (using SMO *asArray()*), resulting in **Version 2** in Figure 4.1(b).

Subsequently, we applied SMOs *deleteMapping()* and *createMapping()* to change the logical representation of the data, i.e., we restructured kinds *User* and *Friends* into separate relational tables (violet). Finally, we embedded kind *Checkin* into kind *Business* via *updateMapping()* and deleted the original kind *Checkin*. These SMOs resulted in the categorical schema **Version 3** in Figure 4.1(c). $\square$

4.3.2 Propagation to Data

Regarding propagation to data, introduced in [82], the SMOs can be further divided into heavy and light. A *heavy operation* is an SMO propagated from the conceptual level to the mapping, the logical level, and the data instances.

³<https://www.yelp.com/dataset>

SMO	Behavior	Prerequisite
$create() \rightarrow O$	Creates object O with a unique identifier and default settings.	–
$create(D, C) \rightarrow m$	Creates morphism $m : D \rightarrow C$ with a unique signature and default settings.	$D, C \in \mathcal{O}_{\mathbf{S}}$
$delete(O)$	Deletes object O .	$O \in \mathcal{O}_{\mathbf{S}}$; O can participate only in $1_O \in \mathcal{M}_{\mathbf{S}}$.
$delete(m)$	Deletes morphism m .	$m \in \mathcal{M}_{\mathbf{S}}$
$rename(O, n)$	Changes the name of object O to n .	$O \in \mathcal{O}_{\mathbf{S}}$
$relabel(\mathcal{M}, l)$	Changes the label of morphisms in $\mathcal{M}$ to l .	$\mathcal{M} \subseteq \mathcal{M}_{\mathbf{S}}$; If $l = \#role$, then $ \mathcal{M} > 1$ and $m \in \mathcal{M}$ share <i>dom</i> .
$move(m, p)$	Sets $m.dom$ to $p[last].cod$, i.e. to the end of a path p .	$m \in \mathcal{M}_{\mathbf{S}}$; $p[first].dom = m.dom$
$copy(O) \rightarrow O'$	Creates a copy O' of object O (with a new unique identifier).	$O \in \mathcal{O}_{\mathbf{S}}$
$copy(m, D, C) \rightarrow m'$	Creates a copy $m' : D \rightarrow C$ of morphism m (with a new unique signature).	$D, C \in \mathcal{O}_{\mathbf{S}}$; $D = copy(m.dom)$; $C = copy(m.cod)$

Table 4.1 Basic SMOs

SMO	Behavior
$createProperty(l, P) \rightarrow (O)$	Creates object O and morphism $m : P \rightarrow O$ with label #property .
$deleteProperty(O)$	Removes property O , i.e., deletes object O and all incoming morphisms. If O is part of an identifier of an object P , it is removed from the respective set $P.ids$.
$deleteComplexProperty(O)$	(1) Deletes all nested properties without path from another complex property. Otherwise, it deletes the respective morphism. (2) Deletes all incoming morphisms. (3) If O participates in a relationship which would be broken, the relationship is deleted too. (4) Deletes O .
$createIdent(O, \{I_1, \dots, I_n\}) \rightarrow T$	Creates an identifier T from properties $I_1, \dots, I_n$ of a complex property O .
$deleteIdent(T)$	Removes identifier T of an object $O \in \mathcal{O}_S$.
$createRelationship(l, \{O_1, \dots, O_n\}) \rightarrow R$	(1) Creates object R representing a <i>relationship</i> object with label l . (2) For each O_i creates morphism $m_i : R \rightarrow O_i$ with label #role .
$deleteRelationship(R)$	(1) Deletes all outgoing morphisms. (2) Deletes R .
$createArray(P, l) \rightarrow A$	For object P , creates an <i>array</i> complex property A with label l and properties index I and element E .
$deleteArray(A)$	(1) Deletes all outgoing morphisms. (2) Deletes A .
$createSet(P, l) \rightarrow S$	For object P , creates a <i>set</i> complex property S with label l and a property E .
$deleteSet(S)$	(1) Deletes all outgoing morphisms. (2) Deletes S .
$createMap(P, l) \rightarrow M$	For object P , creates a <i>map</i> complex property M with label l and properties key K and value V .
$deleteMap(M)$	(1) Deletes all outgoing morphisms. (2) Deletes M .

Table 4.2 Composite SMOs

SMO	Behaviour
$group(l, \{O_1, \dots, O_n\}) \rightarrow G$	(1) Creates object G representing a <i>group</i> with label l . (2) For each O_i creates morphism $g_i : G \rightarrow O_i$. (3) For $P \in \mathcal{O}_S$, a parent of objects O_i , creates $g : P \rightarrow G$.
$ungroup(\{O_1, \dots, O_n\})$	Inverse operation to group.
$join(\{O_1, \dots, O_n\}, \Phi) \rightarrow O'$	(1) Creates object O' such that its active domain is formed of results of transformation function Φ applied to values from the active domains of $O_1, \dots, O_n$. (2) For $P \in \mathcal{O}_S$, a parent of objects O_i , creates morphism $m : P \rightarrow O'$. (3) Deletes morphisms $m_i : P \rightarrow O_i$.
$split(O', \Phi') \rightarrow \{O_1, \dots, O_n\}$	Inverse operation to join.
$asArray(P)$	Converts a complex property P to an <i>array</i> .
$asArray(P, \Psi)$	Converts a simple property P to an <i>array</i> so that the values of the active domain of P are separated using separator ψ and become the values of the active domain of the array.
$asMap(P)$	Converts a complex property P to a <i>map</i> .

Table 4.3 Complex SMOs

Conversely, a *light operation* only triggers propagation to the mapping. Each SMO can be heavy. Operations that do not consist of *delete(O)* or *delete(m)* can be also light, e.g., SMOs *createProperty()*, *asArray()*, or *asMap()*.

There are multiple strategies for heavy SMO propagation, namely *eager*, *lazy*, and *hybrid*. In the first case, we update all the data instances at once. In the second case, we update them one by one only when we need them. The last option is a combination of the first two [13, 47]. Our approach [82] is compatible with all of them; however, we focus only on the eager strategy at the moment.

4.3.3 Propagation to Queries

The propagation of changes to queries depends on how we handle the schema and data changes. The first option involves versioning the schema and the data, allowing the original queries to continue working with the original schema version. For new queries and applications, we use the most recent schema version. The second option we focus on involves propagating changes to both the data [82] and queries [83] and maintaining only the latest version. Generally, the propagation strategies follow the SMOs and the respective mappings when possible. However, the core problem is resolving cases that cannot be propagated straightforwardly.

More formally, let I be a database instance consisting of data D structurally corresponding to categorical schema $\mathbf{S}$ (represented as a functor $I : D \rightarrow \mathbf{S}$). The impact of SMO $O : \mathbf{S} \rightarrow \mathbf{S}'$ propagated to instances as $P_O : \text{Inst}(S) \rightarrow \text{Inst}(S')$ can be characterized using the concept *information capacity* [105, 106]:

- If I' is structurally equivalent to I , then it is *information capacity-preserving* operation. An example is SMO *rename()*.
- If for each $I \in \text{Inst}(S)$ there is at most one instance $I' \in \text{Inst}(S')$, i.e., the functor P_O is surjective on objects, the operation is *decreasing information capacity*. An example is SMO *deleteProperty()*.
- If for each $I \in \text{Inst}(S)$ there is at least one instance $I' \in \text{Inst}(S')$, i.e., the functor P_O is injective on objects, the operation is *increasing information capacity*. An example is SMO *createProperty()*.

In this paper, we primarily focus on applying strategies for synchronizing queries in the context of information capacity-decreasing SMOs. These operations are the most critical as they cause information loss, fundamentally altering the query. When reconstructing a query, several strategies [42, 43, 44] can be employed. Strategy-selection criteria include:

1. Whether the query can be reconstructed using the remaining data.
2. The increased complexity of the new version of the query, including the expression complexity and the time/memory complexity of its evaluation.
3. The metadata we maintain about the data (e.g., a *catalog*) and dependencies, including the size and memory intensity of this metadata and how efficiently it can be extracted, maintained, and updated as data changes.

4. The fidelity of the new query version, or how much the new query results differ from the original, such as returning different, fewer, or more records.

Additionally, changes to integrity constraints can impact query performance. For example, modifying the primary or secondary key (i.e., corresponding indices) impacts queries using the old index (e.g., within a join/filtering). Without optimal propagation, query performance deteriorates.

Example 4.3. Let us introduce an example of the impact of capacity-decreasing SMOs on representative queries. Figure 4.2(a) illustrates a fragment of the Version 3 schema from Figure 4.1(c), where modifications are performed using the SMOs *deleteProperty()*, *createProperty()*, and *rename()*. Additionally, Figure 4.2(a) presents three specific queries: Q_1 : “Retrieve the *text* and *stars* from *Reviews* where *review_id* exceeds 100”, Q_2 : “Retrieve pairs of (*state*, *countOfBusinesses*) representing the number of businesses in each state”, and Q_3 : “Retrieve all *Reviews* ordered by *review_id*”.

The capacity-decreasing SMO *deleteProperty(review_id)* affects queries Q_1 and Q_3 , while SMO *deleteProperty(state)* impacts Q_2 . $\square$

A common basic strategy is to utilize an *impact catalog* for each column, recording the impact of its removal on queries and allowing users to assess column importance. Such information can be variously complex. For example, paper [107] classifies attributes as *replaceable* or *dispensable* upon removal, based on metadata annotations specifying whether an attribute, condition, or relation can be replaced in specific query contexts (e.g., selection predicates, projection lists, join conditions). These metadata elements, represented as Boolean values, indicate replacement feasibility and associated constraints. The subsequent EVE project [108], which serves as an inspiration for our work, employs an extended metadata for query synchronization. When a column or table is deleted, the system identifies affected queries and views, determines their dependencies, and selects the best replacement strategy using recorded metadata (e.g., to preserve query result, minimize query complexity and execution time). The impact catalog further characterizes columns using factors such as distinct values, value ranges, default values, functional dependencies, containment relationships, and integrity constraints. These attributes inform replacement selection to maintain query correctness. Metadata can be manually specified by database administrators or inferred from schema constraints and query patterns.

Generally, such metadata facilitates the selection of appropriate replacement methods. We introduce and utilize several representatives:

A. Phantom Strategy The first propagation option involves replacing the removed column with a *phantom*, wherein the original column values are substituted with those from the minimal domain (smallest number of values and memory size) to preserve query functionality. This process entails analyzing queries, utilizing metadata, and substituting original values with new ones.

The representation of phantom values varies based on the query Q where property p is removed. If p appears only in filtering conditions, the phantom can be a bitmap indicating whether each row was part of the original result by modifying the filter condition to $p = \text{True}$. For clauses such as **DISTINCT**, aggregation functions, and **JOIN**, the original values are replaced with enumerated values from a less memory-intensive domain (e.g., replacing strings with integers).

For the **ORDER BY** clause, the order is maintained using ordered unsigned integers while retaining original values for projection. A minimal memory *footprint* is sought since the phantom size matches the number of original records. For each query, a phantom of the deleted column can be created (e.g., using multiple bitmaps for greater memory efficiency). For some queries, phantoms can be consolidated into fewer representations.

Example 4.4. Consider capacity-decreasing operations from Figure 4.2(a) that are applied to *review_id* in kind *Review* and *state* in the address of kind *Business*, impacting queries Q_1 – Q_3 . Let us analyze them to determine the optimal phantom representation based on query composition.

For example, removing *review_id* impacts both selection *reviews* > *rew*#100 in Q_1 and sorting in Q_3 . For selection, a binary phantom (True/False) suffices to indicate document inclusion. However, the phantom must also capture sequence for ordering, requiring string substitution with a more memory-efficient type.

Aggregation in Q_2 adds additional complexity. If the aggregated value is excluded from the result, replacing strings with primitive enumeration values conserves memory (as a resulting phantom representation, as illustrated in Figure 4.2(b)). Conversely, if included, the original values must be retained, resulting in phantoms as memory intensive as the original data.

In all cases, the modified queries still yield results identical to their originals. $\square$

B. Functional-Dependency Strategies Let us first recall the basic definition of a functional dependency. Consider a schema $R(A_1, A_2, \dots, A_n)$ with attributes $A_1, A_2, \dots, A_n$. A *functional dependency* (FD) $X \rightarrow Y$, where $X, Y \subseteq R$, holds in R if for any tuples $t_1, t_2 \in R$, $t_1[X] = t_2[X]$ implies $t_1[Y] = t_2[Y]$, i.e., the attributes in X uniquely determine those in Y . A *relaxed functional dependency* (RFD) generalizes an FD by introducing flexibility through a parameter ϵ , allowing the dependency to hold with some tolerance, as a strict FD may not be applicable across the entire database.

Inspired by the approach in [42], we can synchronize queries using (relaxed) functional dependencies to maintain the original query semantics. Furthermore, we categorize (R)FDs as surjective or injective. A surjective FD enables the replacement of all values in the deleted column, while an injective FD ensures that the replacement values are unambiguous.

Example 4.5. To assess the substitutability of a removed property, valid (relaxed) FDs are utilized (see Figure 4.2(c) and 4.2(d)). For example, removing *review_id* from kind *Review* is governed by FD $user_id, business_id, date \rightarrow review_id$, allowing the replacement with $(user_id, business_id, date)$. This substitution preserves review identification but complicates queries, making filtering operation inefficient (see Q_1 in Figure 4.2(c)) and significantly altering ordering in result (i.e., Q_3 cannot be reconstructed). A more efficient alternative uses relaxed FD $date \rightarrow review_id$, improving filtering (see Q_1 in Figure 4.2(d)) but possibly including additional documents when multiple reviews share the same date and causing slight discrepancies in ordering (see Q_3 in Figure 4.2(d)).

For aggregation queries, employing FD $city, postalCode \rightarrow state$ changes the aggregation root but permits result recovery through transformation and external knowledge (see Q_2 in Figure 4.2(c)). However, the relaxed FD $city \rightarrow state$ may not accurately reproduce results when cities share names between states (see Q_2 in Figure 4.2(d)). $\square$

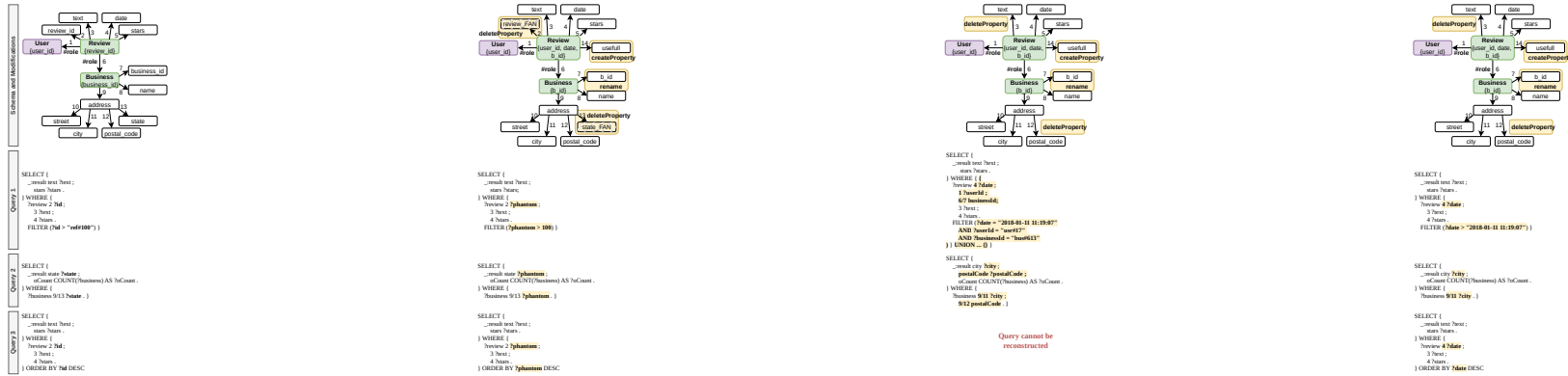


Figure 4.2 An example of application of various strategies to propagate structural changes to the queries

Additional Considerations Beyond the outlined strategies, several factors must be addressed. First, different sequences of identical operations can yield varying query propagation effects [107]. For instance, executing *delete(A)* followed by *create(A)* may negate each other, rendering the sequence ineffective.

Additionally, deciding whether to synchronize queries with the initial or the latest version is crucial. As proposed in [109], referencing the first query version requires backtracking a history of queries and changes to restore deleted columns, which is memory-intensive due to storing original-sized column phantoms.

Moreover, information capacity-decreasing operations can be classified as no longer needed (e.g., remove query) or destructive (i.e., requiring replacement). A preliminary what-if analysis, derived from the metadata catalog and visually indicated to the user, can determine the impact of propagating the SMO.

If query synchronization is unfeasible due to capacity-decreasing operations without an efficient replacement, decisions must be made to drop queries or restrict schema evolution [106]. Typically, three policies are adopted: propagate (i.e., apply changes and synchronize queries), block (i.e., prohibit schema changes), or prompt (i.e., consult the database administrator). The application of individual strategies may result in:

- The query is fully reconstructed to return identical results, though it may require more execution time or increased memory due to complex queries or extensive metadata. Nonetheless, the metadata size remains smaller than the original deleted data, reducing the overall memory footprint.
- The query is partially reconstructed to approximate original results, potentially causing time inefficiency with non-injective or non-surjective FDs.
- The query cannot be reconstructed, necessitating a decision to either propagate the SMO and delete the query or block the SMO propagation.

Utilization of AI AI-based approaches can be used to select an appropriate strategy, e.g., considering the frequency of execution of that query (for example, frequently executed and important queries will lead to a phantom strategy, even if it is memory intensive) while considering the available system resources. In other words, it is an optimization problem where the AI agent operates in a state space defined by the system’s current resource availability (e.g., memory, CPU usage, etc.) and the historical performance of previously used strategies. The actions then correspond to selecting specific propagation strategies.

A promising solution is to use reinforcement learning because we do not know which actions to take, but we can easily measure their outcome (e.g., consumption of resources during and after the propagation). Another option is to use simpler methods (e.g., heuristics). The advantages of this approach include a lower need for training data, less training time, and better explainability of the resulting policy.

4.4 Implementation and Evaluation

Our solution maintains versions of a schema category **S** and a chain of applied SMOs. We also maintain the mapping between queries and schemas. So, the

propagation is straightforward since each such mapped query targets a subset or a view of the data. The benefits of our approach lie in the use of MMQL abstraction and its translation to Domain Specific Languages of the underlying DBMSs. We do not have to deal with propagation to individual languages and their specific constructs. Depending on the queries an SMO affects, we can always use the most appropriate strategy for the given property. Currently, we heuristically calculate the price per change and operation of new versions of queries. In the future, we will utilize a suitable statistical model.

The experimental implementation of the described idea is based on a family of tools systematically developed in our research group for managing multi-model data based on the category theory. Around this representation, we have already built the following tools:

- *MM-cat* [12] enables multi-model modeling, i.e., the manual creation of a categorical model and its mapping to any combination of the supported logical models. It also supports the respective data transformations between the logical and categorical models.
- *MM-infer* [17] enables automatic inference of the categorical schema from sample multi-model instances. In addition, as the data set can be large, the data is processed in parallel.
- *MM-evocat* [82] extends *MM-cat* with the propagation of changes in the categorical schema to the data, i.e., it extends the SMOs with the respective propagation strategies.
- *MM-quecat* [14] adds support for querying over the categorical schema category **S** using MMQL. The queries are then decomposed according to the mapping. The subqueries are evaluated in the underlying DBMSs, and the partial results are combined to produce the final result.
- *MM-evoque* [83] extends *MM-quecat* for propagation of changes in the categorical schema to queries in MMQL.

4.4.1 Propagation to Data Instances

The toolset was implemented as a proof of concept to demonstrate that the proposed strategy enables (1) the model-agnostic conceptualization of any multi-model logical schema, (2) mapping to specific data logical schemas, (3) querying using MMQL, and (4) schema evolution based on user requirements (SMOs) with correct and complete propagation of changes to data instances and queries.

Experimental evaluations of various schema-to-data propagation strategies were conducted on a 14" MacBook Pro (2021) featuring an Apple M1 Pro processor (8 cores), 16 GB LPDDR5 memory, and a 512 GB SSD. To mitigate potential biases from resource fluctuations and initial overhead, each experiment configuration was executed 30 times, excluding initial runs and averaging the remaining measurements. The experiments utilized five *Yelp Academic Dataset* collections in *MongoDB: Review* (6,990,247 records), *User* (1,987,897 records), *Business* (150,346 records), *Tip* (908,915 records), and *Checkin* (131,930 records) – for sample data see Figure 4.1(a).

The modifications correspond to the SMOs depicted in Figure 4.1(b) and Figure 4.1(c). Three eager strategies were utilized for substantial SMO propagation:

- *insertMany*: Drops integrity constraints, deletes records with schema **S**, performs a bulk insert of records with schema **S'**, and restores constraints.
- *insertOne*: Similar to *insertMany*, but inserts records individually.
- *update*: Retains integrity constraints and updates affected documents using update or set operations based on the target system.

Moreover, the *insertOne* and *insertMany* strategies are implemented as *standard*, i.e., proactively preventing database driver exceptions such as integrity constraint violations, and *alter*, i.e., disabling integrity constraints before insertion and re-enabling them afterwards.

Table 4.4 illustrates the time requirements of the three SMO propagation strategies applied to kinds (*Business*, *CheckIn*, *User*, and *Friends*) during schema evolution from Version 1 to 2 and 2 to 3, as illustrated in Figure 4.1. The times are divided into the *preparation phase PreP* (i.e., creating tables/collections or disabling constraints), *transformation phase TraP* (i.e., updating data to match the target structure and representation), *propagation phase ProP* (i.e., inserting data into the DBMS), and the *alter IC phase AltP* (i.e., re-enabling constraints).

Change	Strategy	PreP	TraP	ProP	AltP	Total Time
<i>Business Address</i> SMOs	updateOne	0.000	1.841	80.018	-	81.858
	insertOne	0.000	1.746	73.764	-	75.511
	insertMany	0.000	1.865	3.246	-	5.111
<i>Checkin Dates</i> SMOs	updateOne	0.000	3.446	88.863	-	92.310
	insertOne	0.000	3.706	86.362	-	90.068
	insertMany	0.000	3.976	9.201	-	13.178
<i>User Friends</i> SMOs	updateOne	0.001	101.064	1,443.758	-	1,544.823
	insertOne	0.004	88.367	1,378.334	-	1,466.705
	insertMany	0.002	82.045	129.150	-	211.197
<i>Business Checkin</i> SMOs	updateOne	0.000	6.369	104.918	-	111.286
	insertOne	0.000	7.531	102.470	-	110.001
	insertMany	0.000	8.070	14.712	-	22.782
<i>User Relational</i> SMOs	insertOne	0.001	75.102	130.275	-	205.378
	insertOneAlter	0.025	74.468	116.480	2.159	193.131
	insertMany	0.001	54.723	18.762	-	73.486
	insert-ManyAlter	0.023	54.315	9.466	1.942	65.745
<i>Friends Relational</i> SMOs	insertOne	2.023	304.041	1,605.221	-	1,911.285
	insertOneAlter	2.121	305.847	1,355.017	27.931	1,690.916
	insertMany	2.053	161.603	381.701	-	545.357
	insert-ManyAlter	2.094	162.392	55.954	28.212	248.652

Table 4.4 Time [s] needed for propagation of SMOs

As observed, solely updating data is the slowest strategy by an order of magnitude between systems. The fastest approach involves creating a new collection or deleting data and bulk inserting a new version, achieving up to eight times greater performance than individual inserts or updates when propagating SMOs to

the entire collection. Performance is additionally enhanced by disabling integrity constraints before insertion despite the increased effort in the preparation and transformation phases.

4.4.2 Propagation to Operations

Table 4.5 illustrates the effectiveness of phantom and (relaxed) FD strategies on high-impact filtering, aggregation, and ordering queries from Figure 4.2. As we can see, in the phantom strategy, queries can be fully reconstructed with minimal alterations to the original MMQL expression, thus preserving execution time and memory usage. However, this strategy effectively substitutes the removed property with a memory-efficient placeholder, which may be seen as a disadvantage. In contrast, the FD and relaxed FD strategies remove properties entirely but often increase the complexity of the MMQL expression (e.g., the expression contains numerous UNION clauses), potentially hindering performance and resource efficiency. The fidelity of query outcomes in these approaches depends on the properties of the FD, such as injectivity, surjectivity, and degree of relaxation. When a dependency is both injective and surjective, the resulting dataset remains identical to the original query. One notable benefit of these methods is their minimal metadata requirements, relying solely on query synchronization with SMO propagation.

As for data changes, they affect all strategies differently. In the phantom strategy, newly introduced data remain excluded as if the corresponding rows were not part of the original query. Conversely, FD-based approaches may include such data, thereby risking the invalidation of the original functional dependency. Once a property is removed, its validity can no longer be verified.

Query	Strategy	QR	QF	SC	ET	EM	MM	ME	MU
Q_1	Phantom	Yes	Good	Good	Good	Good	Average	Good	Good
	FD	Yes	Good	Worse	Worse	Worse	Good	Worse	Good
	Relaxed	Yes	Average	Worse	Good	Worse	Good	Worse	Good
	FD								
Q_2	Phantom	Yes	Good	Good	Good	Good	Worse	Good	Good
	FD	Yes	Good	Worse	Worse	Worse	Good	Worse	Average
	Relaxed	Yes	Average	Average	Good	Good	Good	Worse	Average
	FD								
Q_3	Phantom	Yes	Good	Good	Good	Good	Worse	Good	Good
	FD	No	n/a	n/a	n/a	n/a	n/a	n/a	n/a
	Relaxed	Yes	Average	Average	Good	Good	Good	Worse	Average
	FD								
General case	Phantom	Yes	Good	Good	Good	Good	Worse	Good	Good
	FD	Yes	Good	Worse	Worse	Worse	Good	Worse	Average
	Relaxed	Yes	Average	Average	Average	Average	Good	Worse	Average
	FD								

^{QR} Query reconstruction
^{QF} Query fidelity
^{SC} Statement complexity
^{ET} Execution time

^{EM} Execution memory
^{MM} Metadata memory
^{ME} Metadata extraction
^{MU} Metadata updating

Table 4.5 An experimental comparison of query synchronization strategies

4.5 Related Work

The evolution management is generally challenging, even for single-model data. In a multi-model world, we have to cope with additional problems such

	<i>CGOOD</i> [102]	Spivak et al. [46]	<i>Darwin</i> [47]	<i>MM- evolver</i> [97]	<i>Orion</i> [110]	<i>MM- evocat</i> [82]
Schema modification	Yes	Yes	Yes	Yes	Yes	Yes
Data models	Relational, object, object- relational	Relational, RDF	Graph, key/value, document, columnar	Relational, graph, key/value, document, columnar	Relational, graph, key/value, document, columnar	Relational, array, graph, RDF, key/value, document, columnar
Multi-model	No	No	No	Yes	No	Yes
Propagation to IC	No	No	No	Partial	Partial	Yes
Data migration	Eager	Eager	Eager, lazy, hybrid	Eager	Eager	Eager
Schema inference	No	No	Tracked, untracked	No	Untracked	Yes (using <i>MM- infer</i> [16])
Version jumps	No	No	Yes	No	No	No
Query rewriting	No	No	Yes	No	No	Yes

Table 4.6 A comparison of features of selected evolution management approaches

as a missing unifying interface for both data and operations, distinct ways of combining the models, contradictory features of the models, different propagation strategies for distinct models, cross-model data migrations, etc. Several evolution management approaches currently consider at least two models [102, 46, 47, 97, 110]. However, they have various limitations. Table 4.6 summarizes the comparison of our approach with them.

The main common limitations of existing approaches are targeting only a small fraction of data models and/or (partially) ignoring integrity constraints and/or queries over the data and other features arising from combining multiple data models. Our approach genuinely supports all popular data models in a multi-model way. The set of SMOs is the richest, and the changes are propagated eagerly to data and ensure query synchronization, even for the capacity-decreasing SMOs. The approaches we adopt and utilize/extend [107, 42] were originally proposed only for single-model use cases. To our knowledge, no multi-model solution deals with this challenge and offers such a range of strategies. Only system *Darwin* [47] allows for other data migration strategies, namely lazy and hybrid, and supports optimization using version jumps. However, it focuses only on propagation to data instances.

4.6 Conclusion

Evolution management is one of the core and most complex data management tasks, as most applications need to evolve according to changing user requirements. The situation is much more complex in the context of multi-model data.

This paper presents our strategy based on the idea of capturing multi-model data in a unified, model-agnostic manner using category theory, treating the data naturally as a categorical graph. Having such a representation, we introduce a set of schema modification operations that can modify the categorical graph without dealing with the technical details of the underlying DBMSs and models. These operations are classified based on their complexity, and we define the

respective propagation strategies that adapt the data instances and ensure query synchronization. We especially focus on capacity-decreasing SMOs, where the information loss needs to be reflected respectively. Finally, we empirically evaluate and compare the strategies that were introduced.

Even though the main aim of the introduced toolset is to simplify the critical tasks of data management in the world of multi-model data, the user is still required to make complex decisions. Our near future aim is to exploit suitable AI strategies and extend the toolset towards self-adaptability. The system can learn from the given data, given or extracted metadata, operations, change logs, and user decisions and automatically or proactively make these complex decisions.

Paper V

FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts

Pavel Koupil¹, Jáchym Bártík¹, Stefan Klessinger², André Conrad³,
Stefanie Scherzinger²

Published as Pavel Koupil et al. “FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5227–5230. ISSN: 2150-8097. DOI: [10.14778/3750601.3750612](https://doi.org/10.14778/3750601.3750612)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

²Faculty of Computer Science and Mathematics, University of Passau, Passau, Germany

³Faculty of Mathematics and Computer Science, University of Hagen, Hagen, Germany

Abstract

Functional dependency (FD) discovery is fundamental in data profiling. Inevitably, existing approaches can return fake FDs that hold only coincidentally. Moreover, these approaches fall short of identifying ghost FDs that would be observable in a clean dataset, but that remain undetected because of outliers in the data. We introduce an interactive method for dependency discovery that augments an Armstrong relation with additional tuples. We rely on artificially generated *negative examples* that emulate real-world tuples to help expose fake FDs. In addition, we rely on domain experts to confirm that *positive examples* indeed reflect the characteristics of the original dataset. Our tool prototype *FDepHunter* thus provides a novel human-in-the-loop workflow where the set of discovered FDs can be iteratively refined.

5.1 Introduction

Functional dependencies (FDs) are a foundational concept in data management, with applications in schema matching, dataset similarity, outlier detection, and extraction of both integrity constraints and business rules. Several approaches to facilitate FD discovery have been proposed [112, 113, 114], which differ primarily in performance and suitability for specific datasets.

Which FDs are discovered depends on the quality of the input data. In particular, FDs can be classified into three types [115]: (1) *genuine* FDs, which remain valid across all realistic dataset instances and capture inherent dependencies; (2) *fake* FDs, which hold only within the given dataset but fail when additional valid tuples are introduced; fake FDs tend to occur in datasets with large numerical domains, such as medical or transport data, where most values are effectively unique – for example, the US flights dataset⁴ contains over 900,000 FDs [116], many of which are probably fake; and (3) *ghost* FDs, which remain undetected due to outliers or anomalies but which would hold in a clean dataset.

A fully representative dataset would help in identifying genuine FDs. To better approximate dataset completeness, we introduce two complementary concepts: *negative* and *positive examples*. The former are artificially constructed to simulate realistic but absent data, allowing the removal of fake FDs. This approach is motivated by Gold’s Theorem [117], which highlights the need for negative evidence, such as unseen valid tuples, to infer generalizable properties from incomplete data. The latter reflect tuples of the original dataset and are used to reveal ghost FDs obscured by outliers. *FDepHunter*, the tool presented in this paper, enables users to interactively confirm or reject negative and positive examples.

Most FD discovery approaches return a minimal FD set, i.e., a basis from which other (non-minimal) FDs follow via Armstrong’s axioms. When the dataset is extended (e.g., with negative examples), the minimal set may change, and newly added FDs must be validated to ensure their genuineness. Beyond incompleteness, this introduces the challenge of validating such FDs. We propose a human-in-the-loop workflow to discover genuine FDs and lay the groundwork for follow-up research, e.g., on handling this at scale.

⁴<https://www.transtats.bts.gov>, accessed July 2025

Contributions of this paper include the following:

- We present *FDepHunter*, a data profiling tool designed to support domain experts. The tool provides a novel interactive workflow to detect fake FDs and to facilitate the discovery of ghost FDs.
- *FDepHunter* incrementally constructs an Armstrong relation [45], augmented with so-called *negative* and *positive examples*, to approximate the properties of a complete dataset.
- Users iteratively refine the set of FDs: they identify genuine FDs, expose fake FDs through *negative examples*, and uncover ghost FDs via *positive examples*.
- Negative and positive examples are generated to target multiple fake or ghost FDs simultaneously. This allows to efficiently refine multiple dependencies in a single step.

5.2 Core Concepts

In this section, we introduce and illustrate the core concepts.

For a relational schema R and sets of attributes $X, Y \subseteq R$, a FD $X \rightarrow Y$ holds if, whenever two tuples agree on X they also agree on Y ; c.f. Abiteboul et al. [45] for further details. We call X the left-hand side (LHS) and Y the right-hand side (RHS) of the FD. A FD $X \rightarrow Y$ is *minimal* if there is no strict superset $X' \subseteq R$ of X for which $X' \rightarrow Y$ holds.

We classify FDs by their RHS, grouping FDs determining the same attribute into distinct *classes*. The set of these classes is denoted by $\mathcal{C}$. Within each class $c \in \mathcal{C}$, we identify *maximal sets* [112] of attributes on the LHS that do not form a FD for the corresponding RHS. Formally, a set $X \subseteq R$ is a maximal set for the class of A if $X \rightarrow A$ does not hold, but $X' \rightarrow A$ holds for every strict superset $X' \subseteq R$ of X . The set of all maximal sets for a class c is denoted by $\mathcal{M}_c$. The elements of $\mathcal{M}_c$ and the FDs can be illustrated using a *lattice*, where each node represents a subset of attributes and edges indicate the addition or removal of an attribute.

An *Armstrong relation* satisfies exactly a given set of FDs and no others [45]. We artificially construct an Armstrong relation, beginning with a base tuple t_0 , where all attributes are assigned a default value. For each maximal set $M \in \bigcup_{c \in \mathcal{C}} \mathcal{M}_c$, a unique tuple t_M is added to the relation. In each tuple t_M , attributes belonging to the corresponding maximal set M retain the default value, whereas attributes not in M are assigned distinct real-world values. By substituting abstract identifiers with actual data values, the Armstrong relation mirrors the characteristics of input datasets.

Within the Armstrong relation, rows are categorized as either *positive* or *negative examples*. Both are constructed by modifying a base tuple t_0 using real-world values so that the values of LHS attributes match t_0 while the values of RHS attributes differ. Positive examples collectively uphold exactly the FDs in the input dataset. In contrast, negative examples are artificially generated so that they violate FDs that may hold coincidentally and are likely fake.

Example 5.1. Fig. 5.1 shows a sample from the real-world IMDB title dataset⁵. Fig. 5.2 shows an Armstrong relation created from the sample. The positive examples (marked with red squares) form an Armstrong relation that satisfies twelve minimal FDs (e.g., $CD \rightarrow A$, $CE \rightarrow A$, $C \rightarrow B$). By adding the negative example (yellow), the Armstrong relation violates the FD $C \rightarrow B$. Finally, Fig. 5.3 presents a lattice diagram for the class of A (denoted by c_A), where elements of $\mathcal{M}_{c_A}$ are shown in solid red, their subsets as outlined red nodes, minimal FDs in solid blue, and derived FDs as outlined blue nodes. This visualization provides an intuitive map of the combinations of attributes that determine the RHS within each FD class. $\square$

tconst	primaryTitle	startYear	runtimeMin.	genres
tt0036443	Titanic	1943	85	Action+Drama+History
tt0079836	S.O.S. Titanic	1979	194	Drama+History
tt0115392	Titanic	1996	87	Action+Drama+History
tt0120338	Titanic	1997	194	Drama+Romance
tt0155274	Titanic	1915	null	History
tt0594950	Titanic	1997	null	Documentary+Short
tt0650185	Titanic Tech	2003	46	Documentary+History
tt0771984	Titanic	2006	51	Documentary
tt0902058	The Titanic	1981	null	Documentary+Drama+Fantasy
tt0929378	Titanic's Ghosts	2002	57	Documentary+History+War

Figure 5.1 Input dataset.

A	B	C	D	E
tconst	primaryTitle	startYear	runtimeMin.	genres
tt0036443	Titanic	1943	85	Action+Drama+History
tt0079836	Titanic	1943	194	Drama+History
tt0115392	Titanic	1979	85	Drama+Romance
tt0120338	Titanic	1996	87	Action+Drama+History
tt0155274	S.O.S. Titanic	1997	85	History
tt0594950	Titanic Tech	1943	null	Documentary+Short

BC

BD

BE

D

C

Figure 5.2 Positive (red) and negative (yellow) examples.

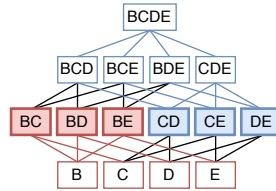


Figure 5.3 An example of lattice for class c_A (i.e., RHS: A).

5.3 Workflow

We now present the workflow of *FDepHunter*, illustrated in Fig. 5.4.

⁵<https://developer.imdb.com/non-commercial-datasets/>, accessed July 2025

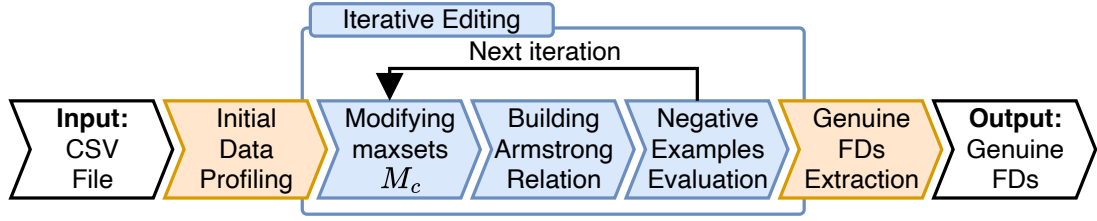


Figure 5.4 A high-level overview of the FDepHunter workflow.

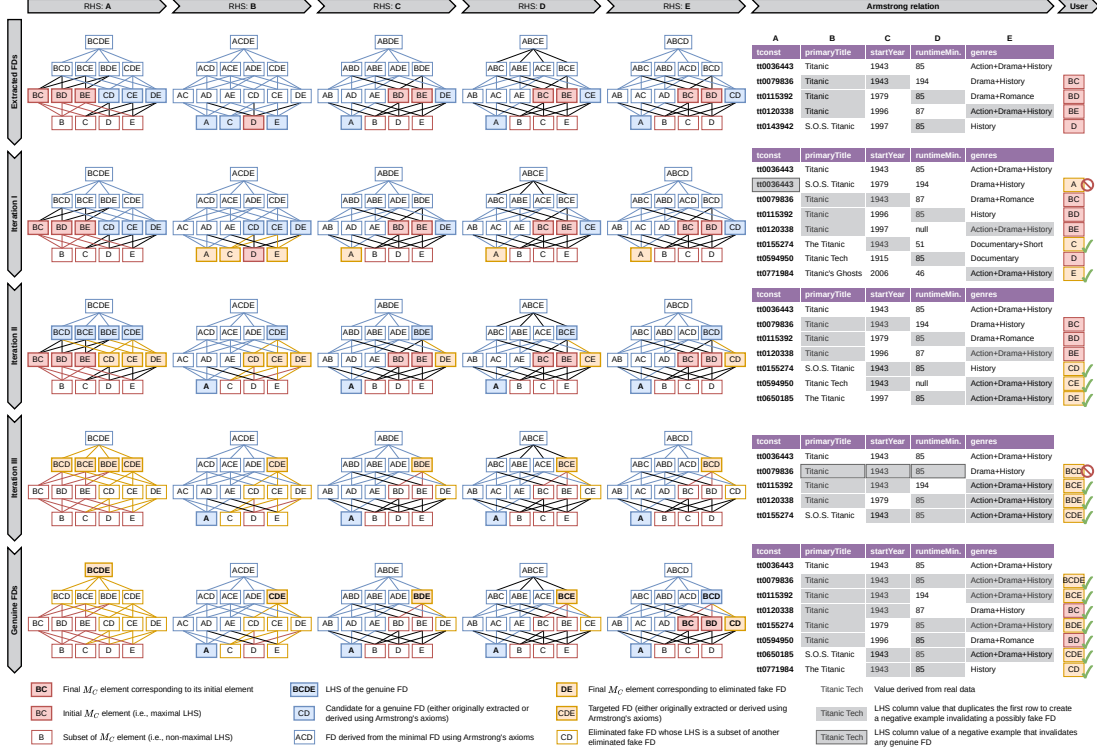


Figure 5.5 Iterative (top-down) editing of maximal sets and elimination of fake FDs utilizing negative examples.

Initial Data Profiling. As part of the initial phase focusing on the extraction of FDs and maximal sets M_c for each class $c \in \mathcal{C}$, we apply a method that fits the properties of the input data. This work employs an approach inspired by Dep-Miner [112]. For each class $c \in \mathcal{C}$, the method initially generates the maximal sets M_c and subsequently reconstructs the FDs. Alternatively, approaches such as HyFD [113] can be employed to extract FDs first, ensuring that even for large data volumes the system is reactive and prompt, allowing for fluent interaction. Maximal sets are then reconstructed by pruning attributes from minimal LHSs until only maximal subsets that do not form an FD remain.

Iterative Editing of Maximal Sets. In the second phase, the goal is to identify *fake* FDs and to uncover *ghost* FDs, by employing heuristics [115] or by incrementally consulting users; we focus on the incremental approach. Fake FDs are examined by iterating from smaller to larger LHSs (as illustrated in Fig. 5.5), assuming pessimistically that the extracted dependencies (shown in blue) are fake. Consequently, an affected M_c is expanded to include a new element M corresponding to the targeted FD (depicted in yellow), and all $M' \subset M$, $M' \in M_c$,

are removed to preserve maximality. This adjustment updates the Armstrong relation to contain one additional row for each newly added element $M \in \bigcup_{c \in C} \mathcal{M}_c$, forming a *negative example* evaluated by the user. If accepted, the targeted FD is deemed fake; if rejected, it implies that at least one corresponding FD could be genuine, so the matching LHS is removed from affected $\mathcal{M}_c$. If confirmed that all FDs with that LHS are genuine, pruning of LHS stops; otherwise, the algorithm identifies genuine FDs, and continues the search for supersets of LHS that fail to invalidate genuine FDs. If any genuine FD is identified, the M corresponding to its LHS is removed from all affected $\mathcal{M}_c$, and previously removed M' are restored. After all negative examples are processed, the algorithm proceeds to the next iteration, increasing the size of the LHS by one, and terminates when every extracted FD has been classified. With this strategy users do not need to validate the full transitive closure of FDs derived via Armstrong’s axioms, but only those with LHSs that are a strict superset of a previously evaluated one.

For ghost dependencies, the process focuses on the original (red) elements M in each $\mathcal{M}_c$, proceeding in reverse (from those containing the largest number of columns to smaller ones). Here too, the user receives pairs of rows – specifically, the first row and one with a red square – to determine whether the latter truly represents a maximal set. If the user accepts the pair, the currently evaluated element M remains valid in each relevant $\mathcal{M}_c$; if rejected, the algorithm presumes an undiscovered FD exists, marks the targeted element as an FD, and continues to target all its immediate predecessors (i.e., subsets of attributes of size one less), verifying them in subsequent iterations. The algorithm terminates once no M (or its subsets) remains to be checked.

Both steps ultimately yield a collection of genuine FDs by incrementally eliminating fake FDs and uncovering ghost FDs. While the worst-case number of examples a user must review is 2^n (lattice size), in our experience this remains manageable in practice, especially for small datasets with many genuine FDs.

Example 5.2. Fig. 5.5 illustrates the iterative workflow for identifying genuine FDs from the dataset in Fig. 5.1. The initial phase outputs maximal sets and FDs depicted as a lattice alongside the corresponding Armstrong relation. Each iteration eliminates FDs, from the smallest to the largest LHS. Negative examples targeting $LHS = A$ and $LHS = BCD$ are rejected as A is an identifier and $BCD \rightarrow E$ is genuine (e.g., two series episodes may share genre E but have distinct identifiers A). Finally, the user accepts the remaining maximal sets, ensuring no ghost FDs are in the dataset. $\square$

Architecture and Implementation *FDepHunter* is implemented as a client-server application. On the server side, it relies on *Java 21* and the *Spring Boot* framework to provide a REST interface. The client employs the *React* library to provide a GUI that displays the current status of the entire workflow.

5.4 Related Work

Several studies have evaluated the quality of discovered FDs using scoring functions. Chu et al. [118] introduce *succinctness* and *coverage* to assess the interestingness of denial constraints, a generalization of FDs. Chiang and Miller [119] propose similar ranking functions for conditional FDs. Andritsos et al. [120] rank FDs by their information content, while Wei and Link [121] use data redundancy

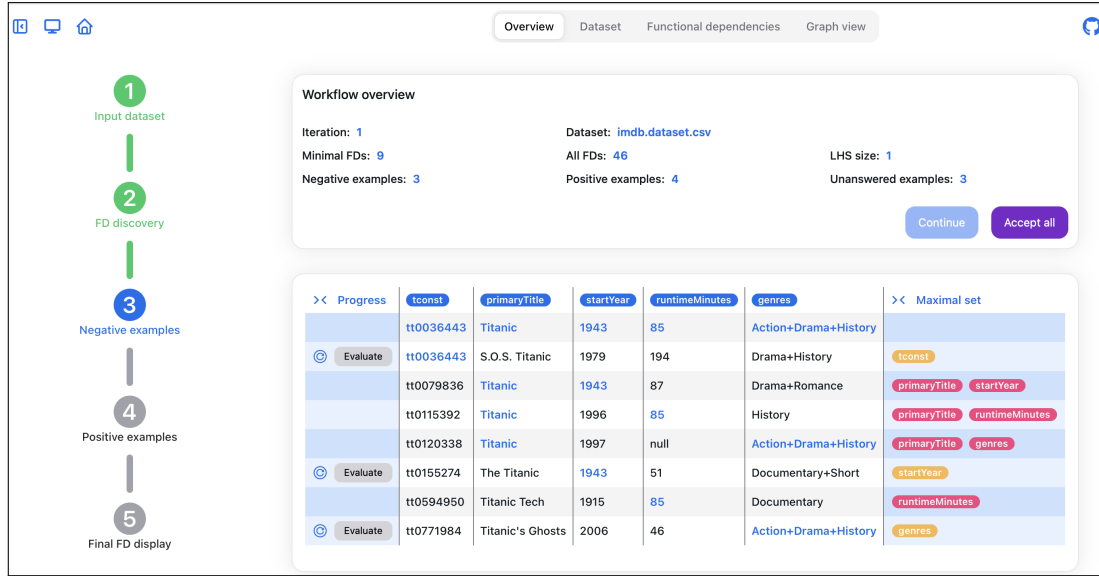


Figure 5.6 Armstrong relation and workflow status.

as a heuristic to identify genuine FDs. Berti-Équille et al. [115] study the impact of missing values on the discovery of genuine, ghost, and fake FDs under various NULL semantics and propose probabilistic scoring.

A separate line of research uses Armstrong relations to identify meaningful FDs [122]. This is highly related to our notion of positive examples. Langeveldt and Link [123] evaluate the usefulness of Armstrong relations for identifying meaningful FDs. They conclude that Armstrong relations are helpful in identifying meaningful FDs that are incorrectly perceived as meaningless, but not vice versa.

Our approach differs from existing work by iteratively refining Armstrong relations through user feedback, and also incorporating artificial negative examples to help eliminate fake FDs.

5.5 Demonstration Outline

In our interactive tool demo, users systematically identify genuine, fake, and ghost FDs through the following steps: (1) Users begin by selecting from a curated collection of real-world datasets, which differ in size, attribute count, and exhibit varying numbers of fake FDs and outliers (e.g., IMDb datasets, or Metanome profiling datasets [116]). Alternatively, users can provide their own dataset. (2) Users iteratively refine maximal sets in ascending size by reviewing and validating examples. Each iteration generates an Armstrong relation in which negative examples target fake FDs (see Fig. 5.6). Negative examples appear sequentially (see Fig. 5.7), and are either accepted, removing a fake FD, or rejected with justification, as users act as domain experts. (3) In the final stage, users assess remaining positive examples and reject outliers to reveal ghost FDs.

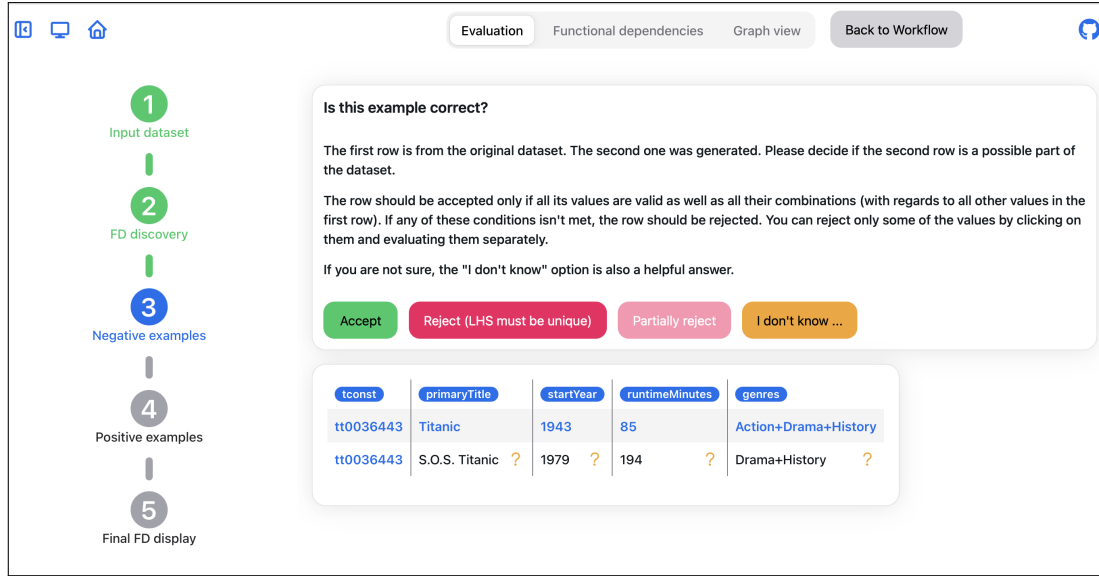


Figure 5.7 Evaluation of negative example.

5.6 Conclusion and Outlook

FDepHunter introduces an interactive workflow to eliminate fake FDs using real-world-like negative examples and to uncover ghost FDs, hidden by outliers, by validating positive examples.

Future work will focus on enhancing the scalability and effectiveness of *FDepHunter*. Specifically, we plan to develop advanced strategies for constructing negative examples with maximal impact by leveraging heuristics to assess the genuineness of FDs. Additionally, we aim to conduct user studies with domain experts to systematically evaluate the tool's usability and its effectiveness in practical applications. A key objective will be to assess the cognitive load perceived by domain experts when evaluating negative and positive examples and investigate whether showing additional context or metadata is perceived as helpful.

Paper VI

Refining Storage Strategy Through Index Selection Methods in Multi-Model Database Systems: A Survey

Filip Mihál¹, Jáchym Bártík¹, Pavel Koupil¹

Published as Filip Mihál, Jáchym Bártík, and Pavel Koupil. “Refining Storage Strategy through Index Selection Methods in Multi-Model Database Systems: A Survey”. In: *Data & Knowledge Engineering* 164 (2026), p. 102582. ISSN: 0169-023X. DOI: [10.1016/j.datak.2026.102582](https://doi.org/10.1016/j.datak.2026.102582)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

Multi-Model database systems combine the advantages of traditional and NoSQL database systems. However, the management of these systems is challenging, as users have to design an appropriate storage strategy for their data. One of the most influential factors in the storage strategy is the selection of indexes. Indexes can significantly improve query performance, but they require additional storage space and maintenance overhead. Index selection problem is well-studied in the context of single-model Database Management Systems (DBMSs), but there is a lack of research in the context of multi-model database systems.

We address this problem by conducting a survey of current state-of-the-art index selection algorithms and evaluating their applicability to other DBMSs. The results reveal the strengths and weaknesses of existing algorithms and highlight the need for specialized algorithms for multi-model database systems. Moreover, we formulate open questions and suggest future research directions in this field. Our research provides a foundation for the development of efficient index selection algorithms for multi-model DBMSs.

6.1 Introduction

The way we store and access data has been changing rapidly over the past few decades. Initially, data was stored in traditional single-node *relational databases*, which were designed to handle *normalised* and *structured data*. Developers had to define the *schema* of the data in advance, storing it in a structured manner. However, as the amount of data grew, relational databases became too slow and expensive to scale. Additionally, the need to store *denormalised* and *unstructured data*, such as logs and multimedia content, emerged, leading to the development of *NoSQL databases*, which were designed to handle unstructured data and scale horizontally across distributed systems. In parallel, new data models and their underlying DBMSs emerged, each optimised for specific data types and use cases.

In recent years, the exponential growth of data generated by various applications and devices has driven the need for more advanced and sophisticated database systems. Developers are increasingly required to handle *multi-model data*, combining different data models within a single system. This has given rise to *multi-model DBMSs*, which integrate multiple data models into a unified backend, allowing users to manage and query relational, document, graph, and key-value data within the same platform. While these databases enable efficient cross-model operations and data consolidation, they present new challenges in terms of storage strategy and performance optimisation and vigorous research has been conducted to find the optimal storage strategy for given multi-model data [10, 125, 13, 17].

In addition to *multi-model data*, modern applications frequently deal with *multi-modal data*, where data from different modalities (such as text, images, videos, and geospatial information) must be stored and queried together. *Multi-modal databases* require specialised indexing techniques, including *inverted indexes* for text, *R-trees* for spatial data, and *multi-index hashing* for high-dimensional data such as images. Handling such diverse data types within a single system introduces complexities that traditional index selection approaches, which were designed for

single-model or structured data, cannot directly address [126]. Consequently, new research is needed to adapt and extend these index selection algorithms for the challenges posed by multi-modal and multi-model databases.

Although selecting the right combination of data models is critical for database performance, it is not the only factor. For example, effective *index selection* and *materialised views* are essential for optimising performance [86]. Moreover, in multi-modal systems, index types must be tailored to each data modality, and automated index selection algorithms must be capable of handling diverse query patterns across multiple data types.

This paper focuses on the *index selection problem* in multi-model and multi-modal DBMSs. It is a well-studied problem in the field of single-model databases [88, 127, 128, 129, 130, 131], where its impact on query performance is well understood and is constantly improving [88]. However, as the complexity of data models grows, particularly with multi-model and multi-modal data, the need for a new level of abstraction in managing indexes becomes evident. We argue that new algorithms must be developed to automate and optimise index selection in these complex environments, ensuring that modern DBMSs can scale efficiently and meet the evolving demands of real-world applications.

The main objectives of the paper can be summarised as follows:

- *Analysis of indexing in various DBMSs* First, we analyse the most popular DBMSs, describe their use cases, and compare their typical workloads. Next, we conduct a comprehensive analysis of indexing strategies these DBMSs use. We mainly focus on the index types they support and the data structures they use to implement these indexes.
- *Analysis of existing approaches* As index selection is already a well-studied problem in the context of relational databases [88], we first analyse the current state-of-the-art algorithms and select the ones that might be suitable for multi-model data. The algorithms we study vary in their approach, implementation, and complexity. We also consider machine learning-based algorithms, which are currently inferior to the other algorithms, but are extremely promising for the future development of index selection algorithms [86].
- *Verification of their broader applicability* We consider the applicability of the selected algorithms to other DBMSs. We also compare the algorithms and suggest the most promising ones for each database system. In case the algorithms are not applicable, we analyse the reasons and formulate ideas for their improvement.
- *Definition of open questions* Finally, we formulate open questions and challenges for future research of index selection in multi-model and multi-modal DBMSs.

Outline Section 6.2 provides an overview of popular data models and their corresponding DBMSs, along with a comparison of these models based on real-world applications, typical queries, advantages, and disadvantages. Section 6.3 introduces the definitions and notations used throughout the paper. Section 6.4 then reviews state-of-the-art index selection algorithms, categorising them by

approach (e.g., machine learning, integer linear programming, greedy algorithms). In Section 6.5, we compare selected approaches based on their applicability to different data models, the range of supported index types, and computational complexity, and assess their suitability for multi-model and multi-modal data. Section 6.6 formulates challenges and open questions for future work in multi-model DBMSs and index selection, followed by the conclusion. Finally, we conclude.

6.2 Review of popular database systems

There are hundreds of DBMS, more than 50 % of which are considered multi-model systems [132]. Each serves a variety of purposes and is suitable for different data. If we want to apply the index selection algorithms to multi-model DBMSs, we need to understand the database systems, supported data models, their characteristics, and how indexes are used to optimise query performance.

This chapter introduces the most popular data models used in multi-model and NoSQL database systems. In particular, a brief overview of the relational, graph, key-value, document, and column models is provided, including a discussion of real-world applications, typical queries, and how indexes are implemented. In addition, a representative DBMS is selected for each model and its index implementation is specifically described. Finally, a comparison of the different DBMSs is presented in the context of the data handled and the types of indexes used.

6.2.1 Use of indexes in database systems

A database index is a data structure that is used to quickly locate data without having to search unnecessarily large parts of the database. This advantage comes with the additional write and memory overhead of maintaining the index data structure. Most modern database systems implement indexes to allow developers to speed up queries and manage data as efficiently as possible. As popular DBMSs use different data models, the indexes they adopt vary. For example, there are DBMSs that use indexes to quickly find relationships between entities, others to immediately find specific values, and yet others to enforce strict constraints. However, the most frequent use cases for indexes include quick lookups, data aggregation, sorting, and range queries.

6.2.2 Relational database systems

The relational model is the most popular data model in the world [132]. It is considered ideal for storing strongly consistent (i.e., ACID transactions are supported), normalised and structured data, where information can be organised into tables with columns and rows. This type of data, for example, includes financial records, inventory lists, and customer information.

Typical queries in the relational model involve projection of specific attributes from one or more tables, joining multiple tables, filtering data based on certain criteria, grouping, and aggregating data. Various types of indexes are used for this purpose, including B-trees, hash indexes, and bitmap indexes [133]. Namely, B-trees are used for range queries and sorting data, hash indexes for fast lookups in large tables, and bitmap indexes for handling data with boolean operations.

Among the most popular representatives of DBMSs that support the relational model are, e.g., MySQL², Oracle³, and PostgreSQL⁴. For example, PostgreSQL provides several indexes, each with its own characteristics and use cases. Namely, it supports B-tree, hash, GiST, GIN, and SP-GiST indexes [134].

- B-tree is the default index type used for indexing simple data types, such as integers, floats, and timestamps. It is well-suited for range queries and sorting data.
- The Hash index is used for indexing simple equality queries on fixed-length data types such as integers. It is implemented as a hash table, allowing fast lookups for exact matches. However, it does not support range queries and sorting.
- GiST index is used for indexing geometric and text data types and implementing full-text search.
- GIN index is used for indexing arrays and composite types, as well as for implementing full-text search.
- SP-GiST index is used for efficient range queries on multidimensional data types such as geographic coordinates and time-series data.

6.2.3 Graph database systems

The graph database systems represents a family of NoSQL systems designed to store and manage data in the form of nodes and edges (i.e., graph), allowing for efficient management and querying of complex and connected data, such as social networks, recommendation engines, and fraud detection systems. Moreover, these systems excel at storing data that is constantly changing and evolving.

As the relationships are not calculated at query time but are stored as physical connections between nodes, it allows for faster and more efficient queries that traverse the relationships between nodes without the need for complex join operations. Hence, typical queries in a graph database include finding the shortest path between two nodes, identifying patterns in the data, and recommending related nodes based on common attributes. In addition to high performance for complex queries, graph databases offer the ability to add or modify relationships and properties without changing the entire schema.

Graph database design offers a large variety of indexes [135]. For example, indexes can be created on node or edge attributes, on a single property, or on multiple attributes. In addition, indexes themselves can be optimised for range, exact match, or text queries. For that, multiple data structures that support these types of indexes are needed. In particular, graph databases typically use B+ trees, hash tables, and inverted indexes.

Without a doubt the most popular representative of DBMS implementing the graph model is Neo4j⁵. It automatically creates lookup indexes for node and

²<https://www.mysql.com/>

³<https://www.oracle.com/database/>

⁴<https://www.postgresql.org/>

⁵<https://neo4j.com/>

edge labels to eliminate the need to search through all entities when executing a query. Moreover, Neo4j supports range, text, point, and full-text indexes [135], as illustrated in Figure 6.1. The range index is the most flexible type as it can be used for exact match, range, and sorting queries. Point indexes only handle predicates that operate on points, such as distance or bounding box queries. The text index is optimised for predicates based on prefixes, suffixes, and other text operations. Full-text index, as the name suggests, is used for full-text search. Unfortunately, only lookup and range indexes can be created on multiple attributes (i.e., composite). The remaining index types, namely full-text, text, and point indexes, are limited to single attribute indexing only.

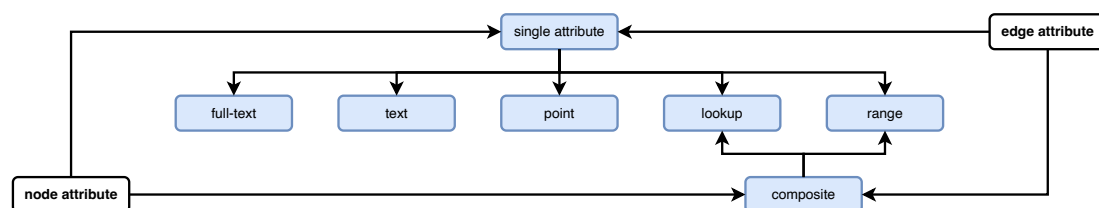


Figure 6.1 Supported index types in Neo4j

6.2.4 Key-value database systems

The key-value model is the simplest (aggregate-oriented) model for storing data. Each piece of data is stored as a *(key, value)* pair, with the *key* acting as a unique identifier and the *value* containing the actual data. It is used for storing semi-structured or unstructured data, such as user profiles, session data, or shopping cart data. The advantages of key-value models include faster read and write operations (e.g., in-memory), scalability, and the ability to handle large volumes of data with low latency.

Typical queries in the key-value model involve retrieving specific values based on their keys, updating or deleting values, and performing basic operations such as incrementing or decrementing values.

A key-value DBMS can be viewed as a hash table that allows fast retrieval of stored values. Unlike other types of DBMSs, in key-value stored, each value is usually treated as black box and the only way to query it is by the associated key.

The key lookups are usually instantaneous, hence there is no or very little need for secondary indexes. However, there exists key-value databases, such as Redis [136], that support secondary indexes, but users are expected to explicitly define them.

Among the most popular representatives of key-value stores are Redis⁶ and Riak⁷. As opposed to typical key-value databases, in Redis, values can contain more complex data types, such as lists, geospatial, and sorted sets. These values are then more likely to be queried on attributes other than their keys. Hence, the need for secondary indexes arises. Redis uses hash tables for primary indexes and sorted sets for secondary indexes. There is no automatic way to create a secondary index in Redis; the user must explicitly define it by implementing a

⁶<https://redis.io/>

⁷<https://riak.com/>

new sorted set of the indexed attributes [136]. The sorted list then maps each attribute to a set of primary keys whose values match the attribute.

6.2.5 Document database systems

The document model represents data in hierarchically structured aggregates (documents), which are self-contained units of information. As such, it allows for flexible schema design and can easily accommodate changes in data structure over time [137]. DBMSs that implement the document model are ideal for storing unstructured or semi-structured data, such as JavaScript Object Notation (JSON) [138] or eXtensible Markup Language (XML) [139] formats. In the industry, document DBMSs are used for content management, social media, and internet of things (IoT) applications.

Typical queries in the document model involve, for example, selecting specific documents based on their fields, filtering data based on specific values, and sorting documents based on certain criteria. The advantages of these queries include the ability to handle complex and dynamic data, faster read and write operations, and theoretically unbounded scalability.

The most common index types used in the document-oriented DBMSs are single field, compound, and multi-key indexes [140, 141]. In particular, single field indexes are used to exact match queries on a specific field and compound indexes to query on multiple fields. Both single field and compound indexes are represented as B-trees. On the other hand, hashed indexes are used to look up data in large collections efficiently and are represented as hash tables. Finally, document-oriented DBMSs also utilise full-text indexes (e.g., inverted index), geospatial indexes, and multi-key indexes.

Among the most popular representatives of document-oriented DBMSs are MongoDB⁸ and Couchbase⁹. Notably, MongoDB is a massively scalable distributed system, which makes it an ideal datastore for handling large amounts of data and traffic. As for indexes, single field, compound, multi-key, geospatial, text, and hashed indexes are implemented [140]. Moreover, hashed indexes support sharding using shard keys to determine how the data is partitioned across the cluster [142].

6.2.6 Columnar database systems

Unlike traditional row-based (relational) models, the columnar model represents data by column. Rows then correspond to a collection of columns, and tables (also known as column families) are represented as a collection of rows. Thus, each row does not need to have the same set of columns as opposed to the relational model. The columnar database systems are well-suited for storing large amounts of structured data, such as financial data or log files.

Typical queries in the columnar database systems involve selecting specific columns and filtering data based on specific values. The advantages of using the columnar DBMSs for these queries include faster read and write operations,

⁸<https://www.mongodb.com/>

⁹<https://www.couchbase.com/>

improved compression and data storage, and the ability to scale horizontally by adding more servers.

As systems that use the column model are usually distributed databases, their index architecture differs from other single-node databases. Primary indexes are comprised of a partition key and a clustering key. The partition key determines which node the data is stored on, while the clustering key determines the order of data within a partition. Secondary indexes are hidden tables whose primary key is the indexed column and whose other columns are the primary key of the original table and additional metadata depending on the index type. Secondary indexes are co-located with the source data on the same nodes [143]. The partition keys are represented as hashed tokens, and the indexed column values are stored in B+ trees.

The most popular representatives of columnar database systems are Apache Cassandra¹⁰ and HBase¹¹. In particular, Apache Cassandra is heavily optimised for write operations. It indexes data using Log-Structured Merge Trees (LSMT) [144], which might not be as efficient as a B+ tree for read operations, but it can write data in constant time [145]. On the other hand, Cassandra does not support multi-attribute indexes, and indexed columns are not supposed to have high cardinality [146, 147]. Hence, rather than use of secondary indexes in Cassandra, it is more common to modify the database schema to make it more suitable for the queries being executed.

6.2.7 Multi-Model data

Multi-model data, i.e., data represented by a combination of logical models, is becoming increasingly common as organisations generate and store large volumes of diverse data types. For instance, a social media platform may need to store user profiles using a relational model, the relationships between users using a graph model, and user-generated content using a document model. In particular, there are generally two approaches to storing multi-model data: academia-driven *polystores* [6] and industry-driven *multi-model databases* [7].

Polyglot persistence Polystore is an approach that involves using multiple database systems to store multi-model data [6] against single integrated interface. These databases are then managed by database administrators, who are responsible for ensuring that the data is stored and managed correctly. The advantages of polyglot persistence include the ability to use the best-suited storage technology for each type of data, improved scalability, and better performance. On the one hand, it requires additional effort and complexity in the design and management of the system. It may also increase development and maintenance costs.

Multi-model database systems On the other hand, multi-model DBMSs are designed to support multiple data models against a single, integrated backend [7]. The system can automatically process multi-model data without the need for manual intervention. Data is consistent and can be queried by a single (possibly extended) query language that supports querying across all involved data models.

¹⁰<https://cassandra.apache.org/>

¹¹<https://hbase.apache.org/>

One of the disadvantages of multi-model databases is that they are complex to design and implement. Similarly to NoSQL systems, multi-model DBMSs are also quite immature compared to traditional database systems, which means that they are still evolving and may miss some features that are available in relational systems.

Two representatives of multi-model DBMSs are, e.g., ArangoDB¹² and OrientDB¹³.

Indexes in multi-model databases In a multi-model DBMS, indexes function similarly to single-model systems. Based on our observation, the most significant difference is that multi-model DBMS must support a broader range of index types than single-model databases, i.e., these systems must support at least the main index types depending on the supported data models to ensure efficient query processing. Another difference is that multi-model databases require more complex query optimisation to determine which secondary index to use for a given query.

6.2.8 Multi-Modal data

In recent years, multi-modal data has gained significant attention due to the increasing need to handle complex data types that go beyond traditional structured data [126]. Multi-modal data includes information from multiple sources, such as text, images, videos, and sensor data, which often coexist in modern applications like social networks, healthcare systems, and autonomous driving.

Indexing in multi-modal databases poses unique challenges due to the heterogeneous nature of the data. For instance, unlike relational databases where data is structured in tables, multi-modal data involves a combination of various data types (i.e., formats), which require specialised indexing techniques. Common queries in multi-modal systems include similarity searches, semantic searches, and cross-modal retrieval (e.g., retrieving an image based on a text description) [148].

Several indexing strategies are employed in multi-modal systems, including inverted indexes, multi-index hashing, and R-trees. These indexing methods are chosen based on the type of data and query patterns:

- Inverted indexes are often used for text-based queries, enabling fast lookups for keywords or phrases across large document collections.
- Multi-index hashing is employed to handle high-dimensional data like images and videos, allowing efficient approximate nearest neighbour searches for similarity queries [149].
- R-trees are frequently used for spatial data, supporting range queries and nearest neighbour searches for data that include geographic coordinates or multidimensional attributes.

Popular database systems supporting multi-modal data include Apache Solr¹⁴ and Elasticsearch¹⁵, both of which utilize inverted indexing for handling large-scale

¹²<https://www.arangodb.com/>

¹³<http://orientdb.org/>

¹⁴<https://solr.apache.org/>

¹⁵<https://www.elastic.co/elasticsearch/>

textual data, along with support for integrating various other data types. Another notable system is Milvus¹⁶, an open-source platform designed specifically for vector similarity search in high-dimensional data, making it well-suited for tasks like image and video retrieval [150].

6.2.9 Comparison of selected database systems

Table 6.1 provides a comparison of selected popular DBMSs. Each data model is represented by a single representative. As we can see, most of the selected systems are built to store semi-structured and unstructured data. This reflects a trend in NoSQL systems, as large volumes of dynamically evolving data cannot be stored in a structured form. Relational (PostgreSQL) DBMSs were on the other hand designed to store structured data.

Generally, the DBMSs share common indexing strategies, but they mainly differ in the implementation details and supported index types. Selected DBMSs optimise indexing strategies for fast read operations with the exception of Cassandra, which takes into account write operations as well. The most common index types are range and lookup indexes. These are usually implemented using B-trees and hash tables, respectively. In addition, MongoDB, PostgreSQL, and Neo4j support various index types ranging from geo-spatial to array indexes.

With a proper index selection, these DBMSs can efficiently process a wide range of queries, but the complexity of the index selection process increases. Another observed pattern is the limited or non-existent support for secondary indexes in columnar and key-value DBMSs. For these DBMSs, it is recommended to modify the database schema instead of using secondary indexes.

6.3 Definitions & Notation

In this section, we provide the basic definitions and notation that we use throughout the paper. There are many different notations used in the literature, therefore we use the notation that is most common in the articles we reference.

Workload Given a set of relational tables $T = \{t_1, t_2, \dots, t_m\}$, where each table t_r (for $r = 1, \dots, m$) is a relation defined over a subset of attributes (i.e., named columns) from the universal set of attributes $A = \{a_1, a_2, \dots, a_n\}$, a *workload* $W = \{q_1, q_2, \dots, q_o\}$ represents a set of queries q_j (for $j = 1, \dots, o$) executed over tables T . Each query q_j accesses a subset of attributes $access(q_j) \subseteq \{a_1, a_2, \dots, a_n\}$.

Index Given a relational table t with n attributes, i.e., $t = \{a_1, a_2, \dots, a_n\}$, a (*multi-attribute*) *index* $i = \{a_1, a_2, \dots, a_o\}$ over a single table t ¹⁷ is an ordered set of attributes such that $i \subseteq t$. The set of all (potential) indexes is denoted as I . The number of attributes o is referred to as the *width* of the index.

¹⁶<https://milvus.io/>

¹⁷In this context, we assume that a single index can only reference attributes from a single table, not from multiple tables.

	PostgreSQL ¹	Neo4j	Redis ¹	MongoDB ¹	Cassandra	Apache Solr
Model	Relational	Graph	Key-value	Document	Columnar	Document
Data type	Structured	Semi-structured, unstructured	Semi-structured, unstructured	Semi-structured, unstructured	Structured	Semi-structured, unstructured
Indexed attribute	Columns	Node/edge properties	Keys	Fields	Columns	Fields
Range indexes	B-tree, SP-GiST	B+ tree	Sorted set	B-tree	LSMT	N/A ³
Fast lookups	Hash, B-tree	Hash, B+ tree	Hash	Hash, B-tree	Hash	Inverted Index
Composite indexes	Yes	Yes	No ²	Yes	No	No
Text indexes	Yes	Yes	No	Yes	Yes	Yes
Other indexes	GiST, BRIN	Point	No	Multi-key, Geospatial	No	Geospatial

¹ This DBMS supports multiple models; in our case, we consider it to represent the selected data model. composite indexes are supported in Redis. However, we can construct composite indexes using sorted sets. inverted indexes and does not natively support range queries like traditional RDBMSs but can handle them

² Neither secondary indexes nor
³ Apache Solr primarily relies on through custom configurations.

Table 6.1 Comparison of different DBMSs

Index Configuration Given a set of all indexes I , an *index configuration* C is a subset of I (i.e., $C \subseteq I$) that is considered and evaluated by index selection algorithms.

Cost Given a workload $W = \{q_1, q_2, \dots, q_o\}$ and a set of indexes C , the *cost* of a query q_j ($j = 1, \dots, o$) is determined by a function $cost(q_j, C)$, that assigns a cost value to the query q_j based on the indexes in C . Note that cost of each query can be computed efficiently using the what-if optimizer in existing database systems.

The *total workload cost* $T(C)$ is the sum of the individual query costs $cost(q_j, C)$, each multiplied by its occurrence frequency f_j (representing the impact of each query), and is formally expressed as:

$$T(C) = \sum_{j=1}^{|W|} f_j \cdot cost(q_j, C)$$

Thus, the total workload cost $T(C)$ measures the overall execution cost for the workload given the selected index configuration, taking into account both query costs and their occurrences.

Size Given a set of potential indexes I , the *size* of an index refers to its storage consumption. The memory consumed by a specific index $i \in I$ is denoted by $mem(i)$, where $mem(i)$ can be defined arbitrarily based on the index characteristics.

Constraint Given a set of relational tables T , and a set of potential indexes I , a *constraint* k is a condition that limits or defines specific properties for the chosen index configuration $C \subseteq I$. The set of constraints K ensures that the selected configuration C meets required criteria, such as storage limits (denoted as k_B , $k_B \in K$), update costs (denoted as k_C , $k_C \in K$), or the number of indexes in the configuration (denoted as k_N , $k_N \in K$). In particular:

1. The storage limit constraint k_B ensures that the total storage consumption of the selected index configuration C does not exceed the available memory budget, that is, $\sum_{i \in C} storage(i) \leq k_B$.
2. The update cost constraint k_C ensures that the total cost of updating the indexes in C remains below a given threshold k_C , that is, $\sum_{i \in C} updateCost(i) \leq k_C$.
3. A cardinality constraint k_N restricts the size of the index configuration C by limiting the maximum number of allowed indexes, that is $|C| \leq k_N$.

Index Selection Problem Given a workload $W = \{q_1, q_2, \dots, q_o\}$, a set of potential indexes $I = C^1 \cup \dots \cup C^n$ for the relational tables $T = \{t_1, t_2, \dots, t_m\}$, and a set of constraints K , the *index selection problem* is to find a subset of indexes $C^* \subseteq I$ that minimizes the total workload cost $T(C^*)$ while satisfying the constraints. Hence, the goal is to solve:

$$C^* = \arg \min_{C \subseteq I} T(C)$$

subject to the constraints K .

Since evaluating all possible indexes I is computationally expensive, algorithms focus on selecting relevant *index candidates*, often prioritizing syntactically relevant indexes, which contain columns that appear together in at least one query q from the workload W . Hence, the solution quality depends on both the chosen set C^* and the candidate subset of I .

To summarize the notation used in this paper, Table 6.2 provides a comprehensive list.

Notation	Description
$T = \{t_1, t_2, \dots, t_m\}$	Relational tables in the database
$A = \{a_1, a_2, \dots, a_n\}$	Attributes of the relational tables
W	Workload, a set of queries executed over T
$q_1, q_2, \dots, q_o$	Individual queries in the workload W
$access(q_j) \subseteq \{a_1, a_2, \dots, a_n\}$	Subset of attributes accessed by query q_j
I	Set of all potential indexes
$C \subseteq I$	Index configuration, a subset of all indexes
$i = \{a_1, a_2, \dots, a_o\}$	Multi-attribute index with o attributes
$cost(q_j, C)$	Cost of executing query q_j using configuration C
$T(C)$	Total workload cost with respect to index configuration C
f_j	Frequency or weight of query q_j
K	Set of constraints for index selection
$mem(i)$	Memory consumption of index i
k_B	Maximum allowable storage (memory) for index configuration
k_C	Maximum allowable update cost for index configuration
k_N	Maximum allowable number of indexes in the configuration
C^*	Optimal index configuration

Table 6.2 Table of Notations

6.4 Index selection algorithms

A proper selection of database indexes, referred as index selection problem (ISP), can have a significant impact on the performance of the database system. When properly chosen and used, indexes can speed up query execution [88]. However, choosing the right indexes requires careful consideration of the data

model, query patterns, and application workload. By adding too many indexes, write performance can slow down and increase storage requirements, while an insufficient number of indexes can lead to slower query performance. Therefore, it is important to carefully evaluate the trade-offs between the performance benefits of indexes and their associated costs in terms of storage and write performance. Much effort has been put into automating the index selection process [129, 151, 128, 127, 130, 131].

Index selection is an NP-complete problem [152]. Finding the optimal solution is therefore computationally expensive and algorithms must balance the quality of the solution and the time required to find it. Not only is the set of possible combinations of indices exponential, but the indices interact with each other, making the benefit of one index affected by the presence of another index [153]. This makes the problem even more complex. In addition, the contribution of an index can only be computed by calling the database API, which is a time-consuming operation. Despite these problems, many algorithms achieve commendable results. We discuss a selection of them in Section 6.4.

After the problem was proven to be NP-complete [152], further efforts in this area have mostly shifted to heuristic algorithms rather than algorithms that try to find the optimal solution [151, 128, 127, 154, 130, 131]. As a result, modern index selection algorithms approximate the solution. However, it is also difficult to approximate the optimal solution [155]. Therefore, the index selection problem is constantly being researched and new algorithms are being developed.

Example 6.1. In the running example, let us consider a simple database structure that involves tables $T = \{t_1, t_2\}$, where $t_1 = \text{Company}$ and $t_2 = \text{Building}$, and a universal set of attributes $A = \{a_1, a_2, a_3, a_4, a_5, a_6\}$, where $a_1 = \text{name}$, $a_2 = \text{revenue}$, $a_3 = \text{profit}$, $a_4 = \text{city}$, $a_5 = \text{street}$ and $a_6 = \text{price}$. As illustrated in Figure 6.2, the table t_1 contains attributes **name**, **revenue** and **profit** (i.e., $t_1 = \{a_1, a_2, a_3\}$, $t_1 \subseteq A$), and the table t_2 consists of attributes **city**, **street** and **price** (i.e., $t_2 = \{a_4, a_5, a_6\}$, $t_2 \subseteq A$). Each of these tables additionally contains some basic data which is subsequently queried.

Company	name	revenue	profit
	Škoda Auto	450	80
	ČEZ Group	200	60
	Pilsner Urquell	100	25
	Zetor Tractors	50	15

Building	city	street	price
	Prague	Wenceslas Square	150
	Brno	Masaryk Street	80
	Plzeň	Republic Square	55
	Olomouc	Horní Square	40

Figure 6.2 Example of relational tables

The workload W consists of three queries:

- q_1 `SELECT * FROM Company WHERE profit <= 30 AND revenue > 60;`, i.e., a projection of all attributes from the table **Company**, returning only records that have profit less than or equal to 30 and revenue greater than 60 (that is, we are interested in interval search).
- q_2 `SELECT Name FROM Company WHERE revenue = 200;`, i.e., a projection of a name of companies which have revenue exactly 200 (that is, we are interested in exact value match).
- q_3 `SELECT count(*) FROM Company, Building WHERE profit > price;`, i.e., the query is executed on both tables to select number of records where profit is greater than price.

6.4.1 General formulation of ISP algorithms

Despite the great variety of index selection algorithms, most of them share some key features. As depicted in Figure 6.3, a general architecture can be constructed to help us understand the main concepts and will be used to describe the algorithms in this section.

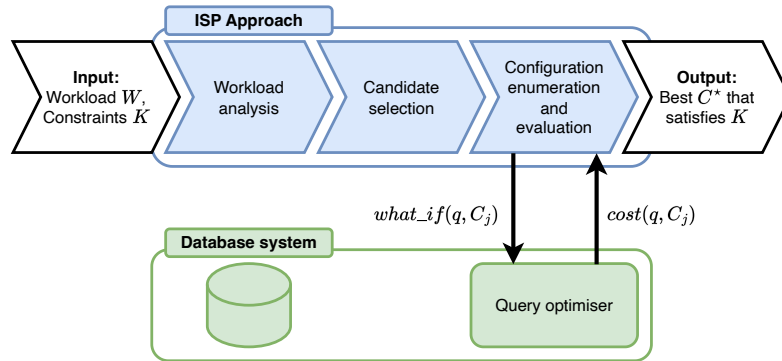


Figure 6.3 General architecture of the index selection algorithms

As the efficient use of indexes depends on queries executed on the database, queries constitute the main input for *index selection algorithms*. The set of input queries forms *workload* $W = \{q_1, q_2, \dots, q_o\}$. Additionally, database systems that require ISP algorithms are usually robust and indexing all attributes is not feasible. Therefore, the algorithms must also be able to handle *constraints* K . In particular, there are several ways to constrain the index selection process. The most common constraints are, e.g., the number of indexes (denoted k_N), the space for storing the indexes (denoted k_B), and the execution time of the algorithm.

ISP algorithms then process the workload and analyse its content. Based on the analysis, a set of indices is selected that might improve the performance of the workload. This set of indexes is referred to as *candidate set*. Then a subset of the candidate set that satisfies the constraints K is selected and *configuration* C is created. It represents a possible solution to the index selection problem. The configurations are then evaluated and the best one is selected as the final result.

As enumerating all possible configurations is usually not possible, the algorithms use *heuristics* to select the best configuration. In particular, heuristics differ based on the given constraints. Usually, the evaluation is done by estimating the cost of the workload execution with the selected configuration. In order to save time and not alter the state of the database, the algorithms usually employ *what-if* calls to the *query optimiser* to estimate the execution cost of a query for the given configuration.

Although most algorithms use the same general architecture, they differ in many aspects. The following subsections introduce three groups of algorithms that solve the index selection problem. Each group has a few selected representatives that will be described in detail.

6.4.2 Linear Programming based algorithms

Integer linear programming allows us to formulate and find a suitable solution to a given problem using a set of linear inequalities. In case of an index selection problem, we attempt to minimise the cost of query execution by implementing suitable indexes that satisfy the given constraints.¹⁸ We can formulate the problem as follows:

$$\arg \min_C \{ \sum_{q \in W} f_q \cdot \text{cost}(q, C) | C \subseteq I^* \}$$

where W represent the workload, $q \in W$ be a particular query, and I be the set of index candidates. The cost of a query q for a configuration $C \subset I$ is $\text{cost}(q, C)$. The weight f_q represents the frequency of query q in the workload. To find the optimal solution, the algorithm must consider all subsets of indexes I satisfying the constraints K and return the set whose cumulative cost is the lowest (i.e., C^*).

CoPhy

A representative of algorithms that solve the index selection problem using linear programming (LP) is CoPhy [129]. The core of CoPhy is based on a binary integer program (BIP). It works with multi-attribute indexes and allows multiple indexes per query. CoPhy is originally designed to be limited by storage space, but the nature of LP allows us to use any kind of linear constraints. For each query, candidate indices are selected and then merged into a single set. The main difference between CoPhy and other LP algorithms is in the way the variables are constructed.

The model of CoPhy is represented by the set of equations illustrated in Figure 6.4. In detail, CoPhy considers configurations in the context of a query, i.e., multiple configurations are created for each query. To minimise the number of what-if calls, the configurations are set to be atomic. That is, each configuration contains only one or zero indexes per table. The set of configurations for query q is referred to as C_q . Candidate indexes that belong to the same table t_j are grouped into a single set S_j . The binary variables $x_{q,C}$ indicate whether a given configuration C is used to query q . The variables $y_{q,j,i,C}$ ensure atomicity of the configurations. The binary variables z_i model whether a particular index $i \in I$ is used. Program constraints ensure that the following four conditions are met:

1. each query must use a single C ,
2. C must be an atomic configuration,
3. each z_i that is included in a used C must be set to 1,
4. the storage constraint k_B must be satisfied as well.

The size of an index i is denoted by $\text{mem}(i)$. The storage budget is denoted as k_B .

¹⁸Constraints such as storage budget and number of indexes can be easily formulated utilising a set of linear inequalities.

minimise:

$$\sum_{q \in W} \sum_{C \in C_q} x_{q,C} \cdot \text{cost}(q, C)$$

subject to:

$$\begin{aligned} \sum_{C \subseteq I} x_{q,C} &= 1 & \forall q \in W \\ \sum_{i \in S_j \cup I_\emptyset} y_{q,j,i,C} &= x_{q,C} & \forall q \in W, j \in T, C \in C_q \\ z_i &\geq y_{q,j,i,C} & \forall q \in W, j \in T, i \in S_j \cup I_\emptyset, C \in C_q \\ \sum_{i \in I} z_i \cdot \text{mem}(i) &\leq k_B \end{aligned}$$

parameters:

$$W, C_{q \in W}, T, S_{j \in T}, k_B$$

variables:

$$x_{q,C} \in \{0, 1\}, y_{q,j,i,C} \in \{0, 1\}, z_i \in \{0, 1\}$$

Figure 6.4 CoPhy binary integer program

CoPhy partially replicates the general architecture of index selection algorithms (see Figure 6.3). It generates index candidates for each query using well-known heuristics from literature [156]. However, it creates configurations for each query separately. The query cost is estimated by calling the query optimiser. The binary program is then constructed and solved by the selected BIP solver.

The modularity of CoPhy is a great architectural choice. In case a module becomes obsolete, it can be easily replaced by its superior version. CoPhy, along with other LP algorithms, guarantees an optimal solution for any storage budget at the cost of an exhaustive enumeration of all relevant candidate combinations. Therefore, it is a suitable choice for systems with a small number of candidate indexes. When CoPhy is used on large workloads, a rigorous selection of candidates is required. Otherwise, the number of configurations may be too large for the BIP solver to handle.

Example 6.2. Let us consider the simple database structure and workload $W = \{q_1, q_2, q_3\}$ from 6.1. The first CoPhy module creates a set of index candidates for each query (i.e., C_1, C_2, C_3). These sets are based on the attributes used in the queries. An exhaustive list of indexes that affect the query cost is used. The $\{ \}$ represents a set of indexes, $()$ represents a single multi-attribute index, and $[]$ represents an atomic configuration. In this particular case, the index candidates look like this:

- Candidates for q_1 : $\{ (\text{profit}, \text{revenue}), \text{profit}, \text{revenue} \}$
- Candidates for q_2 : $\{ \text{revenue} \}$
- Candidates for q_3 : $\{ (\text{profit}, \text{revenue}), \text{profit}, \text{price} \}$

Hence, the set of candidates is:

$$I := \{(\text{profit}, \text{revenue}), \text{profit}, \text{revenue}, \text{price}\}$$

And the workload is:

$$W := \{q_1, q_2, q_3\}$$

CoPhy then constructs atomic configurations for each query:

$$\begin{aligned} C_1 &:= \{[], [\text{profit}], [\text{revenue}], [(\text{profit}, \text{revenue})]\} \\ C_2 &:= \{[], [\text{revenue}]\} \\ C_3 &:= \{[], [\text{profit}], [\text{profit}, \text{price}], [\text{price}], \\ &\quad [(\text{profit}, \text{revenue}), \text{price}]\} \end{aligned}$$

Afterwards, the following is calculated: (1) cost for each of the candidate combinations (see Table 6.3 for C_1 , Table 6.4 for C_2 and Table 6.5 for C_3) and (2) the query for which it is used. In this particular case, PostgreSQL is used and 100,000 rows in each table is queried. The costs are given in milliseconds. Storage consumption is given in Table 6.6. Values are in kilobytes. Moreover, our storage budget k_B is 3000.

Configuration	Cost
[]	35
[profit]	3
[revenue]	18
[(profit, revenue)]	2

Table 6.3 Table of costs for C_1

Configuration	Cost
[]	20
[revenue]	5

Table 6.4 Table of costs for C_2

Configuration	Cost
[]	> 100000
[profit]	1985
[price]	1648
[profit, price]	1232
[(profit, revenue), price]	1762

Table 6.5 Table of costs for C_3

Finally, a BIP is constructed according to the model shown in Figure 6.4, minimising the implementation cost while satisfying the budget constraint. Assuming we set the same weights for each query, the optimal solution would be: {price}. Although profit helps for multiple queries, price reduces the cost most for q_3 , which is the most expensive query. $\square$

Index	Storage consumption
revenue	920
profit	920
(profit, revenue)	2208
price	2100

Table 6.6 Table of storage consumptions

Summary

Besides CoPhy, there exist several other algorithms that solve the index selection problem using linear programming, namely Generalized Uncapacitated Facility Location Problem (GUFLP) [157] and Integer Linear Program by Ailamaki et al. (ILP) [158]. However, GUFLP allows only one index per query, which makes it unsuitable for modern workloads. ILP, on the other hand, is similar to CoPhy, but creates significantly more variables, leading to longer solution times. Hence, CoPhy is the best-performing algorithm in this category [129].

6.4.3 Greedy algorithms

In modern database systems, the number of potential index combinations can be prohibitively large, making it impossible to search the entire space of possible index selections. To address this challenge, researchers have proposed various greedy algorithms [151, 128, 127, 154]. These algorithms can produce results that are close to the optimal solution, while operating significantly faster than optimal algorithms. Although the algorithms are not guaranteed to find the optimal solution, they provide an efficient way to approach the best choice of indices. Therefore, they are widely used in practice [127].

In this subsection, we discuss the application of greedy algorithms, including their characteristics, different types and limitations.

Drop heuristics

The Drop heuristics [151] is one of the oldest index selection algorithms. It does not guarantee an optimal solution, but it is fast and easy to implement. The main idea behind the Drop heuristic (see Algorithm 1) is to consider all possible index candidates and then eliminate indexes that are not useful for query processing. It works in a series of iterations, discarding in each iteration the index that contributes the least to query processing performance. The main drawback of this algorithm is that it considers only single-column indexes. The original version drops indexes until there is no cost reduction in the workload execution time. However, this is not suitable for modern, read-intensive workloads. Therefore, a modified version of this algorithm was introduced. This drops indexes until a certain index set size is reached [88].

Example 6.3. Let us consider the simple database structure and workload from 6.1. Once again we are given two tables, namely **Company** and **Building**, and a workload

Algorithm 1: Modern Drop Heuristics

Input: W – list of queries (workload)
 I – list of all index candidates
 k_N – number of indexes to be selected

```
1  $RESULT := I$ 
2 while length of  $RESULT > k_N$  do
3    $lowest\_cost := \infty$ 
4    $index\_to\_drop := \text{None}$ 
5   for  $index$  in  $RESULT$  do
6      $cost := \text{calculate\_cost}(W, RESULT - \{index\})$ 
7     if  $cost < lowest\_cost$  then
8        $lowest\_cost := cost$ 
9        $index\_to\_drop := index$ 
10    end
11  end
12   $RESULT := RESULT - \{index\_to\_drop\}$ 
13 end
14 return  $RESULT$ 
```

$W = \{q_1, q_2, q_3\}$.

The Drop algorithm starts with an exhaustive set of all index candidates, i.e., $I = \{\text{profit}, \text{revenue}, \text{price}\}$, from which we want to select only 2 indexes, i.e. $k_N = 2$. Thus, the algorithm 1 iteratively discards the least useful index until the desired number of indexes is reached. The first iteration considers the configurations listed in Table 6.7.

Configuration	q_1 cost	q_2 cost	q_3 cost	W cost
{profit, revenue}	3	5	1985	1993
{profit, price}	3	20	1232	1255
{revenue, price}	18	5	1648	1671

Table 6.7 Costs for configurations of size $|I| - 1$

This configuration is then used by the `calculate_cost` function. The algorithm greedily discards the least useful index. In this case, it is **revenue**. As the algorithm reaches the desired number of indices, it stops and returns the resulting index configuration, that is **{profit, price}**. $\square$

AutoAdmin

The AutoAdmin [128] algorithm follows a slightly modified general architecture of index selection algorithms (the modified architecture is illustrated in Figure 6.5, with the differences highlighted in orange). The main difference is that it starts with single-column indexes and then iteratively increases the index width until the desired number of indexes is reached or until the cost of the index set cannot be further reduced. Although there exist several versions of this algorithm [159, 127], the original version uses the number of indexes as the main constraint.

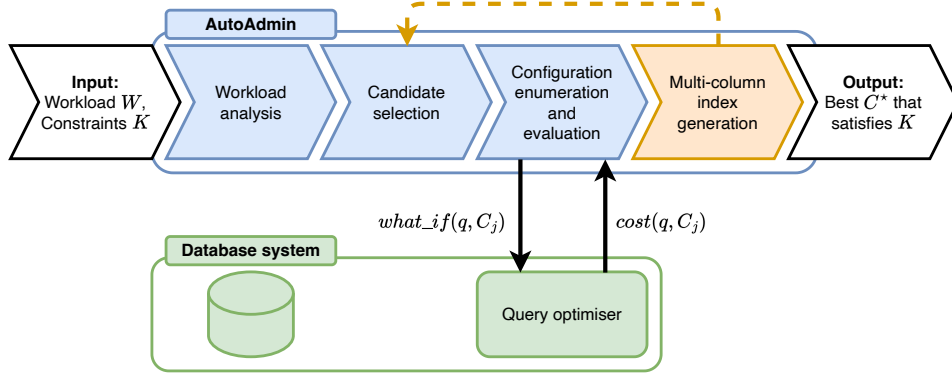


Figure 6.5 AutoAdmin architecture

The algorithm (see Algorithm 2) first determines the index candidates for each query and then merges them into a single set of candidates.

Algorithm 2: AutoAdmin

Input: W – list of queries (workload)
 I – list of index candidates
 k_N – number of indexes to be selected
 k_X – size of naive enumeration
 k_W – maximum index width

```

1 function auto_admin( $W, I, k_N, k_X, k_W$ ):
2    $P := \{\}$  // list of potential candidates
3    $S := \{\}$  // list of indexes on top of which we will build
   multi-attribute indexes
4   for  $w \in [1, \dots, k_W]$  do
5      $I := \text{select\_index\_candidates}(W, P, k_N)$ 
6      $S := \text{enumerate\_configurations}(W, I, k_N)$ 
7     if  $w < k_W$  then
8        $P := S \cup \text{create\_multicolumn\_indexes}(W, S)$ 
9     end
10  end
11  return  $S$ 

```

Candidate selection is implemented in function `select_index_candidates()` (see Algorithm 3). The function considers potential candidates for each query, performs greedy enumeration, and merges them into a final list of candidates.

The algorithm then utilising function `enumerate_configurations()` greedily selects the best indexes from the candidate set until the required number of indexes is reached or the cost stops decreasing (see Algorithm 4). Utilising function `create_multicolumn_indexes()`, the second iteration of the algorithm uses the index set from the previous iteration to build multi-attribute indexes of +1 width (see Algorithm 5).

To reduce the number of new multi-attribute index candidates, only indexes from the previous set (i.e., within the current iteration) can be the leading columns of new multi-attribute indexes. In addition to approximating the query cost, AutoAdmin uses two additional tricks to improve its performance. First, it

Algorithm 3: AutoAdmin – function *select_index_candidates()*

Input: W – list of queries (workload)
 P – list of potential candidates
 k_N – number of indexes to be selected

```
1 function select_index_candidates( $W, P, k_N$ ):  
2    $S := \{\}$   
3   for  $q \in W$  do  
4     // Select only those candidates from  $P$  that belong to  $q$   
5      $P_q := \text{filter\_candidates\_for\_query}(P, q)$   
6      $S := S \cup \text{enumerate\_configurations}(W, P_q, k_N)$   
7   end  
8   return  $S$ 
```

Algorithm 4: AutoAdmin – function *enumerate_configurations()*

Input: W – list of queries (workload)
 I – list of index candidates
 k_N – number of indexes to be selected

```
1 function enumerate_configurations( $W, I, k_N$ ):  
2    $\text{temp\_indexes} := \text{enumerate\_naive}(W, I)$   
3    $\text{indexes} := \text{enumerate\_greedy}(W, \text{temp\_indexes}, I - \text{temp\_indexes}, k_N)$   
4   return  $\text{indexes}$ 
```

Algorithm 5: AutoAdmin – function *create_multicolumn_indexes()*

Input: W – list of queries (workload)
 S – list of indexes on top of which we will build multi-attribute indexes

```
1 function create_multicolumn_indexes( $W, S$ ):  
2    $\text{multicolumn\_candidates} := \{\}$   
3   for  $\text{index} \in S$  do  
4      $\text{indexable\_columns} := \text{get\_indexable\_columns}(\text{index.table}())$   
5      $\text{indexable\_columns} := \text{indexable\_columns} \setminus \text{index.columns}()$   
6     for  $\text{column} \in \text{indexable\_columns}$  do  
7        $\text{new\_index} := \text{index} \cup \{\text{column}\}$   
8        $\text{multicolumn\_candidates} := \text{multicolumn\_candidates} \cup$   
         $\{\text{new\_index}\}$   
9     end  
10  end  
11  return  $\text{multicolumn\_candidates}$ 
```

uses function `enumerate_naive()`, i.e., a naive enumeration of candidate indexes, for small configuration sizes (see Algorithm 6). Second, in the candidate selection phase, it uses function `select_index_candidates()` itself to determine the best candidates for a single query. Finally, the algorithm performs union of naively enumerated candidates and multi-attribute indexes.

Example 6.4. Let us consider the simple database structure and workload W from 6.1. We would like to select two indexes for this structure (i.e., $k_N = 2$), and set the other parameters to $k_X = 0$, $k_W = 2$.

The algorithm starts by finding suitable candidates for the single-column index.

Algorithm 6: AutoAdmin – function *enumerate_naive()*

Input: W – list of queries (workload)
 I – list of candidates that need to be enumerated
 k_X – size of naive enumeration

```
1 function enumerate_naive( $W, I, k_X$ ):  
2    $n := \min(k_X, |I|)$   
3    $best\_configuration := \{\}$   
4    $best\_cost := \infty$   
5   for  $i \in [1, \dots, n]$  do  
6     for  $C \in \binom{I}{i}$  do  
7        $cost := \text{calculate\_cost}(W, C)$   
8       if  $cost < best\_cost$  then  
9          $best\_configuration := C$   
10         $best\_cost := cost$   
11      end  
12    end  
13  end  
14  return  $best\_configuration$ 
```

Algorithm 7: AutoAdmin – function *enumerate_greedy()*

Input: W – list of queries (workload)
 I – list of already enumerated candidates
 C – list of candidates that must be enumerated
 k_N – number of indexes to be selected

```
1 function enumerate_greedy( $W, I, C, k_N$ ):  
2    $index\_number := \min(k_N, |I|)$   
3    $best\_index := \text{None}$   
4    $best\_cost := \infty$   
5    $initial\_cost := \text{calculate\_cost}(W, I)$   
6   if length of  $I \geq index\_number$  then  
7     return  $I$   
8   end  
9   for  $index \in C$  do  
10     $cost := \text{calculate\_cost}(W, I \cup \{index\})$   
11    if  $cost < best\_cost$  then  
12       $best\_index := index$   
13       $best\_cost := cost$   
14    end  
15  end  
16  if  $best\_cost < initial\_cost$  then  
17     $C := C - \{best\_index\}$   
18     $I := I \cup \{best\_index\}$   
19    return enumerate_greedy( $W, I, C, k_N$ )  
20  end  
21  return  $I$ 
```

This is done for each query separately:

- Candidates for q_1 : {profit, revenue}

- Candidates for q_2 : {profit}
- Candidates for q_3 : {profit, price}

Next, a union of all candidates is formed to obtain the set of candidates I , which is {profit, revenue, price}. Afterwards, a greedy algorithm is executed to select the best indices from I . To compute the total cost of each index, the cost values from Table 6.3, Table 6.4 and Table 6.5 are used. Here, the index **profit** achieves a cost of 2008, the index **revenue** achieves a cost of > 100000 and the index **price** achieves a cost of 1703. The best candidate index, i.e., **price**, is selected by AutoAdmin. After that, the greedy algorithm is re-executed to select the second index. In this case, the index **profit** is selected as the total cost of {price, profit} is 1255 according to Table 6.7.

Finally, the algorithm generates two-column candidates. In this particular case, we have a single two-column candidate (**profit, revenue**). As the next algorithm iteration does not lead to any greedy improvement, the algorithm yields the final solution, that is {price, profit}. $\square$

Anytime Database Tuning Advisor

Anytime Database Tuning Advisor (DTA) [127] is an extended version of AutoAdmin. The DTA also selects index candidates for each query, but does so once, including multi-attribute indexes. It then considers indexes that may not be useful in the context of a single query, but may be useful in the context of a workload. DTA then greedily selects the most useful indexes. These indexes are then processed by a combination of several greedy algorithms. The indexes are ranked according to their benefits, so that each time the program stops the process, it returns the current best solution.

Extend algorithm

Extend [154] is one of the most recent greedy index selection algorithms. It criticises the premature pruning of candidates that occurs in the general architecture candidate selection step (see Figure 6.3). Although pruning speeds up the algorithms, it usually leads to sub-optimal selection as pruned candidates may appear to be useful. Therefore, the Extend algorithm does not follow a general architecture, but uses a sequential constructive approach. Pruning of candidates occurs as late as possible to maximise the chance of finding an optimal solution. This approach allows the Extend algorithm to consider a larger pool of candidates and make better informed decisions during the selection process.

At each step of the algorithm, Extend considers either an addition of a single-column index to the current set of indexes C_j or an extension of one of the existing indexes in C_j with a new attribute. It does so by calculating the ratio of the execution cost and memory consumption of the new index set. In the original paper [154], the cost of re-configuring the new index set is also considered, which is denoted as $R(C_{j+1}, C_j)$. In this context, C_{j+1} is the new index set and C_j is the current index set. The reconfiguration cost is the cost of changing the current index set to the new one. The final ratio for the index i is described by the equation 6.1:

$$\frac{R(C_j, \emptyset) + \sum_{q \in W} \text{cost}(q, C_j) - \sum_{q \in W} \text{cost}(q, C_j \cup \{i\}) - R(C_j \cup \{i\}, \emptyset)}{\text{size}(C_j \cup \{i\}) - \text{size}(C_j)} \quad (6.1)$$

The index with the best ratio is added to the current index set C_j . The complete algorithm is described here:

1. Start with an empty set C_j .
2. Find an index i from the set of single-column candidates I that minimises the ratio of its cost and memory consumption. Add the index to the set C_j (i.e., $C_j \cup \{i\}$).
3. For each single-column index $i \in I$ and each index $k \in C_j$, consider the following options:
 - (a) Add the selected index $i \in I$ to the set C_j as a new single attribute index, if $I \cap \{i\} = \emptyset$.
 - (b) Append the selected single-attribute index $i \in I$ to an index $k \in C_j$ to form a multi-attribute index.

Both steps yields new candidates to a new set of indexes C_{j+1} .

4. From all candidates to C_{j+1} created in step 3, select one that maximises the Ratio 6.1 and replace set C_j by the selected C_{j+1} .
5. Repeat steps 3 and 4 until the budget k_B is reached or no improvement can be made.

Note that the algorithm can also be used for online index selection. However, being focused on offline index selection, we do not consider reconfiguration costs.

Example 6.5. Let us consider the simple database structure and workload $W = \{q_1, q_2, q_3\}$ from 6.1 and let our storage budget $k_B = 3000$.

First, the algorithm compares the different benefit ratios of the single-column index candidates (see Table 6.8). The first selected index is **profit**. Other algorithms would select **price** as it minimises the total cost. Extend, however, prefers indexes with a higher benefit per unit of storage.

Index candidate	Total cost	Storage size	Benefit ratio
profit	2008	920	~ 107
revenue	> 100000	920	~ 0
price	1703	2100	~ 46

Table 6.8 Table of single-column index candidates

As the current price of C_j is 2008 and the current storage size is 920, we proceed to step 3, construct new index configurations and calculate the benefit ratios relative to the current C_j (see Table 6.9).

Although **{profit, price}** has the highest relative benefit ratio, it exceeds the storage budget. Therefore, **{profit, revenue}** is selected as the next index. This leads to a sub-optimal solution, as **{price}** would have been a better choice. $\square$

Index candidate	Total cost	Storage size	Benefit ratio
{profit, revenue}	1993	1840	0.02
{profit, price}	1255	3020	0.35
{(profit, revenue)}	2208	2022	-0.01

Table 6.9 Benefit ratios of extended index candidate sets

6.4.4 Reinforcement Learning based algorithms

Reinforcement Learning (RL) is a branch of machine learning that focuses on how an agent can learn to maximise its reward in a given environment. That is, an agent interacts with the environment and receives rewards or punishments based on the actions it performs. In the index selection problem, query costs are used to measure the performance of the system and gradually improve the performance by selecting new combinations of indexes. RL models can use these costs to optimise decisions and adjust their behaviour accordingly, making them suitable for the index selection problem.

NoDBA

Researchers at the Saarland Informatics Campus present the deep reinforcement learning (DRL) algorithm called NoDBA [130], which learns to select indexes for a given workload. At a high level, it simulates the work of database administrators (DBAs) who change database parameters based on rewards or punishments they receive from the system. The main components of the NoDBA algorithm are:

- *Input to the neural network*, i.e., workload W and the current index configuration C_j .
- *Actions x* that agent can take, i.e., create a secondary index.
- *Reward function $f(C_{j+1})$* , i.e., total workload run-time improvement after adding the index.
- *Hyper parameters*, e.g., the number of hidden layers, neurons, and iterations.

The essential part of the algorithm is the learning process (see Figure 6.6), which employs the main ideas of RL. In particular, it is an iterative process where each step j is described as follows:

1. Based on the workload and the current index configuration C_j , the neural network predicts the next action x .
2. The predicted action x is applied to the current index configuration. A new configuration C_{j+1} is created.
3. Reward r is computed using the reward function f on the new configuration C_{j+1} .
4. The neural network is trained utilizing quadruple (C_j, C_{j+1}, x, r) . The program then returns to step 1.

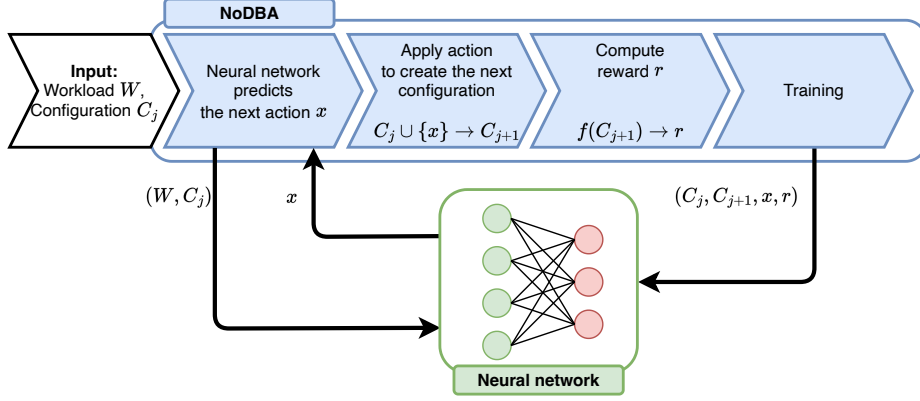


Figure 6.6 NoDBA learning process

The input to the network is represented as a matrix of size $n \times m$, where n is the number of queries and m is the number of columns in the database. Each cell then describes the selectivity of a column based on the current query. It is the ratio of selected rows based on the column predicate to the total number of rows in the table where the column is located. If the query does not select a particular column, the value is set to 1. The current index configuration is also included in the network input and is encoded as a list of m bits. As could be already deduced, the algorithm considers only single-column indexes. Although it is possible to encode multi-attribute indices, the combination of the current design and the increased index width leads to extremely high learning times [131].

Limited experiments indicate that the algorithm performs well on highly selective workloads [130]. However, it does not scale well to larger workloads and has many limitations. For example, it lacks support for multi-attribute indexes and has long training times. However, this work was the first to use RL for index selection, making it a good starting point for further research.

Budget-aware RL algorithm

Despite the great complexity of ISP algorithms, most of their run-time is consumed by what-if calls. Experiments show that what-if calls take between 75 % and 93 % of the total execution time of modern ISP algorithms [160]. Therefore, there has been recent interest in budget-aware algorithms that can reduce the number of database optimiser calls. A novel approach utilises reinforcement learning to select indexes while meeting the budget [131]. The approach follows the general architecture (see Figure 6.3) of the cost-based index selection algorithm and focuses on the rebuilding of the candidate enumeration component. Therefore, it assumes that the index candidates are already selected, and only needs to select the best combination of indexes.

To reduce the number of what-if calls, an upper bound approximation of the actual cost per query is used. The algorithm assumes that the index configuration costs are monotonic, i.e., having $C_1 \subseteq C_2$ and arbitrary q , then $cost(q, C_2) \leq cost(q, C_1)$ (see Proof [131]). Thus the approximation is computed as the minimum cost over all subset configurations of the current configuration (i.e., for each

$C' \subseteq C$). This approximation is referred to as *derived cost*:

$$derived(q, C) = \min_{C' \subseteq C} cost(q, C')$$

The main goal of the algorithm is to decide what configurations should be approximated and what configuration cost should be calculated by the database optimiser. Information about the known what-if costs is stored in the so-called *budget allocation matrix*. That is, a matrix $n \times m$, where n is the number of queries and m is $2^{|I|} - 1$, where I represents the set of candidate indexes. A cell with a value of 1 indicates that the cost of query q for the configuration C is known, whereas a value of 0 means the cost needs to be approximated.

To set the values in the matrix, the algorithm must find a balance between allocating the budget to configurations that contain the best indexes so far and configurations that do not contain the best indexes but contain unexplored indexes with similar costs. This problem is known as the exploration vs. exploitation dilemma and is solved by formulating it as a Markov decision process (MDP). The algorithm then uses Monte Carlo tree search (MCTS) to find the actions that maximise the expected reward.

The formulation of the MDP is quite straightforward. The states $\mathcal{C}$ are defined as all index configurations in the searched space. More precisely, it is the set of all subsets of index candidates I . The actions $\mathcal{A}(c)$ for a state $c \in \mathcal{C}$ are defined by the missing indexes in the current state. Thus, from a single state whose configuration size is s , we can access all subsets of index candidates of size $s+1$. As transitions are deterministic, the transition probabilities $\mathcal{P}$ of actions are defined as $Pr(\text{from state } s \text{ by action } a \text{ to state } s \cup \{a\}) = 1$. The expected return of state s is the expected cumulative reward of starting from state s and taking actions according to some policy. The algorithm computes the expected return as the expected percentage improvement of all configurations that contain the indexes of state s as a subset. The percentage improvement of state s represented by configuration C is calculated as:

$$\left(1 - \frac{\sum_{q \in W} derived(q, C)}{\sum_{q \in W} derived(q, \emptyset)}\right) \times 100$$

The objective of RL is to find the optimal policy that maximises the expected return at the initial state $\emptyset$.

MCTS is then used to find the optimal action selection strategy. It is a tree search algorithm that uses a Monte Carlo method to traverse the tree. Part of MCTS is based on random sampling of index candidates. The authors' implementation selects actions randomly, but assigns higher probability to actions that have lower average costs. To further optimise the number of what-if calls, the cost of the configuration X selected in the episode is approximated by the derived cost, and what-if costs are called only for a single query q . The probability of selecting a particular query is proportional to its derived cost. The final costs are then as follows:

$$cost(q, C) + \sum_{q' \in W \setminus \{q\}} derived(q', C)$$

6.5 Comparison of the algorithms

The algorithms discussed in Section 6.4 were originally developed and evaluated on a relational model. In this Section, we compare key characteristics relevant to multi-model databases and verify which algorithms are suitable for non-relational data.

Table 6.10 presents a comparison of algorithms and their characteristics that are important for multi-model DBMSs. As already shown, most of the examined algorithms belong to the family of greedy algorithms (as the index selection problem is NP-complete and the search for an optimal solution is not feasible [152]). One of the key properties of the described algorithms is whether they follow a general architecture (see Figure 6.3).

Each of the algorithms compared supports single column indexes, which is applicable to various data models as all support the concept of an indexable attribute. However, algorithms such as Drop and NoDBA do not support multi-attribute indexes. This causes a performance loss in relational models and makes them less suitable for those data models that support multi-attribute indexes. None of the algorithms support partition keys. This is understandable as the authors developed them exclusively for single-node relational databases. However, for distributed DBMSs, such as document and columnar stores, this poses a potential concern.

In addition, the algorithms do not consider multiple index types, a typical feature of many DBMSs. This limitation can be solved, for example, in the candidate selection phase by selecting multiple index types for a single attribute and then enumerating the candidates.

The way the candidates are selected is another important feature of the studied algorithms. While older proposals, such as AutoAdmin and CoPhy, select candidates based on individual queries, more recent algorithms, such as NoDBA and DTA, select candidates based on workload. Both approaches have advantages and disadvantages. The workload-based approach is more complex but can find beneficial indexes for the entire workload. On the contrary, the query-based approach is more straightforward but may miss indexes that are not beneficial for individual queries but are beneficial for the entire workload.

Non-essential features include anytime support and online selection support. The anytime feature ensures that the algorithm can stop at any time and still have a valid solution. This is quite useful for NoSQL and multi-model DBMSs as they may work with very large datasets and the index selection process can take a long time. The online selection feature allows the algorithm to select indexes in a system where the workload and database architecture is constantly changing. The algorithm must then take these changes into account and drop or add indexes depending on the current workload. Of the algorithms discussed, only NoDBA and Extend support online selection.

6.5.1 Index selection in Relational DBMSs

Since all the surveyed algorithms were developed for relational DBMS, they are highly optimised for this particular model. However, Drop and NoDBA have limitations when considering multi-attribute indexes. Otherwise, there are

	CoPhy [129]	Drop [151]	AutoAdmin [128]	DTA [127]	Extend [154]	NoDBA [130]	MCTS [131]
Approach type	ILP	Greedy	Greedy	Greedy	Greedy	ML	ML
Follows architecture 6.3	Yes	No	Yes	Yes	No	No	Yes
Main constraint	Storage	Indexes	Indexes	Many ¹	Storage	Indexes	What-if calls
Single-column indexes	Each algorithm supports single-column indexes						
multi-attribute indexes	Yes	No	Yes	Yes	Yes	No	Yes
Index types	B-Tree, Hash, B+ Tree, Bitmap, Inverted index						
Best index type	No	No	No	No	No	No	No
Partitioning keys	No	No	No	No	No	No	No
Optimal solution	Yes ²	No	No	No	No	No	No
Stop any time	Yes	No	Yes	Yes	Yes	No	No
Candidates context	Query	Workload	Query	Workload	Workload ⁵	Workload	Depends
Online selection	No	No	No	No	Yes	Yes	No
Relational DBMSs	Yes	Partial	Yes	Yes	Yes	Partial	Yes
Graph DBMSs	Yes ³	Partial	Yes ⁴	Yes ⁴	Partial	Partial	Yes ⁴
Key/Value DBMSs	None of the algorithms supports manual construction of secondary indexes						
Document DBMSs	Yes ³	Partial	Yes	Yes	Yes ³	Partial	Yes
Columnar DBMSs	Each algorithm supports single-column indexes						
Multi-model DBMSs	Partial	Partial	Partial	Partial	Partial ⁶	Partial ⁶	Partial
Multi-modal DBMSs	Partial	Partial	Partial	Partial	Partial ⁷	Partial	Partial

¹ DTA can be configured to use many constraints at the same time. It supports indexes, storage, and running time.

² CoPhy does find the optimal solution based on the candidates that were provided. Usually, the number of candidates is limited.

³ Generally supports the data model but is not suitable for it. ⁴ Generally supports the data model, however, at least three components of the algorithm need to be adjusted. ⁵ Extend does not contain a candidate selection phase. However, it considers potential indexes in the context of the workload. ⁶ Extend and NoDBA have adaptive elements that make them flexible for multi-model systems by allowing different index types.

⁷ Extend can potentially be adapted to support multi-modal data indexing by incorporating different indexing strategies.

Table 6.10 Comparison of index selection algorithms

no major drawbacks to using these algorithms for the relational model. When implementing the algorithms with specific database systems such as PostgreSQL, Oracle or MySQL, we have to consider minor changes, e.g., in database optimiser utilisation.

6.5.2 Index selection in Graph DBMSs

The graph model also groups data into independent units, but in this case the units are represented as nodes instead of rows in a table. Then, edges can easily be added, allowing for more flexible and expressive modelling of complex relationships.

For graph models, we can still employ the surveyed algorithms. However, we need to make some modifications based on the general architecture (see Figure 6.3). First, we need to change the workload analysis phase. Graph databases such as Neo4j use the Cypher query language. This is a declarative language that allows us to express complex graph traversal in a single query. Therefore, the query parsing component must be modified to support parsing of node and edge property predicates, such as the example in Figure 6.7:

```
MATCH
  (t:Actor {name: "Tom"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(p:Actor)
WHERE
  p.gender = "male" AND p.born <= 1985
RETURN
  DISTINCT p.name
```

Figure 6.7 A Cypher query example.

Secondly, the selection phase needs to be adjusted. The scenario is similar to that of document DBMSs, where we need to add new index types and consider their limitations to avoid generating invalid candidates. Finally, the candidate enumeration does not need to be changed, except for algorithms that do not follow the general architecture, such as Extend, Drop and NoDBA. The fundamental change that affects each of the algorithms researched is the cost estimation. Cost estimation in graph DBMSs is more complex than in other systems because queries can be based on graph traversal and complex relationships between nodes. However, Neo4j provides a sufficient API for cost estimation [161]. A query plan does not provide an expected execution plan, but provides all the operations that the query will perform. After the above modifications, we can use any of the algorithms for graph models. As the number of index candidates can be large, CoPhy must implement a more rigorous candidate selection phase. However, this may lead to sub-optimal solutions, making CoPhy less suitable for graph models.

6.5.3 Index selection in Key-Value DBMSs

Key-value stores represent the simplest of all the database systems that we discuss. Therefore, there is little need to use secondary indexes as queries are usually based on the primary key. However, there exist key-value databases, such

as Redis, that support more complex data structures, which additionally may require secondary indexes. In such case, secondary indexes can be implemented manually using appropriate data structures (e.g., hash tables, sorted sets). Thus, the problem of selecting indexes is replaced by the problem of selecting appropriate data structures and modifying queries to fit them. Although we might be able to find a bijection between the index selection problem and the data structure selection problem, the researched algorithms are not originally designed for this problem. Thus, creating a new algorithm that constructs data structures and modifies queries is beyond the scope of this paper.

6.5.4 Index selection in Document DBMSs

Workloads in document DBMSs do not use relationships between collections, making the queries generally simpler than in relational DBMSs. On the other hand, document DBMSs use new kinds of indexes, such as inverted indexes and geospatial indexes, which are not explicitly supported by any of the algorithms. Therefore, we need to take these new types of indexes into account when applying the algorithms to document DBMSs. We can do this, for instance, by changing the candidate selection phase to include the new types of indexes. In addition, index types such as hash and text indexes do not support multi-attribute indexes. Therefore, we need to modify Extend algorithm to omit these indexes when extending existing candidates. Drop and NoDBA only support single-item indexes, making them impractical for document stores. AutoAdmin and DTA support multi-attribute indexes and follow the general architecture, making them the most suitable for document stores. CoPhy can also be used for document DBMSs, but the increased number of index types makes the candidate enumeration phase more expensive. With proper candidate selection, we can use MCTS for document stores as well. This can be a good choice because what-if calls are expensive in document models and MCTS can reduce the number of them. ML-based algorithms show great potential for document models as well, as there are many types of indexes and they can learn strategies to avoid inappropriate ones.

6.5.5 Index selection in Columnar DBMSs

Distributed column-based systems use distributed indexes that store the distribution key along with the indexed value. This key is used to partition the data within the cluster. This poses a challenge for index selection algorithms because they do not support data partitioning. Many modern distributed columnar databases solve this problem by partitioning the data based on the primary key. The secondary indexes are then placed co-located with the original data across the cluster [143]. Index selection algorithms therefore do not need to take into account data partitioning. However, one of the problems that the algorithms need to address is that they can no longer rely on cost estimates because the distributed nature of the system cannot guarantee the same performance every time the same query is executed. Queries in the columnar model are much simpler than in the relational model, which eases the index selection problem. Cassandra also supports multi-attribute indexes, distinguishing between partition keys, clustering keys, and composite keys. However, despite this support, algorithms like DTA,

MCTS, and NoDBA are unsuitable as they fail to differentiate between index types. Among the remaining approaches, Drop only considers single-column indexes, while CoPhy can efficiently find the optimal solution when the number of index candidates is relatively small.

6.5.6 Index selection in multi-model and multi-modal data

The surveyed index selection approaches have primarily been designed for relational databases and the specific challenges they face, such as choosing the best set of indexes to optimize query performance in a structured environment. However, while these methods excel in relational settings, they require substantial adaptation to be directly applicable for multi-modal data. Multi-modal data involves a mix of structured, semi-structured, and unstructured data, including text, images, videos, and even sensor data. The heterogeneity of the data requires different types of indexes (e.g., inverted indexes for text, multi-dimensional indexes like R-trees for spatial data, and hashing for high-dimensional data). Thus, simply applying relational-based index selection strategies without modification would not be sufficient.

To extend these approaches for multi-modal data, the methodologies would need to be adapted to handle multiple data representations. For example, AutoAdmin or DTA could potentially be modified to select across a broader array of index types, such as inverted indexes for textual data or R-trees for spatial/geometric data. Heuristic-based methods like Drop Heuristics could be adapted by including additional factors such as data modality and query patterns specific to multi-modal queries, such as similarity searches or cross-modal retrieval. Another example is NoDBA, which focuses on adaptive indexing; it could be enhanced to handle real-time data in a multi-modal context, particularly for streaming data like video feeds or IoT sensors, which require dynamic indexing strategies. In all cases, the tuning process would need to consider how different index types (e.g., inverted, R-tree, multi-index hashing) interplay with the complex query workloads found in multi-modal environments.

6.6 Open questions and challenges

Based on our analysis of state-of-the-art index selection algorithms and their applicability to different data models, several open questions and issues were revealed. In this section, we discuss these questions and challenges and suggest possible directions for future research. We have divided the questions and challenges into five categories: *general*, *load distribution*, *candidate selection*, *query cost*, and *candidate enumeration*. The last four categories are based on the key components of the general architecture of ISP algorithms (as illustrated in Figure 6.3).

General questions and challenges

- As the volume of (big) data increases, iterative index selection methods are becoming increasingly impractical due to high computational cost and time consumption nature. We found CoPhy [129] to be unsuitable for most of the researched DBMSs. Its time complexity increases rapidly with the

number of index types. DRL algorithms such as NoDBA [130], although only partially successful, represent a promising direction for future research.

- In columnar and key-value DBMSs such as Cassandra and Redis, it is more important to design an appropriate *schema* than to use secondary indexes [143, 136]. Therefore, ISP algorithms for multi-model data should be able to recommend a schema for the database. This is a challenging task because schema design is a complex process that requires a deep understanding of the data model and workload. Another solution could be to find a bijection between the recommended secondary index and the schema. For example, in Cassandra, a multi-attribute index can be interpreted as a composite primary key.

Workload parsing

- Different query languages are used by various DBMSs, such as, e.g., Cypher, Cassandra Query Language (CQL) and MongoDB query language. We need to be able to parse the queries and extract index candidates from them. This can be solved, for example, by using a specific query parser for each query language and then passing the index candidates to the candidate selection component.

Candidate selection

- The algorithms we have examined do not formally distinguish between different types of indexes. This poses a challenge for DBMSs that support a wide variety of index types and their workloads can benefit significantly from their use. Algorithms should be able to find the proper index types for each candidate. This problem can be solved, for example, by generating a set of candidates for each attribute according to the available index types. However, this will increase the number of candidates and thus the computational cost of the algorithm. Therefore, ML algorithms that can learn strategies for preferring certain index types over others might be useful for these DBMSs.

Query cost

- Query cost estimation is the main challenge of the index selection problem. It requires index construction and query execution. This process is time consuming and not feasible to perform for every index configuration. Current state-of-the-art algorithms speed up this process by approximating the query cost using what-if calls and virtual indexes. These tools are available in relational DBMSs, but their availability in other database systems is limited. In order to use these algorithms in other DBMSs, we first need to explore possible ways of estimating query costs for each model.
- Even by having tools for estimating costs on demand, they would still be slow and lacking in accuracy. Therefore, algorithms that are optimised to minimise the number of what-if calls, such as MCTS [131], would be a great choice. In addition, we can try a new approach that uses machine learning to predict the cost of a query [162]. This might be the best solution because

modern DBMSs have robust schemas, so the learning time can easily match the time required to make all the what-if calls.

- All the surveyed algorithms assume a read-intensive workload and do not optimise write time. Some of them, such as Extend, consider reconfiguration costs [154]. As NoSQL systems may be write-intensive, we may need to consider write costs and include insert queries and their weights in the workload.

Candidate enumeration

- The enumeration is the part that is easiest to generalise for all data models. It just receives a set of candidates and their costs and returns the optimal configuration. The only challenge is to find the most optimal enumeration algorithm for each model. Some models may benefit from a greedy approach that selects the candidate with the absolute highest cost reduction, while others may benefit from a more sophisticated approach, such as the benefit ratio presented in Extend [154]. In addition, as the enumeration component interacts closely with the query cost, we need to minimise the number of what-if calls. Therefore, MCTS [131] may be a good choice for this component as well.

Conclusion

In this paper, we have taken the next step towards the development of efficient and possibly autonomous multi-model DBMSs. We have done so by analysing state-of-the-art index selection algorithms currently used in relational DBMSs. We then verified their applicability in other database systems, specifically graph, key-value, document and columnar DBMSs. Through our analysis, we were able to formulate open questions, challenges, and ideas that could help in the future development of index selection algorithms for multi-model database systems.

Additionally, we have highlighted the growing need for adapting traditional index selection approaches to handle heterogeneous data types demanding different indexing strategies in multi-model and multi-modal data.

In our future work, we plan to address the open questions (see Section 6.6). This will involve further investigation into how index selection algorithms can be generalised to handle multi-model and multi-modal data simultaneously. We will focus primarily on issues related to the general architecture of ISP algorithms, as they concern key components of the algorithms under investigation. These issues include handling diverse query workloads, supporting a variety of indexing types, and dynamically tuning indexes based on the evolving data. Once we have resolved these challenges, we will be able to implement an improved algorithm that works well on both multi-model and multi-modal database systems.

Paper VII

MM-mapsearch: Workload-Aware Mapping Selection

Pavel Koupil¹, Bedřich Mazourek¹, Jáchym Bártík¹, Irena Holubová¹

Published as Pavel Koupil et al. “MM-mapsearch: Workload-Aware Mapping Selection”. In: *Proceedings of EDBT 2026*. OpenProceedings.org, 2026. DOI: [10.48786/edbt.2026.68](https://doi.org/10.48786/edbt.2026.68)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

Modern database systems offer several alternative representations of the same data, both within one model and across multiple models. These choices have a strong influence on performance. But manually identifying an efficient mapping is challenging, especially in multi-model environments that combine heterogeneous operators.

In this paper, we present *MM-mapsearch*, an automatic advisor for selecting workload-aware multi-model mappings. A simplified categorical schema defines the search space and enables uniform reasoning about structural alternatives. *MM-mapsearch* analyses a weighted workload, explores feasible schema variants using Monte Carlo Tree Search, and estimates their performance with a plan-structured neural predictor that generalizes across relational, document, and graph systems.

7.1 Introduction

Modern database management systems (DBMSs) usually allow several ways to represent the same data. Even within one data model, the data can be organized using different structural designs that vary in their level of nesting, granularity, or normalization. Prior work has shown that such representation choices can lead to substantial performance differences across workloads [52]. Many systems also support multiple data models² simultaneously, which enables the combination of representations across models [7]. While all these options provide flexibility, they also introduce significant performance differences.

Choosing an appropriate mapping is, therefore, a challenging task. Multi-model workloads may contain heterogeneous access patterns and operator families, and each representation affects operator behavior, indexing, and traversal differently. As the number of possible mappings grows rapidly, manual evaluation becomes time-consuming, error-prone, and often unrealistic without a deep understanding of the underlying systems.

Schema evolution and workload-driven optimization have been studied across various DBMSs. For multi-model databases, recent work shows that deep reinforcement learning can tune system parameters [55, 54], but these approaches do not modify the schema or the underlying data representation. In NoSQL systems, schema evolution has been explored extensively: [164] surveys practical challenges of schema evolution and migration, MigCast [57] estimates the cost and performance implications of data-model changes, [110] introduces a formal taxonomy of schema modifications, [31] proposes generic evolution techniques applicable across both NoSQL and relational models, [165] presents a categorical framework for migrations between NoSQL models, Darwin [47] offers a platform for multi-model data evolution, and [104] defines basic operations for both evolution and migration of multi-model data. Although these works highlight the importance of structural adaptation, none address *query-driven* schema changes in multi-model settings.

The natural solution – automatically suggesting or adapting data representations in multi-model settings – introduces several challenges. (1) The search space of possible mappings grows exponentially with the number of entities and their

²<https://db-engines.com/en/ranking>

possible embeddings, inlinings, or placements across models. (2) Multi-model query workloads may combine different operator families (relational, document, graph), and the impact of a representation change often propagates across model boundaries. (3) Evaluating each candidate mapping requires estimating performance for heterogeneous operators, which is difficult without a unified model of query-plan behavior.

To address these challenges, we introduce *MM-mapsearch*, a tool that analyses multi-model workloads and recommends performance-oriented schema or mapping changes. Its main contributions are:

- A simplified categorical layer [13] that provides a uniform representation of relational, document, and graph structures and enables principled cross-model reorganization.
- Automatic analysis of weighted multi-model workloads, with schema and mapping changes derived directly from query patterns, removing the need for manual inspection.
- Systematic exploration of model assignments and embedding variants via Monte Carlo Tree Search (MCTS), with a reward function balancing predicted latency and migration effort
- Performance estimation for each candidate representation using an extended plan-structured neural predictor [53], steering the search toward high-quality schema variants.
- A full-featured prototype implementation integrating workload analysis, schema-variant generation, neural performance prediction, and an interactive interface for inspecting and comparing recommended mappings.

7.2 Proposed Approach

The goal of *MM-mapsearch* is to identify an optimal representation across logical models – relational, document, and graph – for a given workload of queries. It searches for mappings that minimize query latency and migration cost by exploring alternative schema variants and predicting their performance. Fig. 7.1, 7.2, and 7.3 shows the overall workflow, composed of three main stages:

1. **Categorical representation:** The schema is converted into a simplified categorical form capturing complex properties and their embedding or reference links, which together with the weighted workload define the search space.
2. **MCTS space exploration:** The algorithm explores model assignments and embedding variants, maintaining a directed acyclic graph (DAG) of visited states and using a reward function balancing predicted latency and migration effort.
3. **Performance prediction:** A trained neural model estimates query latency for each candidate mapping based on system-specific plans represented in a unified operator-level form.

Example 7.1. Consider a simplified TPC-H³ scenario in Fig. 7.1, where order data are stored in a relational DBMS (purple), while information about parts and suppliers resides in a graph DBMS (blue). The workload contains three queries (Q_1, Q_2, Q_3) with associated execution frequencies (w_1, w_2, w_3). The goal is to find a representation that minimizes overall latency, whether by keeping the current placement, migrating dataset(s), or reorganizing all data under a single model.

MM-mapsearch evaluates the alternatives automatically. The schema and workload in Fig. 7.1 are encoded into the root state of the MCTS search: each complex property is assigned a model and an optional embedding parent, represented by two vectors that form the initial configuration. MCTS (Fig. 7.2) then explores neighboring states, where each edge corresponds to a single change in model assignment or embedding. During search, the four MCTS phases are applied repeatedly: a promising state is selected, one modification is expanded, its workload latency is estimated using the neural predictor (Fig. 7.3), and the resulting average value, total reward, and visit counts are backpropagated along the path to the root. The final output is a recommended mapping together with predicted performance gains. $\square$

Contrary to the example, the initial mapping doesn’t have to be multi-model. *MM-mapsearch* can also start from a single-model schema and explore its multi-model alternatives. The resulting mapping can then directly replace the original representation, keeping the same query interface while improving performance.

7.2.1 Categorical Multi-Model Representation

The input to the framework is a categorical schema $\mathcal{S} = (O, M, \circ, 1)$, where O is the set of objects representing data properties, M is the set of morphisms describing connections between properties, $\circ$ denotes morphism composition, and 1 is the identity morphism. In the general form [13], morphisms capture a rich set of structural relationships, such as “has an identifier”, “has a property”, “has a role”, or “is a”. The categorical schema $\mathcal{S}$ can be created manually [12] or inferred automatically from sample data instances [16].

For the purpose of mapping optimization *MM-mapsearch* operates on a simplified variant of $\mathcal{S}$ that retains only *complex properties* and their *embedding* or *reference* morphisms. Complex properties correspond to entire structural units such as tuples, (nested) documents, graph vertices, or edges (e.g., *Orders* or *Supplier* in Fig. 7.1).

The resulting schema can be viewed as a directed graph $\mathcal{G} = (V, E)$ induced by the objects and morphisms of $\mathcal{S}$, where vertices $v \in V$ represent complex properties and edges $e = (v_i, v_j) \in E$ denote embedding, inlining, or reference links. Together with this simplified schema, the user provides a workload $Q = \{(q_i, w_i)\}$ of queries q_i and their execution frequencies w_i . Additional constraints may be attached to edges to specify whether embedding, inlining, or referencing is permitted or mandatory, thus defining the feasible search space to be explored.

7.2.2 Mapping-Space Exploration using MCTS

Given the simplified schema and workload, *MM-mapsearch* employs MCTS to explore possible model assignments and embedding configurations that jointly

³<https://www.tpc.org/tpch/>

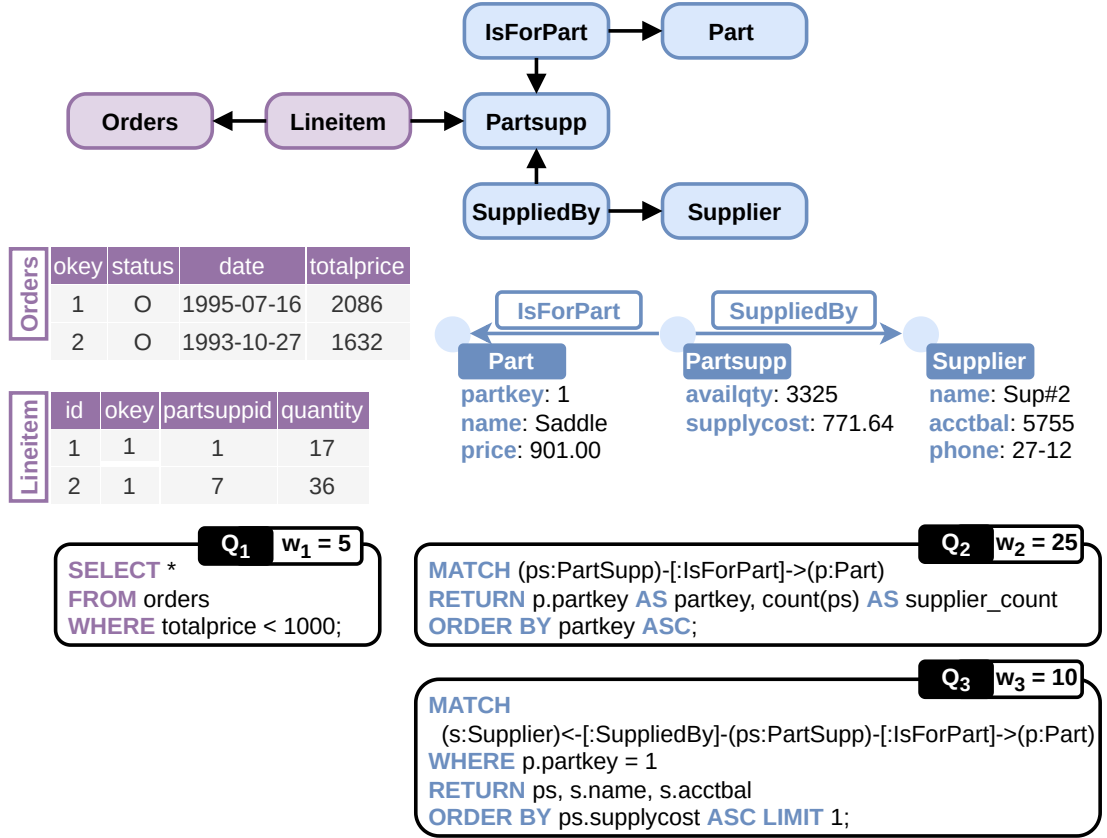


Figure 7.1 Categorical input representation of the schema and workload.

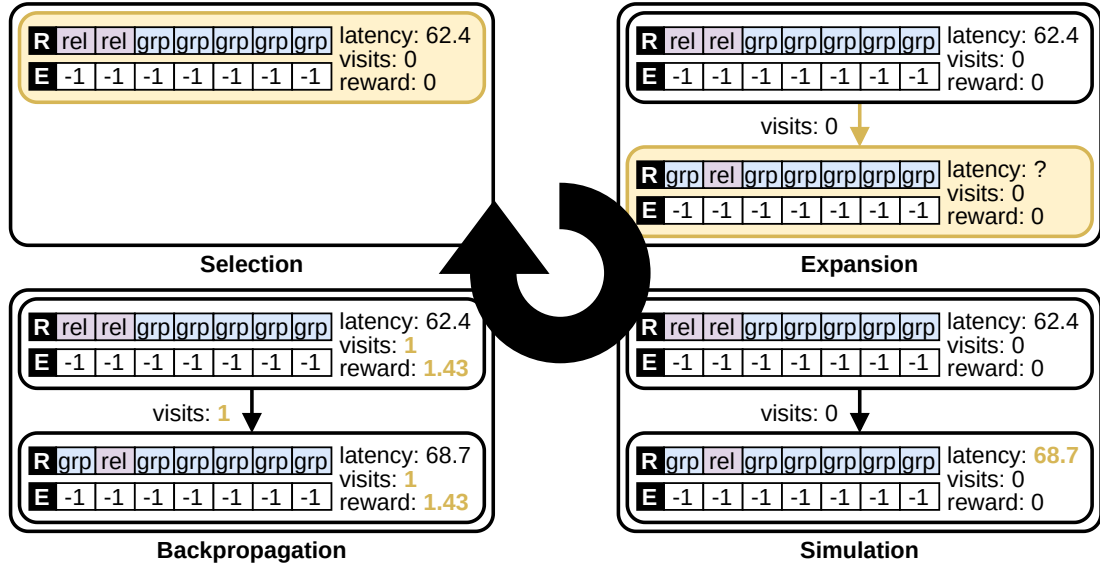


Figure 7.2 Exploration of the mapping space using MCTS.

determine the multi-model mapping. Each *state* $S = (R, E)$ consists of two vectors: $R[i] \in \{\text{rel}, \text{doc}, \text{grp}\}$ encodes the logical model of the i -th complex property, and $E[i] \in \{-1, 0, \dots, n-1\}$ specifies its embedding parent, with $E[i] = -1$ indicating a top-level property. *Actions* modify the state by changing a model assignment or altering an embedding relation. Each node in the search graph maintains its *visit count*, *total reward*, and *average reward*, while edges store their own visit counts.

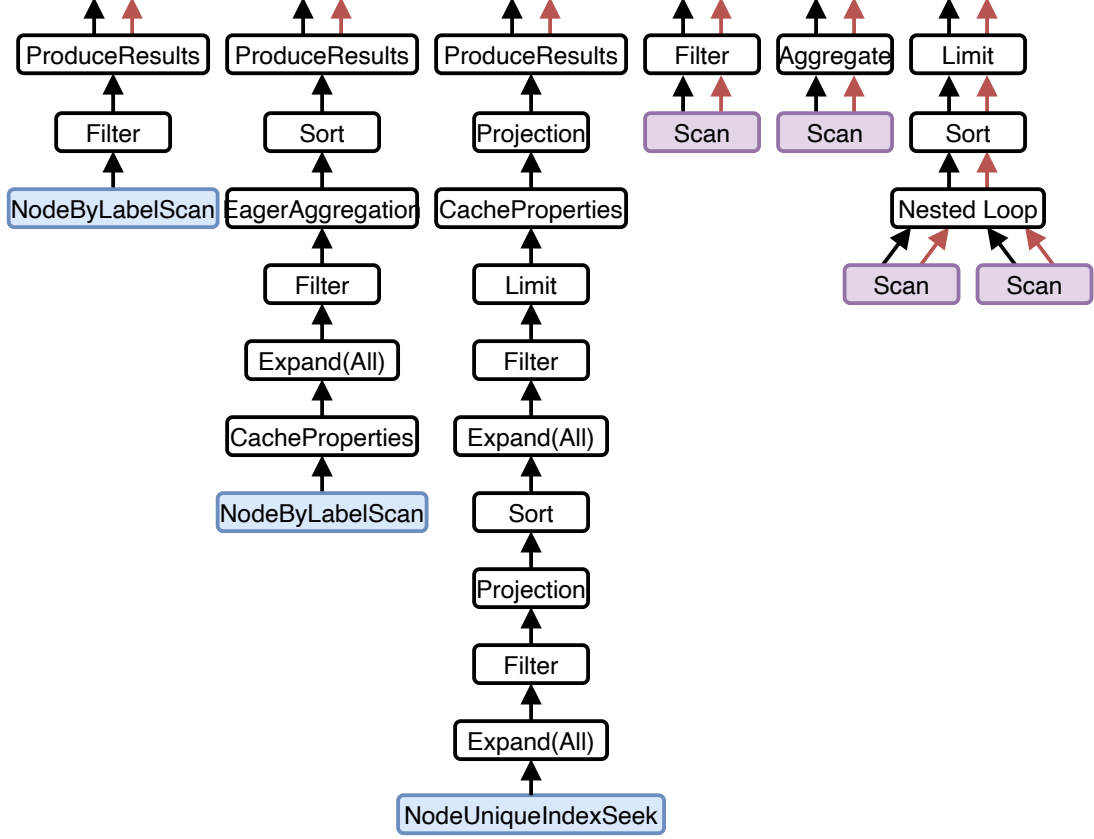


Figure 7.3 Plan-structured neural predictor for estimating query performance.

This enables a UCT2-style [166] adaptation of the *upper confidence bound* (UCB) for DAGs, ensuring consistent exploration even when different action sequences lead to the same decomposition. Before the search begins, the predicted latency of the root state is computed to establish the initial baseline. The reward is defined as $r = \frac{100}{t_m + t_e + 0.001}$, where t_m is the migration cost and t_e is the weighted query latency (both in seconds).

The MCTS iterates through four phases. During *selection*, the algorithm traverses the search space from the root, choosing at each step the child that maximizes a UCB score. Exploitation uses a node’s average reward, whereas exploration relies on the visit count of the connecting edge. Selection stops once a state with untried actions is reached. In *expansion*, one such action is applied, usually modifying a single component of (R, E) to create a new mapping candidate. During *simulation*, the neural predictor estimates the workload latency for this candidate, which is converted into a reward. In *backpropagation*, every node and edge along the selected path updates its visit count and reward value.

Unlike standard tree-based MCTS, our implementation builds a DAG of visited states because many action sequences produce identical mappings. This avoids redundancy and significantly reduces the search space. To prevent cycles (e.g., repeatedly migrating a property back and forth), visited paths are recorded and actions leading to previously seen states are excluded.

Example 7.2. Figure 7.2 illustrates a single MCTS iteration. The root mapping is selected, and a new variant is generated by modifying one model assignment (here, $R[0] = \text{rel}$). The neural predictor estimates its workload latency (68.7 ms), producing a

reward of 1.43, which is then backpropagated to update statistics along the visited path. Iterations continue until the time budget is reached, at which point the best mapping found is recommended. $\square$

7.2.3 Plan-Structured Neural Predictor

During the *simulation* phase, MCTS invokes a learned performance predictor that estimates workload latency for the current mapping. The model extends the *plan-structured neural network* of [53] to support multiple logical models. As illustrated in Fig. 7.3, the predictor is built from *operator-level neural units*, each specialized for a single operator type (e.g., **Projection**, **Selection**, **Join**). These units are dynamically assembled into a network mirroring the query plan. A unit receives an operator’s *feature vector* (red arrow) and a *data vector* (black arrow) from its children and outputs (i) a latency contribution and (ii) a new data vector summarizing intermediate statistics. Computation proceeds bottom-up: leaf units (e.g., **Scan**, **NodeByLabelScan**) use only operator features, internal units concatenate their children’s outputs with their own features, and the root (**ProduceResults**) emits the final latency estimate. The left three trees (blue leaves) correspond to Neo4j operators (e.g., **NodeByLabelScan**, **Expand(All)**), while the right three trees (purple leaves) represent PostgreSQL plans (e.g., **Scan**, **Filter**).

To support multi-model workloads, equivalent operator families across PostgreSQL, MongoDB, and Neo4j are mapped to unified operator types inspired by MMQL translation [15]. The feature extractors normalize numeric parameters (e.g., **Plan Rows**, **Total Cost**), convert categorical fields into vectors, and represent yes/no properties as binary flags. PostgreSQL features come from **EXPLAIN ANALYZE**; MongoDB features from JSON plans; and Neo4j features from **PROFILE**. As Neo4j lacks per-operator timings, internal nodes emit only data vectors, while the root predicts total latency.

Training data are obtained by executing benchmark workloads and recording plans and runtimes. For systems with operator-level timings, the loss aggregates errors across all plan nodes; for Neo4j, it compares only the total latency. The resulting unified model serves as a fast cost oracle, enabling MCTS to evaluate mapping alternatives without executing the workload physically.

7.3 Architecture and Implementation

*MM-mapsearch*⁴ is implemented as a standalone microservice within the *MM-evocat* toolset [82] (see Fig. 7.4), a broader ecosystem for modeling, querying, evolving, and propagating changes in multi-model data. The toolset follows a modular, service-oriented architecture in which components communicate through REST APIs and exchange data using a unified categorical representation. The *MM-mapsearch* service is written in Python and exposes an endpoint that accepts the simplified schema category (Section 7.2), mapping constraints, and a weighted query workload. Based on this input, the service produces a mapping recommendation together with estimated performance characteristics. Due to its declarative

⁴<https://github.com/mmcatdb/aimm>

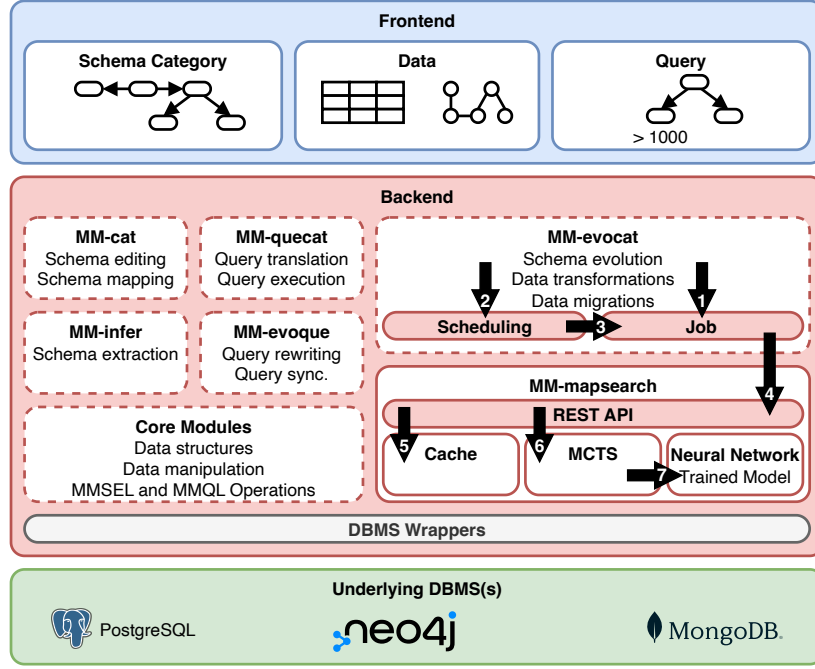


Figure 7.4 Architecture.

interface and reliance on the common representation layer, the advisor can be easily extended to support new logical models, workload types, or integration scenarios within the *MM-evocat* ecosystem.

The microservices cooperate through a request-response workflow. Mapping analysis can be initiated either on demand (1) or through the scheduling component (2), which periodically triggers advisory jobs (3). A job instance communicates with *MM-mapsearch* via its REST API, submitting a request for a new recommendation (4). Before launching a computation, the service consults an internal cache to reuse previously generated results (5), ensuring fast responses when the schema and workload have not changed. Otherwise, *MM-mapsearch* performs a fresh optimization (6-7) and returns the resulting mapping plan to the invoking component, where it is visualized and may be further propagated to schema-management or migration pipelines.

Internally, the service organizes optimization tasks as isolated Python processes. At most one optimization job is executed for a project at a time, preventing conflicting updates or excessive load. The trained neural model used for performance estimation is trained on query plans from PostgreSQL, MongoDB, and Neo4j using the TPC-H benchmark.

7.4 Demonstration Outline

Our interactive demo presents the complete *MM-mapsearch* workflow. Users can start with curated examples (e.g., TPC-H), which include predefined schemas and workloads, or import their own data into PostgreSQL, Neo4j, and/or MongoDB, and define custom properties and queries. Through the interface (Fig. 7.5), they select optimization targets, set mapping constraints (embedding, inlining, reference), assign workload weights, and specify a time budget for the search.

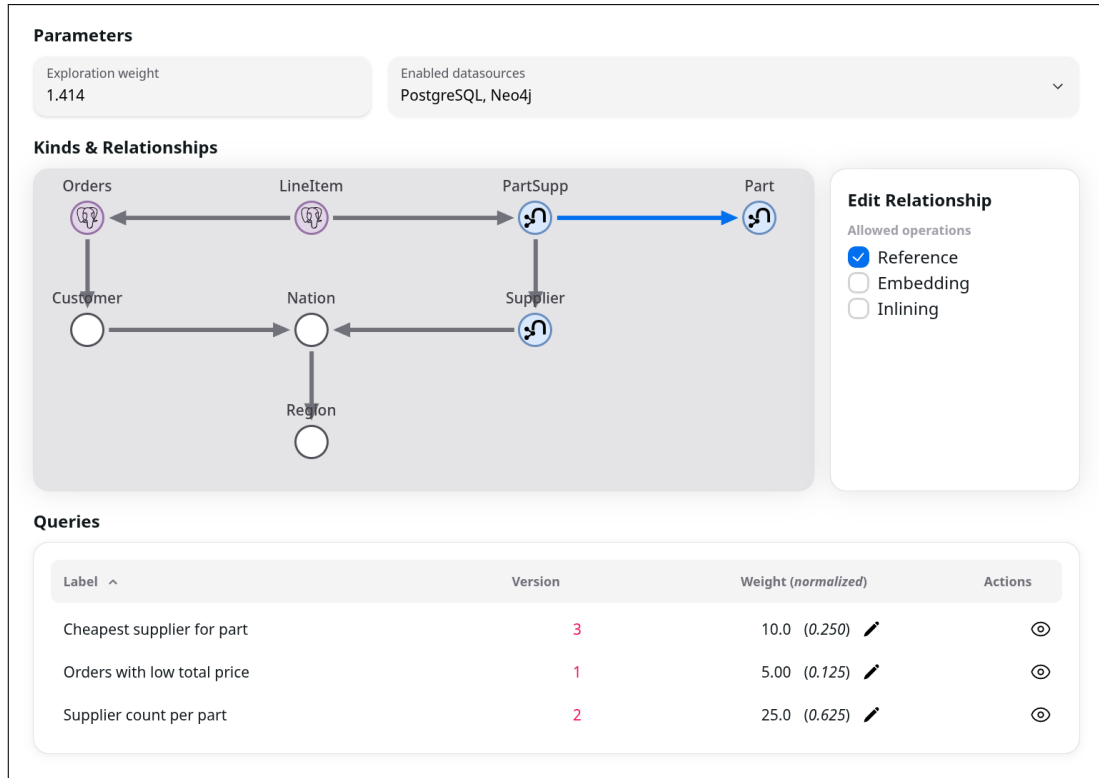


Figure 7.5 Input and workload selection.

During optimization, the system provides continuous interactive feedback (Fig. 7.6). Users can track MCTS progress, inspect the current top candidates, and see how each proposed representation differs from the original. Graphical summaries highlight estimated latency gains, structural reorganization steps, and migration implications.

After the search terminates, or at any time when the user accepts a suggested mapping, the interface enables direct execution or scheduling of data migration. Up to three candidates can be inspected side by side, with detailed statistics that clarify their respective trade-offs. The demo highlights the interactivity and transparency of *MM-mapsearch*, offering a concise yet hands-on exploration of workload-aware multi-model mapping selection.

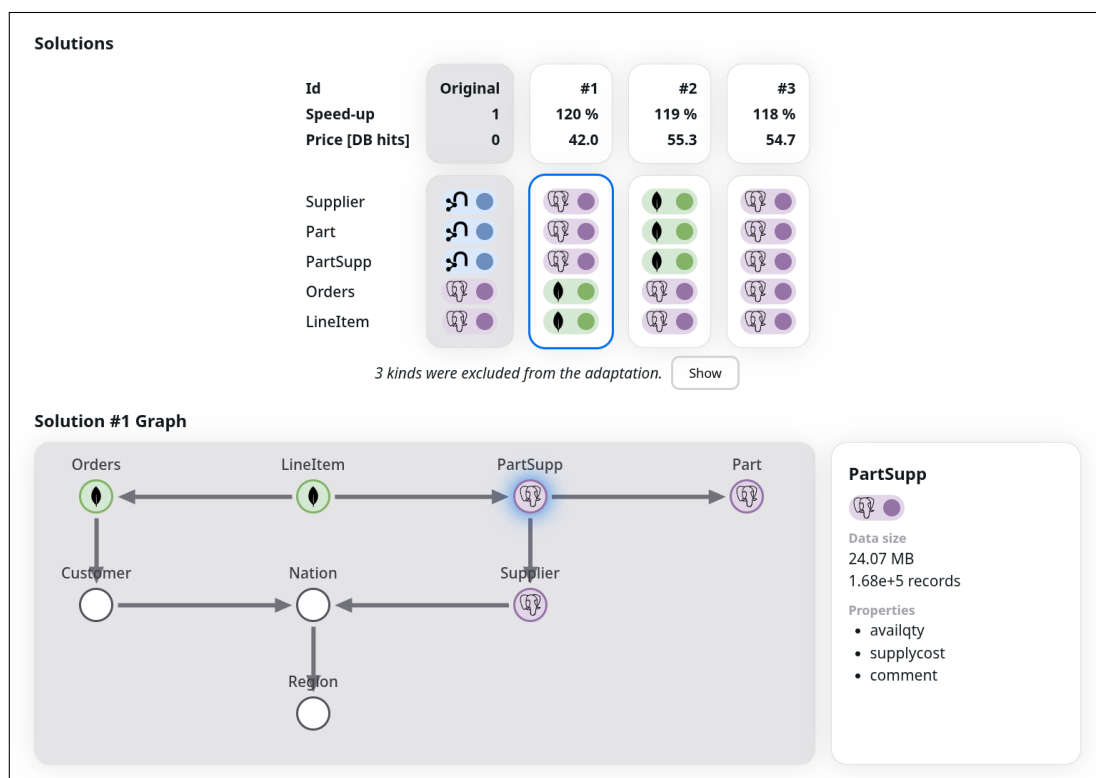


Figure 7.6 Mapping results and statistics.

Paper VIII

Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs

Irena Holubová¹, Alžběta Šrůtková¹, Jáchym Bártík¹

Published as Irena Holubová, Alžběta Šrůtková, and Jáchym Bártík.
“Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs”. In: *Proceedings of ENASE 2025*. Jan. 2025, pp. 174–184. DOI:
[10.5220/0013395300003928](https://doi.org/10.5220/0013395300003928)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

The *variety* characteristic of Big Data introduces significant challenges for verified single-model data management solutions. The central issue lies in managing the *multi-model data*. As more solutions appear, especially in the database world, the need to benchmark and compare them rises. Unfortunately, there is a lack of available real-world multi-model datasets, the number of multi-model benchmarks is still small, and their general usability is limited. This paper proposes a solution that enables creation of multi-model data from virtually any given single-model dataset. We introduce a framework that enables automatic inference of the schema of input data, its user-defined modification and mapping to multiple models, and the data generation reflecting the changes. Using the well-known Yelp dataset, we show its advantages and usability in three scenarios reflecting reality.

8.1 Introduction

Although the traditional relational data model has been the preferred choice for data representation for decades, the advent of Big Data has exposed its limitations in various aspects. Many technologies and approaches considered mature and sufficiently robust have reached their limits when applied to Big Data. One of the most daunting challenges is the *variety* of data, which encompasses multiple types and formats that originate from diverse sources and are inherently adherent to different models. There are structured, semi-structured, and unstructured formats; order-preserving and order-ignorant models; aggregate-ignorant and aggregate-oriented systems; models where data normalization is critical or the redundancy is naturally supported; etc.

The naturally contradictory features of the so-called *multi-model data* introduce an additional dimension of complexity to all aspects of data management, including modelling, storing, querying, transforming, integrating, updating, indexing, and many more. Hence, several multi-model tools for data management have emerged. For example, considering the storage of multi-model data, more than 2/3 of the 50 most widely used database management systems (DBMSs)² now fall under the category of *multi-model* following the Gartner prediction [168] made almost 10 years ago. Unfortunately, no standards exist on which models to combine and how, so each DBMS provides a proprietary solution.

Similarly, there exist *polystores* [37, 38], sometimes denoted as *multi-database systems*. The general idea is that several distinct data management systems (usually single-model) live under a common, integrated schema provided to the user. Polystores can be further classified [169] depending on various aspects, such as the number of query interfaces or the types of underlying systems (homogeneous or heterogeneous), the level of autonomy of the underlying systems, etc. So, again, the variety of choices is wide.

Choosing the optimal tool for the particular use case is highly challenging, considering the range of each area's approaches. Naturally, we need to be able to compare the selected set of tools for *all* target use cases, and benchmarking comes into play. Despite many single-model benchmarks and data generators for

²<https://db-engines.com/en/ranking>

all the common models (see Section 8.2), the shift to the multi-model world is not straightforward. The multi-model test cases must cover the required subset of models and their mutual relations, such as multi-model embedding, cross-model references, or multi-model redundancy. In addition, the variety of use cases grows with the number of distinct models combined. Hence, the number of truly multi-model benchmarks is small, and their versatility and coverage are limited.

In response to this problem, we propose a solution that enables the creation of virtually any possible multi-model data set together with the respective operations. To ensure the data sets have realistic characteristics, we do not utilize the classical approach of exploitation of generators, providing values with a required distribution. Instead, this paper proposes a framework for transforming given single- (or multi-) model data and queries to any possible combination of multi-model data and queries.

Our approach is based on utilizing the toolset we have developed in our research group for various aspects of multi-model data management based on the unifying categorical representation of multi-model data – the so-called *schema category* [13]. This abstract graph representation backed by the formalism of category theory enabled us to propose and develop tools for categorical schema modeling [82], categorical schema inference [16], querying using SPARQL-based query language MMQL [15], or query rewriting [83]. We show that selected features of the tools, when appropriately extended and integrated, can form a framework whose outputs enable the simulation of virtually any multi-model use case.

Outline In Section 8.2 we overview related work. In Section 8.3, we introduce the categorical representation of multi-model data and the tools we utilize in the proposal. In Section 8.4, we introduce the multi-model transformation framework and provide an illustrative example using the Yelp dataset. In Section 8.5, we conclude and outline future steps.

8.2 Related Work

Two main obvious approaches to benchmark data management tools exist. We can use existing, preferably real-world datasets or a data generator that outputs synthetic, pseudo-realistic datasets. Although we can find many representatives of both, most focus on a single selected model. The number of multi-model representatives is very low.

8.2.1 Repositories

Considering the well-known repositories of real-world datasets, the most popular model is relational, reflecting the history and popularity of relational DBMSs. The second most popular model is hierarchical, expressed usually in JSON [138], the main format supported in NoSQL document DBMSs. There are also repositories of graph data, as this model represents specific use cases, hardly captured by the previous two.

The most popular repositories are usually related to research activities. There

are general repositories such as the Kaggle repository³ of datasets for data science competitions (involving, e.g., Titanic survival data), the UCI Machine Learning Repository⁴ for machine learning research (involving, e.g., census data), the IEEE DataPort⁵, or the Harvard Dataverse⁶. The open-access repository Zenodo⁷, developed under the European OpenAIRE program, enables researchers to share datasets and other research outputs. For graph data, there are popular repositories such as the Stanford Large Network Dataset Collection⁸, the Network Data Repository⁹, or the Open Graph Benchmark¹⁰.

The open data movement naturally provides another good source of data. Many governments (e.g., US¹¹, UK¹², EU¹³, etc.) provide open data portals hosting various datasets on demographics, economics, transportation, and public health. Similarly, Amazon Web Services (AWS) host a variety of open datasets¹⁴ that can be accessed and analyzed directly in the cloud.

Various datasets can be found also in GitHub¹⁵, or related projects, such as DataHub¹⁶. Or, one can search the whole Internet, e.g., using the Google Dataset Search¹⁷.

8.2.2 Generators

Often, we cannot easily find a suitable real-world dataset. In that case, we can use a data generator or a comprehensive benchmark with a data generator capable of producing pseudo-realistic datasets with required natural features (e.g., distribution of values or structural features). However, to our knowledge, most existing generators are limited to a single, specific data model or format, or they are constrained to a fixed set of one or a few use cases, each represented by a dataset and related operations. For example, popular benchmarks, such as TPC-H and TPC-DS¹⁸, are naturally focused on the relational data model. Similarly, benchmarks like XMark [60] or DeepBench [61] are tailored to the document data model involving basic NoSQL or path-finding queries. A comprehensive review of purely graph data generators is presented in [63]. For instance, GenBase [170] focuses on the array data model and queries for array manipulation.

Considering multi-model data, only a few representatives fall into this category. BigBench [64] covers semi-structured and unstructured data and the relational data model, but it lacks support for both graph and array data models. UniBench [65]

³<https://www.kaggle.com/>

⁴<https://archive.ics.uci.edu/>

⁵<https://ieee-dataport.org/>

⁶<https://dataverse.harvard.edu/>

⁷<https://zenodo.org/>

⁸<https://snap.stanford.edu/data/>

⁹<https://networkrepository.com/>

¹⁰<https://ogb.stanford.edu/>

¹¹<https://data.gov/>

¹²<https://www.data.gov.uk/>

¹³<https://data.europa.eu/>

¹⁴<https://registry.opendata.aws/>

¹⁵<https://github.com/>

¹⁶<https://datahub.io/>

¹⁷<https://datasetsearch.research.google.com/>

¹⁸<https://www.tpc.org/>

does not support the array data model either, and it considers only a single use case within the benchmark. Finally, M2Bench [66] encompasses relational, document, graph, and array data models. Nevertheless, despite each covered benchmark task involving at least two data models, the benchmark is designed to fit within one of three predefined use cases.

8.3 Categorical View and Management of Multi-Model Data

Multi-model data refers to data represented by multiple interconnected logical models within a single system. The interconnection can be done in several ways:

1. The two (or more) models can be mutually *embedded*. For example, a JSONB column in PostgreSQL¹⁹ enables embedding a JSON document into a relational table.
2. A *reference* can exist between two entities residing in different modes.
3. The same part of data can be represented *redundantly* using multiple models.

Integrating different data models within a larger system, such as a polystore or a multi-model DBMS, allows for using the most appropriate model for specific tasks. For example, structured data with slight variations might best suit the document model. Data with numerous relationships requiring efficient path queries may fit the graph model. Or, rapidly generated data with simple querying needs could be handled by the key/value model.

8.3.1 Categorical Representation of Multi-Model Data

First, to unify the terminology from different models, we use the following terms: A *kind* corresponds to a class of items (e.g., a relational table or a collection of JSON documents), and a *record* corresponds to one item of a kind (e.g., a table row or a JSON document). A record consists of simple or complex *properties* having their *domains*.

To grasp the popular models' specific features, we utilize the so-called *schema category* [13], a unifying abstract categorical representation of multi-model data to manage any possible combination of known models.

Let us first remember the basic notions of category theory. A *category* $\mathbf{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of objects $\mathcal{O}$, set of morphisms $\mathcal{M}$, and a composition operation $\circ$ over the morphisms ensuring transitivity and associativity. Each morphism is modelled as an arrow $f : A \rightarrow B$, where $A, B \in \mathcal{O}$, $A = \text{dom}(f)$, $B = \text{cod}(f)$. And there is an *identity* morphism $1_A \in \mathcal{M}$ for each object A . The key aspect is that a category can be visualized as a multigraph, where objects act as vertices and morphisms as directed edges.

The *schema category* is then defined as a tuple $\mathbf{S} = (\mathcal{O}_{\mathbf{S}}, \mathcal{M}_{\mathbf{S}}, \circ_{\mathbf{S}})$. Each schema object $o \in \mathcal{O}_{\mathbf{S}}$ is internally represented as a tuple $(key, label, superid, ids)$, where *key* is an automatically assigned internal identity, *label* is an optional

¹⁹<https://www.postgresql.org/>

user-defined name, $superid \neq \emptyset$ is a set of attributes (each corresponding to a signature of a morphism) forming the actual data contents a given object is expected to have, and $ids \subseteq \mathcal{P}(superid)$, $ids \neq \emptyset$ is a set of particular identifiers (each modelled as a set of attributes) allowing us to distinguish individual data instances uniquely. Each morphism $m \in \mathcal{M}_{\mathbf{S}}$ is represented as a tuple $(signature, dom, cod, label)$. The explicitly defined morphisms are denoted as *base*, obtained via the composition $\circ_{\mathbf{S}}$ as *composite*. The *signature* allows us to distinguish all morphisms except the identity ones mutually. For base morphism, we use a single integer number. For composite morphism, we use the concatenation of signatures of respective base morphism using the $\cdot$ operation. *dom* and *cod* represent the domain and codomain of the morphism. Finally, $label \in \{ \#property, \#role, \#isa, \#ident \}$ allows us to further distinguish morphisms with semantics “has a property”, “has an identifier”, “has a role”, or “is a”. (We provide explanatory examples in Section 8.4).

8.3.2 Categorical Multi-Model Data-Management Toolset

The schema category (together with its mapping to the underlying models) allows us to seamlessly handle any combination of models and process them independently of the system. When a specific operation needs to be performed at this abstract level, it is passed down to the underlying database system for execution.

During the last couple of years, our research group has developed a family of tools that enable one to manage multi-model data represented using category theory. The tools whose selected functionality we will utilize for our proposed purpose are the following:

- *MM-evocat* [82] enables the manual creation of the schema category representing the conceptual model, its mapping to a selected combination of the logical models, and propagation of further changes in the categorical schema to data instances.
- *MM-infer* [16] enables (semi-)automatic inference of the schema category from sample multi-model data instances.
- *MM-evoque* [83] enables querying over the schema category using the *Multi-Model Query Language* (MMQL) [15], which is based on well-known SPARQL [22] notation. The queries are then decomposed according to the mapping to logical models. The subqueries are evaluated in the underlying DBMSs, and the partial results (if any) are combined to produce the final result. In addition, the changes in the schema category are propagated to the queries.

8.4 Multi-Model Transformation Framework

The original aim of the listed tools is different, and so is their interface and overall functionality. However, if we utilize and extend their selected functionality, integrate the tools thanks to the common categorical representation of multi-model data, and add the respective GUI, we can gain a framework that enables

a user-friendly and efficient way to generate pseudo-realistic multi-model data. On the input, we assume a real-world single-model data set (or, eventually, a synthetic one with reasonable characteristics or a multi-model dataset we want to modify). On the output, we want to get multi-model data created from the input data based on user requirements. Eventually, the users can also provide a query over the input data, and we want to output its respective modification reflecting the data transformation (if it exists). We can identify several scenarios where such a framework is applicable:

- **Scenario A:** The users provide input data with model X , and they want to transform it to model X' .
- **Scenario B:** The users provide input data having model X , and they want to transform its part to model X' and the rest to X'' , whereas a multi-model DBMS that supports both X' and X'' exists.
- **Scenario C:** The users provide input data having model X , and they want to transform its part to model X' and the rest to X'' , whereas none of the DBMSs we consider supports both X' and X'' . So, the data is stored in two DBMSs.

Our framework covers all three scenarios. To explain the ideas, we provide a running example based on a subset of the Yelp Open Dataset²⁰. The data describes Yelp’s businesses, reviews, and user data, all represented using the JSON format.

Example 8.1. Fig. 8.1 involves a part of the input dataset. We can see JSON document collections **User**, **Review**, **Checkin**, **Business**, and **Tip**, i.e., the data represented in the original JSON document model (green). Next to the documents, we can see the initial schema category automatically inferred from the data by *MM-infer*. The green nodes represent the roots of the respective kinds. In the compound brackets, we can see the identifiers of the kinds (e.g., the property `review_id` for kind **Review**, or the pair of properties `user_id`, `business_id` for kind **Tip**). The arrows represent morphisms – in this simple example, only the most common type “has a property” (whose label we omit for simplicity), i.e., leading to simple/complex properties of the kinds.

As we can see, the quality of the initial schema category is limited by the input data quality, the input model’s specific features, and the capabilities of automatic schema inference of *MM-infer*. In particular, the properties denoted with red color bear values of identifiers of various kinds, as they probably represent the respective references. E.g., kind **Review** is identified by `review_id`, but it also involves `user_id` of the user who created the review and `business_id` of the reviewed business. This cannot be captured using JSON, but we want to capture this information in the schema category and use it later. Similarly, kind **User** has a set of properties (denoted with pink color) that have the same (in this case simple) structure and semantics (as we can guess from their names `compliment_*`) and differ only in type. And there might be lots of such properties. So, at the categorical (conceptual) level, expressing them as a single property with a particular type might make more sense and can be represented better in another logical model. $\square$

²⁰<https://www.yelp.com/dataset>

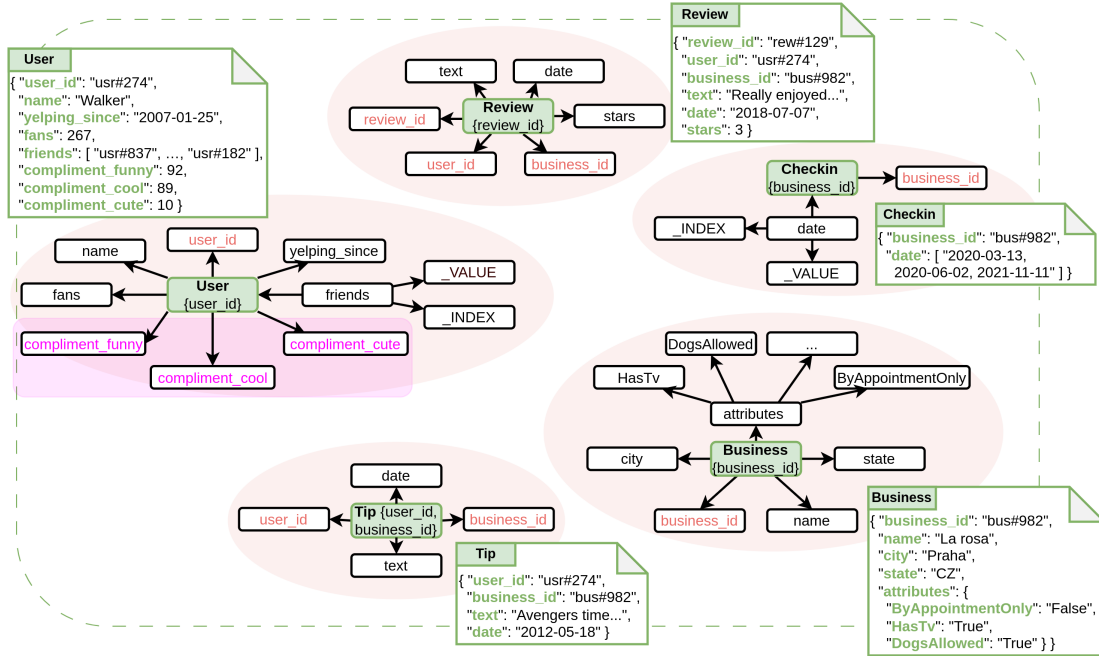


Figure 8.1 Input single-model JSON document collections and inferred initial schema category

Example 8.2. Fig. 8.2 depicts the situation after the users visualized the initial schema category in *MM-cat* and edited it using its extension *MM-evocat* to solve the issues.²¹ First, the users replaced repeating occurrences of properties `business_id` and `user_id` and expressed the references using the morphisms with the respective direction (the new morphisms are emphasized with dotted arrows). Second, the properties of kind `User` that are structurally and semantically equivalent were merged and transformed into a single property with a respective property `_TYPE`. □

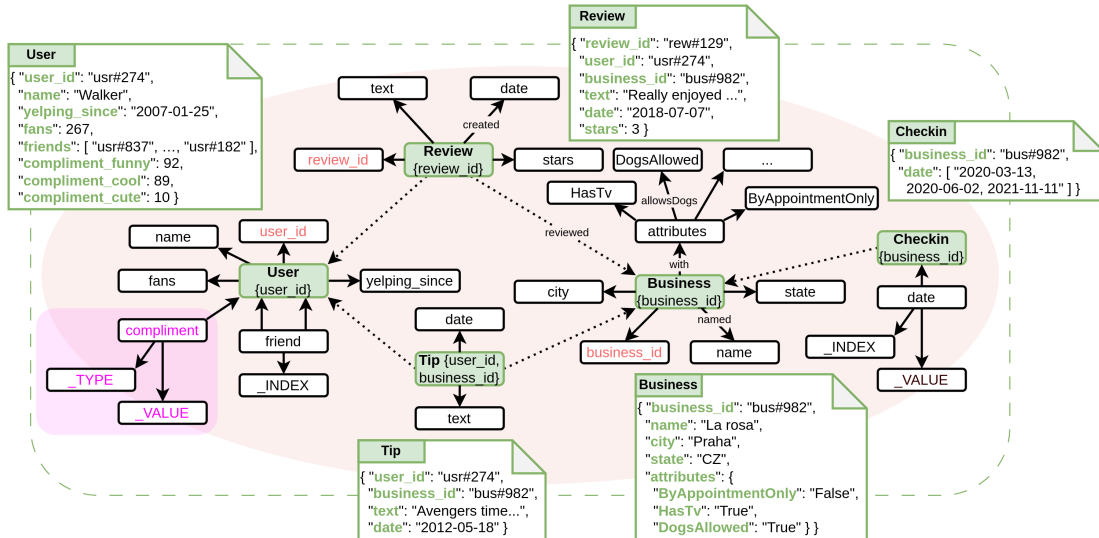


Figure 8.2 Edited (improved) schema category from Fig. 8.1

Example 8.3. Having the edited schema category, we can use *MM-evocat* again to modify the mapping (initially to the input document model). Following **scenario A**, we

²¹Some issues can be solved in *MM-infer* (semi-)automatically, we use them just for illustration.

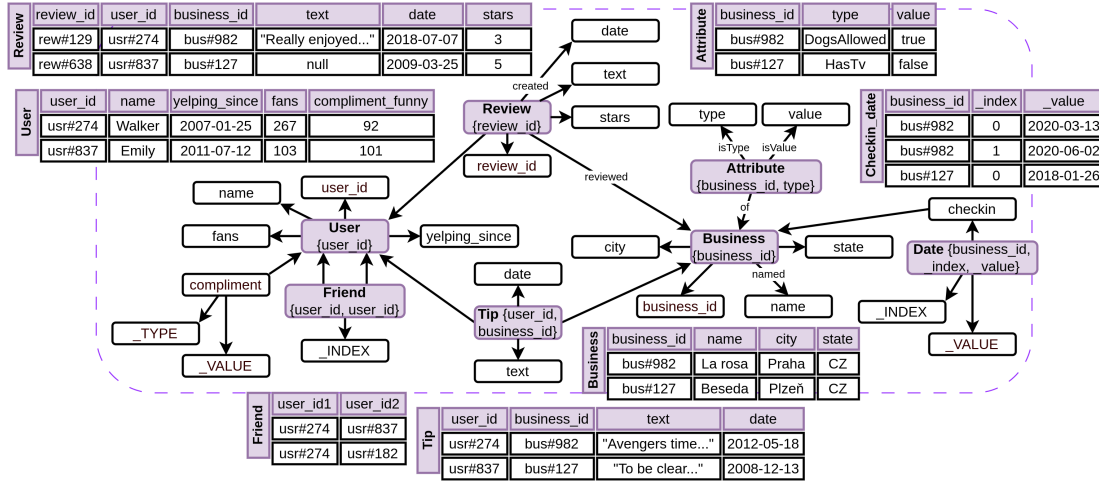


Figure 8.3 Schema category from Fig. 8.2 mapped to the relational model (scenario A)

want to transform all the JSON document data into the relational model. The situation is depicted in Fig. 8.3, where the users changed the mapping of the whole schema category to the relational model (violet). Namely, the original kinds **User**, **Review**, **Business**, and **Tip** were mapped to the respective relational tables instead of JSON collections. Regarding the features of the relational models, also the property **friend** of kind **User** and property **date** of kind **Checkin** had to be mapped to separate kinds **Friend** and **Date** (and, therefore, to respective separate relational tables). Similarly, the property **attribute** of kind **Business** was mapped to a map of attributes and thus a separate table. □

Example 8.4. Following **scenario B**, the users might find out that transforming all the data to the relational model is not optimal, and they decide to use the best of both worlds. As depicted in Fig. 8.4, they kept the mapping of kinds **User** and **Friend** to the relational models, each to a separate table, like in Fig. 8.3. They also want to keep a mapping of kinds **Review** and **Tip** to the relational model but to merge them into a single table because **Tip** is just a subset of **Review**. So, they create a new kind **Comment** that covers both of them and map it to a single table. The new kind requires property **comment_id**, which we can reuse (for records of kind **Review**) or generate by a simple algorithm (for records of kind **Tip**).

Finally, they decided to embed the kinds **Attribute** and **Date**, which required separate relational tables, to the relational table of kind **Business**. So, they mapped them to the document model and embedded them to the kind **Business**. (Such a combination of models is supported, e.g., in PostgreSQL.) This transformation reduced the overhead of joining the same tables each time while keeping the kind **Business** mapped to the relational model.

Fig. 8.4 depicts the result, where we get truly multi-model data represented in two logical models – violet relational and green document. □

Example 8.5. Finally, following **scenario C** and as depicted in Fig. 8.5, the users might further transform the multi-model data from a combination of two to a combination of three logical models and map the kind **Comment** to the wide-column model (red). This model is better suited for frequent data analysis, i.e., the type of queries the users might want to do with the comments. It also more naturally represents that tips do not have all the attributes of reviews. □

So, as we can see, by using the framework, it is very simple to transform the

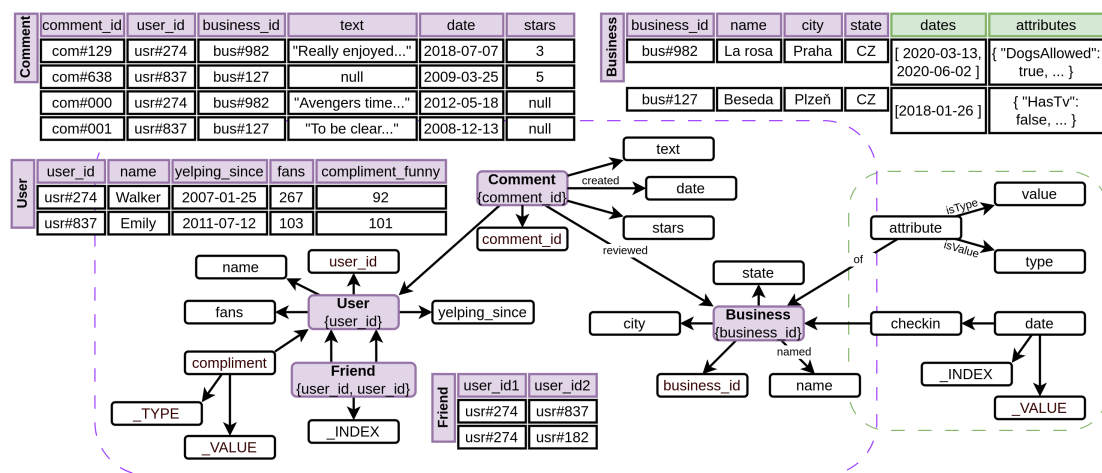


Figure 8.4 Schema category from Fig. 8.2 mapped to relational and document model (scenario B)

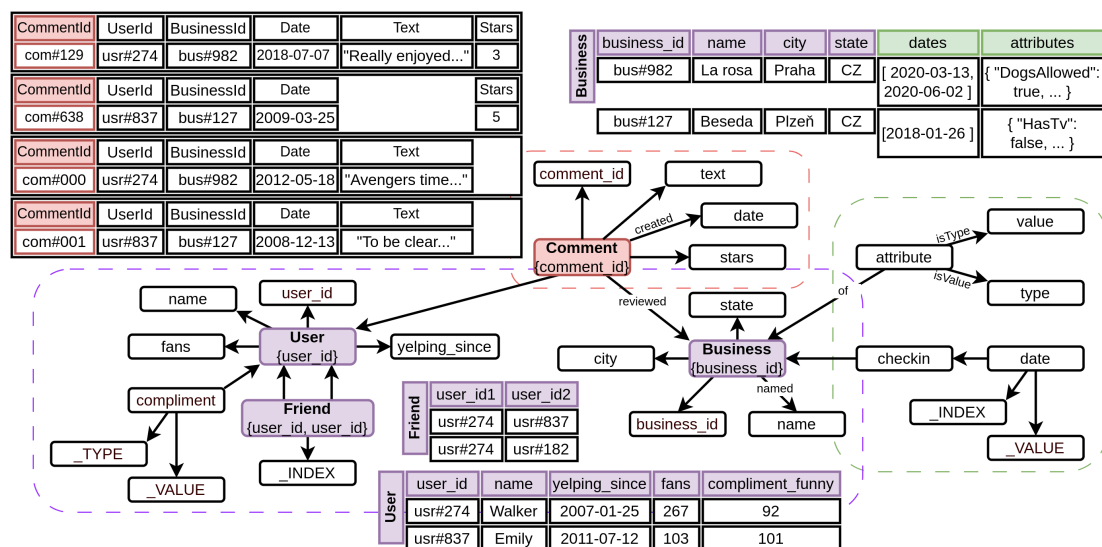


Figure 8.5 Schema category from Fig. 8.4 mapped to relational, document, and wide-column model (scenario C)

input data to any multi-model data only by modification of the schema category and its mapping to the logical models. Nevertheless, we may also want a similar functionality for the queries. Extending the framework further with *MM-quecat* makes it possible to query over the schema category using MMQL [15], a graph query language utilizing the SPARQL notation to query over the schema category. Depending on the specified mapping of the schema category to the logical models, the MMQL query can be translated using *MM-quecat* to be evaluated in the underlying DBMS. But, for our purposes, instead of querying, we only retrieve the query with the transformed data and use it for benchmarking.

Example 8.6. For example, the users may want to query for “*names of businesses which have been reviewed since January 1st, 2023 and allow dogs*”. Its expression in MMQL over the improved schema category in Fig. 8.2 is provided in Fig. 8.6. If the input data in Fig. 8.2 were stored in MongoDB²², its translation to MongoDB QL is provided in Fig. 8.7. □

```
SELECT {
  ?business name ?name .
}
WHERE {
  ?business -reviewed/created ?date ;
    with/allowsDogs "true" ;
    named ?name .
  FILTER(?date > "2023-01-01")
}
```

Figure 8.6 MMQL query over the improved schema category in Fig. 8.2

```
db.review.aggregate([
  { $match: {
    date: { $gt: ISODate('2023-01-01') }
  } },
  { $lookup: {
    from: "business",
    localField: "business_id",
    foreignField: "business_id",
    as: "business"
  } },
  { $match: {
    attributes: { DogsAllowed: true }
  } },
  { $project: {
    _id: 0,
    name: "$business.name"
  } },
])
```

Figure 8.7 MongoDB QL query over data from Fig. 8.2

If we change the mapping to another model (or a combination of models) represented in another DBMS (or multiple DBMSs), we get the query expressed using the respective query language(s). In addition, if we change the part of the schema category accessed by the query, the modification of MMQL is ensured along with the modification of the mapping.

²²<https://www.mongodb.com/>

Example 8.7. When we unify the business attributes to a map, as depicted in Fig. 8.3, the MMQL query is modified to reflect the change, as depicted in Fig. 8.8. In addition, in Fig. 8.3, we also changed the mapping to the relational model (**scenario A**). Assuming that now the data is stored in PostgreSQL, the respective mapping to the relational model ensures the translation of MMQL query to the SQL query provided in Fig. 8.9. $\square$

```
SELECT {
  ?business name ?name .
}
WHERE {
  ?business -reviewed/created ?date ;
    -of ?attribute ;
    named ?name .
  ?attribute isType "DogsAllowed" ;
    isValue "true" .
  FILTER(?date > "2023-01-01")
}
```

Figure 8.8 MMQL query over schema category from Fig. 8.3

```
SELECT business.name AS name
FROM business
JOIN review ON business.business_id = review.business_id
JOIN attribute ON business.business_id = attribute.business_id
WHERE review.date > '2023-01-01'
AND attribute.type = 'DogsAllowed'
AND attribute.value = true
```

Figure 8.9 SQL query over data from Fig. 8.3 (scenario A)

Example 8.8. If we use the combination of the document and relational model (**scenario B**) depicted in Fig. 8.4, we can assume that the data is still stored in PostgreSQL. As SQL in PostgreSQL is extended towards the support of cross-model queries over both relational and document data, i.e., SQL/JSON, the evaluation process again translates the MMQL query to a single, this time cross-model query, as depicted in Fig. 8.10. Note that despite the mapping change, the parts of the schema category accessed by the MMQL query remain untouched, so the MMQL query remains the same. $\square$

```
SELECT business.name AS name
FROM business
JOIN comment ON business.business_id = comment.business_id
JOIN attribute ON business.business_id = attribute.business_id
WHERE comment.date > '2023-01-01'
AND attributes->>'DogsAllowed' = 'true'
```

Figure 8.10 SQL/JSON query over data from Fig. 8.4 (scenario B)

Example 8.9. Finally, suppose we use a combination of models unsupported by a single multi-model DBMS (**scenario C**) depicted in Fig. 8.5. In that case, the evaluation consists of the decomposition of the query to two subqueries for the respective subsystems – SQL for PostgreSQL and, e.g., CQL for Apache Cassandra²³ – as depicted in Fig. 8.11. Thus, we can also test a family of DBMSs, together with the need to use an additional

²³<https://cassandra.apache.org/>

tool to merge the results. However, because the schema category did not change, the MMQL query stays the same again. $\square$

```
SELECT business_id
FROM comment
WHERE date > '2023-01-01'

SELECT name AS name
FROM business
WHERE business_id IN (/* CQL query result */)
AND attributes->>'DogsAllowed' = 'true'
```

Figure 8.11 CQL and SQL queries over data from Fig. 8.5 (scenario C)

8.4.1 Architecture

Fig. 8.12 provides the schema of the architecture of the proposed framework. In general, we utilize selected parts of the functionality of the existing and verified tools, extend them, integrate them, and roof the whole framework with a GUI to create the target framework. The expected work with the framework is as follows:

1. The users provide the input single-model data to be transformed. The data can be stored in one of the supported DBMSs or provided in files.
2. *MM-infer* parses the data and infers a schema that a new schema conversion module transforms to the initial schema category.
3. The users can modify the schema category depending on their requirements. The users can change the mapping of the schema category to selected combinations of logical models, or they can also change the structure of the schema category itself. When the modification is finished, *MM-evocat* transforms the data according to the new mapping.
4. In addition, the users can specify an MMQL query, which is updated using *MM-evoque* according to the changes in the mapping or the schema category to reflect the changes.

8.4.2 Evaluation of the Proposed Solution

Table 8.1 provides an overview of the advantages of framework utilization compared to manual data/query transformation. On average, depending on the complexity of the data, it is much faster. The framework enables us to infer the initial schema category and, thus, get the overall view of the data structure quickly. Also, all special cases and outliers are immediately provided to the users in a visual form. Also, the specification of the requested output is fast, and the transformation is performed automatically without the need to know the specific features of the underlying systems.

Of course, we assume the framework supports all the required systems for which we want to create the testing data. However, integrating a new DBMS is simple, as it only requires implementing a respective wrapper. Once we have

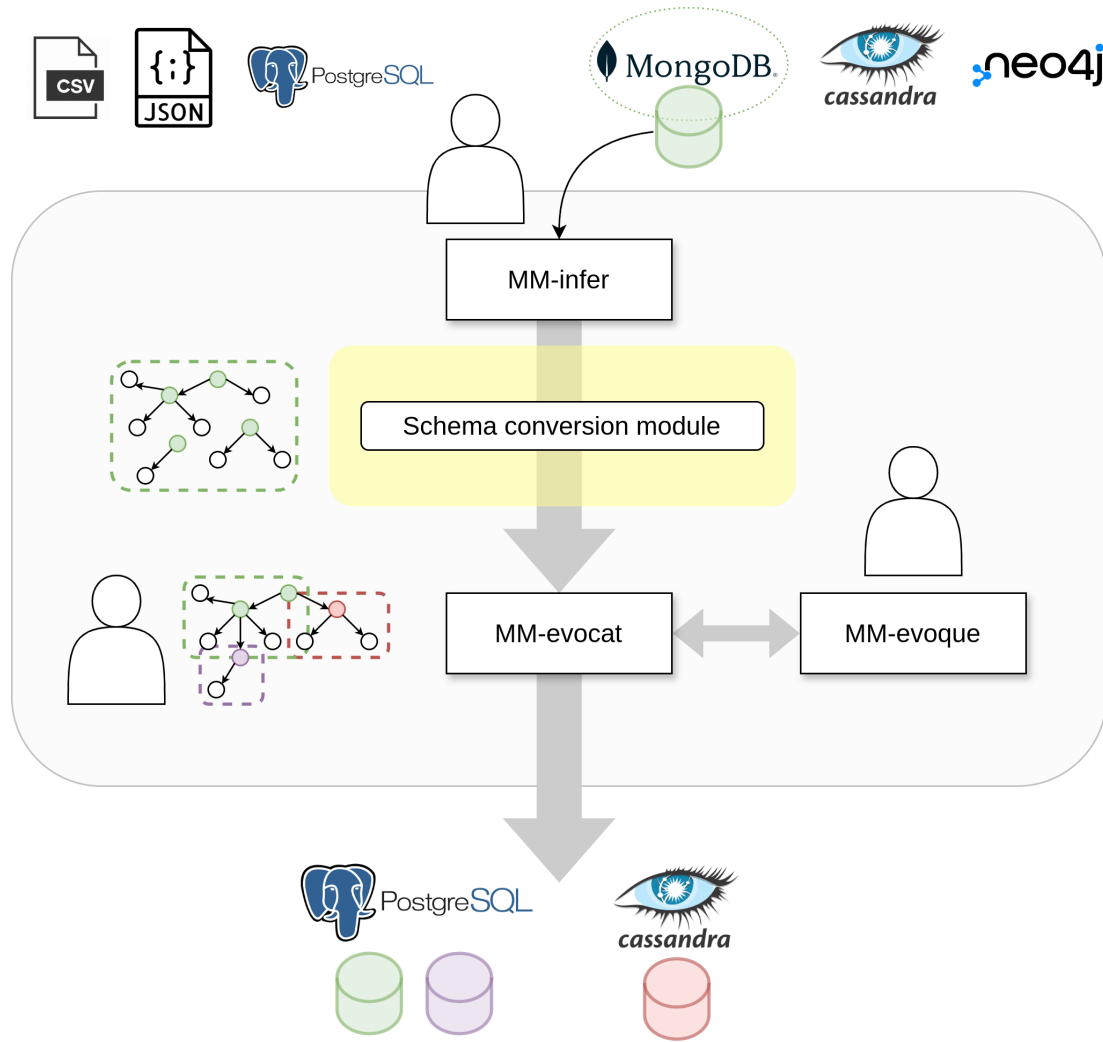


Figure 8.12 Architecture of the framework

it, we do not need to implement any transformation script, and we can express the modification only by interacting with the framework tools. Consequently, we avoid numerous user-defined errors, as the users are shielded from the technical details. Thus, we do not require an expert familiar with the specifics of various DBMSs.

The flexibility of the framework compared to manual data transformation is not limited. As mentioned above, although the framework currently supports MongoDB, PostgreSQL, neo4j, Apache Cassandra, JSON files, or CSV files, new DBMSs and data formats can be easily added using wrappers.

Finally, Table 8.2 illustrates the time required for user interactions across scenarios A, B, and C depicted in Figs. 8.3, 8.4, and 8.5 when processing the Yelp dataset, comparing the conventional manual approach versus using the proposed framework tool. The framework streamlines the workflow by automating every step of the process. In Step 1, it infers the schema from the data. Following this, the framework facilitates editing the schema in Step 2 by providing a user-friendly interface that allows users to make necessary adjustments with minimal effort. In Step 3, it supports creating custom mappings between different data models. It then moves on to generate multi-model data in Step 4. Finally, it

Metric	Without framework	Using framework
Time Required (hours)	10+ (estimation)	0.5 (estimation)
Lines of Code	200+	0
Potential for Errors	High (coding, manual transformation)	Low (tool has been tested)
User Expertise Required	Advanced	Beginner / Intermediate
Flexibility / Customization	High	High

Table 8.1 Comparison of approaches using different metrics

Scenario	Step	Without framework (min)	Using framework (min)	Difference (min)
A	Step 1	120	5	+115
	Step 2	120	10	+110
	Step 3	120	12	+108
	Step 4	180	2	+180
	Step 5	180	0	+180
B	Step 3	180	12	+168
	Step 4	240	2	+238
	Step 5	240	0	+240
C	Step 3	240	16	+234
	Step 4	300	2	+298
	Step 5	300	0	+300

Table 8.2 User interaction needed in particular scenarios for the Yelp dataset

translates queries to operate across different data models in Step 5. The results demonstrate a substantial reduction in the time required for each step when using the framework, highlighting its efficiency and effectiveness in reducing user input and eventual errors.

8.5 Conclusion

This paper proposes a solution to the problem of lack of real-world multi-model data (and the respective queries). We use a different approach instead of the common strategy of generating a synthetic dataset despite having numerous realistic features. Using a specific utilization of our previously created toolset, we introduce the idea of a transformation framework that can transform a given, preferably real-world, dataset into a preferred multi-model dataset. Using a well-known dataset, Yelp, we demonstrate the advantages and applicability of the idea.

Our future work will focus primarily on implementing a common interface that will cover the whole functionality of the proposed framework and simplify the integration of the tools. In addition, we want to focus on the *simulation of the evolution* of the resulting datasets, either through user specification or through the detection of changes in the input single-model data or operations. Lastly, we want to create a *repository of the resulting multi-model datasets* to provide a robust source of test cases to be immediately used. We also want to perform *extensive experiments with the datasets* to provide unbiased benchmarking results for elected multi-model databases.

Paper IX

TransforMMer: A Universal Multi-Model Data Generator

Jáchym Bártík¹, Alžběta Šrůtková¹, Irena Holubová¹

Published as Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová.
“TransforMMer: A Universal Multi-Model Data Generator”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 1150–1153. DOI:
[10.48786/edbt.2025.111](https://doi.org/10.48786/edbt.2025.111)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

The **V**ariety aspect of Big Data poses challenges for most established single-model data management solutions. However, as multi-model systems increase, especially in the data-management domain, the demand for benchmarking and comparison grows. However, real-world multi-model datasets are scarce, and existing benchmarks are limited in number and versatility. This demo paper introduces *TransforMMer*, a tool to generate pseudo-realistic multi-model data from given single-model (or multi-model) datasets. It is based on a pipeline leveraging a subset of our multi-model data management tools to automate schema inference, enable user-defined schema modifications, and generate data reflecting these changes. In the demonstration, we will introduce the tool, the repository of created multi-model datasets, their challenges, and future extensions.

9.1 Introduction

The traditional relational data model has long been the standard for data representation, but Big Data has revealed its limitations. A key challenge is the **V**ariety of Big Data, which encompasses multiple types and formats that originate from diverse sources and are inherently adherent to different models. There are structured, semi-structured, and unstructured formats; order-preserving and order-ignorant models; aggregate-ignorant and aggregate-oriented systems; etc. The naturally contradictory features of the so-called *multi-model data*, combining distinct models with contradictory features, complicates all aspects of data management, from modelling to querying.

Many tools now support multi-model capabilities – e.g., more than two-thirds of the top 50 database management systems (DBMSs)² align with Gartner’s decade-old prediction of support for multiple models [168]. However, the lack of standards for combining models leaves each DBMS with proprietary solutions. Even the variety of query languages [11] highlights the state of multi-model data systems.

Selecting the optimal tool for a specific use case is challenging, and extensive benchmarking is essential to compare tools across all target use cases. While single-model benchmarks and data generators exist for standard models, their adaptation to the multi-model paradigm is complex. Multi-model test cases must account for numerous combinations of different models using various strategies such as embedding, cross-model references, or redundancy. And this limits the availability and versatility of truly comprehensive multi-model benchmarks. [7, 65]

Let us illustrate the problem with an example. We are building a web application that depends on various types of data. When it comes to selecting a database solution, we face a dilemma. We want the reliability of a relational database (e.g., for products and orders), but we also need to be able to perform complex graph queries (e.g., about the users, their mutual connections, and interactions with our products). We can leverage a multi-model database to store such data; however, we are uncertain whether this approach is more efficient than storing everything in a single database system. Many single-model datasets are similar to our expected

²<https://db-engines.com/en/ranking>

data, so we can use them to benchmark the performance of the single-database solution. However, there are very few multi-model datasets available, nor there are tools for generating them.

To address this challenge, we introduce *TransformMer*, a tool for generating virtually any multi-model dataset. Instead of relying on traditional value generators with predefined distributions, our approach uses real-world single-model (or multi-model) datasets as input. It infers a schema of the input data, enables its user-specified modification to reflect the required combination of models, and transforms the output to the respective form. It also supports redundancy and versioning, which further increase its versatility. For its implementation, we integrate and extend selected tools from our toolset based on the unifying categorical representation of multi-model data [13]. Combining and extending tools for modelling and transformations [12], schema inference [16], and evolution management [82] with appropriate integration modules and interfaces allows us to ensure the transformation universally and correctly.

Outline In Section 2, we review related work. Section 3 introduces the categorical representation of multi-model data. Section 4 introduces *TransformMer* and Section 5 its demonstration.

9.2 Related Work

Two main approaches to benchmarking data management tools involve real-world data sets or synthetic data generators. Many focus on single models, but there are very few multi-model options.

Real-world datasets are primarily relational due to the dominance of relational DBMSs, accompanied by hierarchical models (e.g., JSON for NoSQL DBMSs) and graph data. Popular sources include Kaggle³ or Harvard Dataverse⁴. Graph data can be found in collections like SNAP⁵. Governments (e.g., US⁶, EU⁷, etc.) provide open data portals involving distinct data formats. Moreover, various datasets can also be found on GitHub or Google Dataset Search.

However, often we cannot easily find a suitable real-world dataset, and we need to use a data generator or benchmark. However, most are limited to specific data models, formats, or fixed use cases. For example, TPC-H and TPC-DS⁸ focus on the relational model. Benchmarks such as XMark [60] are tailored to the document model. And also, many graph data generators exist [63].

Regarding multi-model data, BigBench [64] covers semi-structured and unstructured data, but lacks support for graph and array data models. UniBench [65] does not support the array data model either, and it considers only a single use case. M2Bench [66] encompasses relational, document, graph, and array data models but also involves only three predefined use cases.

³<https://www.kaggle.com/>

⁴<https://dataverse.harvard.edu/>

⁵<https://snap.stanford.edu/data/>

⁶<https://data.gov/>

⁷<https://data.europa.eu/>

⁸<https://www.tpc.org/>

9.3 Categorical Model of Data

First, to unify the terminology from different models, we use the following terms: A *kind* corresponds to a class of items (e.g., a relational table or a collection of JSON documents). A *record* corresponds to one item of a kind (e.g., a table row or a JSON document). A record consists of simple or complex *properties* having their *domains*.

We also recall the basic notions of category theory. A *category* $\mathbf{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of objects $\mathcal{O}$, set of morphisms $\mathcal{M}$, and a composition operation $\circ$ over the morphisms ensuring transitivity and associativity. Each morphism is modelled as an arrow $f : A \rightarrow B$, where $A, B \in \mathcal{O}$, $A = \text{dom}(f)$, $B = \text{cod}(f)$. And there is an *identity* morphism $1_A \in \mathcal{M}$ for each object A . A category can be visualized as a multigraph, with objects represented as vertices and morphisms as directed edges.

The core and the integration point of all our tools is an abstract representation of multi-model data called *schema category* [13]. It is defined as a tuple $\mathbf{S} = (\mathcal{O}_{\mathbf{S}}, \mathcal{M}_{\mathbf{S}}, \circ_{\mathbf{S}})$. *Objects* in $\mathcal{O}_{\mathbf{S}}$ correspond to the domains of the ER model’s entity types, attributes, and relationship types. Each schema object $o \in \mathcal{O}_{\mathbf{S}}$ is internally represented as a tuple $(\text{key}, \text{label}, \text{superid}, \text{ids})$, where *key* is an automatically assigned internal identity, *label* is an optional user-defined name, $\text{superid} \neq \emptyset$ is a set of attributes (each corresponding to a signature of a morphism) forming the actual data contents a given object is expected to have, and $\text{ids} \subseteq \mathcal{P}(\text{superid})$, $\text{ids} \neq \emptyset$ is a set of particular identifiers (each modeled as a set of attributes) allowing us to distinguish individual data instances uniquely. *Morphisms* connect appropriate pairs of objects. Each morphism $m \in \mathcal{M}_{\mathbf{S}}$ is represented as a tuple $(\text{signature}, \text{dom}, \text{cod}, \text{label})$. The *signature* allows us to distinguish all morphisms except the identity ones mutually. *dom* and *cod* represent the domain and codomain of the morphism. Finally, $\text{label} \in \{ \# \text{property}, \# \text{role}, \# \text{isa}, \# \text{ident} \}$ allows us to further distinguish morphisms with semantics “has a property”, “has an identifier”, “has a role”, or “is a”. The explicitly defined morphisms are denoted as *base*, obtained via the composition $\circ$ as *composite*.

The decomposition of a schema category $\mathbf{S}$, eventually partial or overlapping, is defined via a set of *mappings*. For each kind, the mapping specifies where and how its records are stored in a selected single-/multi-model DBMS using a so-called *access path*, which recursively describes the structure of a kind.

For example, in Fig. 9.3, we can see a sample schema category with kinds *User*, *Business* and *Tip* having root objects denoted with green. They are mapped to JSON document collections from Fig. 9.1. For simplicity, we omit the most common edge label “has a property” and replace uninteresting properties with dots in an oval.

9.4 TransforMMer

To solve the indicated open problems, *TransforMMer* enables the transformation of virtually any input dataset (single- or multi-model) to a required output multi-model dataset.

9.4.1 Sample Scenario

Let us consider a simple scenario to demonstrate the purpose and advantages of *TransforMMer*. We want to use the subset of the Yelp Open Dataset⁹, namely the data that describes businesses, tips, and users, represented using the JSON format as depicted in Fig. 9.1.

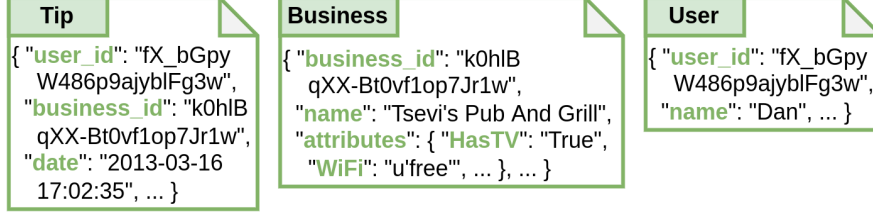


Figure 9.1 Input Yelp dataset in the document model

We want to transform the data into a combination of the relational and graph models, e.g., the one depicted in Fig. 9.2.

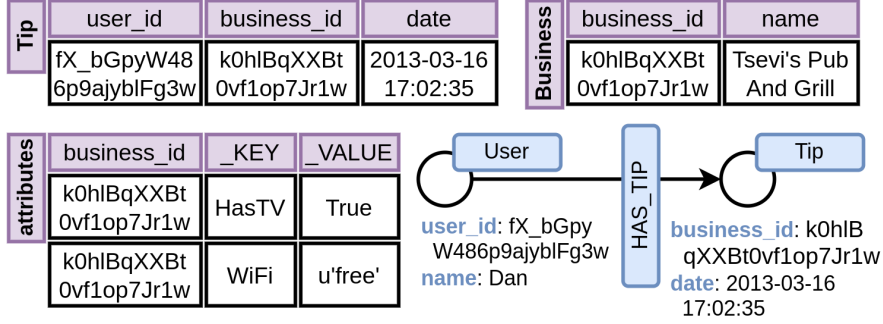


Figure 9.2 Yelp dataset in the graph and relational model

Initial Schema Category

TransforMMer first infers the *initial schema category* S_{ini} depicted for the input data in Fig. 9.3. As we can see, each original JSON document corresponds to one kind in the schema category, namely *User*, *Business*, and *Tip*.

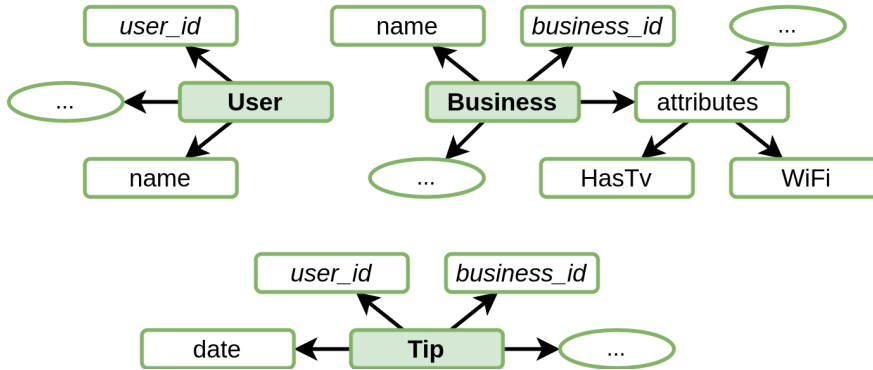


Figure 9.3 Initial schema category S_{ini} inferred from the sample data in Fig. 9.1

⁹<https://www.yelp.com/dataset>

Improved Schema Category

As $\mathbf{S}_{\text{ini}}$ is very simple and even incorrect, *TransforMMer* enables its improvements. During the process of inference of $\mathbf{S}_{\text{ini}}$, it also infers a list of candidates for basic integrity constraints (identifiers and references), clustering, or recursion. The user can confirm or refuse them or add his/her suggestions to create an improved schema category $\mathbf{S}_{\text{imp}}$.

Identifiers First, we need an identifier for each kind to create a correct schema category. The inference process of *TransforMMer* creates a list of simple properties that are candidate identifiers, i.e., they have a unique and compulsory value in each respective record. The user can select the correct ones from them. The candidates do not include composite identifiers, as finding all possible combinations would be too computationally expensive. However, the user can define composite identifiers manually.

In the sample $\mathbf{S}_{\text{ini}}$ in Fig. 9.3, kind *Business* has only one candidate identifier *business_id*, so we choose it as its identifier. Kind *User* also has only one candidate identifier *user_id*. Kind *Tip* does not have any (simple) candidate identifier, so we create a composite identifier consisting of properties *user_id* and *business_id*. (The identifiers are depicted in Fig. 9.4 in curly brackets in root nodes of the kinds.)

References Next, we can define references between the kinds to the respective identifiers. *TransforMMer* also automatically infers candidates for the references (based on the inclusion of values of the referenced properties).

In the sample $\mathbf{S}_{\text{ini}}$ in Fig. 9.3, we identify a reference from kind *Tip* to kinds *User* and *Business* based on the values of properties *user_id* and *business_id*. The result is depicted in Fig. 9.4, where the two objects *user_id* were merged into one. The same happened for objects *business_id*. However, the primary identifier of kind *Tip* is still the same; it is just represented by different schema morphisms.

Clustering of Properties Further improvements in the schema category involve the integration of various more complex constructs. One of them is replacing (huge) clusters of properties with maps. According to our analyses of real-world data, they may often contain multiple properties which have the same structure but different names (such as the property *attributes* of kind *Business* depicted in Fig. 9.3). They make the schema unnecessarily complex, so *TransforMMer* allows the user to transform them into a single set of key-value pairs. The user must just select all the properties to be represented as a map.

The result is depicted in Fig. 9.4. As we can see, the direction of morphism between objects *Business* and *attributes* changed. This is because initially, the object *attributes* corresponded to a complex property containing all the attributes, but now, it represents an array of key-value pairs. So, the cardinality specified by the morphism changed, and the morphism had to be updated (for details, see [13]).

Recursion Similarly to clustering of properties, i.e., generally discovering a repeating pattern, we can also identify the recursion, i.e., a pattern occurring not

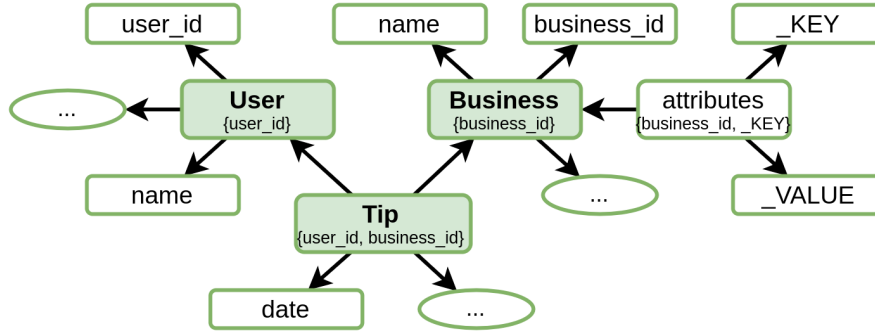


Figure 9.4 The improved schema category $\mathbf{S}_{\text{imp}}$ of the Yelp data containing identifiers, a reference, and a map of properties

within the same parent property but mutually nested. For example, consider a complex property *comment* consisting of properties *author*, *content*, and *replies*, where property *replies* is an array of properties *comment*. Such structure can be repeated in the inferred schema as many times as is the maximum depth of recursion in the source data.

We can significantly simplify this schema by defining a pattern for the property *comment*. *TransformMer* then creates only a single complex property for each unique property in the pattern and a morphism from property *replies* to property *comment*. Thus, the resulting schema category contains a cycle, allowing infinite recursion. An example of such a transformation is depicted in Fig. 9.5, which simultaneously provides sample screenshots of *TransformMer*.

Mapping Modification

Having the final improved schema category $\mathbf{S}_{\text{imp}}$ (see Fig. 9.4), the user can now define its new mapping. As depicted in Fig. 9.6, for example, the original document model (green) can be entirely discarded, and instead of it, the user may define four new kinds. Kind *User* is mapped to the graph model (blue), where each user represents a node in the graph, and edges represent mutual friendship. Kinds *Business*, *Tip*, and *attributes* are mapped to the relational model (purple), i.e., tables. Considering the relational model without arrays, the kind *attributes* had to be established. The output data are depicted in Fig. 9.2.

9.4.2 Workflow

The whole process of transforming the input (single- or multi-model) data to required multi-model data is depicted in Fig. 9.7.

Step 1 First, the users need to specify input data sources. *TransformMer* supports the input from a file system, a DBMS, or their combination. It then infers the schema of each kind (table, collection, etc.) in the data sources. The output of this step is $\mathbf{S}_{\text{ini}}$ and its mapping to the logical structures of the original data. The algorithm also infers candidates for identifiers, references between the kinds, and repeating patterns.

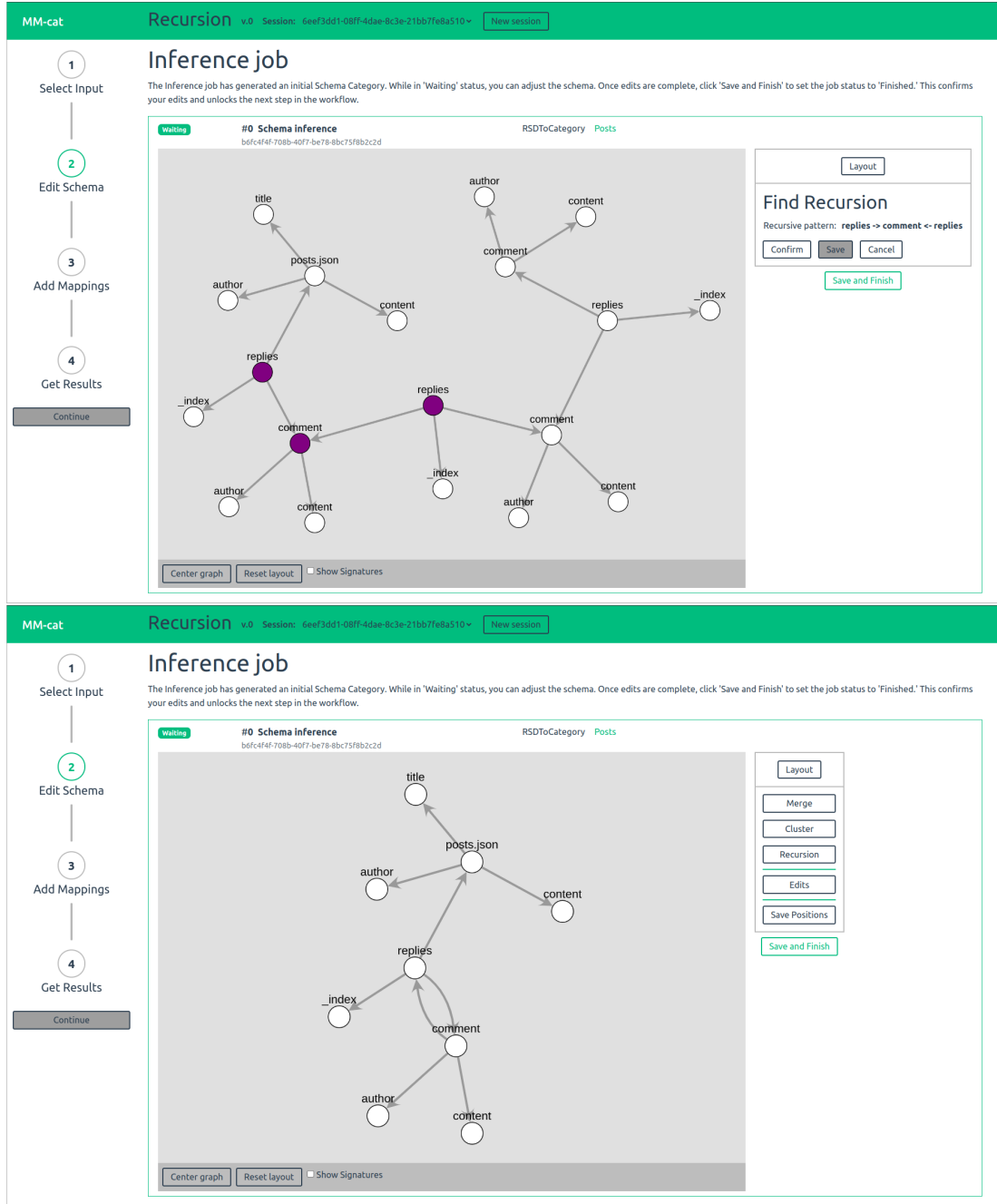


Figure 9.5 Screenshot of *TransformMer* before and after the elimination of a recursive structure

Step 2 S_{ini} is improved by interacting with the user, i.e. S_{imp} is created. The user can specify identifiers, references, and repeating patterns to be merged from the candidate set or his/her own. The user can also further modify the schema category, e.g., by grouping a set of properties into a new property or deleting a property. With every change, *TransformMer* automatically updates the mappings.

Step 3 S_{imp} is mapped to the new requested combination of models. The user selects parts of the schema category and specifies the requested, possibly overlapping mapping (e.g., by modifying an existing one or creating a new one from scratch). *TransformMer* ensures all the respective modifications.

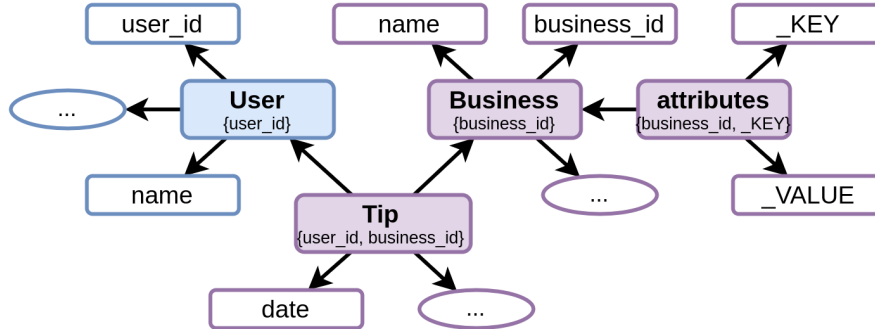


Figure 9.6 S_{imp} of the Yelp data re-mapped to the graph (blue) and relational (purple) model to represent data in Fig. 9.2

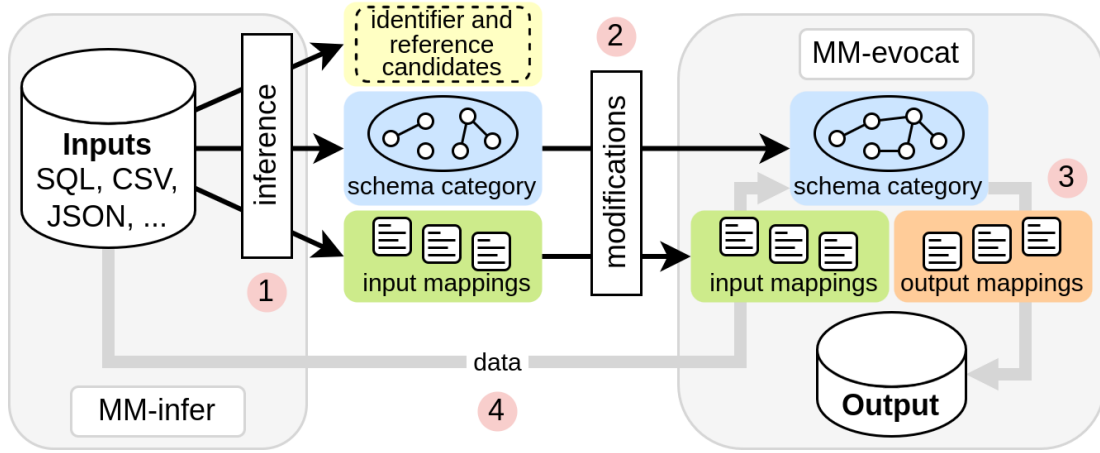


Figure 9.7 The workflow of *TransforMMer*

Step 4 *TransforMMer* loads the original data according to the input mappings into an in-memory categorical representation and then stores it according to the new user-specified mappings into the output. It can be stored in a specified file system or a DBMS.

9.4.3 Architecture

As mentioned in the introduction, *TransforMMer* leverages selected members of our toolset for multi-model data management, namely *MM-infer* [16] for categorical schema inference, *MM-evocat* [82] for schema modifications, and *MM-cat* [12] (now a part of *MM-evocat*) for schema modelling and data transformations. Fig. 9.7 also indicates their integration in *TransforMMer*. *TransforMMer* consists of the backend (written in Java), which contains all the algorithms, and a single-page client (written in Vue), which provides the user interface. The code repository is available at GitHub.¹⁰ And a demo is also available.¹¹ Lastly, we have a short video tutorial.¹²

All of the algorithms mentioned above use abstract wrappers, which provide a unified interface to the data sources instead of working with them directly. This

¹⁰<https://github.com/mmcatdb/mmcat>

¹¹<https://demo.mmcatdb.com>

¹²<https://www.youtube.com/watch?v=Aiw8150aAb4>

allows us to easily add support for new data sources by implementing a new wrapper. For now, we support PostgreSQL, MongoDB, Neo4j (i.e., representatives of the relational, document, and graph model), CSV and JSON files (i.e., the most common file formats).

Because the users might want to work with large datasets, all processes are implemented as asynchronous jobs.

9.4.4 Dataset Repository

As a proof of concept, we have used *TransformMer* to generate a repository¹³ of popular multi-model datasets (to be gradually extended). The current datasets involve, e.g., the Yelp dataset, the IMDb dataset,¹⁴ or the dataset of NASA open-source code projects.¹⁵ In the next steps, we want to extend the repository with the popular datasets and their variations and using a comparative survey demonstrate the wide usability of *TransformMer* for comparing multi-model databases.

9.5 Demonstration Outline

In our presentation, we will first demonstrate the described functionality of *TransformMer* using the Yelp dataset. We will also introduce the repository of multi-model data sets created using *TransformMer* and show selected specifics and challenges we encountered in the real-world datasets, the support for redundancy, and the applicability of versioning to simulate data evolution. Finally, we will overview future plans for the generation of respective queries. The interested audience can also experiment with *TransformMer* and own datasets.

¹³<https://dare.mmcatdb.com>

¹⁴<https://developer.imdb.com/non-commercial-datasets/>

¹⁵<https://data.nasa.gov/Software/NASA-open-source-code-projects-with-A-I-generated-/3efg-u4v8/about-data>

Paper X

DortDB: Bridging Query Languages for Multi-Model Data Ponds

Filip Ježek¹, Pavel Koupil¹, Michal Kopecký¹, Jáchym Bártík¹, Irena Holubová¹

Published as Filip Ježek et al. “DortDB: Bridging Query Languages for Multi-Model Data Ponds”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5335–5338. ISSN: 2150-8097. DOI: [10.14778/3750601.3750665](https://doi.org/10.14778/3750601.3750665)

¹Department of Software Engineering, Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

Multi-model data encompasses structurally distinct data, including relational, document, graph, key/value, columnar, etc., managed within a single system, such as a multi-model database or a data lake. Querying multi-model data requires strategies that balance unification and integration across diverse models and query languages. This paper presents DortDB, an extensible framework enabling cross-model queries combining well-known query languages and offering intuitive flexibility and optimization via a unified algebra. Though a small-scale in-memory prototype is to be demonstrated, its principles can be extended to distributed systems.

10.1 Introduction

Multi-model data refers to different types of data – relational, document, graph, key/value, columnar, etc. – being managed as a whole. They can be stored in a multi-model database management system (DBMS) [7] designed to handle diverse data formats while ensuring optimized performance and querying capabilities. An alternative approach represents data lakes [68], which store raw data in its native format without requiring a predefined schema.

Choosing between multi-model DBMSs and data lakes depends primarily on particular business needs – structured querying and performance favour DBMSs, while large-scale, unstructured data analysis benefits from data lakes. But, data lakes, of course, also offer various querying and data extraction approaches, including SQL-based query engines (e.g., Apache Presto²), data lakehouses utilising the analytical queries of a data warehouse (e.g., Delta Lake³), or search engines (e.g., Elasticsearch⁴). Some approaches (e.g., AWS Athena⁵) enable federated queries over data lakes and relational or NoSQL DBMSs. However, in all these cases, we must consider the multi-model aspect of data, i.e., integration of data with highly contradictory features and thus data-access methods, such as e.g. flat relational data vs. hierarchical JSON documents or table joins vs. graph traversals.

A common strategy for multi-model querying, often employed in polystores [169], follows the principle of *unification at front-end and integration at back-end*. This approach presents a unified view of multi-model data while maintaining mappings to the original structures. Users then interact with this unified view using a single-model-specific language. The queries are translated into the underlying models' query languages, whereas an integration module combines the intermediate results into the final result.

Conversely, multi-model DBMSs [11] typically extend the query language of the original model with constructs supporting additional models (e.g., PostgreSQL's SQL extension for accessing JSON documents). This can be seen as an approximation of the inverse strategy, i.e., *integration at front-end and unification at back-end*. Several approaches exist that enable the combination of two query languages (e.g.,

²<https://prestodb.io/>

³<https://delta.io/>

⁴<https://www.elastic.co/elasticsearch>

⁵<https://aws.amazon.com/athena/>

SQL/XML or SPARQL embedding in SQL in Virtuoso). However, this approach could be generalized to allow users to flexibly combine multiple query languages, each suitable for a particular data model, with recursive embedding as needed.

Example 10.1. Consider the example in Fig. 10.1a, which illustrates data integration from heterogeneous sources. Customer information is obtained from a Neo4j database representing a social network, address data is stored locally in CSV format as relational tables, and order data is retrieved from a remote web service in XML.

A sample query might “retrieve the most frequently ordered products that appear first in the orders of customers’ friends, limited to friends residing in Prague and products ordered by more than two such friends”. The result is returned as a relational table in CSV format to support, e.g., personalized recommendations and targeted advertising. Intuitively, the query requires integration of Cypher for graph traversal, SQL for relational aggregation, and XQuery for filtering hierarchical data – see Fig. 10.1b. □

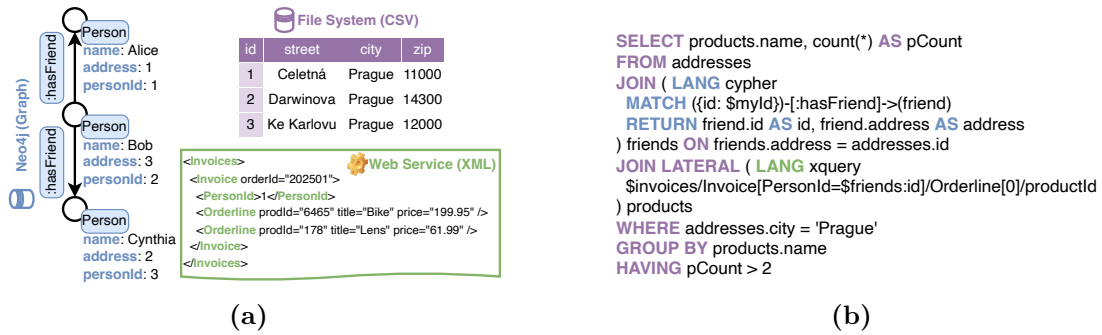


Figure 10.1 Sample multi-model data (a), cross-model query (b)

This paper introduces DortDB⁶, an extensible framework that enables querying multi-model data using virtually any combination of query languages. Its main advantages are as follows:

- Users can seamlessly combine various data formats without the need to predefine schemas, providing unmatched flexibility in data integration.
- Users can intuitively switch between query languages based on their preferences or the specific data fragments they are working with, ensuring ease of use.
- By selecting the top-level language, the user specifies the output data format. He/she can thus transform the given data to any supported format (or a combination of formats).
- Queries are translated into a unified algebra, allowing cross-model optimization to enhance query performance.

Currently, DortDB is a small-scale prototype designed for use within a web browser as a multi-model in-memory *data pond*. However, the underlying concept can be extended to large-scale environments, such as distributed systems utilizing shared RAM.

⁶Inspired by Josef Čapek’s fairy tale “How Doggie and Kitty were making a cake”.

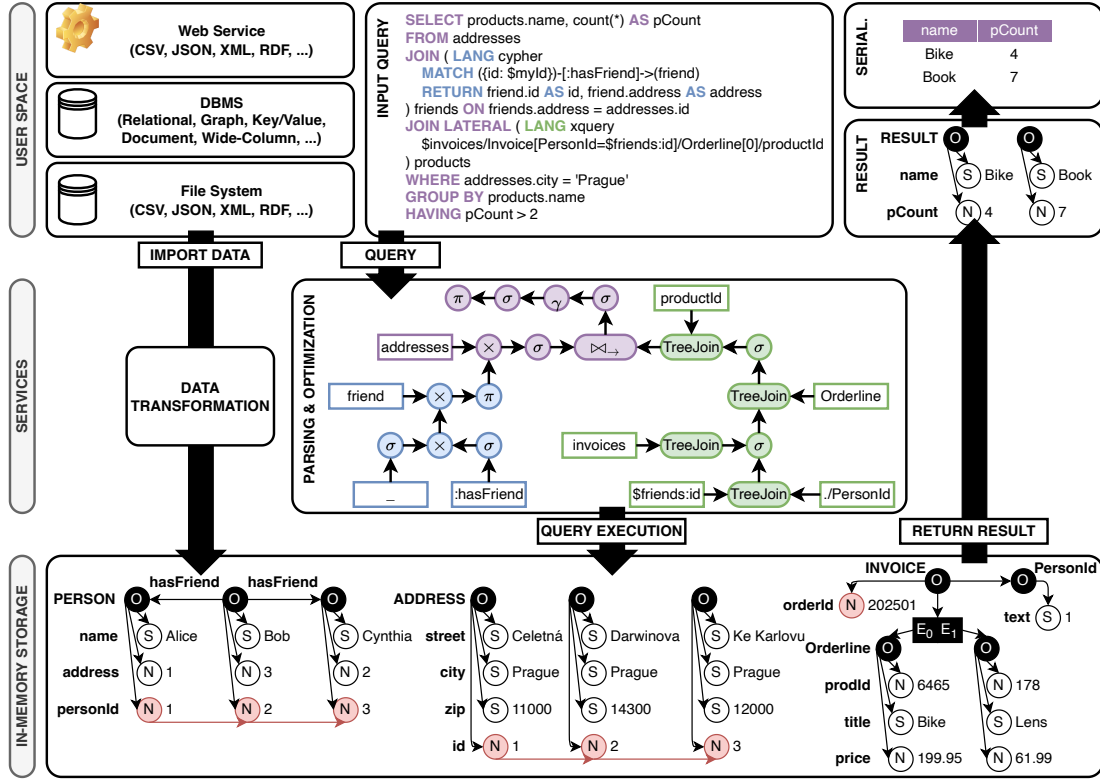


Figure 10.2 Data import and query evaluation workflow in DortDB.

10.2 Basic Concepts

The primary objective of DortDB is to offer a flexible and optimized query execution mechanism for in-memory multi-model data. Data can be imported from various sources, including web services, DBMSs, or local/remote file systems. Once imported, data is represented as an object model, which can be serialized into JSON. DortDB adopts a schema-free approach, enabling dynamic and flexible data management without the constraints of predefined schemas. Users can define secondary indexes in advance using expression-based specifications to optimise query performance.

DortDB queries are constructed using multiple query languages, currently SQL, XQuery, and Cypher (see Fig. 10.1b). These languages can be repeatedly nested within one another, allowing for the creation of complex and sophisticated queries. Once a query is formulated, the system parses it to generate a *syntactic tree* that captures the structure of the query. It is then transformed into a *logical plan*, represented as a directed graph. Its nodes correspond to algebraic operators (see Section 10.2.1). Its edges define the execution order and dependencies between operations, with a bottom-up orientation indicating data flow from sources to the final query result.

The logical plan undergoes optimization, such as operator reordering or index utilization. Then, it is executed over the object data representation following the structure defined by the logical tree and ensuring that operations are performed in the correct sequence. Data streams are processed in memory, benefiting from client-side caching to reduce redundant data retrieval from storage. The query output consists of a list of objects (e.g., a list of dictionaries or simple values).

The result remains in object representation within memory. DortDB does not enforce a specific output format but allows users to transform the results into desired formats.

Example 10.2. A sample workflow of DortDB is depicted in Fig. 10.2. The user first imports data into DortDB from various sources (e.g., a web service providing invoice data, a local Neo4j database with person and friendship data, and a local file containing address data – see Fig. 10.1a). Heterogeneous data formats are normalized into a unified TypeScript object representation, where data types are denoted by letters (O = object, S = string, N = number, E = array element). The user then submits a query, which is parsed and transformed into the unified algebraic form, visualized as a simplified tree (consisting of operators and data sources), and subsequently optimized. The query is executed over the object-based data, and the result is returned. The result can be serialized into a desired format (e.g., CSV). $\square$

10.2.1 Unified Algebra

The *Unified Algebra* (see Table 10.1) defines a formal and extensible framework for query processing, supporting both structured (e.g., relational) and semi-structured (e.g., graph or hierarchical) data. It integrates relational and XQuery algebras [78], path algebras for graphs [79], and optimizations for nested queries [80].

Class	Name	Signature
Tuple	distinct	$\delta(\text{atts}, \text{src})$
	groupBy	$\gamma(\text{keys}, \text{aggs}, \text{src})$
	cartesianProduct	$\times(\text{left}, \text{right})$
	join	$\bowtie(\text{left}, \text{right}, \text{leftOuter}, \text{rightOuter}, \text{condition})$
	orderBy	$\tau(\text{orders}, \text{src})$
	projection	$\pi(\text{attrs}, \text{src})$
	projectionConcat	$\bowtie \rightarrow (\text{mapping}, \text{outer}, \text{src})$
	projectionIndex	$\text{index}(\text{name}, \text{src})$
	selection	$\sigma(\text{expr}, \text{src})$
	recursion	$\phi(\text{min}, \text{max}, \text{condition}, \text{src})$
	tupleSource	name
	tupleFnSource	name (params)
Item	mapFromItem	$\text{fromItem}(\text{key}, \text{src})$
	calculation	$\text{calc}(\text{args})$
	conditional	$\text{cond}(\text{expr}, \text{whenthens}, \text{default})$
	fnCall	$\text{fn}(\text{args})$
	literal	$\text{literal}(\text{value})$
	quantifier	$\text{quant}(\text{type}, \text{query})$
	itemSource	name (params)
	tupleFnSource	name (params)
Ambivalent	mapToItem	$\text{toItem}(\text{key}, \text{source})$
	union	$\cup(\text{left}, \text{right})$
	intersection	$\cap(\text{left}, \text{right})$
	difference	$\setminus(\text{left}, \text{right})$
	limit	$\text{limit}(\text{skip}, \text{limit})$
XQuery	nullSource	$\square$
	projectionSize	$\text{size}(\text{name}, \text{source})$
	treeJoin	$\text{treeJoin}(\text{expr}, \text{source})$

Table 10.1 Operators of the Unified Algebra

Tuple operators manipulate *streams of named tuples*, which are analogous to, e.g., relational tuples. Core operators include *selection* (σ), *projection* (π), *join* ($\bowtie$), *cartesianProduct* ($\times$), *groupBy* (γ), *orderBy* (τ), and *recursion* (ϕ). Additional

operators, such as *projectionConcat* and *projectionIndex*, facilitate advanced tuple transformations. Furthermore, *tupleSource* and *tupleFnSource* generate streams of tuples dynamically. *Item operators* process values as *opaque items*, supporting expression evaluation, function calls, and SQL-style quantifiers. The *calculation* (*calc*) operator represents general expressions, while, e.g., *conditional*, *fnCall*, and *literal* operators construct complex computations. *Ambivalent operators* operate on both tuple and item streams. The *union* ($\cup$), *intersection* ($\cap$), and *difference* ($\setminus$) operators enable set operations. The *limit* operator constrains output size, while *nullSource* emits a null value. Finally, *XQuery-specific operators* extend tuple operations to hierarchical data structures. The *treeJoin* operator generalizes dependent joins by incorporating hierarchical context, whereas *projectionSize* computes the cardinality of tuple streams.

In DortDB’s GUI, the algebra is visualized as a tree, where each inner node represents a logical plan operator. The root of the tree corresponds to the final query, while the leaves represent the individual data sources. Tuple operators display their output schema at their respective nodes. The tree structure uses two types of edges: a *solid edge*, which indicates that the parent operator is bound to an existing data stream, and a *dashed edge*, which signifies that the child operator is dynamically created or reevaluated multiple times.

10.3 Architecture and Implementation

DortDB is a three-layer library for querying multi-model data (in formats like, e.g., CSV, XML, JSON) directly within a web browser. It is implemented entirely on the client side as a browser module, with no server communication, assuming all data resides in memory. Its architecture comprises an *in-memory storage* layer, a *services* layer, and a *user-space* layer (see Fig. 10.2), accessible via a unified API for data import and query execution.

The *in-memory storage* layer provides a uniform TypeScript object representation for heterogeneous data formats, enabling model-independent processing. The *service* layer is responsible for data acquisition and query processing. It supports loading data from external sources such as web services, file systems, and local or remote DBMSs. It performs internal transformations of source-specific models to the unified object model. For querying, it accommodates multiple languages through translation wrappers that convert input queries into a unified algebraic form, which is subsequently optimized and executed. Query results are returned as TypeScript objects, with optional serialization to standard formats such as JSON. Finally, the *user-space* layer includes graphical and programmatic interaction. A graphical interface facilitates the construction and validation of queries.

10.4 Related Work

Embedding one query language within another is commonly used but remains limited in scope. A survey of query languages for multi-model DBMSs [11] highlights a typical pattern – a primary language, typically SQL, embeds (often restricted) subqueries from other data models. This approach is widely adopted by relational DBMSs such as MS SQL Server, Oracle, and PostgreSQL through

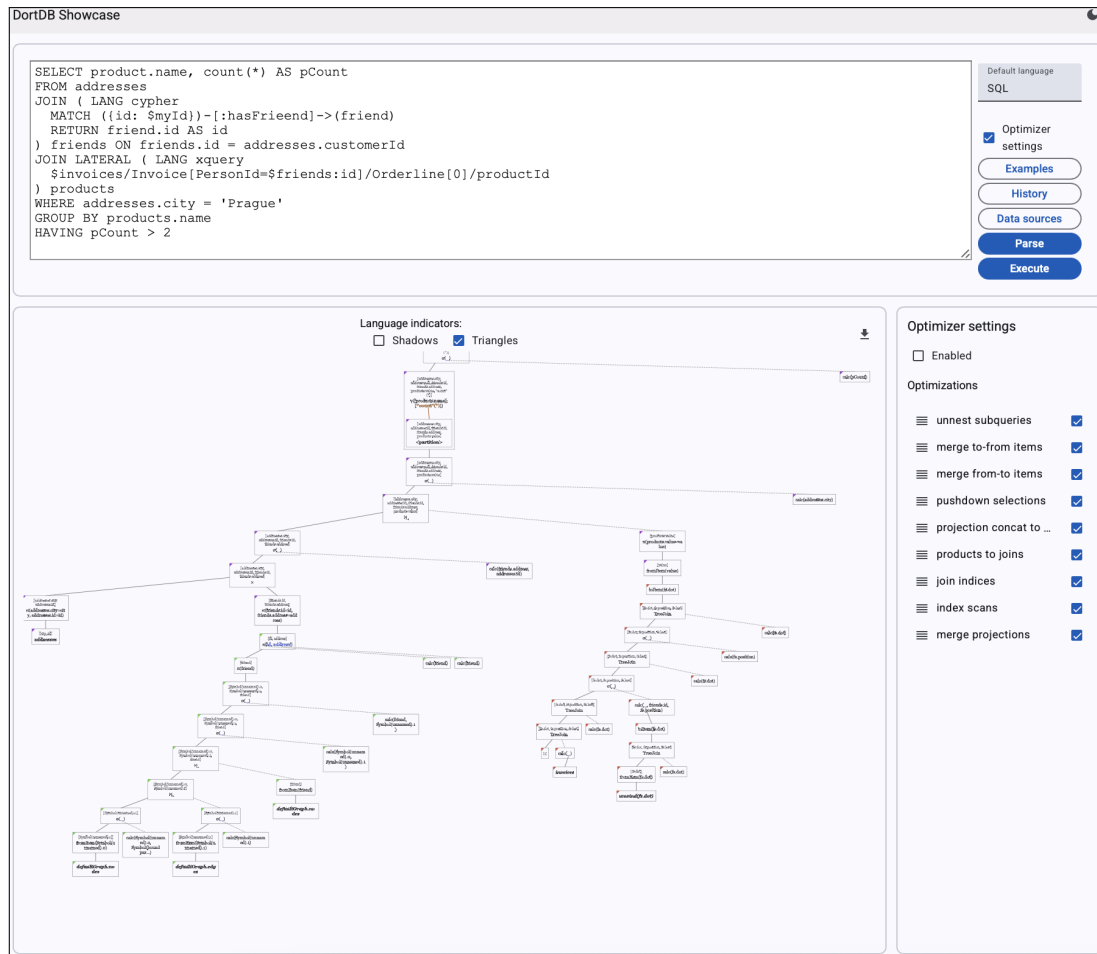


Figure 10.3 Scenario A – testing and validation GUI for DortDB

SQL/XML, SQL/JSON, and SQL/PGQ. It is also employed by key/value systems like Redis and multi-model DBMSs such as Myria [69] or Apache Drill [70], which extend SQL to access non-relational sources. Similarly, CloudMdsQL [71], a polystore, represents data as tables and accesses NoSQL sources via named table expressions. Apache AGE⁷, a PostgreSQL extension, enables Cypher queries via the embedded `cypher()` function. However, it only supports translation from Cypher to SQL, without bidirectional interoperability.

In contrast, DortDB allows any supported query language to be nested within any other, removing the constraint of a fixed top-level language (as seen in polystore BigDAWG [76]). Furthermore, nesting is not restricted to a single level but can extend to arbitrary depths. Unlike, e.g., OrientDB and ArangoDB, which rely on custom syntax extensions, DortDB supports multiple query languages natively, enabling seamless reuse and reducing the learning curve. Additionally, DortDB does not restrict specific query languages to particular data models, allowing users to choose based on their preferences – whether for efficiency, backward compatibility, or familiarity.

⁷<https://age.apache.org/>

```

const db = new DortDB({
  mainLand: SQL(),
  additionalLangs: [
    Cypher( { defaultGraph 'defaultGraph' } ), XQuery(),
  ],
});

db.registerSource(['addresses'], addressCSV);
db.registerSource(['invoices'], invoicesService.firstChild);
db.registerSource(['friends'], friends);

const result = db.query(`
SELECT product.name, count(*) AS pCount
FROM addresses
JOIN ( LANG cypher
  MATCH ({id: $myId})-[:hasFriend]->(friend)
  RETURN friend.id AS id, friend.address AS address
) friends ON friends.address = addresses.id
JOIN LATERAL ( LANG xquery
  $invoices/Invoice[PersonId=$friends.id]/Orderline[0]/productId
) products
WHERE addresses.city = 'Prague'
GROUP BY products.name
HAVING pCount > 2`);

```

Figure 10.4 Scenario B – TypeScript API for DortDB

10.5 Demonstration Outline

In the DortDB demonstration, the users will engage in querying multi-model data using the representation and query language they consider intuitive and/or most efficient. The system supports iterative refinement to improve performance and expressiveness.

Scenario A: Users can interact with a GUI designed for testing and validation (see Fig. 10.3). They can create their own queries by combining SQL, Cypher, and XQuery or select predefined queries from the UniBench benchmark [65] for benchmarking multi-model applications, executed on preloaded UniBench datasets. Upon submission, DortDB parses the query and generates a logical execution plan visualised as a tree, allowing users to inspect and verify its correctness without executing the query. This scenario emphasizes syntactic and structural validation in a multi-model context.

Scenario B: The second scenario offers a programmatic interface through the TypeScript API (see Fig. 10.4). Users may work with the UniBench dataset or upload and query a dataset of their choice imported from external web services, remote databases, or local files. After data import, users construct and execute queries using the API, which internally translates them into a unified algebra and applies query optimization. The results are returned in an object-oriented format, enabling further processing or serialization.

Conclusion

This thesis addresses the problem of managing multi-model data in a way that preserves both semantic coherence and practical usefulness across heterogeneous database systems. Starting from the existing categorical foundations for modeling, transformations, and querying, we extended the approach toward the parts that make a multi-model system viable in practice: evolution, automation, workload-aware optimization, and evaluation. The central idea throughout the thesis is that a shared conceptual representation should not erase differences among individual data models but should instead provide a stable semantic layer from which diverse logical representations, queries, and adaptations can be systematically derived.

Let us summarize the contributions of this thesis with respect to the research questions formulated in Section 0.1.3.

Evolution The first main contribution is a unified approach to evolution management. We showed that schema changes in a multi-model system can be described by a common set of atomic operations and then propagated consistently across conceptual schemas, logical schemas, and queries. On top of these atomic operations, we defined more user-friendly composite and complex operations that capture higher-level restructuring intent while still remaining reducible to basic propagation steps. This made it possible to formulate evolution as a single, platform-independent process rather than a collection of isolated, model-specific migrations. An important extension of this work was the propagation to queries, including cases where information capacity decreases, and the original query cannot be preserved by a trivial rewrite. In this setting, we also introduced FDepHunter as a separate supporting tool that can provide better semantic evidence from data, enabling ambiguous synchronization decisions to be guided by more reliable functional dependencies instead of purely ad hoc choices.

Self-adaptation The second main contribution is the move from reactive propagation to proactive self-adaptation. Before proposing our own solution, we studied workload-aware index selection in heterogeneous systems and showed that existing methods remain strongly tied to the assumption of a fixed logical representation. This observation motivated a broader formulation of the optimization problem—instead of tuning one already chosen schema, the system should search over alternative mappings, embeddings, and redundancies of the same conceptual schema across multiple databases. We therefore defined a finite configuration space based on a state matrix, proposed an automated search process over this space, and implemented it in MM-mapsearch using MCTS guided by learned cost estimation. This contribution shows that self-adaptation in a multi-model setting can be treated as a principled optimization problem over representations, rather than over physical structures within a single database.

Benchmarking The third contribution concerns the practical conditions under which such ideas can actually be tested and deployed. Through TransforMMer, we addressed the lack of realistic multi-model benchmarks by showing how a real dataset can be transformed into multiple semantically equivalent multi-model

representations, along with the corresponding queries. This provides a reproducible workflow for evaluating querying, evolution, and self-adaptation across varied workloads and model combinations.

Schema-free Query Unification At the same time, DortDB explored the limits of the overall research direction. While the main line of this thesis relies on explicit schemas and mappings, DortDB showed that there is also value in a more lightweight approach centered on language interoperability—heterogeneous sources can be queried by recursively combining established languages and translating them into a shared algebraic layer.

Overall, the thesis demonstrates that multi-model data management can be approached as one coherent problem spanning semantic modeling, query processing, evolution, and optimization. The proposed framework establishes a consistent architecture, supported by proof-of-concept implementations, in which these parts fit together—a conceptual schema provides the semantic anchor, mappings connect it to concrete logical representations, queries can be formulated and optimized across models, changes can be propagated through the whole stack, and workloads can eventually drive automatic reconfiguration. In this sense, the thesis shifts the research agenda from isolated multi-model techniques toward an integrated, increasingly automated system.

Future Work

The remaining work follows directly from the research challenges identified in the previous chapters. Some of them are only partially addressed by the current results, while the rest remain mostly open.

E3 – Propagation to data is still incomplete. The current framework can preserve consistency by transforming data to the common representation of the new schema, but it does not yet provide a general way to perform the same changes in place through efficient sequences of **DDL** and **DML** statements. A major next step is therefore to synthesize database-level migration plans that keep the guarantees of the current approach while avoiding full rematerialization whenever possible. The general idea is to define a set of model-independent operations over data, compile the **SMOs** into a sequence of such operations, optimize it, and then translate it into a model-specific **DDL** and **DML** sequence.

E4 – At the moment, we cannot infer changes in the schema of the data. Although we can infer its new schema, we are still missing the ability to compare it with the old schema and detect the actual changes. This is a crucial step for enabling bottom-up evolution, in which changes originate in the data rather than the conceptual schema. We plan to use existing approaches to graph and tree diffing [173, 174] to find the smallest sequence of **SMOs** that explains the difference. Additionally, we might develop a cost model that accounts for the cost of propagating changes and/or their plausibility based on semantic evidence in the data.

E7 – Migration strategy selection remains only partially resolved. Currently, we consider only eager propagation, even though there are use cases for lazy and hybrid alternatives [40, 41]. We intend to first survey decision mechanisms that

can choose among them according to data size, workload intensity, redundancy, and the expected cost of query reconstruction and maintenance (e.g., [57, 101]). Then we will adapt them to our categorical framework.

A3 – Latency estimation for self-adaptation is only partially solved. We have already learned estimators and plan-based evaluation, but the current approach still depends on materializing candidate configurations to obtain the query plans needed for prediction. Our plan is to extend the open-source implementations of existing database systems with an interface that allows us to extract query plans solely from metadata, rather than from fully materialized data.

A6 – We did not explicitly explore workload prediction. The current optimization process reasons about a given weighted workload, but a truly autonomous system should also account for how workloads evolve over time, including periodic patterns, long-term shifts, and short-lived anomalies [95]. Without this predictive layer, the system risks adapting too strongly to the present state and missing more beneficial future configurations.

A7 – The current search space captures only a limited set of design choices (i.e., model assignment, embedding, and cross-model redundancy), all of which are at the logical level. This is already enough to make self-adaptation meaningful, but it does not yet include richer physical design choices such as indexes, materialized views, partitioning, replication, or system-level tuning parameters. There are many existing approaches in this area (see Sections 1.3.1 and 6.4); our idea is to combine them with the state-matrix representation.

Taken together, these open directions show that the thesis establishes the conceptual and algorithmic core of the problem, but there is still work to be done before we can turn it into a fully autonomous, practically scalable system.

Bibliography

- [1] Alžběta Šrůtková. “A Tool for Dynamic Benchmarking of Multi-Model Data Management Systems”. Master’s thesis. Charles University, 2025.
- [2] Filip Štrobl. “Querying of Multi-Model Data”. Master’s thesis. Charles University, 2026.
- [3] Filip Ježek. “In-memory database for web browser”. Master’s thesis. Charles University, 2026.
- [4] Bedřich Mazourek. “AI-Assisted Workload-Aware Data Migration in Polyglot Persistence Environments”. Master’s thesis. Charles University, 2026.
- [5] Edgar F. Codd. “Further Normalization of the Data Base Relational Model”. In: *IBM Research Report, San Jose, California* RJ909 (1971).
- [6] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. 1st. Addison-Wesley Professional, 2012. ISBN: 0321826620.
- [7] Jiaheng Lu and Irena Holubová. “Multi-model Databases: A New Journey to Handle the Variety of Data”. In: *ACM Comput. Surv.* 52.3 (June 2019). ISSN: 0360-0300. DOI: [10.1145/3323214](https://doi.org/10.1145/3323214).
- [8] Jérôme Fink, Maxime Gobert, and Anthony Cleve. “Adapting Queries to Database Schema Changes in Hybrid Polystores”. In: *Proceedings of SCAM ’20*. IEEE, 2020, pp. 127–131.
- [9] DB-Engines. *DB-Engines Ranking*. 2026. URL: <https://db-engines.com/en/ranking> (visited on June 25, 2026).
- [10] Irena Holubová, Pavel Koupil, and Jiaheng Lu. “Self-Adapting Design and Maintenance of Multi-Model Databases”. In: *Proceedings of the 26th International Database Engineered Applications Symposium. IDEAS ’22*. Budapest, Hungary: Association for Computing Machinery, 2022, pp. 9–15. ISBN: 9781450397094. DOI: [10.1145/3548785.3548810](https://doi.org/10.1145/3548785.3548810).
- [11] Qingsong Guo et al. “Multi-model query languages: taming the variety of big data”. English. In: *Distributed and Parallel Databases* (May 2023). ISSN: 0926-8782.
- [12] Pavel Koupil, Martin Svoboda, and Irena Holubová. “MM-cat: A Tool for Modeling and Transformation of Multi-Model Data using Category Theory”. In: *Proceedings of MODELS ’21*. New York, NY, USA: IEEE, 2021, pp. 635–639.
- [13] Pavel Koupil and Irena Holubová. “A Unified Representation and Transformation of Multi-Model Data using Category Theory”. In: *Journal of Big Data* 9.1 (2022), p. 61.
- [14] Pavel Koupil, Daniel Crha, and Irena Holubová. “MM-quecat: A Tool for Unified Querying of Multi-Model Data”. In: *Proceedings of EDBT 2023*. OpenProceedings.org, 2023, pp. 831–834.

- [15] Pavel Koupil, Daniel Crha, and Irena Holubová. “A universal Approach for Simplified Redundancy-Aware Cross-Model Querying”. In: *Information Systems* 127 (2025). ISSN: 0306-4379.
- [16] Pavel Koupil, Sebastián Hricko, and Irena Holubová. “MM-infer: A Tool for Inference of Multi-Model Schemas”. In: *Proceedings of EDBT 2022*. OpenProceedings.org, 2022, 2:566–2:569.
- [17] Pavel Koupil, Sebastián Hricko, and Irena Holubová. “A universal approach for multi-model schema inference”. In: *J. Big Data* 9.1 (2022), p. 97. DOI: [10.1186/s40537-022-00645-9](https://doi.org/10.1186/s40537-022-00645-9).
- [18] Object Management Group. *MDA Guide Revision 2.0*. OMG document formal/2014-06-01. 2014.
- [19] Peter P. Chen. “The Entity-Relationship Model—Toward a Unified View of Data”. In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36. DOI: [10.1145/320434.320440](https://doi.org/10.1145/320434.320440).
- [20] ISO/IEC 9075-1:2023 *Information Technology – Database Languages – SQL – Part 1: Framework (SQL/Framework)*. ISO/IEC, 2023.
- [21] Jonathan Robie, Michael Dyck, and Josh Spiegel. *XQuery 3.1: An XML Query Language*. <https://www.w3.org/TR/xquery-31/>. W3C Recommendation, 21 March 2017. Mar. 2017.
- [22] Eric Prud’hommeaux and Andy Seaborne. *SPARQL Query Language for RDF*. <http://www.w3.org/TR/rdf-sparql-query/>. W3C, Jan. 2008.
- [23] Nadime Francis et al. “Cypher: An Evolving Query Language for Property Graphs”. In: *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. 2018, pp. 1433–1445. DOI: [10.1145/3183713.3190657](https://doi.org/10.1145/3183713.3190657).
- [24] ISO/IEC. *ISO/IEC TR 19075-6:2017 Information technology – Database languages – SQL – Part 6: SQL support for JavaScript Object Notation (JSON)*. 2017.
- [25] ISO/IEC. *ISO/IEC TR 9075-16:2023 Information technology – Database languages – SQL – Part 16: Property Graph Queries (SQL/PGQ)*. 2023.
- [26] MongoDB, Inc. *MongoDB Query Language (MQL)*. 2025. URL: <https://www.mongodb.com/docs/manual/reference/mql/> (visited on June 25, 2026).
- [27] Jáchym Bártík, Filip Štrobl, and Irena Holubová. *Reducing the Cross-Model Tax: Query Optimization over Multi-Model Data*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/query-opt.pdf>.
- [28] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Formal specification. 2017.
- [29] Carlos Javier Fernández Candel, Diego Sevilla Ruiz, and Jesús Joaquín García Molina. “A Unified Metamodel for NoSQL and Relational Databases”. In: *Inf. Syst.* 104 (2022), p. 101898.

- [30] Alberto Hernández Chillón, Diego Sevilla Ruiz, and Jesús García Molina. “Towards a Taxonomy of Schema Changes for NoSQL Databases: The Orion Language”. In: *Proceedings of ER '21, Virtual Event*. Vol. 13011. LNCS. Springer, 2021, pp. 176–185.
- [31] Alberto Hernández Chillón et al. “A Generic Schema Evolution Approach for NoSQL and Relational Databases”. In: *IEEE Trans. on Knowl. and Data Eng.* 36.7 (July 2024), pp. 2774–2789. ISSN: 1041-4347.
- [32] Carlos J. Fernández Candel, Jesús J. García-Molina, and Diego Sevilla Ruiz. “SkiQL: A unified schema query language”. In: *Data & Knowledge Engineering* 148 (Nov. 2023), p. 102234. ISSN: 0169-023X. DOI: [10.1016/j.datak.2023.102234](https://doi.org/10.1016/j.datak.2023.102234).
- [33] Valter Uotila and Jiaheng Lu. “A Formal Category Theoretical Framework for Multi-model Data Transformations”. In: *CoRR* abs/2201.04905 (2022). arXiv: [2201.04905](https://arxiv.org/abs/2201.04905).
- [34] Jiaheng Lu. *A Categorical Unification for Multi-Model Data: Part I Categorical Model and Normal Forms*. 2025. arXiv: [2502.19131](https://arxiv.org/abs/2502.19131) [cs.DB]. URL: <https://arxiv.org/abs/2502.19131>.
- [35] Jiaheng Lu. *A Categorical Unification for Multi-Model Data: Part II Categorical Algebra and Calculus*. 2025. arXiv: [2504.09515](https://arxiv.org/abs/2504.09515) [cs.DB]. URL: <https://arxiv.org/abs/2504.09515>.
- [36] ISO/IEC. *ISO/IEC 9075-14:2011 Information technology – Database languages – SQL – Part 14: XML-Related Specifications (SQL/XML)*. 2011.
- [37] Jiaheng Lu, Irena Holubová, and Bogdan Cautis. “Multi-model Databases and Tightly Integrated Polystores: Current Practices, Comparisons, and Open Challenges”. In: *Proceedings of CIKM 2018*. Torino, Italy: ACM, 2018, pp. 2301–2302.
- [38] Carlyna Bondiombouy and Patrick Valduriez. “Query processing in multi-store systems: an overview”. In: *Int. J. Cloud Comput.* 5.4 (2016), pp. 309–346.
- [39] Rana Alotaibi et al. “ESTOCADA: towards scalable polystore systems”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 2949–2952.
- [40] Stefanie Scherzinger, Meike Klettke, and Uta Störl. “Managing schema evolution in NoSQL data stores”. In: *arXiv preprint arXiv:1308.0514* (2013). URL: <https://arxiv.org/abs/1308.0514>.
- [41] Meike Klettke et al. “NoSQL schema evolution and big data migration at scale”. In: *2016 IEEE International Conference on Big Data (Big Data)*. 2016, pp. 2764–2774. DOI: [10.1109/BigData.2016.7840924](https://doi.org/10.1109/BigData.2016.7840924).
- [42] Loredana Caruccio, Giuseppe Polese, and Genoveffa Tortora. “Dependency-Based Query/View Synchronization upon Schema Evolutions”. In: *Proceedings of ER '18 Workshops*. Vol. 11158. LNCS. Springer, 2018, pp. 91–105.
- [43] Elena V. Ravve. “Maintenance of Queries Under Database Changes: A Unified Logic Based Approach”. In: *Proceedings of FoIKS '16*. Vol. 9616. LNCS. Springer, 2016, pp. 191–208.

- [44] Jie Liu and Barzan Mozafari. “Query Rewriting via Large Language Models”. In: *CoRR* abs/2403.09060 (2024). arXiv: [2403.09060](https://arxiv.org/abs/2403.09060).
- [45] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995. ISBN: 0-201-53771-0.
- [46] David I Spivak and Ryan Wisnesky. “Relational Foundations for Functorial Data Migration”. In: *Proceedings of DBPL ’15*. New York, NY, USA: ACM, 2015, pp. 21–28. ISBN: 9781450339025.
- [47] Uta Störl and Meike Klettke. “Darwin: A Data Platform for NoSQL Schema Evolution Management and Data Migration”. In: *Proceedings of the Workshops of the EDBT/ICDT 2022 Joint Conference, Edinburgh, UK, March 29, 2022*. Vol. 3135. CEUR Workshop Proceedings. CEUR-WS.org, 2022.
- [48] Jan Kossmann, Thorsten Papenbrock, and Felix Naumann. “Data Dependencies for Query Optimization: A Survey”. In: 31.1 (June 2021), pp. 1–22. ISSN: 1066-8888.
- [49] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. “Adaptive Query Processing”. In: *Found. Trends Databases* 1.1 (Jan. 2007), pp. 1–140. ISSN: 1931-7883.
- [50] Carlo A. Curino et al. “Update Rewriting and Integrity Constraint Maintenance in a Schema Evolution Support System: PRISM++”. In: *Proc. VLDB Endow.* 4.2 (Nov. 2010), pp. 117–128. ISSN: 2150-8097.
- [51] Mark Lukas Möller et al. “Query Rewriting for Continuously Evolving NoSQL Databases”. In: *Proc. ER 2019*. Vol. 11788. LNCS. 2019, pp. 213–221.
- [52] Stratos Idreos et al. “The Data Calculator: Data Structure Design and Cost Synthesis from First Principles and Learned Cost Models”. In: *Proceedings of SIGMOD ’18*. New York, NY, USA: ACM, 2018, pp. 535–550. ISBN: 9781450347037.
- [53] Ryan Marcus and Olga Papaemmanouil. “Plan-Structured Deep Neural Network Models for Query Performance Prediction”. In: *Proc. VLDB Endow.* 12.11 (July 2019), pp. 1733–1746. ISSN: 2150-8097.
- [54] Jun Sun et al. “Workload-Aware Performance Tuning for Multimodel Databases Based on Deep Reinforcement Learning”. In: *Int. J. Intell. Syst.* 2023 (Jan. 2023). ISSN: 0884-8173.
- [55] Feng Ye et al. “Parameters tuning of multi-model database based on deep reinforcement learning”. In: *J. Intell. Inf. Syst.* 61.1 (Nov. 2022), pp. 167–190. ISSN: 0925-9902.
- [56] Francesco Basciani et al. “TyphonML: A Modeling Environment to Develop Hybrid Polystores”. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. New York, NY, USA: Association for Computing Machinery, 2020, pp. 1–5. ISBN: 9781450381352.
- [57] Andrea Hillenbrand et al. “MigCast: Putting a Price Tag on Data Model Evolution in NoSQL Data Stores”. In: *Proceedings of SIGMOD ’19*. Amsterdam, Netherlands: ACM, 2019, pp. 1925–1928. ISBN: 9781450356435.

- [58] Transaction Processing Performance Council. *TPC Benchmark H (TPC-H) Standard Specification (version 3.0.1)*. 2022. URL: <https://www.tpc.org/tpch/> (visited on May 25, 2026).
- [59] Transaction Processing Performance Council. *TPC Benchmark DS (TPC-DS) Standard Specification (version 4.0.0)*. 2024. URL: <https://www.tpc.org/tpcds/> (visited on May 25, 2026).
- [60] Albrecht Schmidt et al. “XMark: a benchmark for XML data management”. In: *VLDB '02*. VLDB '02. Hong Kong, China: VLDB Endowment, 2002, pp. 974–985.
- [61] Stefano Belloni et al. “DeepBench: Benchmarking JSON Document Stores”. In: *Proceedings of the 2022 Workshop on 9th International Workshop of Testing Database Systems*. DBTest '22. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 1–9. ISBN: 9781450393539.
- [62] Orri Erling et al. “The LDBC Social Network Benchmark: Interactive Workload”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. SIGMOD '15. Melbourne, Victoria, Australia: Association for Computing Machinery, 2015, pp. 619–630. ISBN: 9781450327589.
- [63] Angela Bonifati et al. “Graph Generators: State of the Art and Open Challenges”. In: *ACM Comput. Surv.* 53.2 (Apr. 2020). ISSN: 0360-0300.
- [64] Ahmad Ghazal et al. “BigBench: towards an industry standard benchmark for big data analytics”. In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. SIGMOD '13. New York, New York, USA: Association for Computing Machinery, 2013, pp. 1197–1208. ISBN: 9781450320375.
- [65] Chao Zhang et al. “UniBench: A Benchmark for Multi-model Database Management Systems”. In: *TPCTC 2018*. Cham: Springer International Publishing, 2019, pp. 7–23. ISBN: 978-3-030-11404-6.
- [66] Bogyong Kim et al. “M2Bench: A Database Benchmark for Multi-Model Analytic Workloads”. In: *Proc. VLDB Endow.* 16.4 (Dec. 2022), pp. 747–759. ISSN: 2150-8097.
- [67] Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “Benchmark Engineering as a Design Instrument for Heterogeneous Information Systems”. In: *arXiv* (2026). Manuscript under second-round review. eprint: [2607.07175](https://arxiv.org/abs/2607.07175) (cs.DB). URL: <https://arxiv.org/abs/2607.07175>.
- [68] Amazon Web Services. “What is a Data Lake? - Introduction to Data Lakes and Analytics”. In: *AWS* (2023). URL: <https://aws.amazon.com/what-is/data-lake/>.
- [69] Jingjing Wang et al. “The Myria Big Data Management and Analytics System and Cloud Services”. In: *Proceedings of CIDR 2017*. www.cidrdb.org, 2017.
- [70] Michael Hausenblas and Jacques Nadeau. “Apache Drill: Interactive Ad-Hoc Analysis at Scale”. In: *J. of Big Data* 1.2 (2013), pp. 100–104.

- [71] Boyan Kolev et al. “The CloudMdsQL Multistore System”. In: *Proceedings of SIGMOD 2016* (2016), pp. 2113–2116.
- [72] Apache.org. *Apache AGE*. 2024. URL: <https://age.apache.org/overview/> (visited on Mar. 10, 2025).
- [73] *Graph database optimized for fast analysis and real-time data processing*. <https://github.com/apache/age/>. [Online; Accessed 18-March-2024]. 2024.
- [74] OrientDB Ltd. *OrientDB Documentation*. 2025. URL: <https://orientdb.dev/docs/> (visited on June 25, 2026).
- [75] ArangoDB GmbH. *ArangoDB Documentation*. 2025. URL: <https://docs.arangodb.com/> (visited on June 25, 2026).
- [76] Jennie Duggan et al. “The BigDAWG Polystore System”. In: *SIGMOD Rec.* 44.2 (Aug. 2015), pp. 11–16. ISSN: 0163-5808. DOI: [10.1145/2814710.2814713](https://doi.org/10.1145/2814710.2814713).
- [77] Filip Ježek et al. *One Algebra to Query Them All: A Schema-Agnostic Foundation for Cross-Model Querying*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/unified-algebra.pdf>.
- [78] Christopher Ré, Jérôme Siméon, and Mary F. Fernández. “A Complete and Efficient Algebraic Compiler for XQuery”. In: *Proceedings of ICDE 2006*. IEEE, 2006, p. 14. DOI: [10.1109/ICDE.2006.6](https://doi.org/10.1109/ICDE.2006.6).
- [79] Renzo Angles et al. “Path-based Algebraic Foundations of Graph Query Languages”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 783–795. DOI: [10.48786/EDBT.2025.63](https://doi.org/10.48786/EDBT.2025.63).
- [80] Jürgen Hölsch, Michael Grossniklaus, and Marc H. Scholl. “Optimization of Nested Queries using the NF2 Algebra”. In: *Proceedings of SIGMOD 2016*. ACM, 2016. DOI: [10.1145/2882903.2915241](https://doi.org/10.1145/2882903.2915241).
- [81] Jáchym Bártík, Pavel Koupil, and Irena Holubová. “Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of SAC 2024*. ACM, 2024.
- [82] Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of CIKM 2022*. Atlanta, GA, USA: ACM, 2022, pp. 4892–4896. ISBN: 9781450392365. DOI: [10.1145/3511808.3557180](https://doi.org/10.1145/3511808.3557180).
- [83] Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evoque: Query Synchronisation in Multi-Model Databases”. In: *Proceedings of EDBT 2024*. OpenProceedings.org, 2024, pp. 818–821. DOI: [10.48786/edbt.2024.78](https://doi.org/10.48786/edbt.2024.78).
- [84] Andy Pavlo. *What is a Self-Driving Database Management System?* A. Pavlo blog, 2018. URL: <https://www.cs.cmu.edu/~pavlo/blog/2018/04/what-is-a-self-driving-database-management-system.html>.
- [85] Yuqing Zhu et al. “BestConfig: Tapping the Performance Potential of Systems via Automatic Configuration Tuning”. In: *Proceedings of SoCC ’17*. ACM, 2017, pp. 338–350.

- [86] Guoliang Li, Xuanhe Zhou, and Lei Cao. “AI Meets Database: AI4DB and DB4AI”. In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD ’21. Virtual Event, China: Association for Computing Machinery, 2021, pp. 2859–2866. ISBN: 9781450383431. DOI: [10.1145/3448016.3457542](https://doi.org/10.1145/3448016.3457542).
- [87] Guoliang Li et al. “openGauss: An Autonomous Database System”. In: *Proceedings of VLDB Endow.* 14.12 (2021), pp. 3028–3041. URL: <http://www.vldb.org/pvldb/vol14/p3028-li.pdf>.
- [88] Jan Kossmann et al. “Magic Mirror in My Hand, Which is the Best in the Land? An Experimental Evaluation of Index Selection Algorithms”. In: *Proc. VLDB Endow.* 13.12 (July 2020), pp. 2382–2395. ISSN: 2150-8097. DOI: [10.14778/3407790.3407832](https://doi.org/10.14778/3407790.3407832).
- [89] Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. “Learning a Partitioning Advisor for Cloud Databases”. In: *SIGMOD 2020*. ACM, 2020, pp. 143–157.
- [90] Ji Zhang et al. “CDBTune+: An Efficient Deep Reinforcement Learning-Based Automatic Cloud Database Tuning System”. In: *VLDB J.* 30.6 (2021), pp. 959–987.
- [91] Ji Sun and Guoliang Li. “An End-to-End Learning-based Cost Estimator”. In: *Proceedings of VLDB Endow.* 13.3 (2019), pp. 307–319.
- [92] Ryan Marcus and Olga Papaemmanouil. “Deep Reinforcement Learning for Join Order Enumeration”. In: *Proceedings of aiDM@SIGMOD 2018*. ACM, 2018, 3:1–3:4.
- [93] Jialin Ding et al. “ALEX: An Updatable Adaptive Learned Index”. In: *Proceedings of SIGMOD ’20*. ACM, 2020, pp. 969–984.
- [94] Stratos Idreos et al. “Design Continuums and the Path Toward Self-Designing Key-Value Stores that Know and Learn”. In: *Proceedings of CIDR 2019*. www.cidrdb.org, 2019. URL: <http://cidrdb.org/cidr2019/papers/p143-idreos-cidr19.pdf>.
- [95] Lin Ma et al. “Query-based Workload Forecasting for Self-Driving Database Management Systems”. In: *Proceedings of SIGMOD ’18*. ACM, 2018, pp. 631–645.
- [96] Martin Nečaský et al. “Evolution and Change Management of XML-based Systems”. In: *J. Syst. Softw.* 85.3 (Mar. 2012), pp. 683–707. ISSN: 0164-1212.
- [97] Irena Holubová, Michal Vavrek, and Stefanie Scherzinger. “Evolution Management in Multi-Model Databases”. In: *Data Knowl. Eng.* 136 (2021), p. 101932.
- [98] Marek Polák, Martin Nečaský, and Irena Holubová. “DaemonX: Design, Adaptation, Evolution, and Management of Native XML (and More Other) Formats”. In: *Proceedings of IIWAS ’13*. Vienna, Austria: Acm, 2013, pp. 484–493. ISBN: 9781450321136.
- [99] Uta Störl et al. “Curating Variational Data in Application Development”. In: *Proceedings of ICDE ’18*. 2018, pp. 1605–1608.

- [100] Uta Störl et al. “Enabling Efficient Agile Software Development of NoSQL-backed Applications”. In: *Proceedings of BTW '17*. Demo. 2017.
- [101] Andrea Hillenbrand et al. “Towards Self-Adapting Data Migration in the Context of Schema Evolution in NoSQL Databases”. In: *ICDE Workshops '20*. IEEE, 2020, pp. 133–138.
- [102] Chris Tuijn and Marc Gyssens. “CGOOD, a Categorical Graph-oriented Object Data Model”. In: *Theoretical Computer Science* 160.1 (1996), pp. 217–239. ISSN: 0304-3975.
- [103] Irena Holubová, Pavel Contos, and Martin Svoboda. “Multi-Model Data Modeling and Representation: State of the Art and Research Challenges”. In: *Proceedings of IDEAS '21*. ACM, 2021, pp. 242–251.
- [104] Pavel Koupil, Jáchym Bártík, and Irena Holubová. “Model-Agnostic Evolution Management”. In: *Proceedings of RCIS 2025*. Vol. 547. LNCS. Springer, 2025, pp. 245–261. DOI: [10.1007/978-3-031-92474-3_15](https://doi.org/10.1007/978-3-031-92474-3_15).
- [105] Richard Hull. “Relative Information Capacity of Simple Relational Database Schemata”. In: *SIAM J. Comput.* 15.3 (1986), pp. 856–886.
- [106] Loredana Caruccio, Giuseppe Polese, and Genoveffa Tortora. “Synchronization of Queries and Views Upon Schema Evolutions: A Survey”. In: *ACM Trans. Database Syst.* 41.2 (May 2016). ISSN: 0362-5915.
- [107] Elke A. Rundensteiner, Amy J. Lee, and Anisoara Nica. “On Preserving Views in Evolving Environments”. In: *Proceedings of Workshop KRDB '97*. Vol. 8. CEUR Workshop Proceedings. CEUR-WS.org, 1997, pp. 13.1–13.11.
- [108] Elke A. Rundensteiner et al. “Evolvable View Environment (EVE): Non-Equivalent View Maintenance under Schema Changes”. In: *Proceedings of SIGMOD '99*. ACM Press, 1999, pp. 553–555.
- [109] Andreas Koeller and Elke A. Rundensteiner. “A history-driven approach at evolving views under meta data changes”. In: *Knowl. Inf. Syst.* 8.1 (2005), pp. 34–67.
- [110] Alberto Hernández Chillón et al. “A Taxonomy of Schema Changes for NoSQL Databases”. In: *CoRR* abs/2205.11660 (2022).
- [111] Pavel Koupil et al. “FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5227–5230. ISSN: 2150-8097. DOI: [10.14778/3750601.3750612](https://doi.org/10.14778/3750601.3750612).
- [112] Stéphane Lopes, Jean-Marc Petit, and Lotfi Lakhal. “Efficient Discovery of Functional Dependencies and Armstrong Relations”. In: *Proc. EDBT*. Vol. 1777. LNCS. 2000, pp. 350–364.
- [113] Thorsten Papenbrock and Felix Naumann. “A Hybrid Approach to Functional Dependency Discovery”. In: *Proc. SIGMOD*. 2016, pp. 821–833. DOI: [10.1145/2882903.2915203](https://doi.org/10.1145/2882903.2915203).
- [114] Thorsten Papenbrock et al. “Functional Dependency Discovery: An Experimental Evaluation of Seven Algorithms”. In: *In Proc. VLDB* 8.10 (2015), pp. 1082–1093.

- [115] Laure Berti-Équille et al. “Discovery of Genuine Functional Dependencies from Relational Data with Missing Values”. In: *In Proc. VLDB* 11.8 (2018), pp. 880–892.
- [116] Felix Naumann. *Repeatability*. <https://hpi.de/naumann/projects/repeatability/data-profiling/fds.html>. Online resource. Accessed July 2025. 2025.
- [117] E. Mark Gold. “Limiting Recursion”. In: *J. Symb. Log.* 30.1 (1965), pp. 28–48.
- [118] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. “Discovering Denial Constraints”. In: *In Proc. VLDB* 6.13 (2013), pp. 1498–1509.
- [119] Fei Chiang and Renée J. Miller. “Discovering data quality rules”. In: *In Proc. VLDB* 1.1 (2008), pp. 1166–1177.
- [120] Periklis Andritsos, Renée J. Miller, and Panayiotis Tsaparas. “Information-Theoretic Tools for Mining Database Structure from Large Data Sets”. In: *Proc. SIGMOD*. 2004, pp. 731–742.
- [121] Ziheng Wei and Sebastian Link. “Towards the efficient discovery of meaningful functional dependencies”. In: *Inf. Syst.* 116 (2023).
- [122] Antonio M. Silva and Michel A. Melkanoff. “A Method for Helping Discover the Dependencies of a Relation”. In: *Advances in Data Base Theory*. Vol. 1. 1979, pp. 115–133.
- [123] Warren-Dean Langeveldt and Sebastian Link. “Empirical evidence for the usefulness of Armstrong relations in the acquisition of meaningful functional dependencies”. In: *Inf. Syst.* 35.3 (2010), pp. 352–374. DOI: [10.1016/J.IS.2009.11.002](https://doi.org/10.1016/J.IS.2009.11.002).
- [124] Filip Miháľ, Jáchym Bártík, and Pavel Koupil. “Refining Storage Strategy through Index Selection Methods in Multi-Model Database Systems: A Survey”. In: *Data & Knowledge Engineering* 164 (2026), p. 102582. ISSN: 0169-023X. DOI: [10.1016/j.datak.2026.102582](https://doi.org/10.1016/j.datak.2026.102582).
- [125] Yu Yan et al. “Multi-SQL: An Automatic Multi-model Data Management System”. In: *Web and Big Data*. Ed. by Bohan Li et al. Cham: Springer Nature Switzerland, 2023, pp. 451–455. ISBN: 978-3-031-25201-3.
- [126] Yi-Ming Lin et al. “Federated Learning on Multimodal Data: A Comprehensive Survey”. In: *Mach. Intell. Res.* 20.4 (2023), pp. 539–553. DOI: [10.1007/S11633-022-1398-0](https://doi.org/10.1007/S11633-022-1398-0).
- [127] Surajit Chaudhuri and Vivek Narasayya. *Anytime Algorithm of Database Tuning Advisor for Microsoft SQL Server*. June 2020.
- [128] Surajit Chaudhuri and Vivek R. Narasayya. “An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server”. In: *Proceedings of the 23rd International Conference on Very Large Data Bases*. VLDB ’97. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997, pp. 146–155. ISBN: 1558604707.
- [129] A. Ailamaki D. Dash N. Polyzotis. “CoPhy: A Scalable, Portable, and Interactive Index Advisor for Large Workloads”. In: *Proceedings of the VLDB Endowment* 4 (2011), pp. 362–372.

- [130] Ankur Sharma, Felix Martin Schuhknecht, and Jens Dittrich. *The Case for Automatic Database Administration using Deep Reinforcement Learning*. 2018. DOI: [10.48550/ARXIV.1801.05643](https://doi.org/10.48550/ARXIV.1801.05643).
- [131] Wentao Wu et al. “Budget-Aware Index Tuning with Reinforcement Learning”. In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD '22. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 1528–1541. ISBN: 9781450392495. DOI: [10.1145/3514221.3526128](https://doi.org/10.1145/3514221.3526128).
- [132] DB-Engines. *DB-Engines Ranking*. 2024. URL: <https://db-engines.com/en/ranking> (visited on Oct. 8, 2024).
- [133] Jaroslav Pokorný and Ivan Halaška. *Databázové systémy*. ČVUT, 1998.
- [134] PostgreSQL Development Team. *PostgreSQL: Documentation: 14: Index Types*. <https://www.postgresql.org/docs/current/indexes-types.html>. Accessed: May 3, 2023. 2021.
- [135] Neo4j. *Query Tuning with Indexes*. <https://neo4j.com/docs/cypher-manual/current/query-tuning/indexes/>. Accessed: April 25, 2024. 2021.
- [136] Redis. *Indexes*. <https://redis.io/docs/manual/patterns/indexes/>. Accessed: April 30, 2023. 2021.
- [137] Pavel Contos and Martin Svoboda. “JSON Schema Inference Approaches”. In: *Advances in Conceptual Modeling – ER 2020 Workshops CMAI, CMLS, CMOMM4FAIR, CoMoNoS, EmpER, Vienna, Austria, November 3–6, 2020, Proceedings*. Ed. by Georg Grossmann and Sudha Ram. Vol. 12584. Lecture Notes in Computer Science. Springer, 2020, pp. 173–183. DOI: [10.1007/978-3-030-65847-2_16](https://doi.org/10.1007/978-3-030-65847-2_16).
- [138] Ecma International. *JavaScript Object Notation (JSON)*. <http://www.JSON.org/>. 2013.
- [139] W3C. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. Nov. 2008.
- [140] MongoDB. *MongoDB Manual: Indexes*. <https://www.postgresql.org/docs/current/indexes-types.html>. Accessed: October 8, 2024. MongoDB, Inc., 2021.
- [141] Couchbase. *Indexes*. <https://docs.couchbase.com/server/current/learn/services-and-indexes/indexes/indexing-and-query-perf.html>. Accessed: May 2, 2023. 2023.
- [142] MongoDB. *MongoDB Sharding*. <https://www.mongodb.com/docs/manual/sharding/>. Accessed: April 15, 2023. 2024.
- [143] Duy Hai Doan. *Cassandra Native Secondary Index Deep Dive*. <https://www.datastax.com/blog/cassandra-native-secondary-index-deep-dive>. Accessed: April 22, 2023. 2016.
- [144] Cassandra. *Cassandra Documentation*. <https://cassandra.apache.org/doc/4.1/cassandra/architecture/dynamo.html>. Accessed: April 22, 2023. 2022.

- [145] Patrick O’Neil et al. “The Log-Structured Merge-Tree (LSM-Tree)”. In: *Acta Inf.* 33.4 (June 1996), pp. 351–385. ISSN: 0001-5903. DOI: [10.1007/s002360050048](https://doi.org/10.1007/s002360050048).
- [146] Apache Cassandra. *Secondary Indexes*. <https://cassandra.apache.org/doc/latest/cassandra/cql/indexes.html>. Accessed: April 24, 2023. 2021.
- [147] DataStax. *Using Multiple Secondary Indexes*. https://docs.datastax.com/en/archived/cql/3.3/cql/cql_using/useMultIndexes.html. Accessed: April 24, 2023. 2015.
- [148] Kaiye Wang et al. “A Comprehensive Survey on Cross-modal Retrieval”. In: *CoRR* abs/1607.06215 (2016). arXiv: [1607.06215](https://arxiv.org/abs/1607.06215). URL: <http://arxiv.org/abs/1607.06215>.
- [149] Jingdong Wang et al. “Hashing for Similarity Search: A Survey”. In: *CoRR* abs/1408.2927 (2014). arXiv: [1408.2927](https://arxiv.org/abs/1408.2927). URL: <http://arxiv.org/abs/1408.2927>.
- [150] Jianguo Wang et al. “Milvus: A Purpose-Built Vector Data Management System”. In: *SIGMOD ’21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*. Ed. by Guoliang Li et al. ACM, 2021, pp. 2614–2627. DOI: [10.1145/3448016.3457550](https://doi.org/10.1145/3448016.3457550).
- [151] Kyu-Young Whang. “Index Selection in Relational Databases”. In: *Proc. Int’l Conf. on Foundations of Data Organization*. Kyoto, Japan, May 1985, pp. 369–378.
- [152] Gregory Piatetsky-Shapiro. “The Optimal Selection of Secondary Indices is NP-Complete”. In: *SIGMOD Rec.* 13.2 (Jan. 1983), pp. 72–75. ISSN: 0163-5808. DOI: [10.1145/984523.984530](https://doi.org/10.1145/984523.984530).
- [153] Karl Schnaitter, Neoklis Polyzotis, and Lise Getoor. “Index Interactions in Physical Design Tuning: Modeling, Analysis, and Applications”. In: *Proc. VLDB Endow.* 2.1 (Aug. 2009), pp. 1234–1245. ISSN: 2150-8097. DOI: [10.14778/1687627.1687766](https://doi.org/10.14778/1687627.1687766).
- [154] Rainer Schlosser, Jan Kossmann, and Martin Boissier. “Efficient Scalable Multi-attribute Index Selection Using Recursive Strategies”. In: *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 2019, pp. 1238–1249. DOI: [10.1109/ICDE.2019.00113](https://doi.org/10.1109/ICDE.2019.00113).
- [155] S. Chaudhuri, M. Datar, and V. Narasayya. “Index selection for databases: a hardness study and a principled heuristic solution”. In: *IEEE Transactions on Knowledge and Data Engineering* 16.11 (2004), pp. 1313–1323. DOI: [10.1109/TKDE.2004.75](https://doi.org/10.1109/TKDE.2004.75).
- [156] Nicolas Bruno, César A. Galindo-Legaria, and Milind Joshi. “Polynomial heuristics for query optimization”. In: *Proceedings of the 26th International Conference on Data Engineering, ICDE 2010, March 1-6, 2010, Long Beach, California, USA*. Ed. by Feifei Li et al. IEEE Computer Society, 2010, pp. 589–600. DOI: [10.1109/ICDE.2010.5447916](https://doi.org/10.1109/ICDE.2010.5447916).
- [157] Alberto Caprara and Juan José Salazar González. “A Branch-and-Cut Algorithm for a Generalization of the Uncapacitated Facility Location Problem”. In: *Top* 4 (June 1996), pp. 135–163. DOI: [10.1007/BF02568608](https://doi.org/10.1007/BF02568608).

- [158] Anastasia Ailamaki and Stratos Papadomanolakis. “An Integer Linear Programming Approach to Database Design”. In: *Proceedings of the 2nd International Workshop on Self-Managing Database Systems (SMDB 2007)* (2007). URL: <http://infoscience.epfl.ch/record/210694>.
- [159] Sanjay Agrawal et al. “AutoAdmin: Self-Tuning Database Systems Technology”. In: *IEEE Data Eng. Bull.* 29.3 (2006), pp. 7–15. URL: <http://sites.computer.org/debull/A06sept/deb1.ps>.
- [160] Stratos Papadomanolakis, Debabrata Dash, and Anastasia Ailamaki. “Efficient Use of the Query Optimizer for Automated Physical Design”. In: *Proceedings of the 33rd International Conference on Very Large Data Bases. VLDB ’07*. Vienna, Austria: VLDB Endowment, 2007, pp. 1093–1104. ISBN: 9781595936493.
- [161] Neo4j. *Query Execution plan*. <https://neo4j.com/docs/cypher-manual/current/execution-plans/>. Accessed: April 25, 2023. 2021.
- [162] Jiachen Shi, Gao Cong, and Xiaoli li. “Learned Index Benefits: Machine Learning Based Index Performance Estimation”. In: *Proceedings of the VLDB Endowment* 15 (Jan. 2023), pp. 3950–3962. DOI: [10.14778/3565838.3565848](https://doi.org/10.14778/3565838.3565848).
- [163] Pavel Koupil et al. “MM-mapsearch: Workload-Aware Mapping Selection”. In: *Proceedings of EDBT 2026*. OpenProceedings.org, 2026. DOI: [10.48786/edbt.2026.68](https://doi.org/10.48786/edbt.2026.68).
- [164] Uta Störl, Meike Klettke, and Stefanie Scherzinger. “NoSQL Schema Evolution and Data Migration: State-of-the-Art and Opportunities (Tutorial)”. In: *Proceedings of EDBT ’20*. OpenProceedings.org, 2020, pp. 655–658.
- [165] Annabelle Gillet and Éric Leclercq. “A Categorical Representation of Multi-model Data to Prevent Data Migration Mismatch”. In: *Transactions on Large-Scale Data-and Knowledge-Centered Systems LIX: Special Issue on Data Management-Principles, Technologies, and Applications*. Springer, 2025, pp. 61–93.
- [166] Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. “Transpositions and Move Groups in Monte Carlo Tree Search”. In: *Proceedings of IEEE CIG ’09, Perth, Australia, 15-18 December, 2008*. IEEE, 2008, pp. 389–395.
- [167] Irena Holubová, Alžběta Šrůtková, and Jáchym Bártík. “Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs”. In: *Proceedings of ENASE 2025*. Jan. 2025, pp. 174–184. DOI: [10.5220/0013395300003928](https://doi.org/10.5220/0013395300003928).
- [168] Donald Feinberg et al. *Gartner Magic Quadrant for Operational Database Management Systems, 12 October 2015*. Oct. 2015.
- [169] Ran Tan et al. “Enabling query processing across heterogeneous data models: A survey”. In: *Proceedings of BigData 2017*. IEEE, 2017, pp. 3211–3220.
- [170] Rebecca Taft et al. “GenBase: a complex analytics genomics benchmark”. In: *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data. SIGMOD ’14*. Snowbird, Utah, USA: Association for Computing Machinery, 2014, pp. 177–188. ISBN: 9781450323765.

- [171] Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “TransforMMer: A Universal Multi-Model Data Generator”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 1150–1153. DOI: [10.48786/edbt.2025.111](https://doi.org/10.48786/edbt.2025.111).
- [172] Filip Ježek et al. “DortDB: Bridging Query Languages for Multi-Model Data Ponds”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5335–5338. ISSN: 2150-8097. DOI: [10.14778/3750601.3750665](https://doi.org/10.14778/3750601.3750665).
- [173] Xinbo Gao et al. “A Survey of Graph Edit Distance”. In: *Pattern Analysis and Applications* 13.1 (2010), pp. 113–129. DOI: [10.1007/s10044-008-0141-y](https://doi.org/10.1007/s10044-008-0141-y).
- [174] Sudarshan S. Chawathe et al. “Change Detection in Hierarchically Structured Information”. In: *Proceedings of SIGMOD '96*. ACM, 1996, pp. 493–504. DOI: [10.1145/233269.233366](https://doi.org/10.1145/233269.233366).
- [175] Jakub Klímek et al. “Atlas: A Toolset for Efficient Model-Driven Data Exchange in Data Spaces”. In: *Proceedings of MODELS 2023*. IEEE. 2023, pp. 4–8. DOI: [10.1109/MODELS-C59198.2023.00009](https://doi.org/10.1109/MODELS-C59198.2023.00009).
- [176] Jáchym Bártík. “AI-Powered Orchestration of Multi-Model Data”. In: *Proceedings of VLDB 2024 PhD Workshop*. Guangzhou, China: VLDB Endowment, Aug. 2024. URL: https://vldb.org/2024/files/phd-workshop-papers/vldb_phd_workshop_paper_id_2.pdf.

List of Figures

0.1	Multi-model data (running example). A subset of the Yelp Academic Dataset from Example 0.1, organized into different data models—relational (purple), document (green), graph (blue), and wide-column (red)—and connected via the same identifiers.	3
0.2	Schema category from Example 0.2, modeling the data from Figure 0.1 on the conceptual level.	12
0.3	A mapping between a logical schema for the wide-column database from Figure 0.1 and the conceptual schema from Figure 0.2. Complex types Friend and Attributes are omitted for simplicity. . .	13
0.4	Logical schemas for the relational, document, and graph databases from Figure 0.1.	14
0.5	A MMQL query from Example 0.5 and its translation to the MongoDB query language [26].	16
0.6	A MMQL query from Example 0.6 and its translation to SQL for a relational and a wide-column database.	16
0.7	Architecture of the MM-cat framework, showing the main tools and their interactions.	18
0.8	Hierarchy of components for evolution management. All changes are mediated by atomic operations (SMO , DDL , DML), which are propagated downward. Direct changes originating outside the system (i.e., data, query workloads) are observed and propagated upward by inferring the corresponding atomic operations.	22
0.9	Schema from Figure 0.2 is extended via the createProperty operation. The change is propagated to the logical schema.	23
0.10	The MMQL query from Figure 0.6 is manually updated to use the new deletedAt property added in Figure 0.9.	24
0.11	Schema from Figure 0.2 is transformed via the asMap operation. The change is propagated to the logical schema.	24
0.12	The MMQL query from Figure 0.5 is automatically fixed so that it uses the new Attribute map created in Figure 0.11.	25
0.13	The initial configuration of the running example and its state matrix. Columns are labeled with the first letter of the corresponding complex type.	33
0.14	First few steps of the search process from the initial configuration (0), either by modifying User (left) or Friend (right). Colors around the steps correspond to the databases from Figure 0.13. .	34
0.15	Benchmark creation steps from Example 0.12: inferred conceptual schema, refined conceptual schema, and three alternative logical representations.	42
0.16	Unified algebra representation of the queries from Example 0.13. The colors correspond to the original languages— SQL (purple), Cypher (blue), and XQuery (green). Some operations (e.g., calculations, tree joins) were omitted for clarity.	44

1.1	A sample schema category. Each violet node represents a complex object. In this case, all of them are mapped to respective tables of the relation model, e.g., PostgreSQL.	50
1.2	The example schema category from Fig. 1.1 after an evolution. The schema category did not change, but the affected tables (<i>Items</i> and <i>Orders</i>) were replaced by a collection of documents (<i>Order</i>), e.g., in MongoDB. The change introduced redundancy to query orders more efficiently.	51
1.3	Workflow of the initial framework. The full arrows represent the flow of information with the U symbols marking user inputs. The dashed lines represent the flow of data.	52
1.4	Evolution in the framework.	52
2.1	An example of multi-model data	57
2.2	ER (a) and categorical (b) schema of sample data	60
2.3	Relational-to-conceptual mapping	61
2.4	Architecture of <i>MM-evocat</i>	62
2.5	A screenshot of <i>MM-evocat</i>	62
2.6	EBNF syntax of MMSEL (Non-terminals are capitalised. Terminals are in quotation marks. Variables are in curly brackets.)	63
2.7	Heavy operation ungroup Address	64
2.8	Light operation group name, surname to Personal	65
3.1	A screenshot of <i>MM-evoque</i>	70
3.2	Version 1 – a naive mapping to the relational model (PostgreSQL)	71
3.3	Version 2 – a mapping to a normalized relational schema (PostgreSQL)	72
3.4	Version 3A – a mapping to the document model (MongoDB)	73
3.5	Version 3B – a multi-model mapping (PostgreSQL and MongoDB)	74
4.1	Schema evolution and its propagation to data	83
4.2	An example of application of various strategies to propagate structural changes to the queries	89
5.1	Input dataset.	99
5.2	Positive (red) and negative (yellow) examples.	99
5.3	An example of lattice for class c_A (i.e., RHS: A).	99
5.4	A high-level overview of the FDepHunter workflow.	100
5.5	Iterative (top-down) editing of maximal sets and elimination of fake FDs utilizing negative examples.	100
5.6	Armstrong relation and workflow status.	102
5.7	Evaluation of negative example.	103
6.1	Supported index types in Neo4j	109
6.2	Example of relational tables	117
6.3	General architecture of the index selection algorithms	118
6.4	CoPhy binary integer program	120
6.5	AutoAdmin architecture	124
6.6	NoDBA learning process	130
6.7	A Cypher query example.	134
7.1	Categorical input representation of the schema and workload.	143
7.2	Exploration of the mapping space using MCTS.	143
7.3	Plan-structured neural predictor for estimating query performance.	144

7.4	Architecture.	146
7.5	Input and workload selection.	147
7.6	Mapping results and statistics.	148
8.1	Input single-model JSON document collections and inferred initial schema category	156
8.2	Edited (improved) schema category from Fig. 8.1	156
8.3	Schema category from Fig. 8.2 mapped to the relational model (scenario A)	157
8.4	Schema category from Fig. 8.2 mapped to relational and document model (scenario B)	158
8.5	Schema category from Fig. 8.4 mapped to relational, document, and wide-column model (scenario C)	158
8.6	MMQL query over the improved schema category in Fig. 8.2 . . .	159
8.7	MongoDB QL query over data from Fig. 8.2	159
8.8	MMQL query over schema category from Fig. 8.3	160
8.9	SQL query over data from Fig. 8.3 (scenario A)	160
8.10	SQL/JSON query over data from Fig. 8.4 (scenario B)	160
8.11	CQL and SQL queries over data from Fig. 8.5 (scenario C)	161
8.12	Architecture of the framework	162
9.1	Input Yelp dataset in the document model	168
9.2	Yelp dataset in the graph and relational model	168
9.3	Initial schema category $\mathbf{S}_{ini}$ inferred from the sample data in Fig. 9.1	168
9.4	The improved schema category $\mathbf{S}_{imp}$ of the Yelp data containing identifiers, a reference, and a map of properties	170
9.5	Screenshot of <i>TransforMMer</i> before and after the elimination of a recursive structure	171
9.6	$\mathbf{S}_{imp}$ of the Yelp data re-mapped to the graph (blue) and relational (purple) model to represent data in Fig. 9.2	172
9.7	The workflow of <i>TransforMMer</i>	172
10.1	Sample multi-model data (a), cross-model query (b)	176
10.2	Data import and query evaluation workflow in DortDB.	177
10.3	Scenario A – testing and validation GUI for DortDB	180
10.4	Scenario B – TypeScript API for DortDB	181

List of Tables

1	Papers of the thesis. Rankings are according to CORE/JCR. . . .	1
3.1	Basic SMOs	75
3.2	Basic SMOs and their impact on MMQL query constructs	76
4.1	Basic SMOs	84
4.2	Composite SMOs	85
4.3	Complex SMOs	85
4.4	Time [s] needed for propagation of SMOs	92
4.5	An experimental comparison of query synchronization strategies .	93
4.6	A comparison of features of selected evolution management approaches	94
6.1	Comparison of different DBMSs	114
6.2	Table of Notations	116
6.3	Table of costs for C_1	121
6.4	Table of costs for C_2	121
6.5	Table of costs for C_3	121
6.6	Table of storage consumptions	122
6.7	Costs for configurations of size $ I - 1$	123
6.8	Table of single-column index candidates	128
6.9	Benefit ratios of extended index candidate sets	129
6.10	Comparison of index selection algorithms	133
8.1	Comparison of approaches using different metrics	163
8.2	User interaction needed in particular scenarios for the Yelp dataset	163
10.1	Operators of the Unified Algebra	178

List of Abbreviations

ACID Atomicity, Consistency, Isolation, Durability. [7](#)

BASE Basically Available, Soft state, and Eventual consistency. [7](#)

DAG Directed Acyclic Graph. [34](#)

DDL Data Definition Language. [14](#), [20](#), [21](#), [22](#), [23](#), [184](#), [201](#)

DML Data Manipulation Language. [14](#), [20](#), [21](#), [22](#), [184](#), [201](#)

ER Entity-Relationship. [10](#), [12](#), [18](#)

JSON JavaScript Object Notation. [3](#), [7](#), [13](#)

MCTS Monte Carlo Tree Search. [34](#), [37](#), [183](#)

MDA Model-Driven Architecture. [9](#)

MMQL Multi-Model Query Language. [15](#), [16](#), [17](#), [21](#), [23](#), [24](#), [25](#), [27](#), [35](#), [41](#), [201](#)

NoSQL Not Only SQL. [3](#), [7](#), [14](#), [27](#), [30](#), [36](#)

PIM Platform-Independent Model. [9](#)

PSM Platform-Specific Model. [9](#)

SMO Schema Modification Operation. [21](#), [22](#), [23](#), [25](#), [32](#), [41](#), [184](#), [201](#)

SQL Structured Query Language. [14](#), [15](#), [16](#), [19](#), [42](#), [43](#), [44](#), [45](#), [46](#), [201](#)

UML Unified Modeling Language. [18](#)

List of Publications

1. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of CIKM 2022*. Atlanta, GA, USA: ACM, 2022, pp. 4892–4896. ISBN: 9781450392365. DOI: [10.1145/3511808.3557180](https://doi.org/10.1145/3511808.3557180)
2. Jakub Klímek et al. “Atlas: A Toolset for Efficient Model-Driven Data Exchange in Data Spaces”. In: *Proceedings of MODELS 2023*. IEEE. 2023, pp. 4–8. DOI: [10.1109/MODELS-C59198.2023.00009](https://doi.org/10.1109/MODELS-C59198.2023.00009)
3. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evoque: Query Synchronisation in Multi-Model Databases”. In: *Proceedings of EDBT 2024*. OpenProceedings.org, 2024, pp. 818–821. DOI: [10.48786/edbt.2024.78](https://doi.org/10.48786/edbt.2024.78)
4. Jáchym Bártík. “AI-Powered Orchestration of Multi-Model Data”. In: *Proceedings of VLDB 2024 PhD Workshop*. Guangzhou, China: VLDB Endowment, Aug. 2024. URL: https://vldb.org/2024/files/phd-workshop-papers/vldb_phd_workshop_paper_id_2.pdf
5. Jáchym Bártík, Pavel Koupil, and Irena Holubová. “Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of SAC 2024*. ACM, 2024
6. Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “TransforMMer: A Universal Multi-Model Data Generator”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 1150–1153. DOI: [10.48786/edbt.2025.111](https://doi.org/10.48786/edbt.2025.111)
7. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “Model-Agnostic Evolution Management”. In: *Proceedings of RCIS 2025*. Vol. 547. LNCS. Springer, 2025, pp. 245–261. DOI: [10.1007/978-3-031-92474-3_15](https://doi.org/10.1007/978-3-031-92474-3_15)
8. Irena Holubová, Alžběta Šrůtková, and Jáchym Bártík. “Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs”. In: *Proceedings of ENASE 2025*. Jan. 2025, pp. 174–184. DOI: [10.5220/0013395300003928](https://doi.org/10.5220/0013395300003928)
9. Filip Ježek et al. “DortDB: Bridging Query Languages for Multi-Model Data Ponds”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5335–5338. ISSN: 2150-8097. DOI: [10.14778/3750601.3750665](https://doi.org/10.14778/3750601.3750665)
10. Pavel Koupil et al. “FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5227–5230. ISSN: 2150-8097. DOI: [10.14778/3750601.3750612](https://doi.org/10.14778/3750601.3750612)
11. Filip Mihál, Jáchym Bártík, and Pavel Koupil. “Refining Storage Strategy through Index Selection Methods in Multi-Model Database Systems: A Survey”. In: *Data & Knowledge Engineering* 164 (2026), p. 102582. ISSN: 0169-023X. DOI: [10.1016/j.datak.2026.102582](https://doi.org/10.1016/j.datak.2026.102582)

12. Pavel Koupil et al. “MM-mapsearch: Workload-Aware Mapping Selection”. In: *Proceedings of EDBT 2026*. OpenProceedings.org, 2026. DOI: [10.48786/edbt.2026.68](https://doi.org/10.48786/edbt.2026.68)
13. Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “Benchmark Engineering as a Design Instrument for Heterogeneous Information Systems”. In: *arXiv* (2026). Manuscript under second-round review. eprint: [2607.07175](https://arxiv.org/abs/2607.07175) (cs.DB). URL: <https://arxiv.org/abs/2607.07175>
14. Filip Ježek et al. *One Algebra to Query Them All: A Schema-Agnostic Foundation for Cross-Model Querying*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/unified-algebra.pdf>
15. Jáchym Bártík, Filip Štrobl, and Irena Holubová. *Reducing the Cross-Model Tax: Query Optimization over Multi-Model Data*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/query-opt.pdf>